

Catheter Placement

Which convolutional neural network out of ResNet, DenseNet, UNet or a custom neural network can accurately assess the placement accuracy of the ETT, NGT and CVC?

Bachelor of Applied Data Science | ADS2002 | Data Challenges 4



Source: <https://www.holidayscalendar.com/event/world-radiography-day/>

Report completed by:

Carlos Weffer Perez [33154414]

Emma Kelsall [32592086]

Fiona George [33154430]

Kevin Sun [33049440]

Pooja Kampli [33154759]

Ryan Garg [33154325]

TABLE OF CONTENTS

Executive Summary	03
Introduction	
I. Background.....	04
II. Supplied Data.....	05
Data Quality	06
I. Data Wrangling and Processing	07
Exploratory Data Analysis	08
Model Development and Training	
I. Initial Models	
A. SVM	09
B. Custom-Built Neural Network	10
C. ResNet	13
D. DenseNet	15
E. UNet	16
Results	
I. Model Accuracies	19
Conclusion	20
References	22

EXECUTIVE SUMMARY

Catheter Placement is a critical aspect of modern healthcare, ensuring the correct positioning of life-sustaining medical devices. However, this process is often vulnerable to human error, potentially leading to severe complications. In this project, we tackled this issue by developing deep learning models to precisely detect and classify the placement accuracy of Endotracheal Tubes (ETT), Nasogastric Tubes (NGT), and Central Venous Catheters (CVC). Our objective was to determine which convolutional neural network out of ResNet, DenseNet, UNet, or a custom-built neural network could best accomplish this task.

The project began with a comprehensive exploration of the dataset supplied by Kaggle (*RANZCR CLiP - Catheter and Line Position Challenge*, n.d.), which included chest X-ray images and relevant annotations. These datasets were of high quality, with no missing values, allowing us to focus on data analysis and modelling. We implemented various machine learning models to address the classification task. Our initial SVM model offered a solid foundation, but we transitioned to convolutional neural networks (CNNs) due to their superior image analysis and classification capabilities. We crafted a Custom-Built Neural Network, then adopted the ResNet-50 model, a deep CNN trained on the ImageNet database. Additionally, we explored the DenseNet121 architecture and UNet, a specialised image segmentation model. The ResNet and DenseNet models showcased the potential of convolutional neural networks, revealing both their strengths and limitations. However, it was the UNet model that outperformed the others with an accuracy rate of 80% and excelling in reducing false negatives.

The analysis showcases the potential of utilising deep learning models to enhance patient safety by improving the accuracy of catheter placement assessments. Through these models, the healthcare industry has the opportunity to reduce complications associated with incorrect catheter placements, which is of paramount significance, especially in high-stress medical procedures.

In essence, this project addresses a critical healthcare concern by developing deep learning models for precise catheter placement assessment in medical images. Our multifaceted approach highlights the significant potential of artificial intelligence in medical applications. These achievements emphasise the pivotal role of machine learning in healthcare and its broader implications for solving real-world problems.

INTRODUCTION

BACKGROUND

In the realm of modern healthcare, the correct placement of various lines and tubes in patients is a critical factor in ensuring their well-being. Medical practitioners, including doctors and nurses, often rely on comprehensive checklists to guide the precise placement of life-sustaining equipment. These checklists are designed to follow established protocols for the safe and effective management of patients. However, even with these safeguards in place, the process can still be time-consuming and, more significantly, vulnerable to human error, especially in high-stress situations when hospitals are operating at maximum capacity (RANZCR CLiP - Catheter and Line Position Challenge, n.d.).

Patients admitted to hospitals routinely require the insertion of catheters and lines as part of their treatment. Incorrect placement of these devices can lead to severe complications. For instance, nasogastric tube misplacement into the airways has been reported in up to 3% of cases, with as many as 40% of these instances resulting in complications. Additionally, malpositioning of airway tubes in adult patients intubated outside the operating room occurs in up to 25% of cases. The likelihood of complications is closely linked to the experience level and specialty of the medical practitioner involved. Recognizing malpositioned tubes early is paramount in preventing hazardous complications, including potential fatalities. This is particularly pertinent given the current healthcare landscape, with millions of COVID-19 patients requiring these essential devices.

This project aims to deal with this concern by creating a deep learning model that is capable of precisely identifying the presence or absence of Catheters. Moreover, it seeks to accurately detect instances of malposition. In this context, our objective is to determine which as of the ETT, NGT, and CVC.

SUPPLIED DATA

Kaggle was the supplier of the data for this project. This data consisted of a folder containing two csv files and two folders containing chest x-ray images to be used for training and testing. Two TFRecord (TensorFlow Record) files were also included for training and testing purposes. These

are Tensorflow's own binary storage format and take up less disk space meaning that they can be read more efficiently from a disk (Gamauf, 2018).

The two csv files which were supplied were train.csv and train_annotations.csv. The train.csv file represents the data that our model should be trained on. This dataset features an entry for each image available in the train folder, with the 'StudyInstanceUID' column corresponding to the name of the jpeg file, totalling 30083 entries/images. Each entry also consists of a patientId identifier which represents the patient that the x-ray image corresponds to, there are 3255 unique patients indicating that one patient can have more than one image. There are then 11 binary labels which indicate the presence of 4 different catheters and whether they are correctly or incorrectly placed. The following figure (figure. 1) further outlines the variables featured in our dataset.

Column Name	Data Type	Description
StudyInstanceUID	Object (String)	Unique ID for each image in Train Folder
ETT - Abnormal	int64	Endotracheal Tube placement abnormal
ETT - Borderline	int64	Endotracheal Tube placement borderline abnormal
ETT - Normal	int64	Endotracheal Tube placement normal
NGT - Abnormal	int64	Nasogastric tube placement abnormal
NGT - Borderline	int64	Nasogastric tube placement borderline abnormal
NGT - Incompletely Imaged	int64	Nasogastric tube placement inconclusive due to imaging
NGT - Normal	int64	Nasogastric tube placement normal
CVC - Abnormal	int64	Central venous catheter placement abnormal
CVC - Borderline	int64	Central venous catheter placement borderline abnormal
CVC - Normal	int64	Central venous catheter placement normal
Swan Ganz Catheter Present	int64	A swan-ganz catheter is present
PatientID	Object (String)	Unique ID for each patient in the dataset.

Figure. 1 - Description of train.csv data set

The train_annotations.csv file provides annotations for some images in the train data set. This dataset features 17999 entries with 9095 unique images, indicating that some images contain

annotations for multiple different catheters/tubes.

Using these datasets, the trainWannot.csv dataset was created which merges the train and train_annotations files together on 'StudyInstanceUID' using a left join. This dataset could then be used in exploratory data analysis.

DATA QUALITY

Clean datasets were provided for the purposes of this project. Using .isna().sum() it was found that both train.csv and train_annotations.csv datasets contained no missing values with each 'StudyInstanceUID' correctly corresponding to an image in the train folder. This meant that no imputation was required and data analysis and modelling could be performed with little data manipulation.

To develop a further understanding of the train dataset, the descriptive statistics were pulled. Using the mean, the likely distribution of the labels could be identified. The higher the mean indicating the more heavy the distribution.

	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT - Abnormal	NGT - Borderline	NGT - Incompletely Imaged	NGT - Normal	CVC - Abnormal	CVC - Borderline	CVC - Normal	Swan Ganz Catheter Present
count	30083.000000	30083.000000	30083.000000	30083.000000	30083.000000	30083.000000	30083.000000	30083.000000	30083.000000	30083.000000	30083.000000
mean	0.002626	0.037829	0.240667	0.009274	0.017585	0.091347	0.159459	0.106206	0.281222	0.708839	0.027590
std	0.051179	0.190785	0.427496	0.095857	0.131438	0.288107	0.366110	0.308106	0.449603	0.454305	0.163799
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000
75%	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	1.000000	0.000000
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

Figure 2 - Descriptive Statistics of train.csv

DATA WRANGLING AND PROCESSING

Given the large size of the dataset (consisting 30083 images) it was important to first consider the machine power available to the group. It was decided that using all 30083 images was infeasible and thus the dataset should be subsetted to become more manageable. Upon exploratory data analysis, it was identified that CVC (Central Venous Catheter) had the most

instances combining the Normal, Abnormal and Borderline instances. For all initial models a dataset called CVC_train was used to represent this subset.

```
#extracting just the CVC labels
train_data = pd.read_csv("/content/drive/MyDrive/ADS2002-Project-CatheterPlacement/Data/train.csv")
CVC_train = train_data[["StudyInstanceUID", "CVC - Abnormal", "CVC - Borderline", "CVC - Normal"]]
CVC_train.set_index('StudyInstanceUID', inplace=True)
CVC_train.head()
```

	CVC - Abnormal	CVC - Borderline	CVC - Normal
StudyInstanceUID			
1.2.826.0.1.3680043.8.498.26697628953273228189375557799582420561	0	0	0
1.2.826.0.1.3680043.8.498.46302891597398758759818628675365157729	0	0	1
1.2.826.0.1.3680043.8.498.23819260719748494858948050424870692577	0	1	0
1.2.826.0.1.3680043.8.498.68286643202323212801283518367144358744	1	0	0
1.2.826.0.1.3680043.8.498.10050203009225938259119000528814762175	0	0	1

Figure 3 - Extracting CVC Labels

Balancing the data

As discovered during the exploratory data analysis phase, the large majority of observations out of the CVC catheters are normally aligned. Thus, to balance the data we performed some under sampling so as to ensure an equal number of representations of Abnormal, Normal, and Borderline observations within the training and testing data. This approach prevents the model from developing a bias towards predicting the most prevalent alignment type within the dataset.

Downsizing the images

Given the considerably low initial machine power available to us, each image had to be processed into a format that was consistent and could be used in modelling. This involved ensuring that the images were all the same pixel size. 600 x 600 was chosen as the ideal size. For CVC models 1500 images were able to be processed and used in our models. The images were then stored in a numpy array. Only 500 images were able to be processed for ETT and NGT models due to undersampling methods performed on the data to remain consistent for all models.

EXPLORATORY DATA ANALYSIS

Our journey in understanding the catheter and line placement dataset began with a comprehensive exploration of its key components. This exploration was instrumental in gaining insights into the dataset's structure and characteristics.

The first aspect of the datasets that was explored was their dimensionality. The train dataset featured 30083 entries indicating that the dataset contained this many unique images. Along with this it was identified that these 30083 images belonged to 3255 unique patients. To pair with this, the train annotations data set was found to have 17999 entries with annotations belonging to 9095 unique images indicating that a large majority of these images consisted of multiple catheters/lines.

Next the distribution of different labels was explored. In figure 4 it can be seen that normal catheter placement is most common followed by borderline. It could also be seen that the Central Venous Catheter had by far the most observations with over 30000 indicating that images can not only feature multiple catheters but multiple of the same catheter.

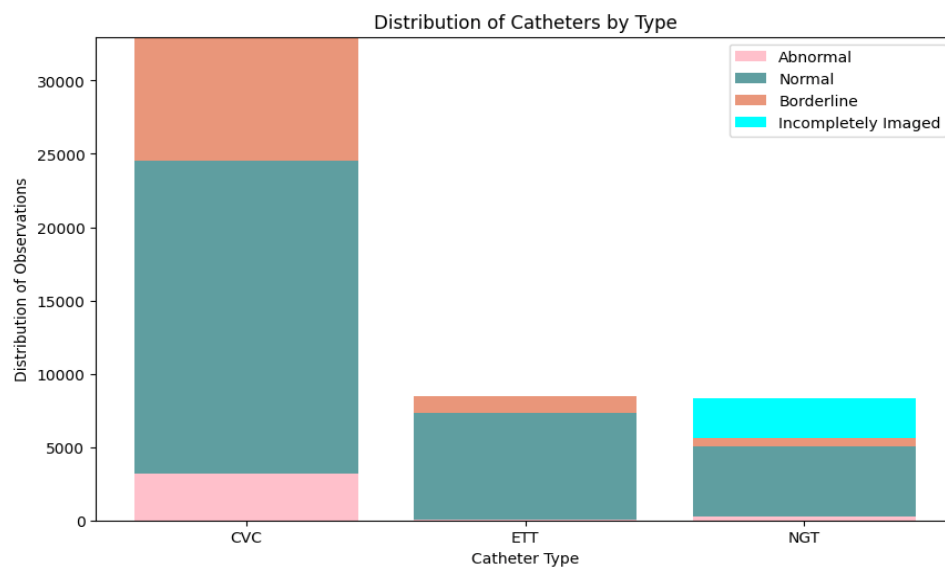


Figure 4 - Distribution of Catheters by type

A histogram (figure 5) was then created to show the number of observations/images per patient. It was interesting to see that there were a couple of patients who had upwards of 100 observations. This presented an interesting question: Why did some patients require so many observations?

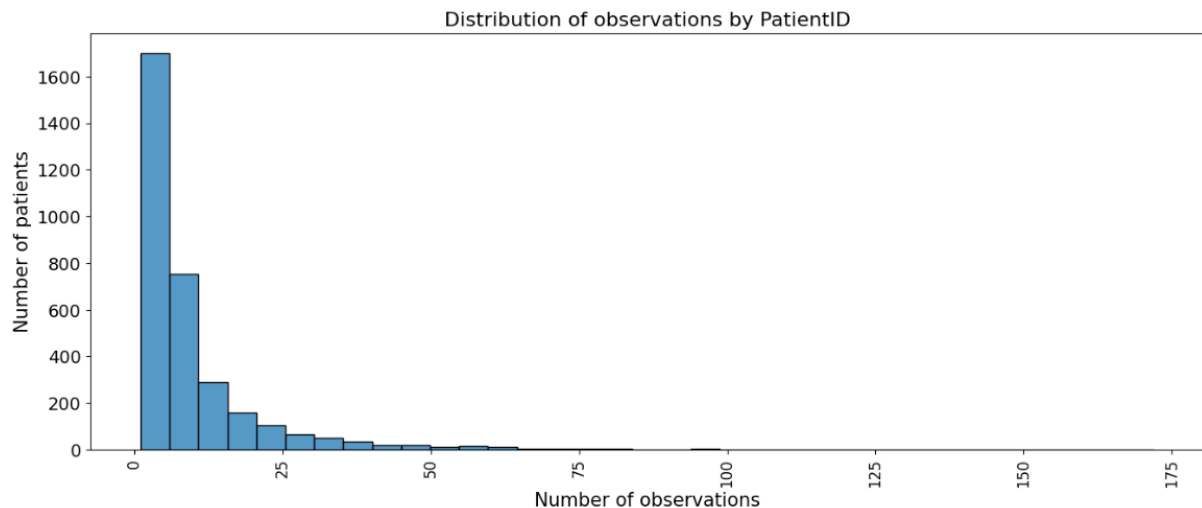


Figure 5 - Distribution of observations by PatientID

Having made these observations, the next thing to observe was the actual placement of catheters in an x-ray image. At first glance each catheter was very difficult to identify. In an attempt to make them easier to see, the annotations from the train annotations dataset were plotted onto x-rays. In doing so, this made it easier to identify what made a catheter borderline vs normal. Each individual catheter was plotted separately then side by side to allow a clear understanding.

MODEL DEVELOPMENT

For the modelling aspect, multiple deep learning methods were produced in an attempt to find the best model for our problem. We pursued five different types of machine learning models: **SVM, Resnet, Densenet, Unet** and a **Custom-Built Neural Network**.

SVM MODEL

The Support Vector Machine (SVM) model was our initial approach to the catheter placement classification task. We pre-processed a limited dataset of 500 images, standardising them to a 600x600 pixel size with 3-channel representations. Labels for CVC catheter placements were structured for multi-class classification.

The SVM model, using a linear kernel, achieved an accuracy of 0.44. While it demonstrated some success, this accuracy fell short of capturing complex spatial relationships within the images. The decision to transition to convolutional neural networks (CNNs) was driven by the need for more advanced models capable of handling high-dimensional image data and capturing intricate spatial features.

The SVM model served as an essential stepping stone, offering initial insights and understanding. However, for the nuanced catheter placement classification task, we needed the depth and capacity of CNNs to effectively analyse complex image features. This transition allowed us to delve deeper into the spatial intricacies and complexity of medical images, enhancing our classification capabilities.

CUSTOM-BUILT NEURAL NETWORK

For our first neural network model, a keras neural network was built using TensorFlow. This model was made up of 4 layers, an input layer which flattened the image, 2 dense ReLu activation layers with 300 and 100 nodes respectively, and an output layer with 3 nodes for the three classifications of the catheters (Normal, Abnormal, Borderline) (Team, n.d.).

We trained this model on 1000 grayscale images, each 600 x 600 pixels wide. 70 epochs were run and the results featured in the plots below.

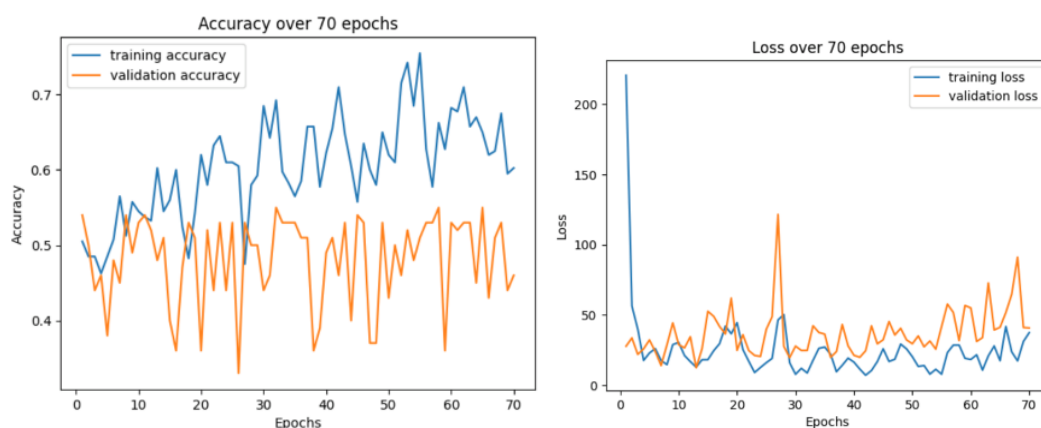


Figure 6 - Accuracy and Loss of Custom Built Model over 70 epochs

The training accuracy continues to increase over the 70 epochs and peaks at around 0.75, whereas the validation accuracy remains around 0.4 – 0.5 over all the epochs.

The training and validation loss both exhibit the same decreasing pattern over the 70 epochs.

Example model prediction:

Here is the model being applied to a specific example and the probabilities it predicted for each class for the chosen catheter image that had borderline alignment.

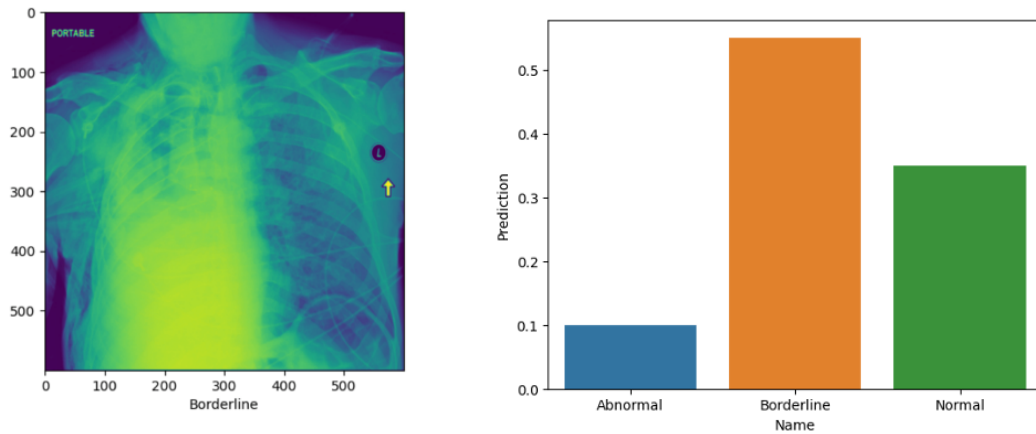


Figure 7 - X-ray image and probabilities for catheter prediction

In the example provided, the model successfully identified a "Borderline" alignment with a probability of around 0.55. However, it also assigned a probability of approximately 0.35 to a "Normal" alignment, indicating a lack of strong confidence in this classification.

Binary Classification With Keras Neural Network

After doing this the task of classification was simplified for the model in order to increase the accuracy. To do this we made it so the model would only have to determine whether the catheter was normal or abnormal, rather than one of the three classifications (Normal, Borderline, Abnormal). Additionally, we utilised annotated images with the catheters outlined when training the model. It was hoped that this would increase the model's ability to detect a catheter.

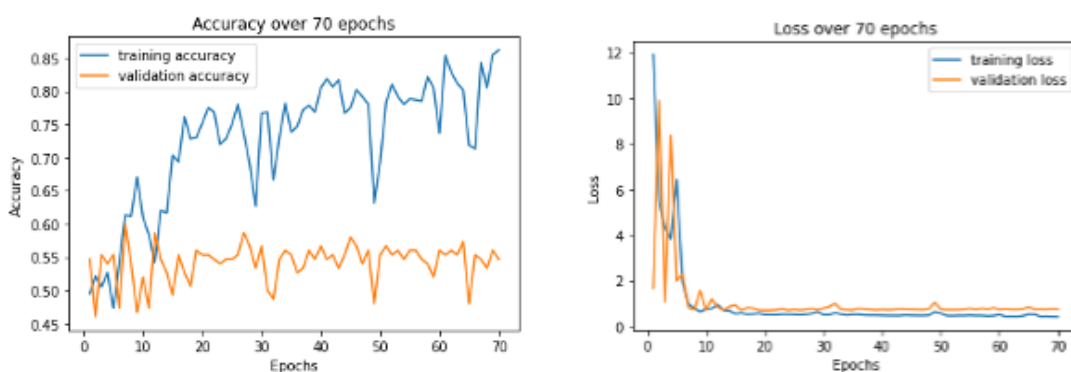
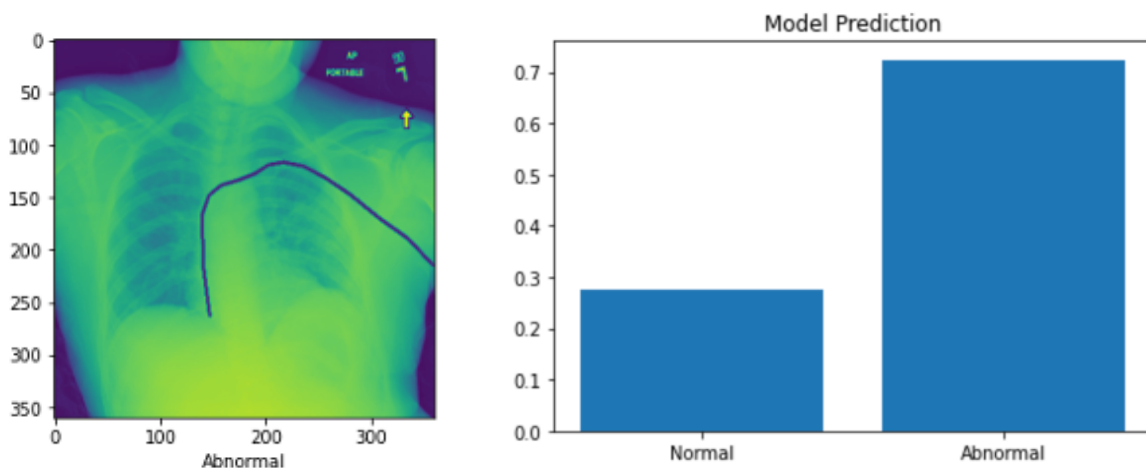


Figure 8 - Accuracy and Loss of Custom Built Model over 70 epochs

Example model prediction:

Here is the model being applied to a specific example and the probabilities it predicted for the catheter having normal or abnormal alignment.

Figure 9 - Example Model Prediction (CVC catheter)



For this example, it was able to correctly classify the catheter's alignment as normal, however it was not very confident in its estimate and resulted in the model not producing consistently accurate results for the rest of the testing data.

These two initial models laid a solid foundation for the beginning of our data analysis, providing a starting point for our work. We used these models as a benchmark for the evaluation of the performance of our future models.

RESNET MODEL

In this project, we utilised the power of ResNet-50, a convolutional neural network designed for image classification tasks and found in tensorflow.keras. This particular model, ResNet-50, is a pre-trained network that delves deep with 50 layers and is trained on the extensive ImageNet Database (*ResNet: The Basics and 3 ResNet Extensions*, n.d.). While ResNet-101 and ResNet-152 were viable alternatives with even more layers, we opted for ResNet-50 for simplicity in working with a pre-built model for the first time .

Our approach for both the ResNet and DenseNet models was to break down the overarching challenge into smaller, more manageable sub-models. Our ultimate goal was to combine these into one comprehensive model which resembled a tree. Although constructing a single, all-encompassing model to predict catheter normality or abnormality proved manageable, unfortunately due to time constraints we were only successful in building three smaller models, one for each catheter type (CVC, ETT, and NGT). Combining these into one was not achieved.

To create these models, we divided the task into two steps. First, we addressed the prediction of whether an X-ray image contained the specified catheter (e.g., CVC - present). This entailed a straightforward binary classification task, where 1 denoted catheter presence and 0 indicated its absence. Subsequently, our focus shifted to predicting catheter placement, with 1 signifying normal placement and 0 representing abnormal placement.

These two distinct tasks were integrated into a single model. Our first job was to customise the ResNet model to suit our specific objectives. This entailed the incorporation of dropout and batch normalisation techniques to enhance the model's generalisation capability. We then established output branches, with one branch dedicated to indicating catheter presence and the other to predicting normality. These output branches were subsequently merged to form the final model. The model was compiled using the Stochastic Gradient Descent optimizer. Notably, we crafted a unique model for each catheter type to address the diverse requirements of CVC, ETT, and NGT catheters.

For this model, undersampling was performed on our training set. It was compiled using x-ray images which featured a 50-50 split of desired catheter to other catheters then those that featured the catheter were split 50-50 between normal and abnormal. This was to ensure that there was no bias in the training set. Due to the small sample size of abnormal ETT and NGT catheters, the training set for these two models was very small (<500 images) thus the results achieved were not very reliable. This was compared to the CVC model which was able to feature 1500 images.

The table below shows the results of the 3 models, including where their training and validation accuracy peaked, as well as their testing accuracy. For each model the training accuracy peaked at 1.00 after 4 epochs, but continued running for 15 epochs. This indicated that all 3 models were

over-fitting the data despite achieving relatively good testing accuracy results as pictured in figure 11.

This is also evident in the plot of model loss (for CVC - Figure 10) where validation loss increases over epochs while the training loss decreases quite rapidly. Despite this, the confusion matrix produced results that we were after, with the majority of predictions being true positives and true negatives. For the CVC model, there were quite a few false negatives which indicate that the model has predicted that a patient's catheter was placed correctly despite it actually being abnormal. This is dangerous and can put patients at risk, thus future models will want to reduce this.

To try to improve these results and reduce overfitting, 3 techniques were added to the model. This included ReduceLROnPlateau which reduces the learning rate when the validation loss plateaus, early stopping which is meant to stop overfitting and model checkpoint which saves the best model during training. Despite adding these features, little improvement was seen, this could potentially be due to the size of the dataset. In the future, it could be useful to try oversampling techniques to increase the minority side of the dataset and thus increase the size of the dataset.

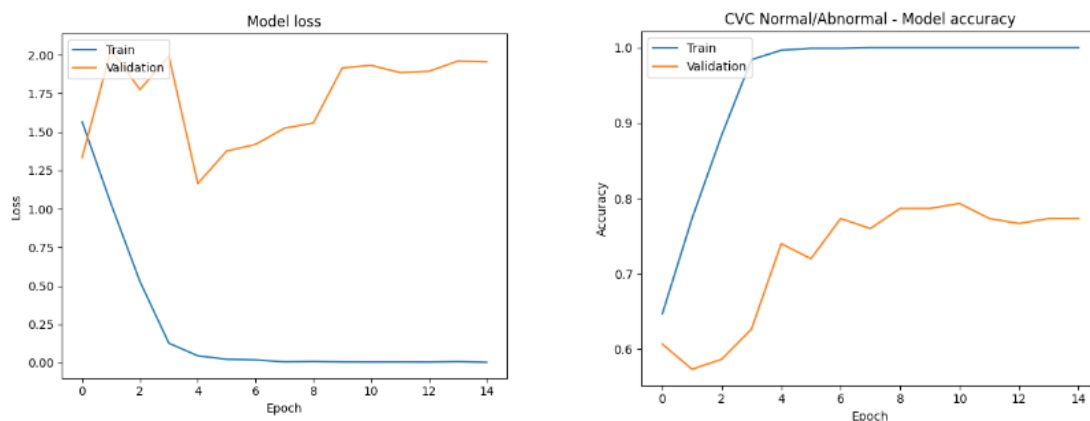


Figure 10 - Loss and Accuracy plots for CVC model

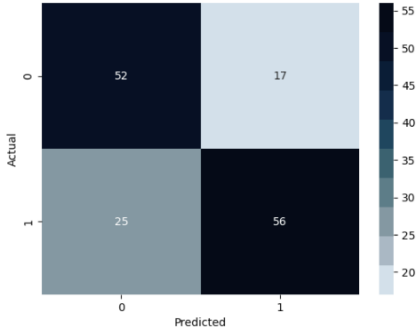
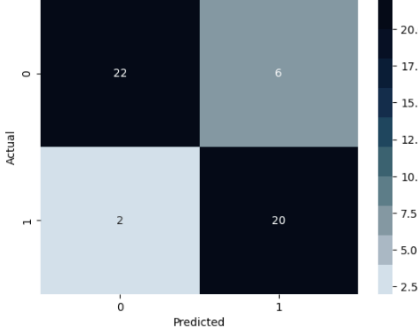
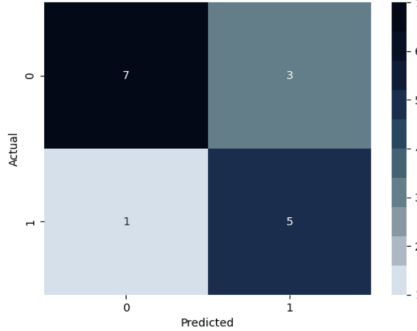
Central Venous Catheter (CVC)	Nasogastric Tube (NGT)	Endotracheal Tube (ETT)																											
Training accuracy peaked at 1.00 Validation accuracy peaked at 0.78 Testing accuracy was 0.72 Confusion Matrix - Normal/Abnormal  <table border="1"> <thead> <tr> <th>Actual \ Predicted</th> <th>0</th> <th>1</th> </tr> </thead> <tbody> <tr> <th>0</th> <td>52</td> <td>17</td> </tr> <tr> <th>1</th> <td>25</td> <td>56</td> </tr> </tbody> </table>	Actual \ Predicted	0	1	0	52	17	1	25	56	Training accuracy peaked at 1.00 Validation accuracy peaked at 0.84 Testing accuracy was 0.84 Confusion Matrix - Normal/Abnormal  <table border="1"> <thead> <tr> <th>Actual \ Predicted</th> <th>0</th> <th>1</th> </tr> </thead> <tbody> <tr> <th>0</th> <td>22</td> <td>6</td> </tr> <tr> <th>1</th> <td>2</td> <td>20</td> </tr> </tbody> </table>	Actual \ Predicted	0	1	0	22	6	1	2	20	Training accuracy peaked at 1.00 Validation accuracy peaked at 0.83 Testing accuracy was 0.82 Confusion Matrix - Normal/Abnormal  <table border="1"> <thead> <tr> <th>Actual \ Predicted</th> <th>0</th> <th>1</th> </tr> </thead> <tbody> <tr> <th>0</th> <td>7</td> <td>3</td> </tr> <tr> <th>1</th> <td>1</td> <td>5</td> </tr> </tbody> </table>	Actual \ Predicted	0	1	0	7	3	1	1	5
Actual \ Predicted	0	1																											
0	52	17																											
1	25	56																											
Actual \ Predicted	0	1																											
0	22	6																											
1	2	20																											
Actual \ Predicted	0	1																											
0	7	3																											
1	1	5																											

Figure 11 - Results of ResNet-50 Models

DENSENET MODEL

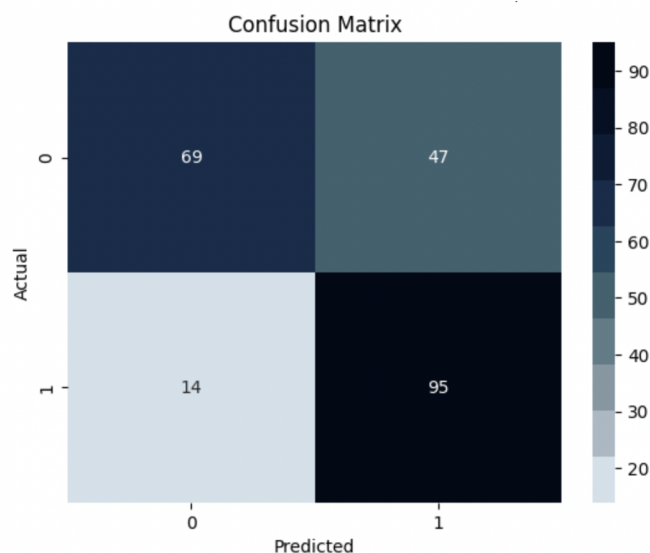
Following the exploration of ResNet, DenseNet was also explored to see if this pre-built model could improve our accuracy scores and reduce overfitting. For our task DenseNet121 was utilised. DenseNet architectures are known for their dense connectivity between layers which aids in mitigating the vanishing gradient problem and strengthens feature propagation.

The approach taken for the DenseNet models was much the same as for the ResNet model following the same structure. This also included the same preparation of the dataset to ensure comparability of results. Given the results of the ETT and NGT models in ResNet it was chosen to simply just explore the CVC model for DenseNet as this would serve as the best comparison.

Our model was trained for up to 30 epochs. After training, the model was evaluated on a separate test dataset, resulting in a test accuracy of 72.89%

To visualise the performance of our CVC model, a confusion matrix was plotted using the Seaborn library. The heatmap provides a visual representation of the True Positives, True Negatives, False Positives, and False Negatives that the model predicted.

Figure 12 - Confusion Matrix (CVC Abnormal or Normal Classification)



From the confusion matrix we could see our model correctly identified 95 samples as normal and 69 samples as abnormal.

Our model made mistakes on 47 samples by predicting them as normal when they were actually abnormal and on 14 samples by predicting them as abnormal when they were actually normal.

In general, a model that has higher values on the diagonal and lower values off the diagonal is performing well. From the heatmap, it seems the model is doing reasonably well with some errors, especially in the False Positive category. While performing much the same as the ResNet model, the amount of false negatives has been reduced using DenseNet indicating that this model could pose a better alternative.

UNET MODEL

The U-Net model was the final model we tried and is a convolutional neural network model specifically designed for image segmentation. The U-Net architecture consists of two main parts, the encoder and decoder. An example architecture diagram for a 572x572 input image is below (figure 14). The encoder is the left side of the U-shaped diagram, this part shrinks the image down through convolutional layers and max pooling. The decoder then upscales the image through the up convolutional layers, also known as the transpose convolution layer. The model then outputs a segmented image (Ronneberger, 2015). Our version of U-Net also outputs a classification, which is done through averaging the last layer of feature maps. The classification is

then interpreted through a sigmoid activation function and a threshold of 0.5 to determine the classification.

Using the `segmented_models_pytorch` package, we loaded in a pre-trained UNet model that was trained on the ImageNet database. The ImageNet database is a collection of 1.2 million images with similar formatting and in order to take full advantage of the pre-trained weights, we formatted the images to be in line with the ImageNet standards. To use the segmented model efficiently, we had to use the annotated images, as this is the optimal training method.

However, to increase the quality of the images, we made the annotations more prominent through a series of transformations. Specifically, we connected the blue dots that the image was provided with, as well as making the annotation colour red, as this made it extremely visible over the darker background. An example image after all transformations is presented below:

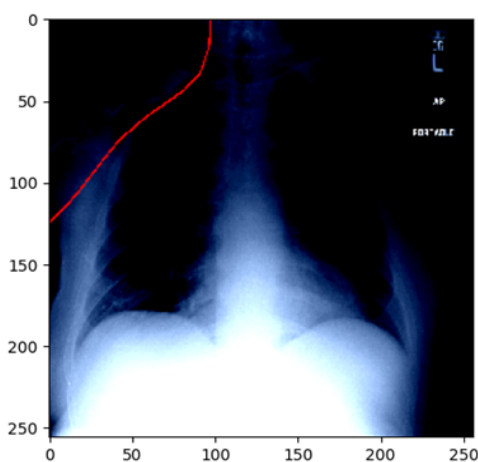


Figure 13 - X-ray image with transformation

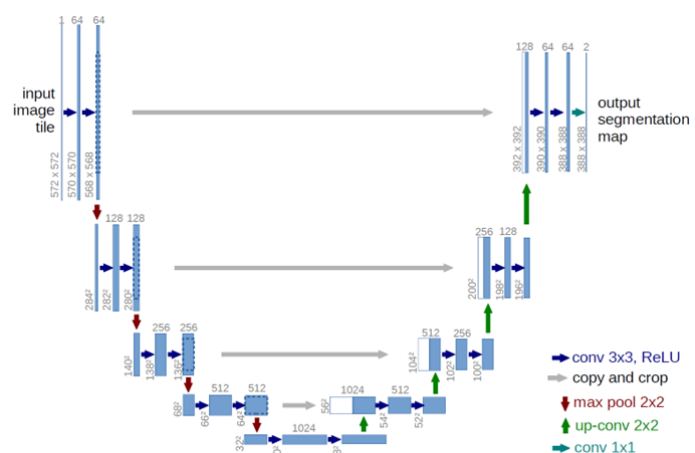


Figure 14 - UNet Architecture Diagram

The outputs of the model consisted of a segmented image as well as the raw logits. By default, the U-Net model only outputs the segmented image but through the aforementioned changes to the architecture, we also obtained the classification. The model was trained and tested on 2000 images, with a split of 1530/170/300 for training/testing validation. Additionally, the images were balanced, with 1000 normal catheters and 1000 abnormal. This ensured that the mode did not suffer from any bias from the data. The loss and accuracy curves are below (figure 15):

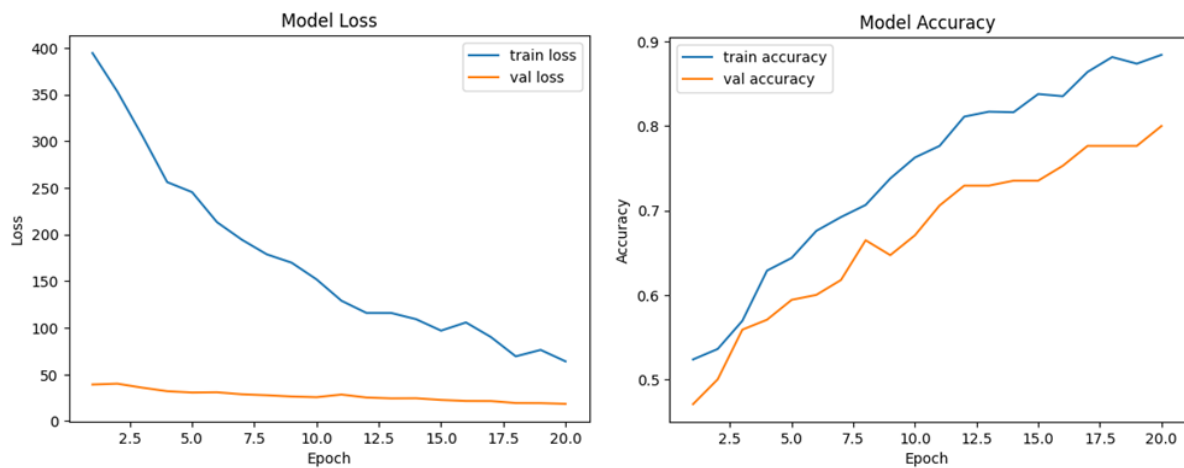
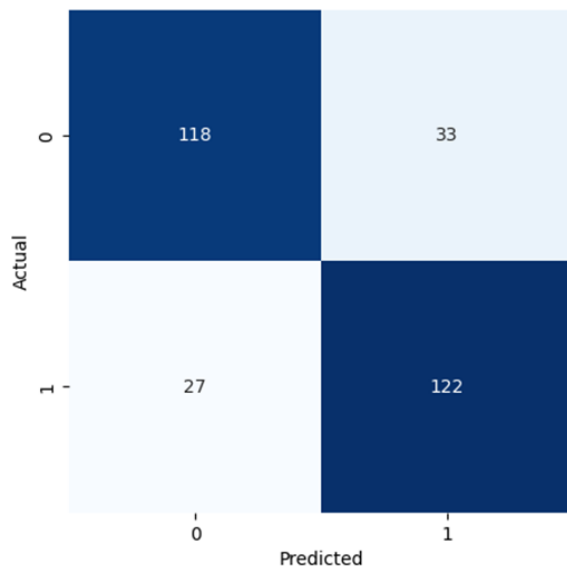


Figure 15 - Model Loss and Accuracy for UNet CVC Classification Model

As we can see, the results are quite good, with no significant signs of overfitting and consistent improvement in both training and validation accuracy. This indicated that we could have trained the model longer for hypothetically better results, however we were limited by time. To visualise the predictions, we created a confusion matrix (figure 16), where 1 indicates a normal catheter placement and 0 indicates an abnormal placement:

Figure 16 - Confusion Matrix for UNet Classification Model



As can be seen by the darker squares on the diagonal, the results are quite good, since most of the values are on the diagonal. For our task, it was especially important to have high precision, since mistakenly calculating a catheter as normal when it is abnormal could be a life-threatening error. The precision and recall scores respectively were 0.78 and 0.81. Considering the balanced nature of the dataset and the limited training images, this was a good result overall.

For future improvements with the U-Net model, we could have explored more encoder options as there were hundreds to choose from, and each one has its own pros and cons, such as higher

number of parameters or efficiency. For our final model, we used the “efficientnet-b3” encoder from the *segmentation_models_pytorch* package. This encoder has 10 million parameters and is a convolutional neural network that is scaled up using compound scaling, which scales the image width, depth and resolution in a coordinated fashion. Another potential improvement/change to our model workflow would be to use an ensemble of models with varying purposes. For instance, the U-Net model required the use of annotated images which requires more time to create rather than the raw x-ray images. However, rather than training the U-Net model to classify the images as normal or abnormal, we could take advantage of its segmentation capabilities and train the model to segment the image and annotate the catheter. This would automate the annotation process and the result would be high-quality annotated images to ultimately classify the catheter position.

MODEL ACCURACIES

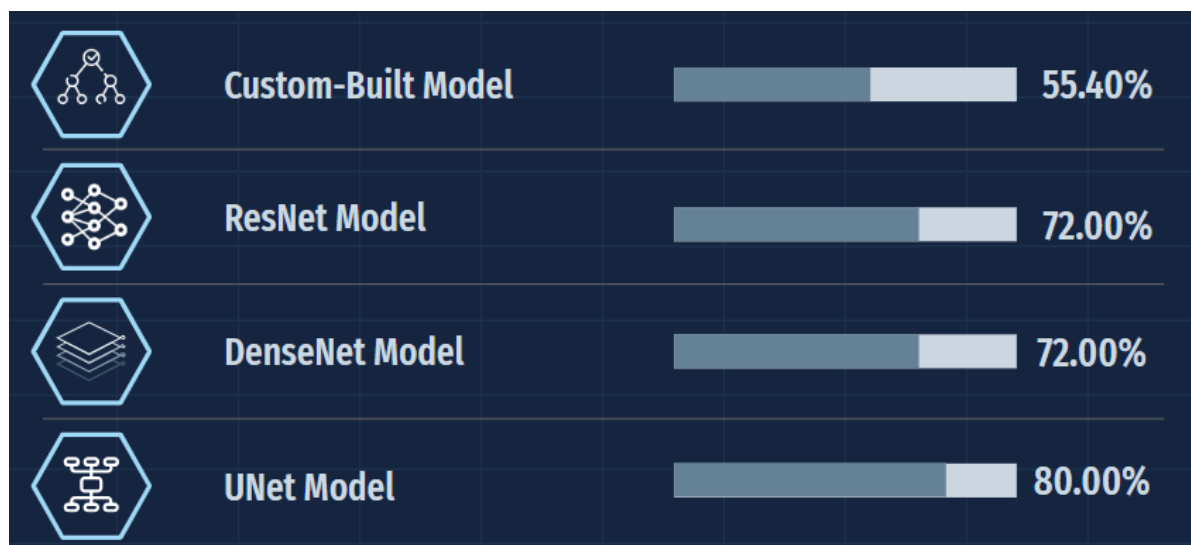


Figure 17 - Neural Network Model Accuracies

In predicting the placement accuracy of the Central Venous Catheter in patient X-rays, all four models achieved an accuracy rate exceeding 0.5. This indicates that our predictive models exhibit a level of performance beyond just random chance. Despite this, the model with the most optimal performance is **UNet**.

The reduction of false negatives is of paramount importance, given the potential risk to patient safety, and the UNet model prioritises exactly this. Not only does it perform significantly better than the other models in terms of minimising the ratio of false-positives + false-negatives against true-positives + true-negatives, but it also has the highest accuracy score of 0.8. UNet also reduces overfitting which was a feature largely present in the ResNet and DenseNet Models. The UNet model exhibits proficiency in accurately predicting abnormality in CVC placements, hence being able to advance healthcare by reducing the risks associated with CVC placement inaccuracies.

CONCLUSION

The aim of our project was to determine which convolutional neural network, among ResNet, DenseNet, UNet, and a custom neural network, could accurately assess the placement accuracy of three critical catheters - ETT, NGT, and CVC. Our efforts involved comprehensive data analysis and model development, all of which contributed to valuable insights regarding the accuracy and performance of these models.

We were able to successfully utilise the potential of various models, each characterised by its unique attributes. Among these, the custom-built neural network and ResNet-50 were notable. Our custom network served as our initial foray into neural networks, emphasising the significance of experimenting with models designed in-house. It provided essential insights into the challenges of training models with limited datasets, a scenario marked by varying complexities in catheter placement classification. Transitioning to the pre-trained ResNet-50 expanded our horizons, shedding light on the advantages of leveraging established neural networks in the domain of medical image analysis.

DenseNet and UNet further enriched our exploratory journey. DenseNet-121, with its densely connected layers, offered depth and capacity, enabling us to discern the complexities of medical images and achieve a test accuracy of 72.89%. UNet, a model tailored for image segmentation, brought valuable insights and promising results, especially regarding false negatives, a crucial factor in patient safety.

While the results showcase the competence of the models, they simultaneously underscore areas for improvement. The overfitting observed in training data demands more data

augmentation techniques and a more extensive dataset for model training. The addition of more encoder options for the UNet model could potentially improve performance. Further experimentation with ensembles of models could enhance the overall classification quality. Moreover, exploring methods to enhance the fine-tuning process of pre-trained models is an avenue for future research.

Following the analysis, the confusion matrices unveiled model strengths and limitations, illuminating the need for precision, especially in medical applications. Each false negative not only signifies a misclassification but also the potential risk to a patient's well-being. It is in the subtle nuances of these results that our journey continues, as we refine our models and venture into the complexity of real-world medical imaging data.

In conclusion, the path taken in this project highlights the increasing significance of machine learning in healthcare. Beyond the confines of our project, it offers a glimpse into the future, where fine-tuned models can stand as critical support tools for medical professionals in their decision-making processes. Our journey is a testament to the broader applications of machine learning, and it underscores the impact that extends to every corner of our society.

REFERENCES

Central Lines (Central Venous Catheters) (for Parents) - Nemours KidsHealth. (n.d.). Kidshealth.org.

<https://kidshealth.org/en/parents/central-lines.html>

Gamauf, T. (2018, October 2). *Tensorflow Records? What they are and how to use them.* Medium | Mostly AI.

<https://medium.com/mostly-ai/tensorflow-records-what-they-are-and-how-to-use-them-c46bc4bbb564>

Garg, A. (2022, September 20). *Image Classification Using Resnet-50 Deep Learning Model.* Analytics Vidhya.

<https://www.analyticsvidhya.com/blog/2022/09/image-classification-in-stl-10-dataset-using-resnet-50-deep-learning-model/>

RANZCR CLiP - Catheter and Line Position Challenge. (n.d.). Kaggle.com. Retrieved October 22, 2023, from <https://www.kaggle.com/competitions/ranzcr-clip-catheter-line-classification>

ResNet: The Basics and 3 ResNet Extensions. (n.d.). Datagen.

<https://datagen.tech/guides/computer-vision/resnet/#>

Ronneberger, O. (2015, May). *Papers with Code - U-Net Explained.* Paperswithcode.com.

<https://paperswithcode.com/method/u-net#:~:text=>

Team, K. (n.d.). *Keras documentation: Image classification from scratch.* Keras.io.

https://keras.io/examples/vision/image_classification_from_scratch/

ADS2002 - Catheter and Line Placement Project

Project Background

The Catheter and Line position project was a Kaggle challenge created by the Royal Australian and NZ College of Radiologists (RANZCR) in an attempt to help clinicians to be able to detect malpositioned catheters and lines early in order to save lives.

OUR OBJECTIVE

The objective of this project is to use machine learning to train and test a model on 40,000 images to categorize whether a catheter or line has been poorly placed.

WHAT IS A CATHETER?

A catheter is a thin tube made from medical grade materials serving a broad range of functions. Catheters are medical devices that can be inserted into the body to treat diseases or as an aid when performing surgical procedures.

The most common catheter is that inserted into the bladder. This allows urine to drain freely. In this project we are not looking at this catheter but rather 4 other types:

1. Endotracheal (inserted between the vocal cords and through the trachea to provide oxygen and inhaled gases to the lungs)
2. Nasogastric (inserted through the nose into the stomach. Used to carry food and medicine directly to the stomach)
3. Central Venous (inserted into a large vein in the neck, chest, groin or arm. Typically used to administer medications over a long period of time as well as directly measure blood pressure)
4. Swan Ganz (inserted into the right of the heart to monitor heart function including blood flow and pressures in and around the heart)

Hospital patients have catheters and lines inserted during the course of their admission and serious complications can arise if they are positioned incorrectly. Millions of COVID-19 patients need tubes and lines inserted with a reported 3% of cases inserted incorrectly.

Chest Radiographs are the gold standard to confirm whether a line or tube is placed correctly. A physician must manually check these chest x-rays to verify the optimal position. This process can be slow and leaves room for human error.

In this project we want to use Deep learning algorithms to automatically detect malpositioned catheters and lines. These will alert clinicians immediately and they can reposition or remove them before complications develop

FILES GIVEN

- **train.csv** - contains images IDs, binary labels and patient IDs
- **train_annotations.csv** - provides annotations for each image in the training set. Will be able to plot where the line or catheter is.
- **test** - Folder with all test images
- **train** - Folder with all train images (IDs same as IDs in train dataset)

- TFRecords - Tensor Flow Records (way of storing a sequence of binary records)

1. Mount to Drive -> will allow access to data set

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')

%cd /content/drive/MyDrive/ADS2002-Project-CatheterPlacement
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

/content/drive/MyDrive/ADS2002-Project-CatheterPlacement

```
In [ ]: !ls -lt
```

```
ls: ranzcr-clip-catheter-line-classification: No such file or directory
total 217
drwx----- 3 root root  4096 Oct 21 10:47 Data
-rw----- 1 root root   182 Oct 20 09:16 'ADS2002 Final presentation .gslides'
-rw----- 1 root root  9161 Oct 19 11:20 catheter_types.png
-rw----- 1 root root   182 Sep  9 01:52 'ADS2002 - Catheter Placement.gdoc'
-rw----- 1 root root 205295 Aug 18 00:49 CatheterPlacementADS2002.pdf
-rw----- 1 root root   1783 Aug 11 06:17 mtcars.csv
lrw----- 1 root root      0 Aug 11 05:22 ranzcr-clip-catheter-line-classification -> /con
tent/drive/.shortcut-targets-by-id/1nr3dkndFZpGTbj671CSST0ofESP0ncpx/ranzcr-clip-catheter-li
ne-classification
```

```
In [ ]: import random
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
from PIL import Image
import ast
```

2. Data Set Exploration

```
In [ ]: train = pd.read_csv('Data/train.csv')
train.head()
```

```
Out[ ]:
```

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT - Abnormal	I Bord
0	1.2.826.0.1.3680043.8.498.26697628953273228189...	0	0	0	0	
1	1.2.826.0.1.3680043.8.498.46302891597398758759...	0	0	1	0	
2	1.2.826.0.1.3680043.8.498.23819260719748494858...	0	0	0	0	
3	1.2.826.0.1.3680043.8.498.68286643202323212801...	0	0	0	0	
4	1.2.826.0.1.3680043.8.498.10050203009225938259...	0	0	0	0	

```
In [ ]: df = train.iloc[:, 1:-2].sum()

df = df.transpose().reset_index()
df.columns = ['Procedure', 'Count']
```

```

df['Category'] = df['Procedure'].apply(lambda x: x.split(' - ')[0])

pivot_df = df.pivot(index='Category', columns='Procedure', values='Count').reset_index()
pivot_df = pivot_df.fillna(0)

pivot_df['Abnormal'] = pivot_df['ETT - Abnormal'] + pivot_df['NGT - Abnormal'] + pivot_df[
pivot_df['Normal'] = pivot_df['ETT - Normal'] + pivot_df['NGT - Normal'] + pivot_df['CVC -
pivot_df['Borderline'] = pivot_df['ETT - Borderline'] + pivot_df['NGT - Borderline'] + piv
pivot_df['Incompletely Imaged'] = pivot_df['NGT - Incompletely Imaged']

```

In []:

```

import matplotlib.pyplot as plt

categories = pivot_df['Category']
abnormal = pivot_df['Abnormal']
normal = pivot_df['Normal']
borderline = pivot_df['Borderline']
incompletely_imaged = pivot_df['Incompletely Imaged']

plt.figure(figsize=(10, 6))
plt.bar(categories, abnormal, label='Abnormal', color='pink')
plt.bar(categories, normal, label='Normal', bottom=abnormal, color='cadetblue')
plt.bar(categories, borderline, label='Borderline', bottom=abnormal + normal, color='darksalmon')
plt.bar(categories, incompletely_imaged, label='Incompletely Imaged', bottom=abnormal + normal + borderline)

plt.xlabel('Catheter Type', color='black')
plt.ylabel('Distribution of Observations', color='black')
plt.title('Distribution of Catheters by Type', color='black')

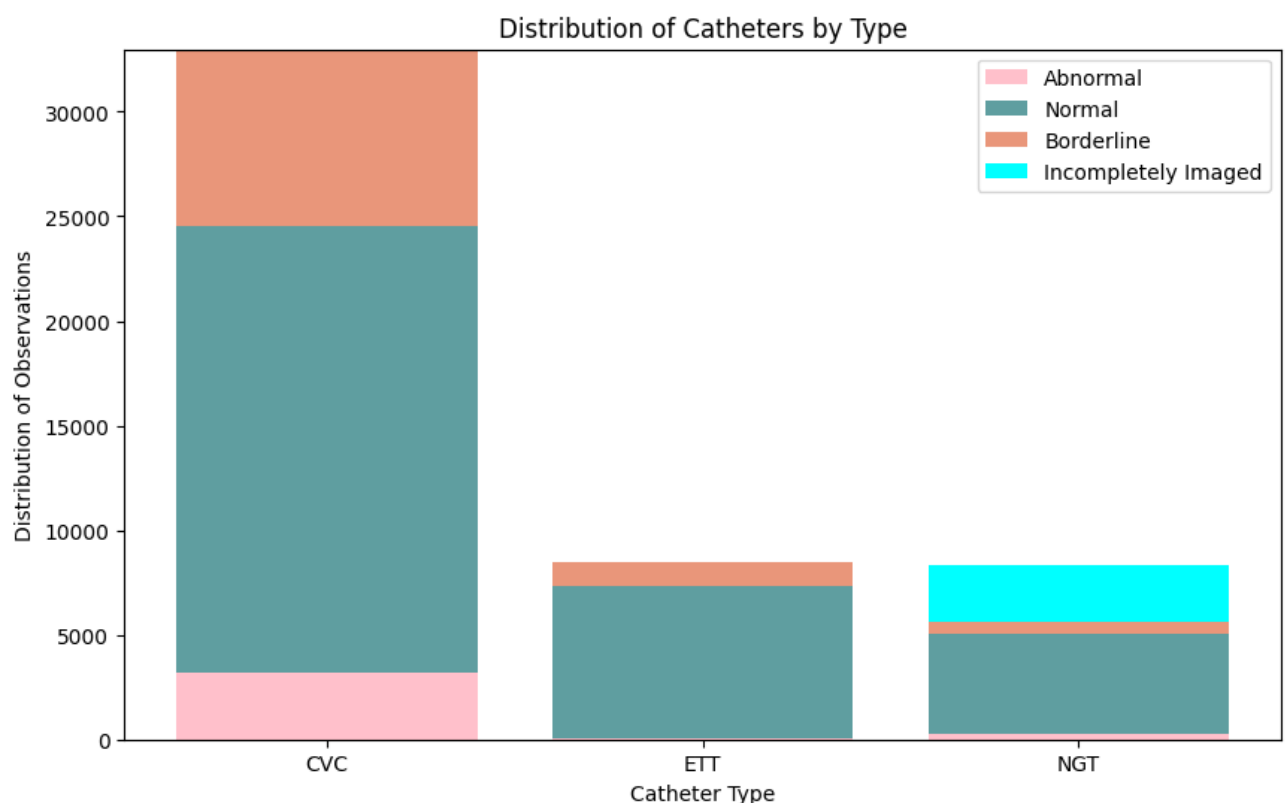
# Customize the tick labels' color to white
plt.xticks(ticks=categories, labels=categories, color='black')
plt.yticks(color='black')

plt.legend()

# Make the plot background transparent
ax = plt.gca()
ax.set_facecolor('none')

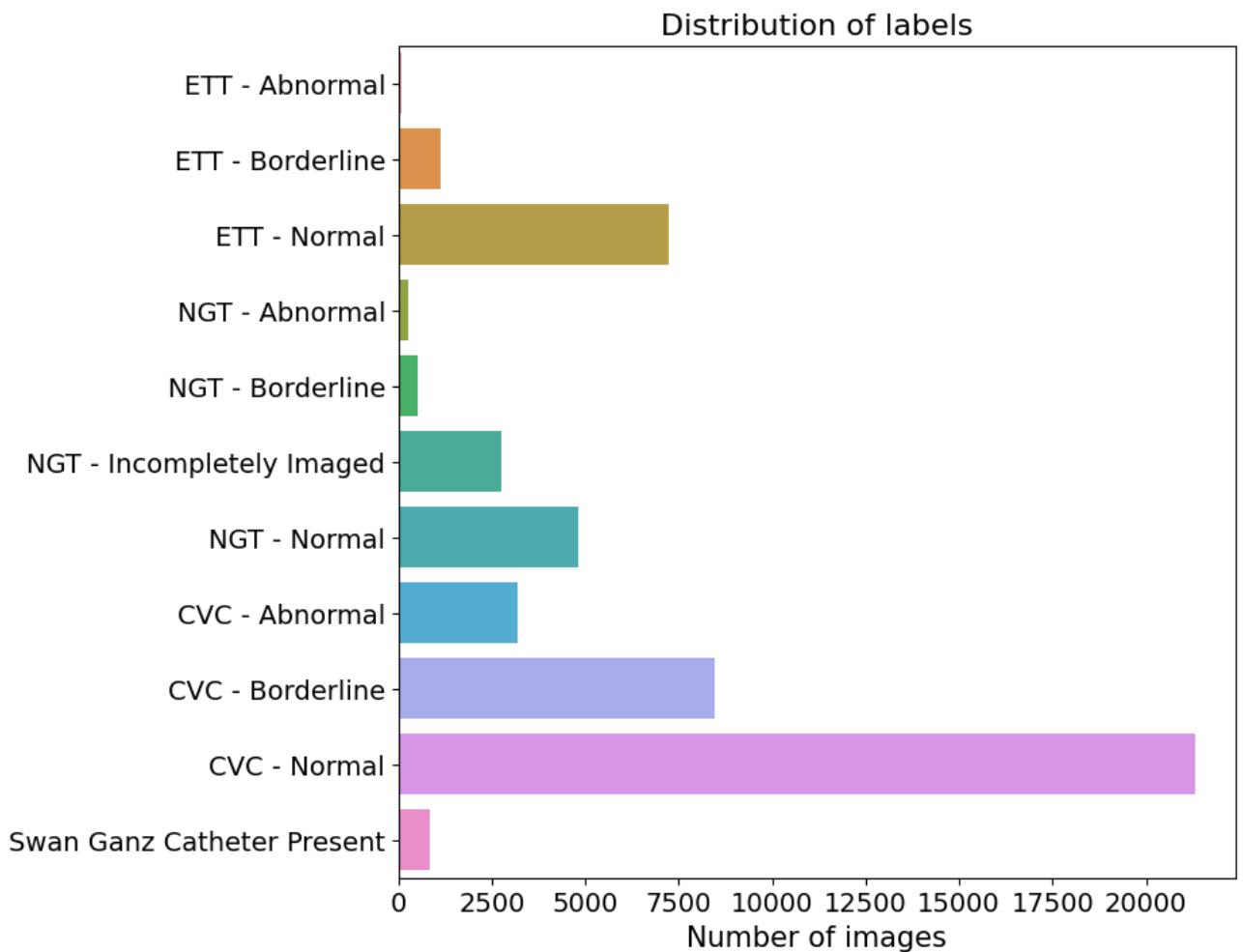
plt.show()

```



```
In [ ]: plt.figure(figsize=(8, 8))
df_tmp = train.iloc[:, 1:-1].sum()

sns.barplot(x=df_tmp.values, y=df_tmp.index)
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
plt.xlabel("Number of images", fontsize=15)
plt.title("Distribution of labels", fontsize=16);
```



COLUMNS

- **StudyInstanceUID** - unique ID for each image
- **ETT - Abnormal** - endotracheal tube placement abnormal
- **ETT - Borderline** - endotracheal tube placement borderline abnormal
- **ETT - Normal** - endotracheal tube placement normal
- **NGT - Abnormal** - nasogastric tube placement abnormal
- **NGT - Borderline** - nasogastric tube placement borderline abnormal
- **NGT - Incompletely Imaged** - nasogastric tube placement inconclusive due to imaging
- **NGT - Normal** - nasogastric tube placement borderline normal
- **CVC - Abnormal** - central venous catheter placement abnormal
- **CVC - Borderline** - central venous catheter placement borderline abnormal
- **CVC - Normal** - central venous catheter placement normal
- **Swan Ganz Catheter Present**
- **PatientID** - unique ID for each patient in the dataset

```
In [ ]: print("Number of unique patients: ", train["PatientID"].unique().shape[0])
```

Number of unique patients: 3255

```
In [ ]: print("Number of unique images: ", train['StudyInstanceUID'].unique().shape[0])
```

Number of unique images: 30083

```
In [ ]: train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30083 entries, 0 to 30082
Data columns (total 13 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   StudyInstanceUID                      30083 non-null  object
 1   ETT - Abnormal                       30083 non-null  int64
 2   ETT - Borderline                     30083 non-null  int64
 3   ETT - Normal                         30083 non-null  int64
 4   NGT - Abnormal                       30083 non-null  int64
 5   NGT - Borderline                     30083 non-null  int64
 6   NGT - Incompletely Imaged            30083 non-null  int64
 7   NGT - Normal                         30083 non-null  int64
 8   CVC - Abnormal                       30083 non-null  int64
 9   CVC - Borderline                     30083 non-null  int64
10   CVC - Normal                         30083 non-null  int64
11   Swan Ganz Catheter Present           30083 non-null  int64
12   PatientID                            30083 non-null  object
dtypes: int64(11), object(2)
memory usage: 3.0+ MB
```

```
In [ ]: train.isna().sum()
```

```
Out[ ]: StudyInstanceUID      0
        ETT - Abnormal        0
        ETT - Borderline      0
        ETT - Normal          0
        NGT - Abnormal        0
        NGT - Borderline      0
        NGT - Incompletely Imaged 0
        NGT - Normal          0
        CVC - Abnormal        0
        CVC - Borderline      0
        CVC - Normal          0
        Swan Ganz Catheter Present 0
        PatientID             0
        dtype: int64
```

```
In [ ]: train.describe()
```

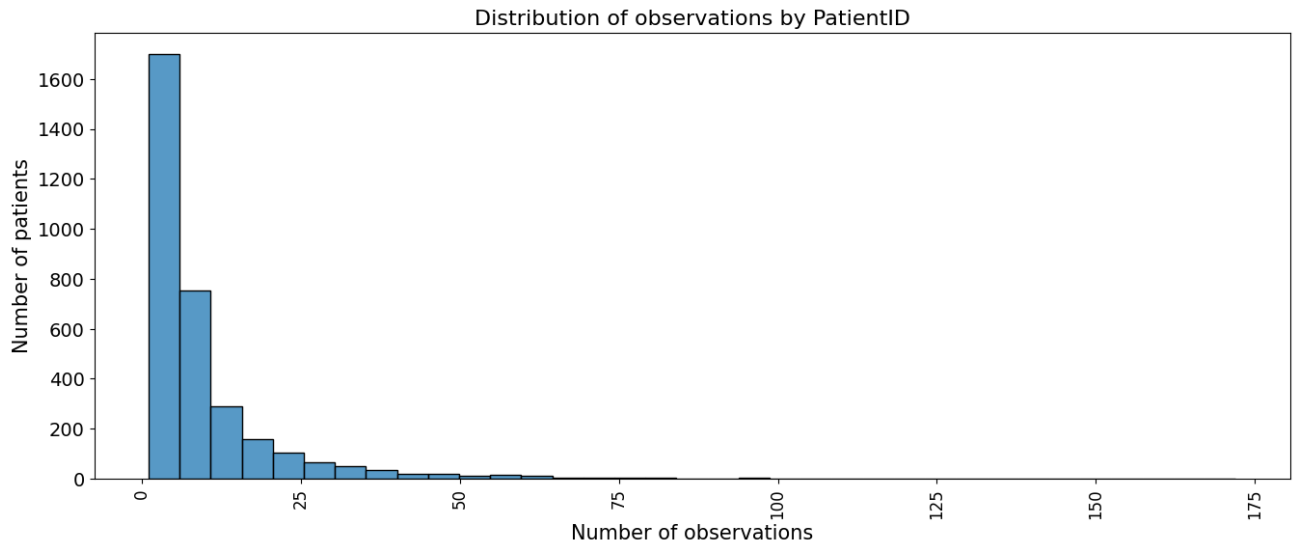
[illegible]

```
In [ ]: plt.figure(figsize=(16, 6))
df_tmp = train["PatientID"].value_counts()
sns.histplot(df_tmp.values, palette='pink', bins=35) # Adjust the number of bins as needed
plt.xticks(fontsize=12, rotation=90)
plt.yticks(fontsize=14)
plt.xlabel("Number of observations", fontsize=15)
plt.ylabel("Number of patients", fontsize=15)
plt.title("Distribution of observations by PatientID", fontsize=16)
```

<ipython-input-30-3502deb40889>:3: UserWarning: Ignoring `palette` because no `hue` variable has been assigned.

```
sns.histplot(df_tmp.values, palette='pink', bins=35) # Adjust the number of bins as needed
```

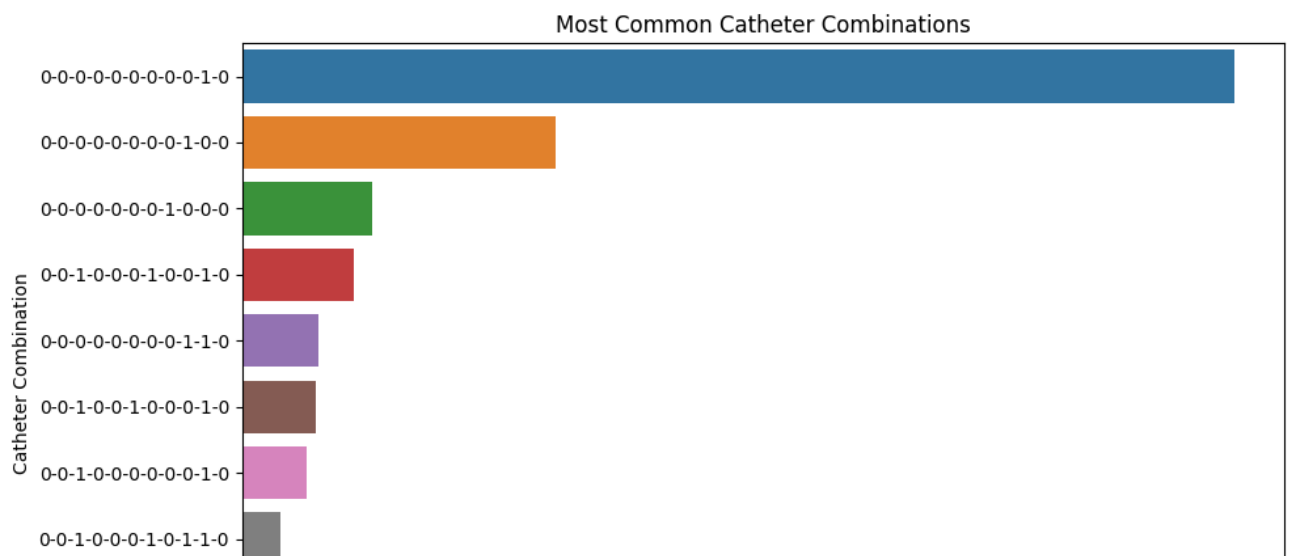
Out[]: Text(0.5, 1.0, 'Distribution of observations by PatientID')

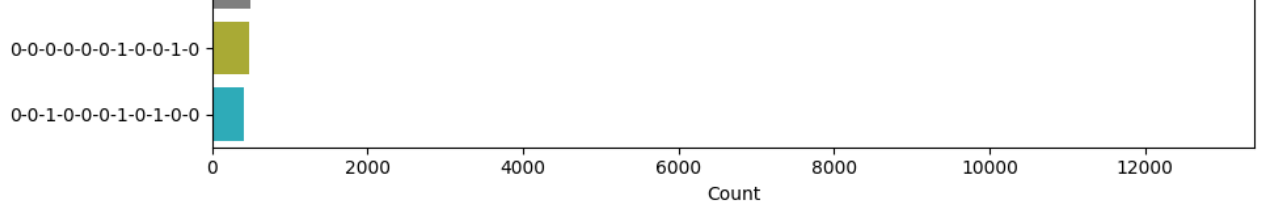


```
In [ ]: # Most common catheter combinations
catheter_columns = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal',
                    'NGT - Abnormal', 'NGT - Borderline', 'NGT - Incompletely Imaged',
                    'NGT - Normal', 'CVC - Abnormal', 'CVC - Borderline',
                    'CVC - Normal', 'Swan Ganz Catheter Present']

# Count the combinations
catheter_combinations = train[catheter_columns].apply(lambda row: "-".join(row.astype(str)), axis=1)
common_combinations = catheter_combinations.value_counts().head(10)

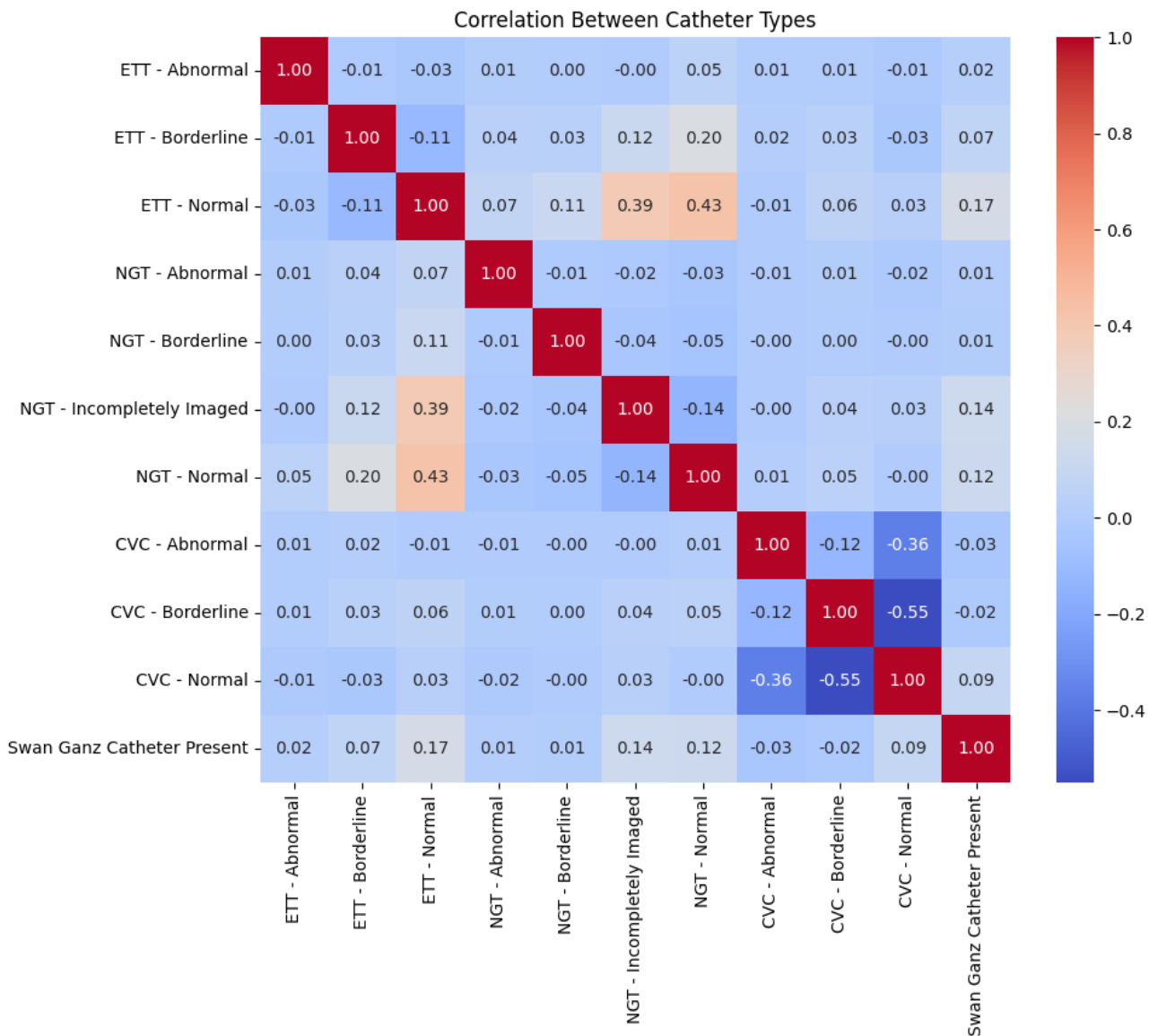
plt.figure(figsize=(10, 6))
sns.barplot(x=common_combinations.values, y=common_combinations.index)
plt.xlabel("Count")
plt.ylabel("Catheter Combination")
plt.title("Most Common Catheter Combinations")
plt.tight_layout()
plt.show()
```





```
In [ ]: # Calculate correlation matrix for catheter columns
catheter_corr = train[catheter_columns].corr()

# Create a heatmap to visualize correlations
plt.figure(figsize=(10, 8))
sns.heatmap(catheter_corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Between Catheter Types")
plt.show()
```



Annotations Dataset

```
In [ ]: annot = pd.read_csv('Data/train_annotations.csv')
annot.head()
```

	StudyInstanceUID	label	data
0	1.2.826.0.1.3680043.8.498.12616281126973421762...	CVC - Normal	[[1487, 1279], [1477, 1168], [1472, 1052], [14...
1	1.2.826.0.1.3680043.8.498.12616281126973421762...	CVC - Normal	[[1328, 7], [1347, 101], [1383, 193], [1400, 2...

2	1.2.826.0.1.3680043.8.498.72921907356394389969...	CVC - Borderline	[[801, 1207], [812, 1112], [823, 1023], [842, ...
3	1.2.826.0.1.3680043.8.498.11697104485452001927...	CVC - Normal	[[1366, 961], [1411, 861], [1453, 751], [1508, ...
4	1.2.826.0.1.3680043.8.498.87704688663091069148...	NGT - Normal	[[1862, 14], [1845, 293], [1801, 869], [1716, ...

```
In [ ]: annot.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17999 entries, 0 to 17998
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   StudyInstanceUID 17999 non-null  object
1   label            17999 non-null  object
2   data             17999 non-null  object
dtypes: object(3)
memory usage: 422.0+ KB
```

```
In [ ]: print("Number of unique images: ", annot['StudyInstanceUID'].unique().shape[0])
        print("9095 images have annotations ")
```

```
Number of unique images: 9095
9095 images have annotations
```

```
In [ ]: # Combine the train and annotations dataset

        trainWannot = pd.merge(train, annot, on="StudyInstanceUID")

        trainWannot.to_csv('Data/trainWithAnnotations.csv')
        print(trainWannot.shape)

        print("Number of unique images: ", trainWannot['StudyInstanceUID'].unique().shape[0])
        print("Number of unique patients: ", trainWannot['PatientID'].unique().shape[0])
        print("9095 images have annotations ")

        trainWannot.isna().sum()
```

```
(17999, 15)
Number of unique images: 9095
Number of unique patients: 3255
9095 images have annotations
```

```
Out[ ]: StudyInstanceUID      0
        ETT - Abnormal        0
        ETT - Borderline      0
        ETT - Normal          0
        NGT - Abnormal        0
        NGT - Borderline      0
        NGT - Incompletely Imaged 0
        NGT - Normal          0
        CVC - Abnormal        0
        CVC - Borderline      0
        CVC - Normal          0
        Swan Ganz Catheter Present 0
        PatientID            0
        label                0
        data                 0
        dtype: int64
```

```
In [ ]: patients = trainWannot['PatientID'].value_counts()
        patients = pd.DataFrame(patients)
        patients = patients.rename(columns={'PatientID': 'Image Count'})
```

patients

Out[]:

	Image Count
05029c63a	354
55073fece	172
b36819098	137
c20ec34df	129
fd3ab6f0e	123
...	...
f06bd0653	1
14d0895c2	1
5005ac431	1
97fdd6be7	1
d31702943	1

3255 rows × 1 columns

In []:

```
def plot_annotations(ind):
    row = annot.iloc[ind]
    image_path = "Data/train/" + row['StudyInstanceUID'] + ".jpg"
    label = row['label']
    data = np.array(ast.literal_eval(row["data"]))

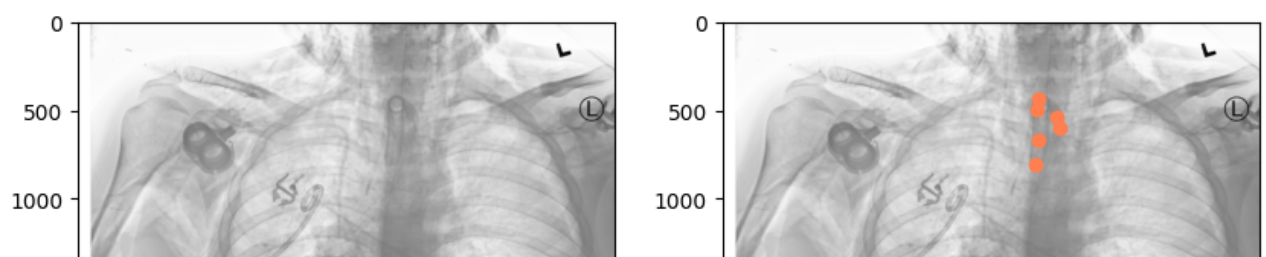
    plt.figure(figsize=(10,5))
    image = Image.open(image_path)
    plt.subplot(1,2,1)
    plt.imshow(image, cmap='Greys')
    plt.subplot(1,2,2)
    plt.imshow(image, cmap="Greys")
    plt.scatter(data[:,0], data[:,1], color='Coral')

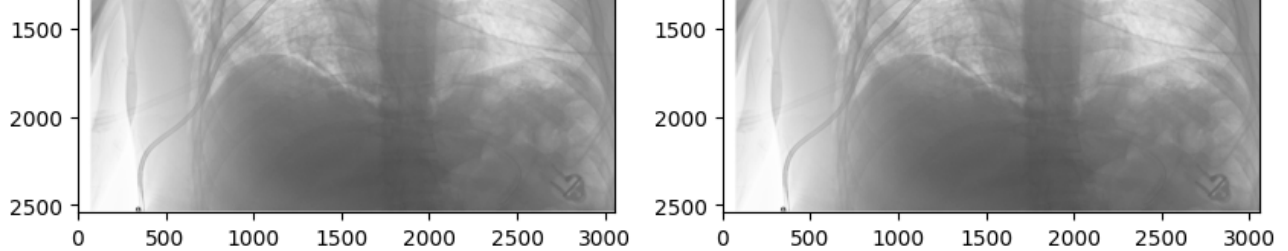
    plt.suptitle(label)
```

In []:

```
plot_annotations(6)
```

ETT - Normal





```
In [ ]: # Function to make multiple annotations on 1 image

def multiple_annotations(img_id):

    plt.figure(figsize=(8,8))

    image = Image.open("Data/train/" + img_id + ".jpg")
    plt.imshow(image, cmap="Greys")

    patient = annot.loc[annot["StudyInstanceUID"] == img_id]

    if patient.shape[0]:
        labels = patient['label'].values.tolist()
        catheters = patient['data'].apply(ast.literal_eval).values.tolist()

        for catheter, label in zip(catheters, labels):
            catheter = np.asarray(catheter)
            if (type is ('ETT - Abnormal' or 'ETT - Normal' or 'ETT - Borderline')):
                plt.scatter(catheter[:, 0], catheter[:,1], s=40, label=label, color='lavender')
            elif (type is ('NGT - Normal' or 'NGT - Borderline' or 'NGT - Incompletely Imaged' or 'NGT - Abnormal')):
                plt.scatter(catheter[:, 0], catheter[:,1], s=40, label=label, color='paleturquoise')
            elif (type is ('CVC - Normal' or 'CVC - Abnormal' or 'CVC - Borderline')):
                plt.scatter(catheter[:, 0], catheter[:,1], s=40, label=label, color='lightsalmon')
            else:
                plt.scatter(catheter[:, 0], catheter[:,1], s=40, label=label, color='palevioletred')

        plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0, prop={'size': 20})

    plt.tick_params(axis="x", labelsize=15)
    plt.tick_params(axis="y", labelsize=15)

    plt.show()
```

```
In [ ]: # Function to find all the x-ray images for 1 patient

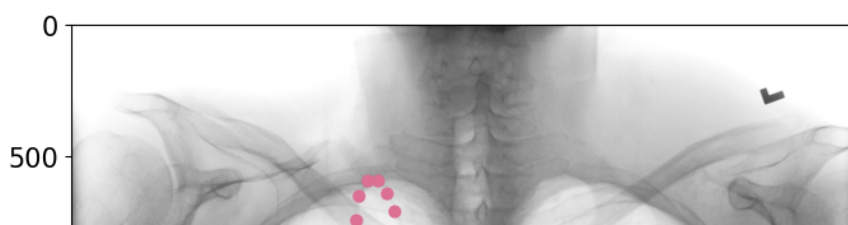
def find_patient_files(patientID):
    patient = trainWannot[trainWannot['PatientID'] == patientID]

    images = patient['StudyInstanceUID'].tolist()
    images = list(set(images))
    return images
```

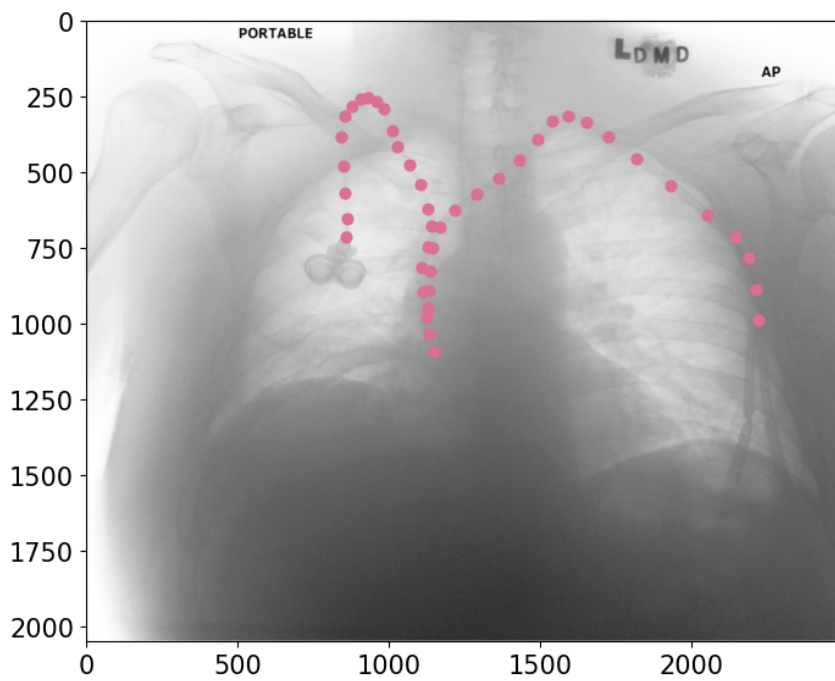
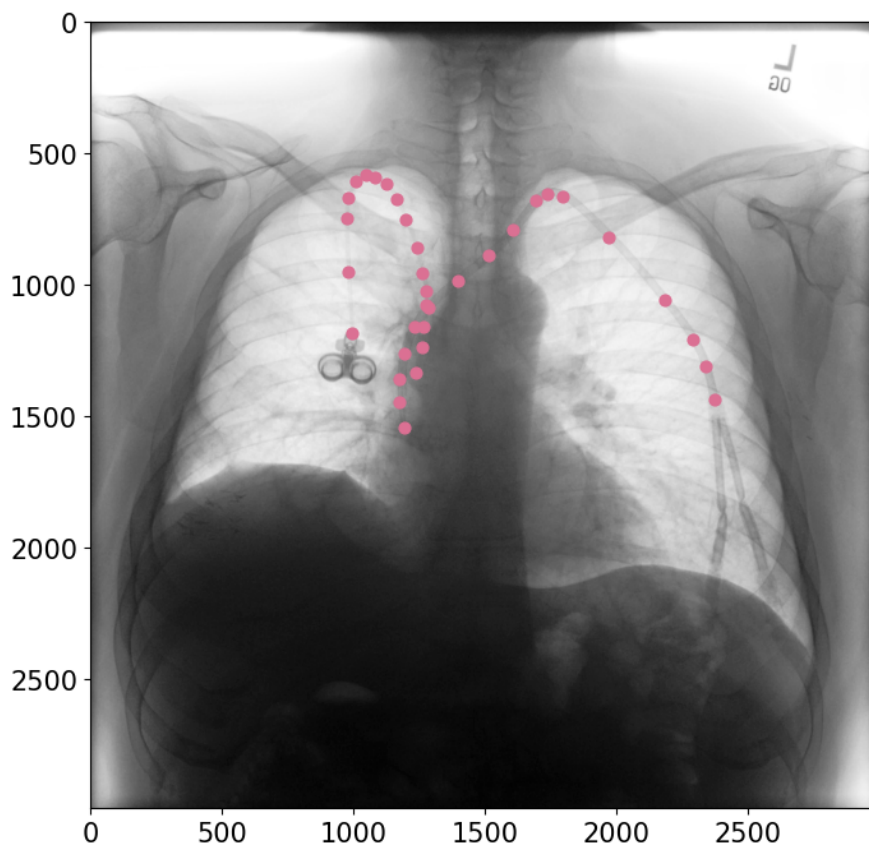
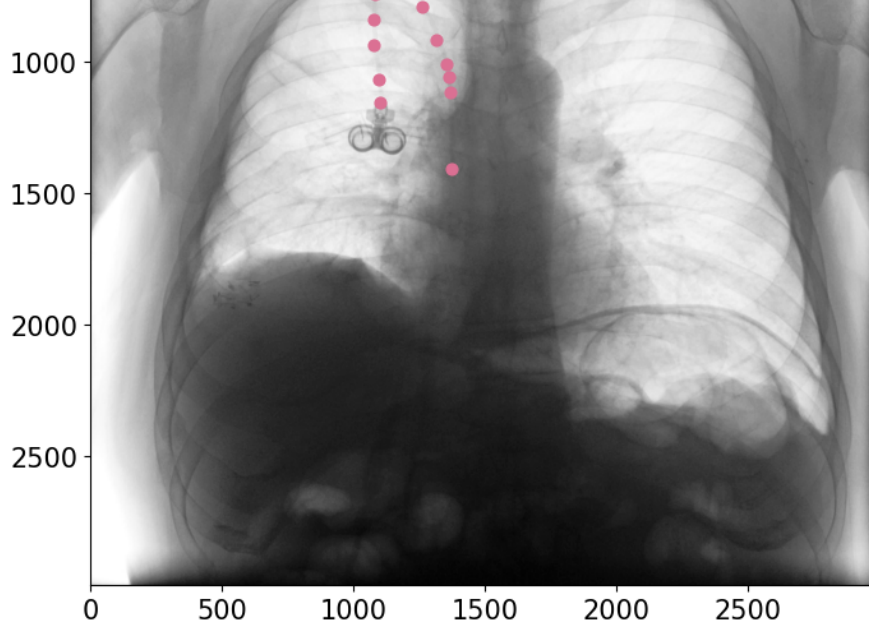
```
In [ ]: image_ids = find_patient_files('248aeb1e5')

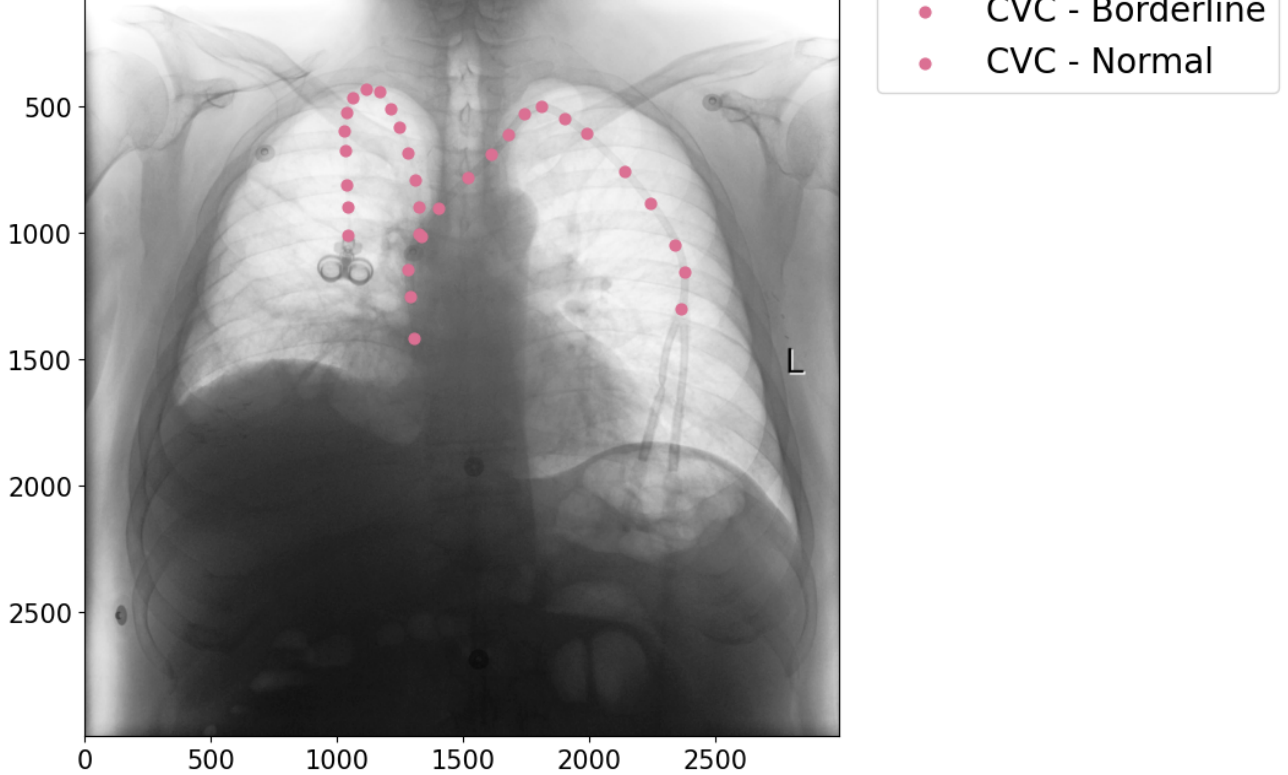
print(image_ids)
for id in image_ids:
    multiple_annotations(id)
```

```
['1.2.826.0.1.3680043.8.498.34106748148648264352761866272322050800', '1.2.826.0.1.3680043.8.498.17230802451173576441578536774255761107', '1.2.826.0.1.3680043.8.498.41663531846873380572788973511404306426', '1.2.826.0.1.3680043.8.498.13400030891771826052457637366763811465']
```



● CVC - Normal





Next Steps:

- Plot annotations for each individual tube placement
 - What do these look like?
- Find the distribution of each type (how many are abnormal, normal, borderline etc.)
- Research ways to start image processing

ETT - ENDOTRACHEAL TUBE PLACEMENT

In []:

```
def plot_distribution(df, type):
    sns.barplot(x=df.values, y=df.index, palette='mako')
    plt.xticks(fontsize=14)
    plt.yticks(fontsize=14)
    plt.xlabel("Number of images", fontsize=15)
    plt.title(f"Distribution of {type}", fontsize=16);

def find_image(tubeType):
    row = train[train[tubeType] == 1]
    print(row)
    return row.index[0]

def visualise_multiple(imgs, title):
    plt.figure(figsize=(16,5))

    for i, img in enumerate(imgs):
        plt.subplot(1,3, i+1)
        image_path = "Data/train/" + img + ".jpg"
        image = Image.open(image_path)
        plt.imshow(image, cmap="Greys")
        plt.axis('off')

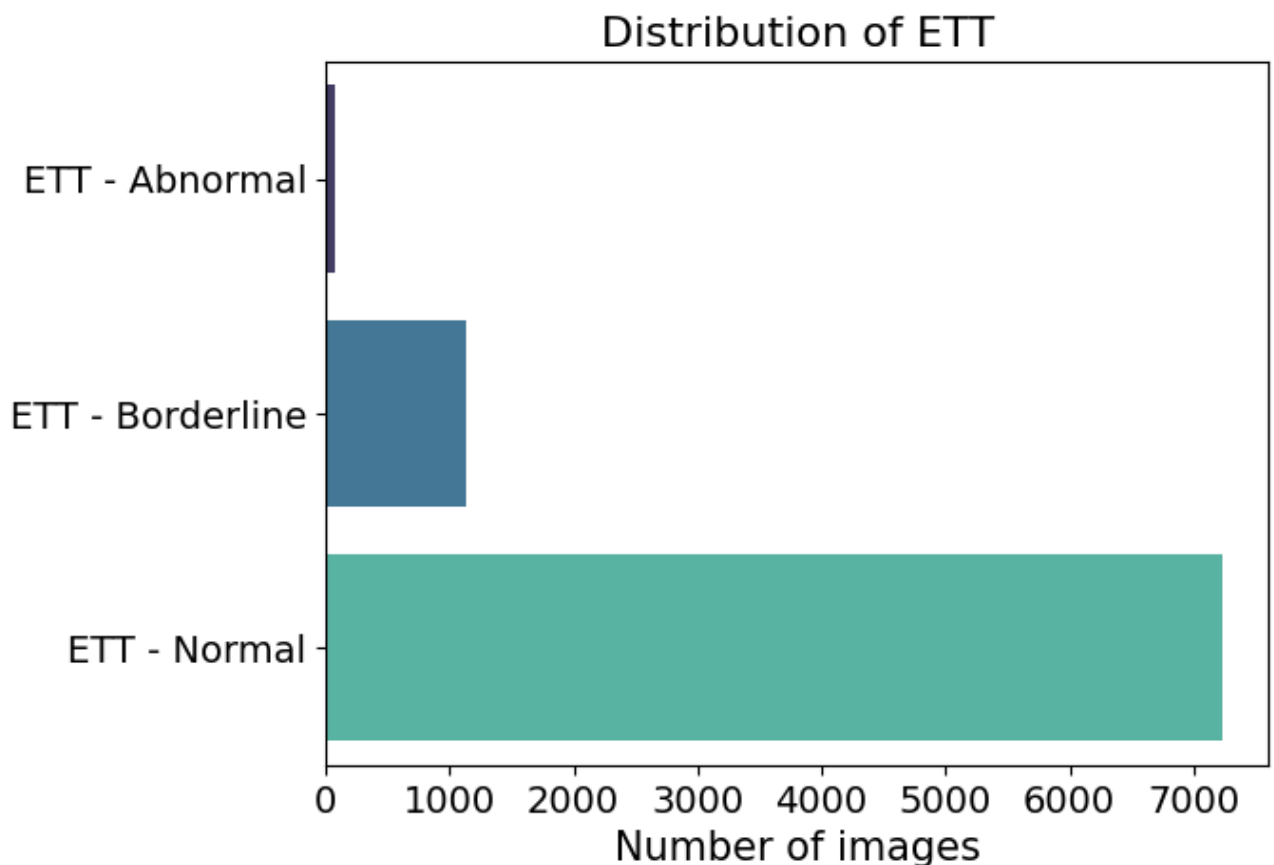
    plt.suptitle(title)
    plt.show()

def process_type(type):
    df = train[train[type] == 1]
    visualise_multiple(random.sample(df['StudyInstanceUID'].tolist(), 3), type)
```

```
def plot_catheter(type):
    row = annot[annot['label'] == type].iloc[0]
    image_path = "Data/train/" + row['StudyInstanceUID'] + ".jpg"
    label = row['label']
    data = np.array(ast.literal_eval(row["data"]))

    plt.figure(figsize=(10,5))
    image = Image.open(image_path)
    plt.subplot(1,2,1)
    plt.imshow(image, cmap='Greys')
    plt.subplot(1,2,2)
    plt.imshow(image, cmap="Greys")
    if (type is ('ETT - Abnormal' or 'ETT - Normal' or 'ETT - Borderline')):
        plt.scatter(data[:,0], data[:,1], color='lavender')
    elif (type is ('NGT - Normal' or 'NGT - Borderline' or 'NGT - Incompletely Imaged' or 'N
        plt.scatter(data[:,0], data[:,1], color='paleturquoise')
    elif (type is ('CVC - Normal' or 'CVC - Abnormal' or 'CVC - Borderline')):
        plt.scatter(data[:,0], data[:,1], color='lightsalmon')
    else:
        plt.scatter(data[:,0], data[:,1], color='palevioletred')
    plt.suptitle(label)
```

```
In [ ]: catheterTypes = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal', '
ETT = train.iloc[:, 1:4].sum()
plot_distribution(ETT, "ETT")
```

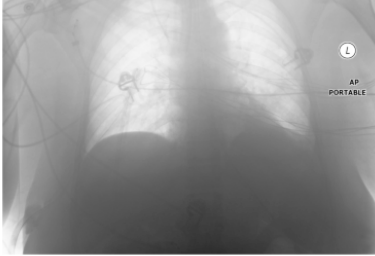


ETT - ABNORMAL

```
In [ ]: process_type('ETT - Abnormal')
```

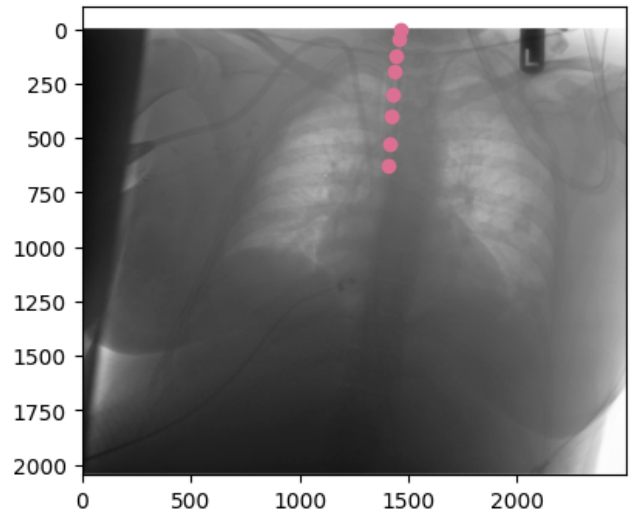
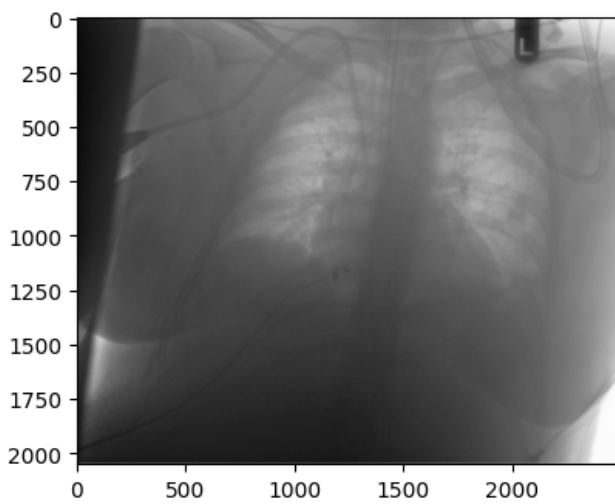
ETT - Abnormal





```
In [ ]: plot_catheter('ETT - Abnormal')
```

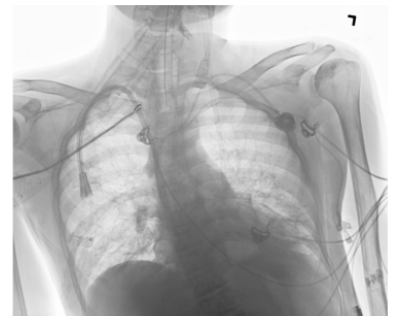
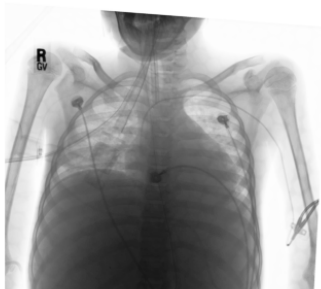
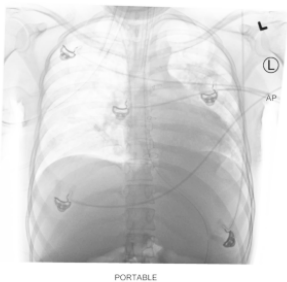
ETT - Abnormal



ETT - BORDERLINE

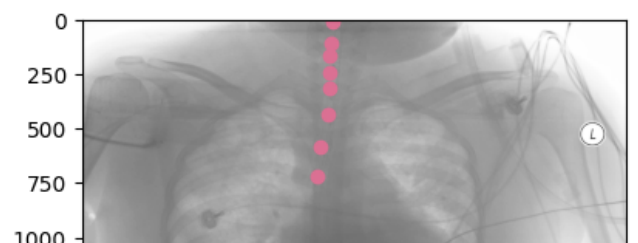
```
In [ ]: process_type('ETT - Borderline')
```

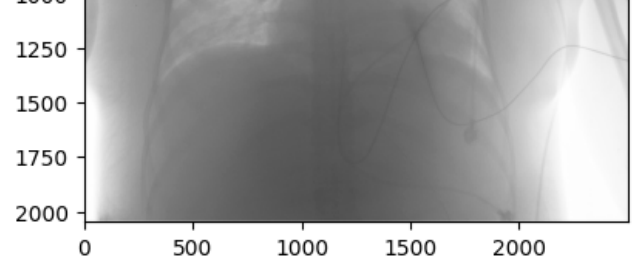
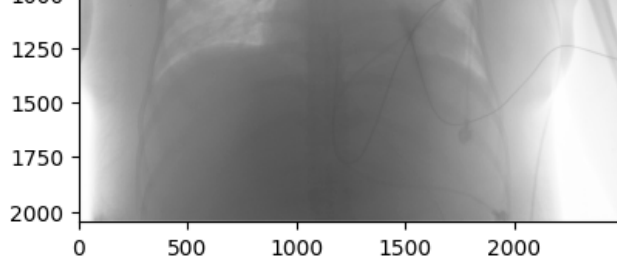
ETT - Borderline



```
In [ ]: plot_catheter('ETT - Borderline')
```

ETT - Borderline

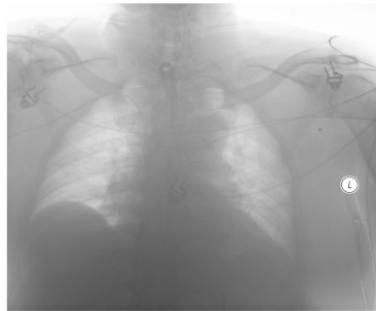




ETT - NORMAL

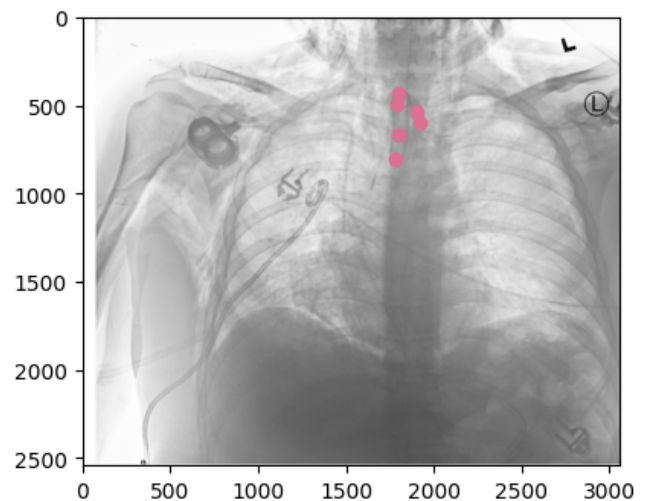
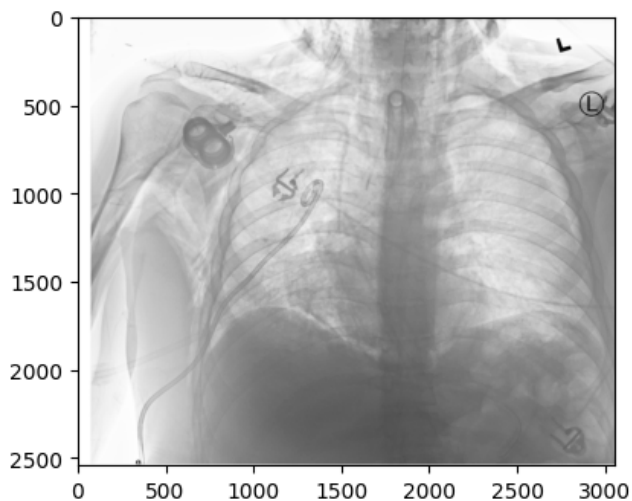
```
In [ ]: process_type('ETT - Normal')
```

ETT - Normal



```
In [ ]: plot_catheter('ETT - Normal')
```

ETT - Normal



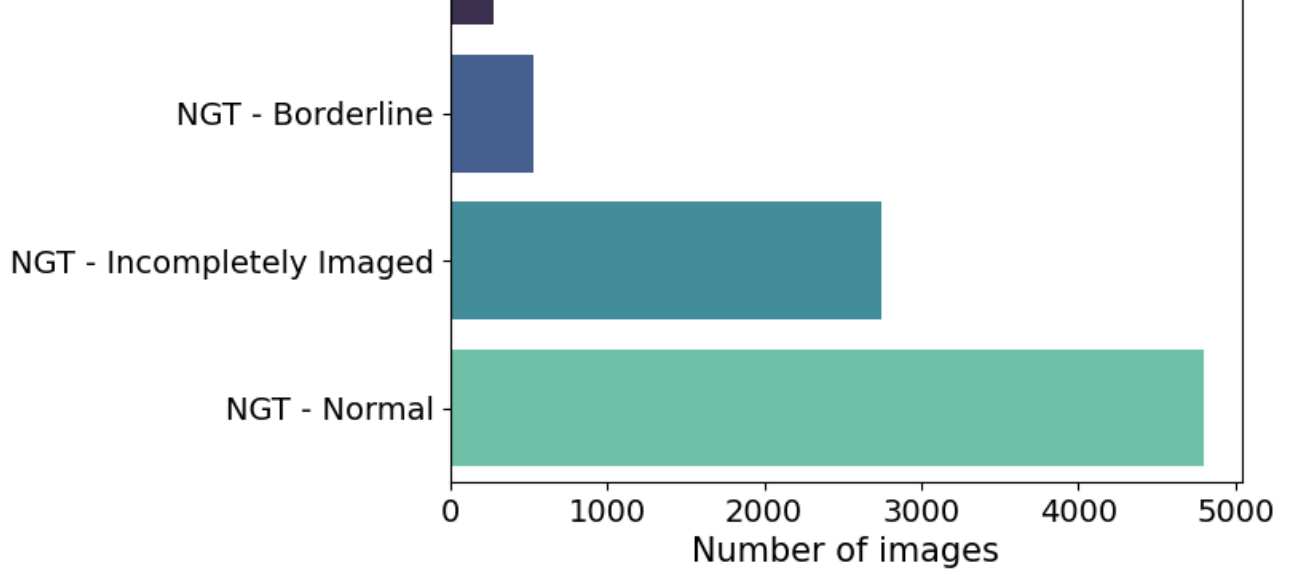
NGT - Nasogastric Tube Placement

```
In [ ]: NGT = train.iloc[:, 4:8].sum()

plot_distribution(NGT, "NGT")
```

Distribution of NGT

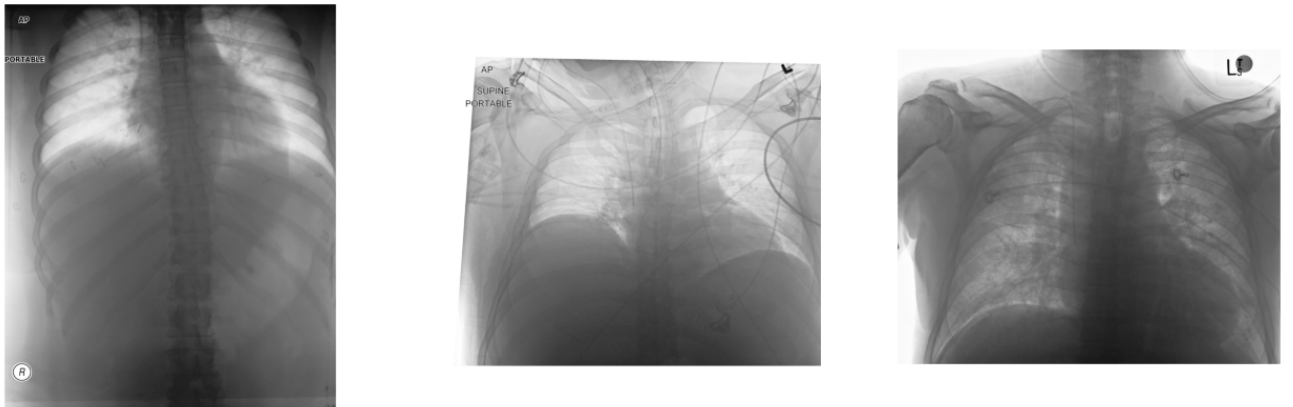
NGT - Abnormal -



NGT - ABNORMAL

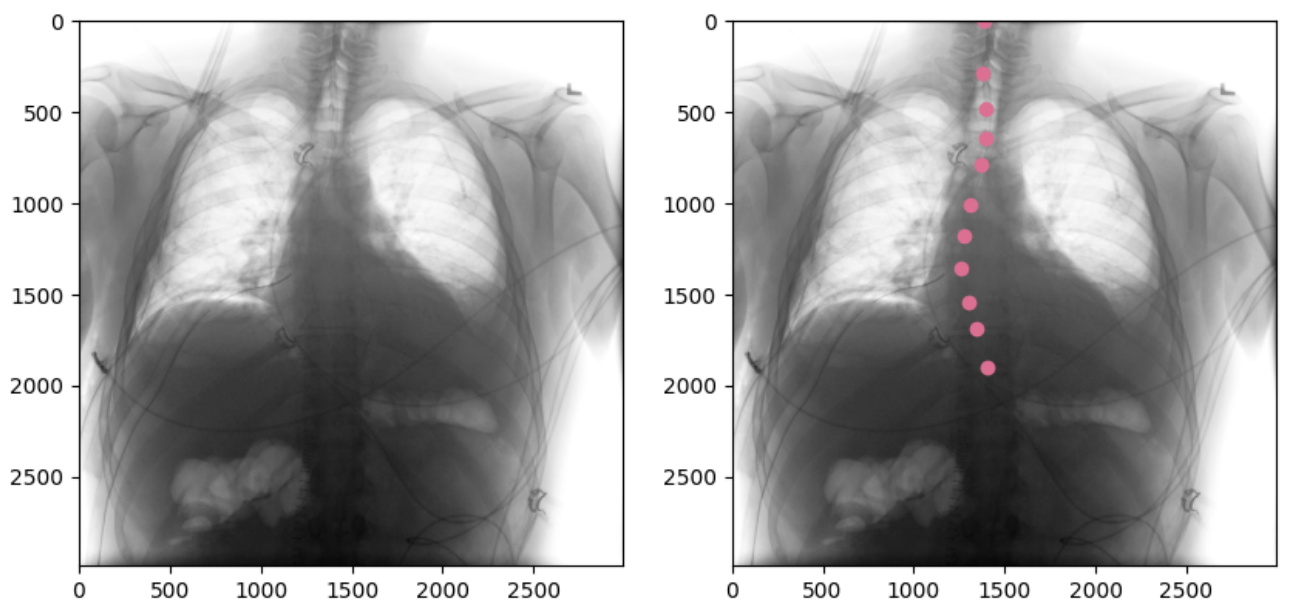
```
In [ ]: process_type("NGT - Abnormal")
```

NGT - Abnormal



```
In [ ]: plot_catheter("NGT - Abnormal")
```

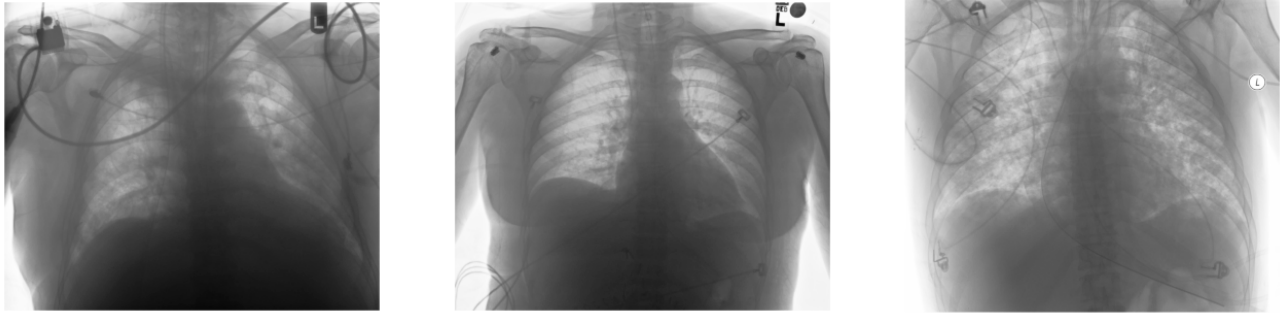
NGT - Abnormal



NGT - BORDERLINE

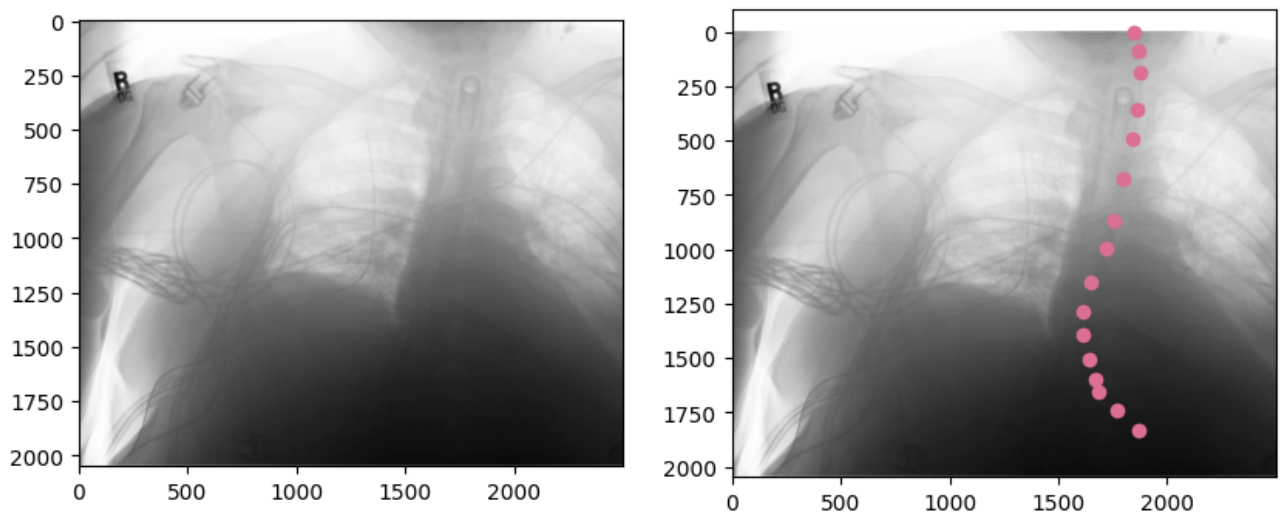
```
In [ ]: process_type('NGT - Borderline')
```

NGT - Borderline



```
In [ ]: plot_catheter('NGT - Borderline')
```

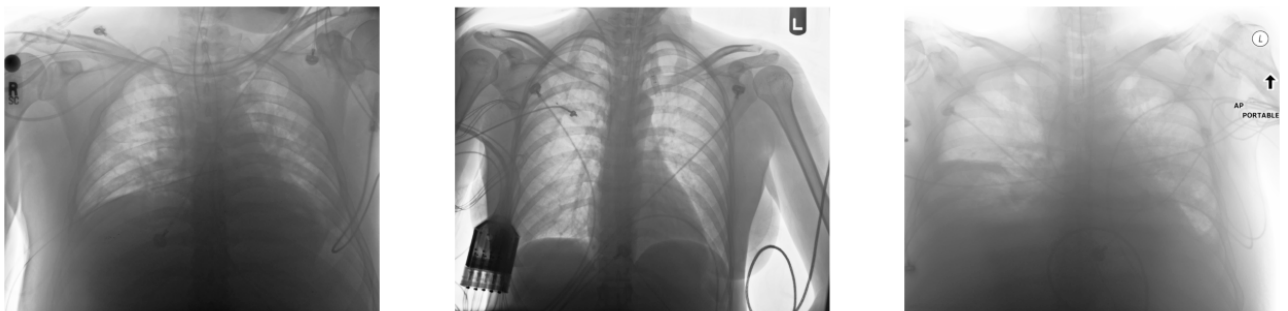
NGT - Borderline



NGT - Incompletely Imaged

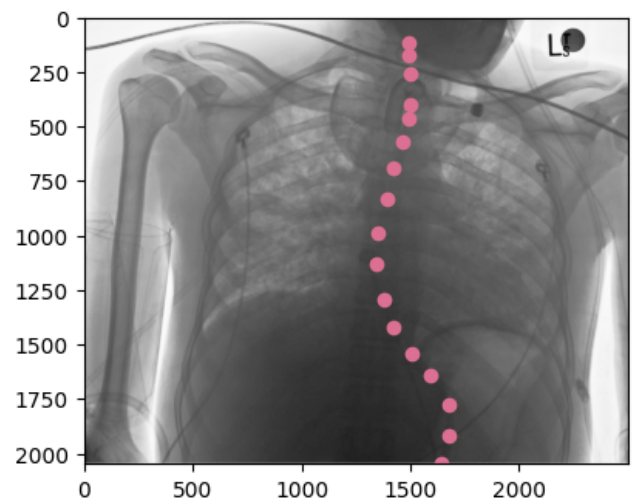
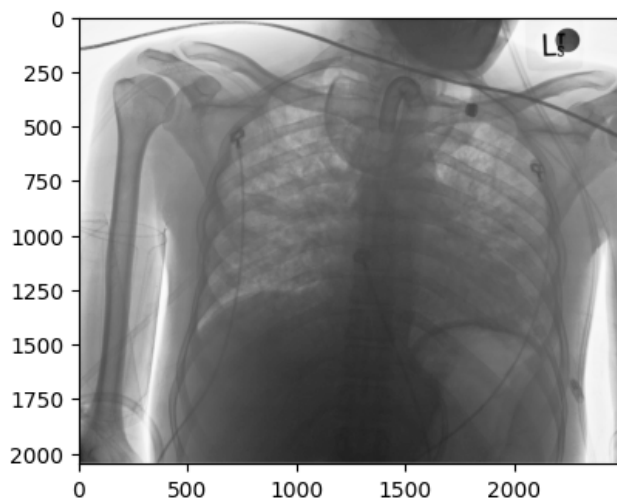
```
In [ ]: process_type('NGT - Incompletely Imaged')
```

NGT - Incompletely Imaged



```
In [ ]: plot_catheter('NGT - Incompletely Imaged')
```

NGT - Incompletely Imaged



NGT - Normal

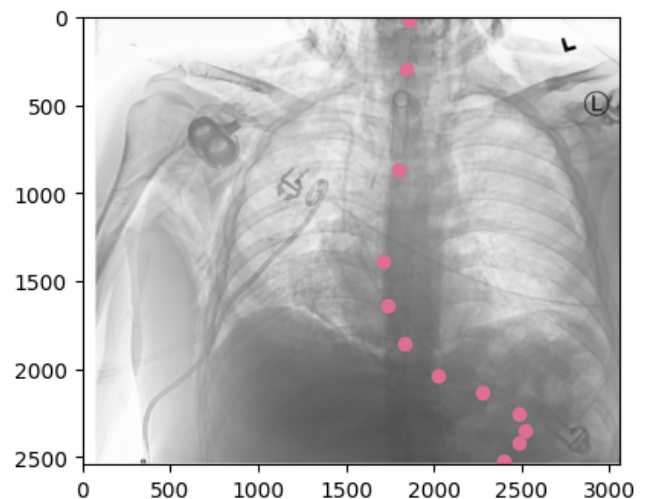
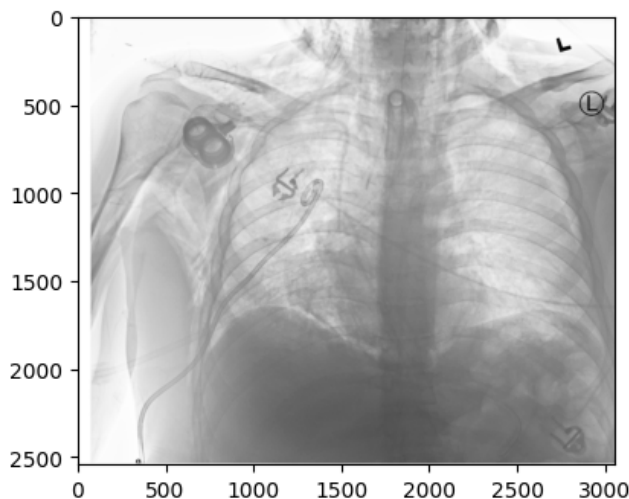
```
In [ ]: process_type('NGT - Normal')
```

NGT - Normal



```
In [ ]: plot_catheter('NGT - Normal')
```

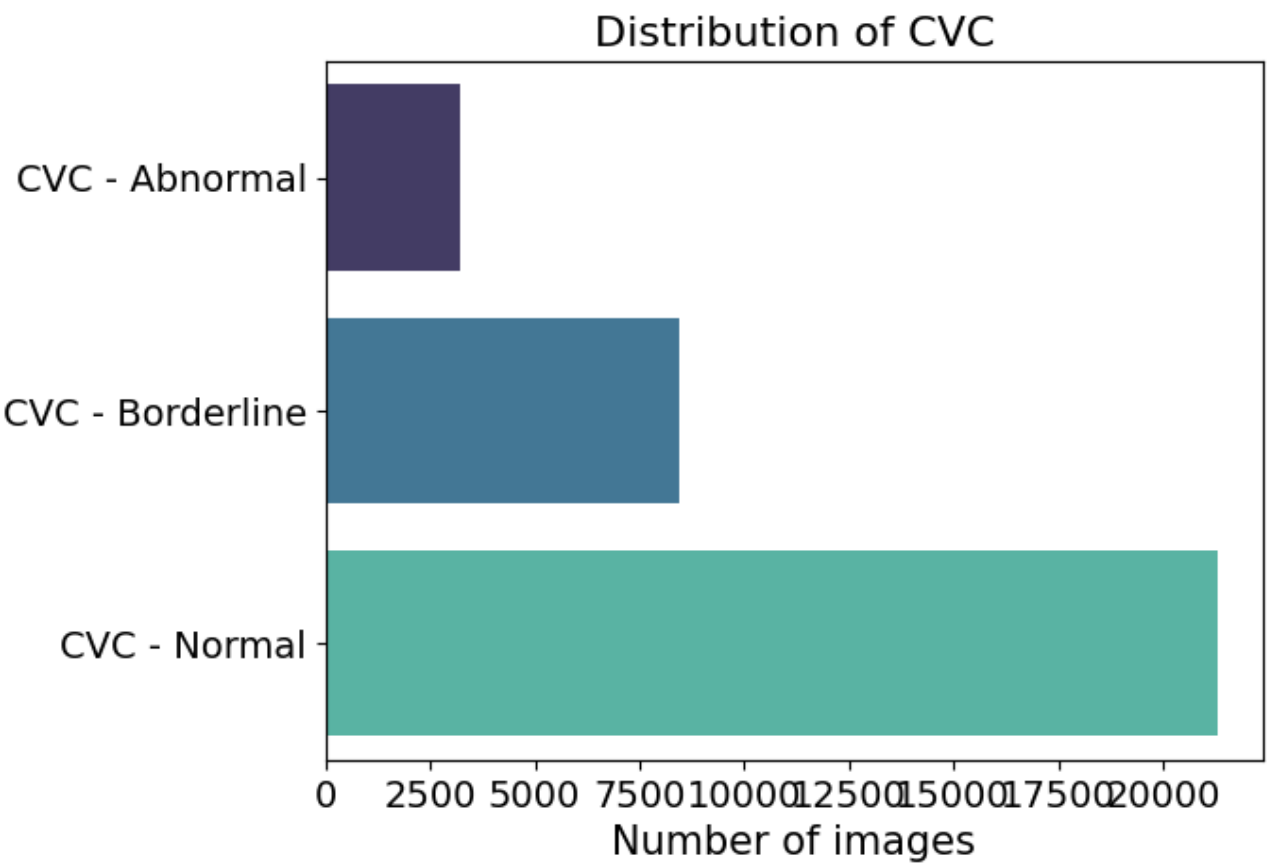
NGT - Normal



CVC - Central Venous Catheter Placement

```
In [ ]: CVC = train_files[8:11].sum()
```

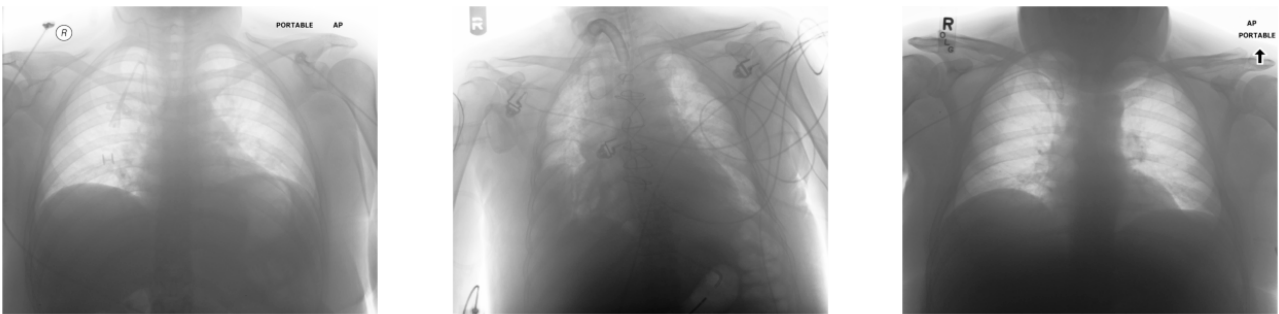
```
CVC = train.iloc[:, 8:11].sum()
plot_distribution(CVC, "CVC")
```



CVC - ABNORMAL

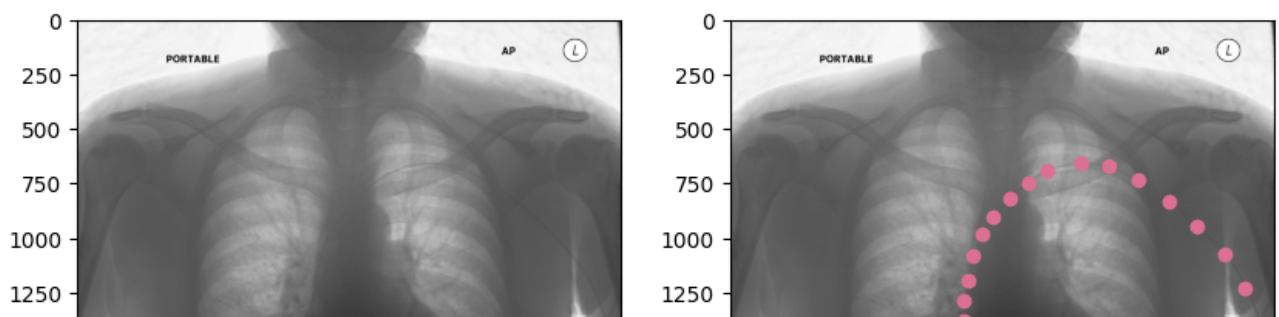
```
In [ ]: process_type("CVC - Abnormal")
```

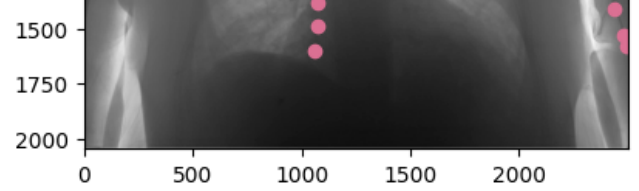
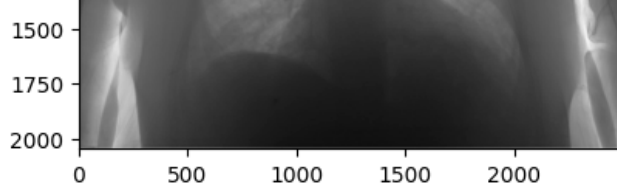
CVC - Abnormal



```
In [ ]: plot_catheter("CVC - Abnormal")
```

CVC - Abnormal

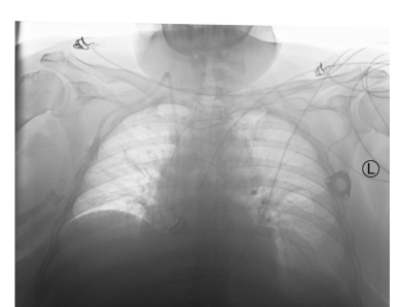




CVC - BORDERLINE

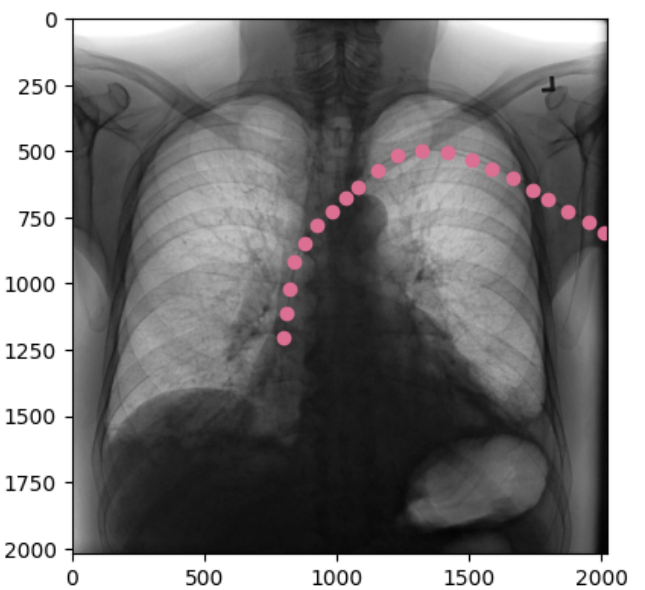
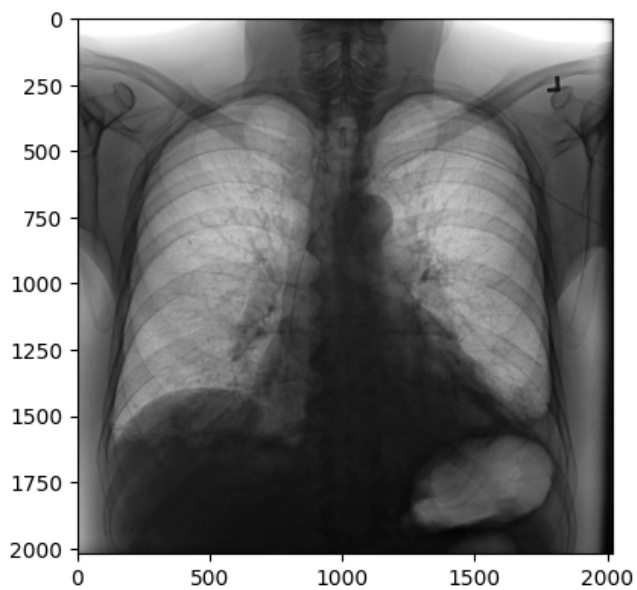
```
In [ ]: process_type("CVC - Borderline")
```

CVC - Borderline



```
In [ ]: plot_catheter("CVC - Borderline")
```

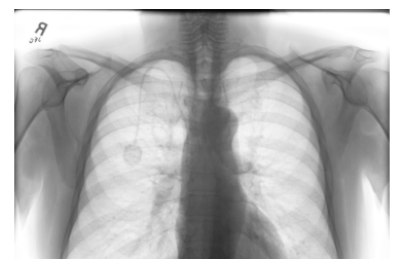
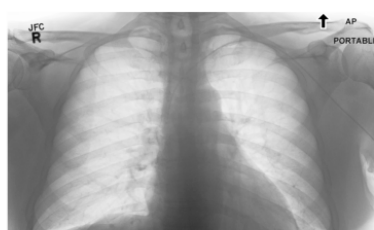
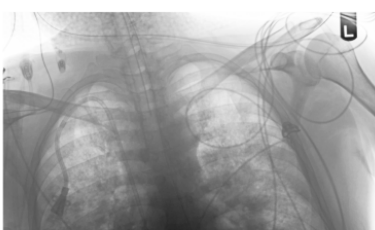
CVC - Borderline



CVC - NORMAL

```
In [ ]: process_type("CVC - Normal")
```

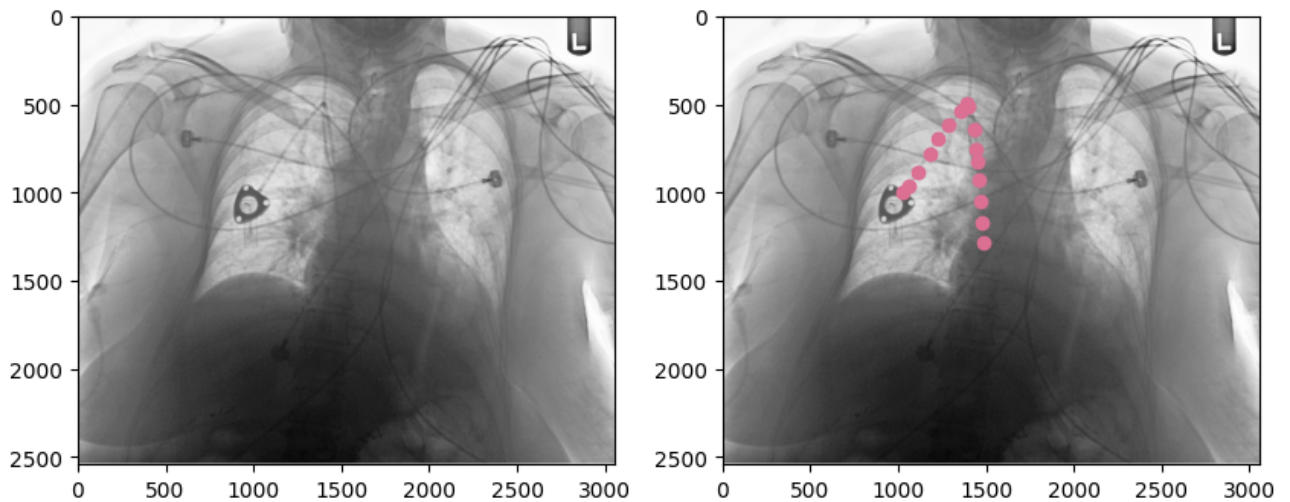
CVC - Normal





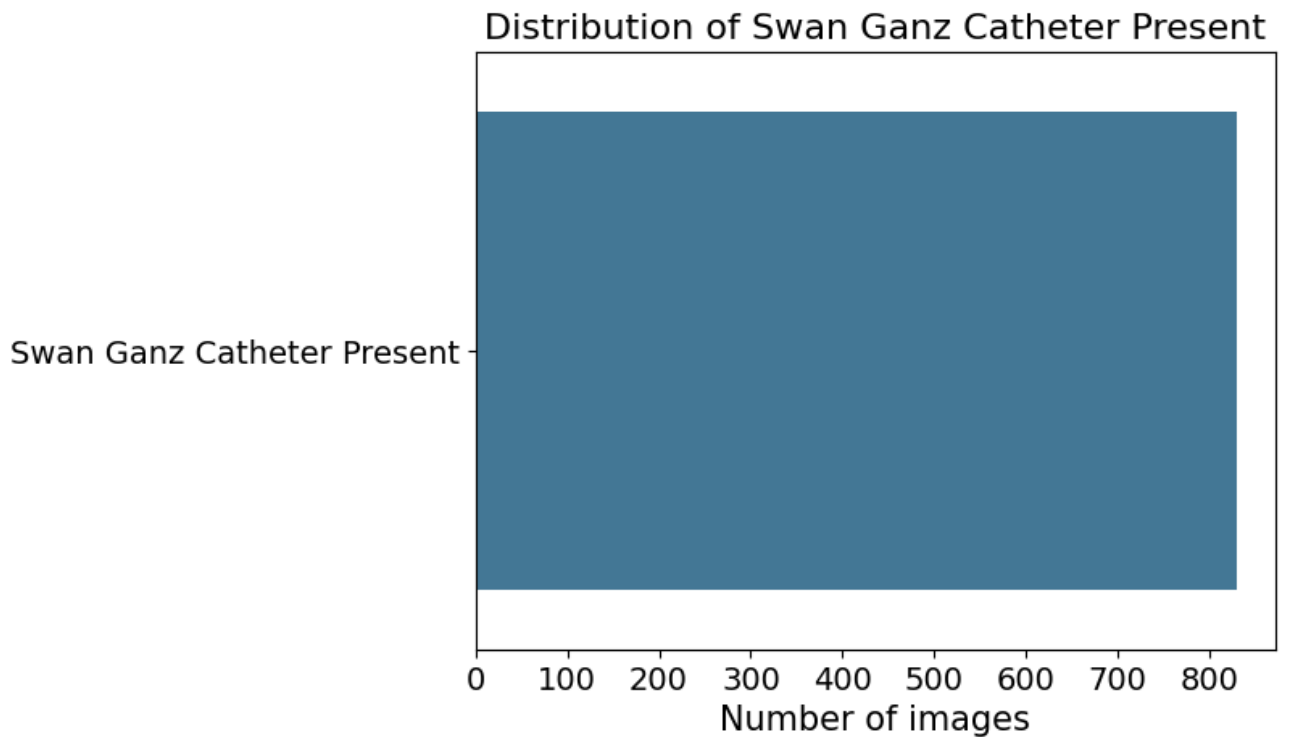
```
In [ ]: plot_catheter("CVC - Normal")
```

CVC - Normal



SWAN GANZ CATHETER PLACEMENT

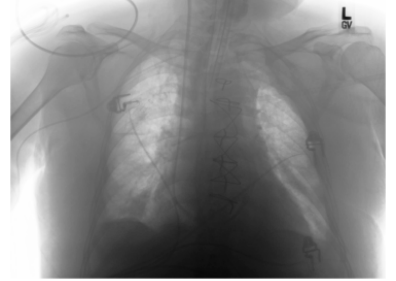
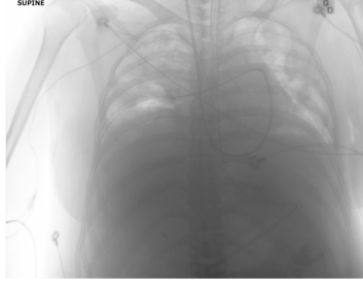
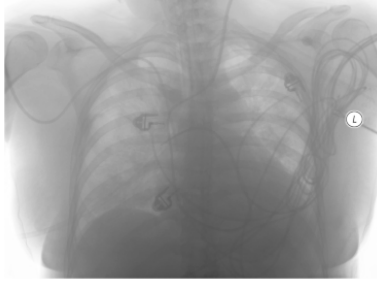
```
In [ ]: SWAN = CVC = train.iloc[:, 11:12].sum()
plot_distribution(SWAN, "Swan Ganz Catheter Present")
```



```
In [ ]: process_type('Swan Ganz Catheter Present')
```

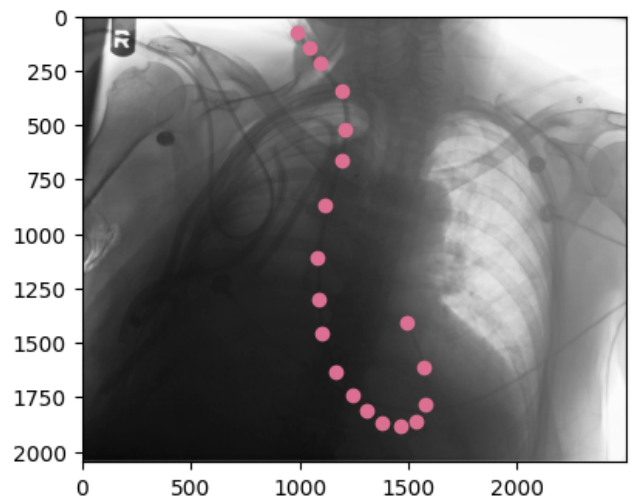
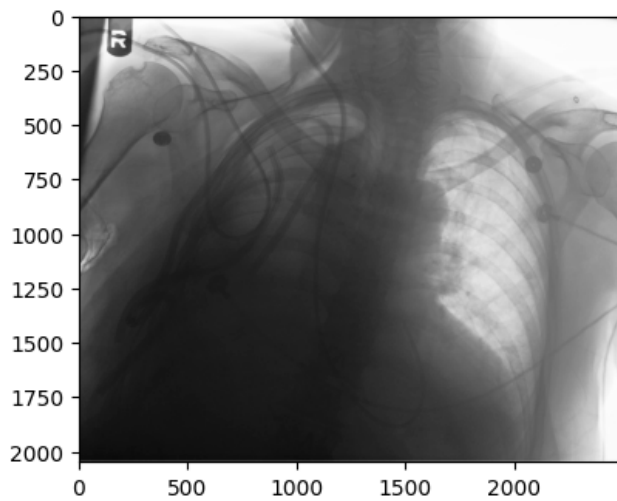
Swan Ganz Catheter Present





```
In [ ]: plot_catheter('Swan Ganz Catheter Present')
```

Swan Ganz Catheter Present



SVM

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')

%cd /content/drive/MyDrive/ADS2002-Project-CatheterPlacement
```

Mounted at /content/drive
/content/drive/MyDrive/ADS2002-Project-CatheterPlacement

```
In [ ]: import pandas as pd
import os
import cv2
import numpy as np
from PIL import Image

filenames = os.listdir("Data/train")
train_imgs = np.empty((0,600, 600))
FileIDs = []
lst = []

count = 0
for file in filenames:
    if count == 500:
        break

    #Making sure the images are the same pixel size
    img = Image.open(f"/content/drive/MyDrive/ADS2002-Project-CatheterPlacement/Data/train/{file}")
    img = img.resize((600,600))
    img = np.array(img)

    #add extra dimension so we can concatenate
    img = np.expand_dims(img,(0))

    #Save filename for later
    FileIDs.append(file)

    #adding to train_imgs array
    train_imgs = np.concatenate((train_imgs, img), axis = 0)

    count += 1

# Now, 'cvc_images' contains your preprocessed CVC catheter images, and 'cvc_labels' contains
```

```
In [ ]: #extracting just the CVC labels
train_data = pd.read_csv("/content/drive/MyDrive/ADS2002-Project-CatheterPlacement/Data/train_data.csv")
CVC_train = train_data[["StudyInstanceUID", "CVC - Abnormal", "CVC - Borderline", "CVC - Normal"]]
CVC_train.set_index('StudyInstanceUID', inplace=True)
CVC_train.head()

#cleaning up the labels
UID = []
for i in range(len(FileIDs)):
    UID.append(FileIDs[i].replace(".jpg",""))
```

```
In [ ]: train_labels = np.empty((0,2))
```

```

train_labels = np.empty((0,3))
data = []
for ID in UID:
    data.append(CVC_train.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)

train_labels = train_labels_df.values

train_labels

```

```

Out[ ]: array([[0, 0, 1],
               [0, 1, 0],
               [0, 0, 1],
               ...,
               [1, 0, 0],
               [0, 1, 1],
               [1, 0, 0]])

```

```

In [ ]: # Assuming you have a list 'Labels' with corresponding labels for each image (Abnormal,
# Example:
# Labels = ["CVC - Abnormal", "CVC - Normal", "CVC - Borderline"] # Make sure this list

from sklearn.model_selection import train_test_split

# Flatten the images in train_imgs
train_imgs_flattened = train_imgs.reshape(train_imgs.shape[0], -1)
train_labels_flattened = train_labels.reshape(train_labels.shape[0], -1)
train_labels_flattened = train_labels.argmax(axis=1)

X_train, X_test, Y_train, Y_test = train_test_split(train_imgs_flattened, train_labels_1

# Now, your SVM classifier is trained and ready for predictions.

# You can use svm_classifier.predict(new_data) to make predictions on new data.

```

```

In [ ]: # Import the SVM classifier from scikit-learn
from sklearn import svm

# Initialize an SVM classifier
svm_classifier = svm.SVC(kernel='linear') # You can choose the kernel type you prefer

# Train the SVM model with your data and labels
svm_classifier.fit(X_train, Y_train)

```

```

Out[ ]: SVC(kernel='linear')
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```

```

In [ ]: # Make predictions on the test set

from sklearn.metrics import accuracy_score, classification_report

y_pred = svm_classifier.predict(X_test)

# Evaluate the model using accuracy and classification report
accuracy = accuracy_score(Y_test, y_pred)
classification_rep = classification_report(Y_test, y_pred)

```

```
print(f"Accuracy: {accuracy}")
print("Classification Report:")
print(classification_rep)
```

Accuracy: 0.44

Classification Report:

	precision	recall	f1-score	support
0	0.11	0.08	0.10	12
1	0.32	0.23	0.27	35
2	0.53	0.66	0.59	53
accuracy			0.44	100
macro avg	0.32	0.32	0.32	100
micro avg	0.44	0.44	0.44	100

Custom Neural Network

```
In [ ]: import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import os
import pandas as pd
import keras

print(tf.__version__)
```

2.12.0

Preparing the Data

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
In [ ]: filenames = os.listdir("/content/drive/MyDrive/ADS2002-Project-CatheterPlacement/Data/tr
```

```
In [ ]: train_imgs = np.empty((0,2048, 2500))
FileIDs = []

#getting the first 100 images
for i in range(100):
    #Making sure the images are the same pixel size
    img = plt.imread(f"/content/drive/MyDrive/ADS2002-Project-CatheterPlacement/Data/train
    img_shape = img.shape

    if img_shape == (2048, 2500):
        #Save filename for later
        FileIDs.append(filenames[i])

        #add extra dimension so we can concatenate
        img = np.expand_dims(img,(0))

        #adding to train_imgs array
        train_imgs = np.concatenate((train_imgs, img), axis = 0)

train_imgs.shape
```

Out[]: (48, 2048, 2500)

```
In [ ]: train_tensor = np.reshape(train_imgs, (48, 2048, 2500, 1))
```

```
In [ ]: # makes img 2x smaller
max_pooling = tf.keras.layers.MaxPooling2D(pool_size = 2, strides = 2, padding = "valid")
pooled_array = max_pooling(train_tensor)
```

```
In [ ]: #extracting just the CVC labels
train_data = pd.read_csv("/content/drive/MyDrive/ADS2002-Project-CatheterPlacement/Data/
CVC_train = train_data[["StudyInstanceUID", "CVC - Abnormal", "CVC - Borderline", "CVC -
CVC_train.set_index('StudyInstanceUID', inplace=True)
CVC_train.head()
```

Out[]:

	CVC - Abnormal	CVC - Borderline	CV Norm
StudyInstanceUID			
1.2.826.0.1.3680043.8.498.26697628953273228189375557799582420561	0	0	
1.2.826.0.1.3680043.8.498.46302891597398758759818628675365157729	0	0	
1.2.826.0.1.3680043.8.498.23819260719748494858948050424870692577	0	1	
1.2.826.0.1.3680043.8.498.68286643202323212801283518367144358744	1	0	
1.2.826.0.1.3680043.8.498.10050203009225938259119000528814762175	0	0	

In []:

```
#cleaning up the labels
UID = []
for i in range(len(FileIDs)):
    UID.append(FileIDs[i].replace(".jpg",""))

UID[0:10]
```

Out[]:

```
['1.2.826.0.1.3680043.8.498.96315500926641444675409331764690933307',
 '1.2.826.0.1.3680043.8.498.96450515721778915344219421534269786290',
 '1.2.826.0.1.3680043.8.498.96359554498564333142663951388648557789',
 '1.2.826.0.1.3680043.8.498.96447675290915183668080373345495953473',
 '1.2.826.0.1.3680043.8.498.96279880013483713441521686300695558821',
 '1.2.826.0.1.3680043.8.498.96349754766067875463134738379718157835',
 '1.2.826.0.1.3680043.8.498.96406955712119661656412685705445497931',
 '1.2.826.0.1.3680043.8.498.96346896468102014556173811602388995798',
 '1.2.826.0.1.3680043.8.498.96332366015730959061321671670587275158',
 '1.2.826.0.1.3680043.8.498.96365165965335417502443200533971674013']
```

In []:

```
train_labels = np.empty((0,3))
data = []
for ID in UID:
    data.append(CVC_train.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)

train_labels = train_labels_df.values
```

Making and training the model

In []:

```
if np.all(train_labels[1] == [0,0,1]):
    print('hello')
```

hello

In []:

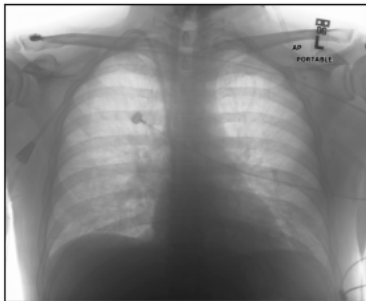
```
def get_alignment(labels):
    result = ""
    if np.any(labels == [2,2,1]):
        result += "Normal"

    elif np.any(labels == [2,1,2]):
        result += "Borderline"

    elif np.any(labels == [1,2,2]):
        result += "Abnormal"

    return result
```

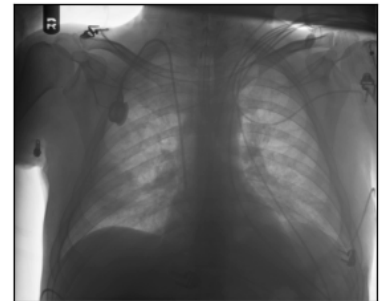
```
In [ ]: #displaying 6 of the images in the pooled_imgs array
plt.figure(figsize=(12,9))
for i in range(6):
    plt.subplot(2,3,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(pooled_array[i], cmap=plt.cm.binary)
    plt.xlabel(get_allignment(train_labels[i]))
plt.show()
```



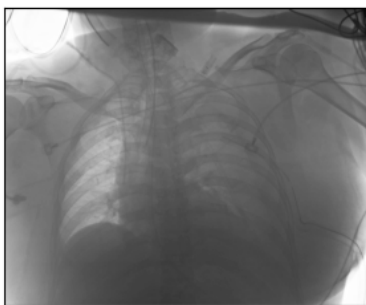
Borderline



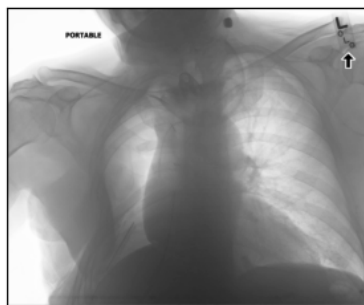
Normal



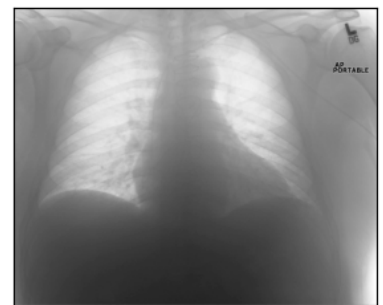
Borderline



Normal



Normal



Borderline

```
In [ ]: pooled_array[0].shape
```

```
Out[ ]: TensorShape([1024, 1250, 1])
```

```
In [ ]: lr = 0.000005
epochs=100
img_size = (1024, 1250)
```

```
In [ ]: #initiating the model
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=img_size),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(3)
])
```

```
In [ ]: #compling the model
optim = keras.optimizers.Adam(learning_rate=lr)
model.compile(optimizer=optim,
               loss=tf.keras.losses.CategoricalCrossentropy(from_logits=True),
               metrics=['accuracy'],)
```

```
In [ ]: #fitting the model
history = model.fit(pooled_array, train_labels, epochs=epochs)
```

Epoch 1/100
2/2 [=====] - 6s 2s/step - loss: 397.0175 - accuracy: 0.3958
Epoch 2/100
2/2 [=====] - 3s 1s/step - loss: 855.8772 - accuracy: 0.5833
Epoch 3/100
2/2 [=====] - 4s 2s/step - loss: 537.0210 - accuracy: 0.5417
Epoch 4/100
2/2 [=====] - 4s 2s/step - loss: 174.6266 - accuracy: 0.5833
Epoch 5/100
2/2 [=====] - 3s 1s/step - loss: 122.2074 - accuracy: 0.3542
Epoch 6/100
2/2 [=====] - 4s 1s/step - loss: 93.0095 - accuracy: 0.5833
Epoch 7/100
2/2 [=====] - 2s 963ms/step - loss: 253.6956 - accuracy: 0.1667
Epoch 8/100
2/2 [=====] - 3s 2s/step - loss: 135.1093 - accuracy: 0.5833
Epoch 9/100
2/2 [=====] - 3s 1s/step - loss: 189.7440 - accuracy: 0.5833
Epoch 10/100
2/2 [=====] - 3s 1s/step - loss: 126.4880 - accuracy: 0.5208
Epoch 11/100
2/2 [=====] - 2s 1s/step - loss: 80.6841 - accuracy: 0.4167
Epoch 12/100
2/2 [=====] - 3s 996ms/step - loss: 58.7510 - accuracy: 0.6042
Epoch 13/100
2/2 [=====] - 3s 1s/step - loss: 55.3757 - accuracy: 0.4792
Epoch 14/100
2/2 [=====] - 3s 2s/step - loss: 36.9668 - accuracy: 0.5833
Epoch 15/100
2/2 [=====] - 2s 964ms/step - loss: 53.7255 - accuracy: 0.6875
Epoch 16/100
2/2 [=====] - 2s 980ms/step - loss: 43.0521 - accuracy: 0.6667
Epoch 17/100
2/2 [=====] - 2s 1s/step - loss: 24.4128 - accuracy: 0.6875
Epoch 18/100
2/2 [=====] - 2s 1s/step - loss: 24.0603 - accuracy: 0.6667
Epoch 19/100
2/2 [=====] - 4s 2s/step - loss: 19.5048 - accuracy: 0.6458
Epoch 20/100
2/2 [=====] - 3s 969ms/step - loss: 12.7744 - accuracy: 0.6875
Epoch 21/100
2/2 [=====] - 2s 1s/step - loss: 22.6980 - accuracy: 0.7292
Epoch 22/100
2/2 [=====] - 2s 951ms/step - loss: 19.1127 - accuracy: 0.6250
Epoch 23/100
2/2 [=====] - 2s 965ms/step - loss: 16.9469 - accuracy: 0.6458
Epoch 24/100
2/2 [=====] - 2s 1s/step - loss: 9.2830 - accuracy: 0.7083
Epoch 25/100
2/2 [=====] - 4s 2s/step - loss: 15.1006 - accuracy: 0.7292
Epoch 26/100
2/2 [=====] - 2s 957ms/step - loss: 14.7963 - accuracy: 0.7292
Epoch 27/100
2/2 [=====] - 2s 970ms/step - loss: 11.8619 - accuracy: 0.6875
Epoch 28/100
2/2 [=====] - 2s 1s/step - loss: 12.6338 - accuracy: 0.7083
Epoch 29/100
2/2 [=====] - 2s 957ms/step - loss: 10.1854 - accuracy: 0.8125
Epoch 30/100
2/2 [=====] - 2s 1s/step - loss: 11.7453 - accuracy: 0.7083
Epoch 31/100
2/2 [=====] - 4s 2s/step - loss: 9.2501 - accuracy: 0.7917
Epoch 32/100
2/2 [=====] - 3s 1s/step - loss: 4.1858 - accuracy: 0.7917
Epoch 33/100
2/2 [=====] - 3s 1s/step - loss: 13.5716 - accuracy: 0.7083
Epoch 34/100
2/2 [=====] - 2s 954ms/step - loss: 6.7785 - accuracy: 0.7083
Epoch 35/100
2/2 [=====] - 2s 983ms/step - loss: 12.0349 - accuracy: 0.6250
Epoch 36/100
2/2 [=====] - 4s 2s/step - loss: 10.0569 - accuracy: 0.6875

Epoch 37/100
2/2 [=====] - 3s 1s/step - loss: 13.1810 - accuracy: 0.6250
Epoch 38/100
2/2 [=====] - 3s 1s/step - loss: 16.4667 - accuracy: 0.6250
Epoch 39/100
2/2 [=====] - 2s 1s/step - loss: 12.5331 - accuracy: 0.7083
Epoch 40/100
2/2 [=====] - 3s 1s/step - loss: 13.2756 - accuracy: 0.7083
Epoch 41/100
2/2 [=====] - 3s 1s/step - loss: 20.9390 - accuracy: 0.6667
Epoch 42/100
2/2 [=====] - 4s 2s/step - loss: 16.4590 - accuracy: 0.6042
Epoch 43/100
2/2 [=====] - 3s 1s/step - loss: 11.6369 - accuracy: 0.7917
Epoch 44/100
2/2 [=====] - 2s 964ms/step - loss: 18.6257 - accuracy: 0.6875
Epoch 45/100
2/2 [=====] - 2s 965ms/step - loss: 5.6847 - accuracy: 0.8333
Epoch 46/100
2/2 [=====] - 2s 1s/step - loss: 6.4516 - accuracy: 0.8333
Epoch 47/100
2/2 [=====] - 4s 1s/step - loss: 7.2442 - accuracy: 0.8125
Epoch 48/100
2/2 [=====] - 2s 965ms/step - loss: 8.5096 - accuracy: 0.7708
Epoch 49/100
2/2 [=====] - 2s 997ms/step - loss: 10.4557 - accuracy: 0.7083
Epoch 50/100
2/2 [=====] - 3s 1s/step - loss: 10.7769 - accuracy: 0.7917
Epoch 51/100
2/2 [=====] - 2s 998ms/step - loss: 15.0131 - accuracy: 0.6667
Epoch 52/100
2/2 [=====] - 3s 1s/step - loss: 20.2162 - accuracy: 0.6250
Epoch 53/100
2/2 [=====] - 3s 1s/step - loss: 10.7835 - accuracy: 0.7708
Epoch 54/100
2/2 [=====] - 2s 970ms/step - loss: 12.0600 - accuracy: 0.7708
Epoch 55/100
2/2 [=====] - 2s 973ms/step - loss: 13.8154 - accuracy: 0.6875
Epoch 56/100
2/2 [=====] - 2s 1s/step - loss: 10.0331 - accuracy: 0.8333
Epoch 57/100
2/2 [=====] - 2s 1s/step - loss: 10.6891 - accuracy: 0.7917
Epoch 58/100
2/2 [=====] - 5s 2s/step - loss: 9.9800 - accuracy: 0.6875
Epoch 59/100
2/2 [=====] - 5s 2s/step - loss: 7.3366 - accuracy: 0.7500
Epoch 60/100
2/2 [=====] - 2s 964ms/step - loss: 10.6622 - accuracy: 0.7917
Epoch 61/100
2/2 [=====] - 2s 975ms/step - loss: 11.9892 - accuracy: 0.7500
Epoch 62/100
2/2 [=====] - 2s 971ms/step - loss: 10.8367 - accuracy: 0.7083
Epoch 63/100
2/2 [=====] - 2s 962ms/step - loss: 8.3898 - accuracy: 0.7917
Epoch 64/100
2/2 [=====] - 4s 2s/step - loss: 20.5695 - accuracy: 0.6667
Epoch 65/100
2/2 [=====] - 3s 955ms/step - loss: 11.9919 - accuracy: 0.6875
Epoch 66/100
2/2 [=====] - 2s 979ms/step - loss: 13.7913 - accuracy: 0.7292
Epoch 67/100
2/2 [=====] - 2s 966ms/step - loss: 6.7140 - accuracy: 0.8125
Epoch 68/100
2/2 [=====] - 3s 1s/step - loss: 9.8374 - accuracy: 0.8125
Epoch 69/100
2/2 [=====] - 4s 2s/step - loss: 4.5649 - accuracy: 0.8333
Epoch 70/100
2/2 [=====] - 3s 1s/step - loss: 10.4958 - accuracy: 0.7500
Epoch 71/100
2/2 [=====] - 3s 1s/step - loss: 6.0589 - accuracy: 0.8750
Epoch 72/100
2/2 [=====] - 2s 974ms/step - loss: 7.3979 - accuracy: 0.8750
Epoch 73/100


```

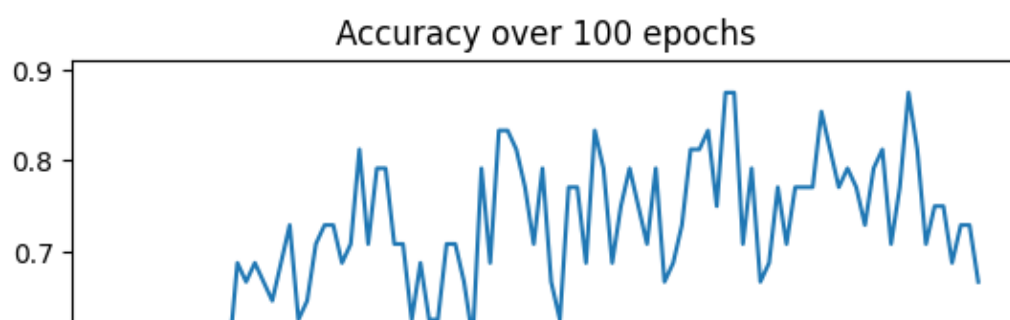
Epoch 75/100
2/2 [=====] - 3s 1s/step - loss: 12.4232 - accuracy: 0.7083
Epoch 74/100
2/2 [=====] - 3s 1s/step - loss: 10.1944 - accuracy: 0.7917
Epoch 75/100
2/2 [=====] - 3s 1s/step - loss: 14.7110 - accuracy: 0.6667
Epoch 76/100
2/2 [=====] - 2s 968ms/step - loss: 24.3729 - accuracy: 0.6875
Epoch 77/100
2/2 [=====] - 2s 958ms/step - loss: 9.4317 - accuracy: 0.7708
Epoch 78/100
2/2 [=====] - 2s 964ms/step - loss: 23.3896 - accuracy: 0.7083
Epoch 79/100
2/2 [=====] - 3s 1s/step - loss: 9.6430 - accuracy: 0.7708
Epoch 80/100
2/2 [=====] - 3s 1s/step - loss: 10.7553 - accuracy: 0.7708
Epoch 81/100
2/2 [=====] - 3s 1s/step - loss: 11.5950 - accuracy: 0.7708
Epoch 82/100
2/2 [=====] - 2s 1s/step - loss: 4.6602 - accuracy: 0.8542
Epoch 83/100
2/2 [=====] - 3s 1s/step - loss: 6.2187 - accuracy: 0.8125
Epoch 84/100
2/2 [=====] - 2s 941ms/step - loss: 10.3964 - accuracy: 0.7708
Epoch 85/100
2/2 [=====] - 2s 1s/step - loss: 8.1115 - accuracy: 0.7917
Epoch 86/100
2/2 [=====] - 4s 2s/step - loss: 9.7994 - accuracy: 0.7708
Epoch 87/100
2/2 [=====] - 3s 1s/step - loss: 8.6359 - accuracy: 0.7292
Epoch 88/100
2/2 [=====] - 2s 968ms/step - loss: 16.8545 - accuracy: 0.7917
Epoch 89/100
2/2 [=====] - 2s 968ms/step - loss: 8.5134 - accuracy: 0.8125
Epoch 90/100
2/2 [=====] - 2s 1s/step - loss: 9.3836 - accuracy: 0.7083
Epoch 91/100
2/2 [=====] - 3s 2s/step - loss: 7.0182 - accuracy: 0.7708
Epoch 92/100
2/2 [=====] - 3s 1s/step - loss: 6.4453 - accuracy: 0.8750
Epoch 93/100
2/2 [=====] - 3s 1s/step - loss: 9.0435 - accuracy: 0.8125
Epoch 94/100
2/2 [=====] - 2s 978ms/step - loss: 9.2163 - accuracy: 0.7083
Epoch 95/100
2/2 [=====] - 3s 1s/step - loss: 12.2373 - accuracy: 0.7500
Epoch 96/100
2/2 [=====] - 3s 1s/step - loss: 9.7253 - accuracy: 0.7500
Epoch 97/100
2/2 [=====] - 4s 2s/step - loss: 10.9290 - accuracy: 0.6875
Epoch 98/100
2/2 [=====] - 3s 1s/step - loss: 16.4481 - accuracy: 0.7292
Epoch 99/100
2/2 [=====] - 2s 1s/step - loss: 9.2781 - accuracy: 0.7292
Epoch 100/100
2/2 [=====] - 2s 954ms/step - loss: 37.2660 - accuracy: 0.6667

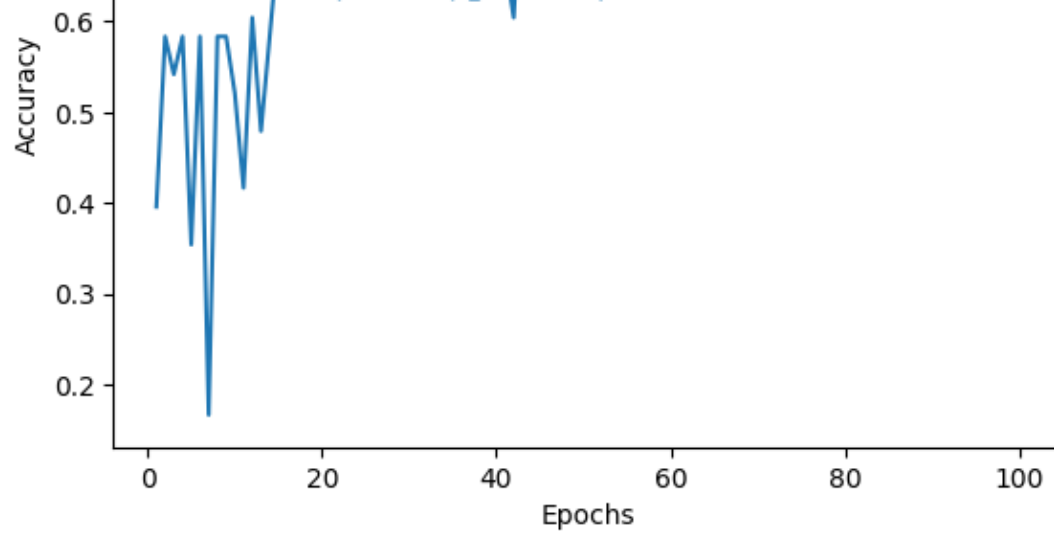
```

```

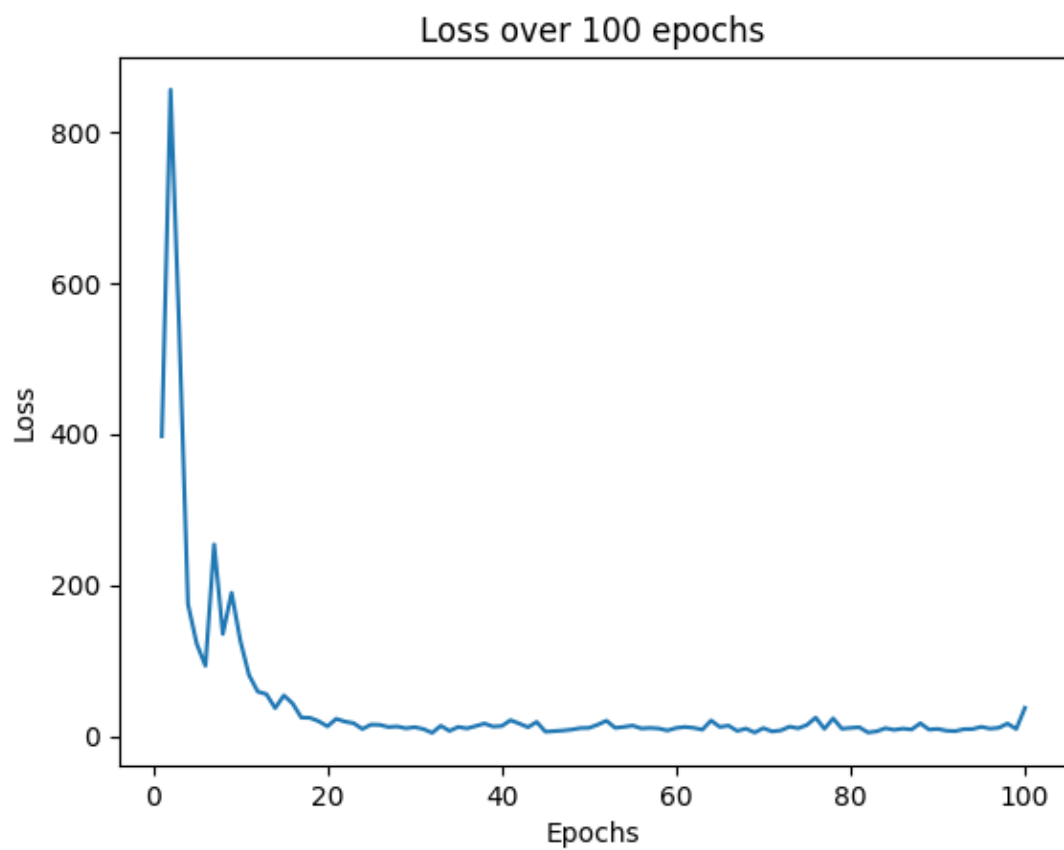
In [ ]: plt.plot(range(1,epochs+1), history.history["accuracy"])
plt.title(f"Accuracy over {epochs} epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy");

```





```
In [ ]: plt.plot(range(1,epochs+1), history.history["loss"])
plt.title(f"Loss over {epochs} epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss");
```



Custom Neural Network - Normal and Abnormal

```
In [ ]: import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import os
import pandas as pd
from tensorflow import keras
from PIL import Image

print(tf.__version__)
```

2.4.0

```
In [ ]: !pip install opencv-python
```

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: opencv-python in /home/carlos/.local/lib/python3.6/site-packages (from opencv-python==4.5.1.40)
Requirement already satisfied: numpy>=1.13.3 in /usr/local/lib/python3.6/dist-packages (from opencv-python==4.5.1.40)
WARNING: You are using pip version 20.3.2; however, version 21.3.1 is available.
You should consider upgrading via the '/usr/bin/python3 -m pip install --upgrade pip' command.

```
In [ ]: import cv2
```

Preparing the Data

```
In [ ]: filenames = os.listdir("/projects/sc73/ranzcr-clip-catheter-line-classification/train")
```

```
In [ ]: #Subsetting the data
train_data = pd.read_csv("/projects/sc73/ranzcr-clip-catheter-line-classification/train.csv")
train_data
```

```
Out[ ]:
```

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT - Abnormal	N Border
0	1.2.826.0.1.3680043.8.498.26697628953273228189...	0	0	0	0	
1	1.2.826.0.1.3680043.8.498.46302891597398758759...	0	0	1	0	
2	1.2.826.0.1.3680043.8.498.23819260719748494858...	0	0	0	0	
3	1.2.826.0.1.3680043.8.498.68286643202323212801...	0	0	0	0	
4	1.2.826.0.1.3680043.8.498.10050203009225938259...	0	0	0	0	
...	...	...	...	...	...	
30078	1.2.826.0.1.3680043.8.498.74257566841157531124...	0	0	1	0	
30079	1.2.826.0.1.3680043.8.498.46510939987173529969...	0	0	0	0	
30080	1.2.826.0.1.3680043.8.498.43173270582850645437...	0	0	1	0	
30081	1.2.826.0.1.3680043.8.498.95092491950130838685...	0	0	0	0	
30082	1.2.826.0.1.3680043.8.498.99518162226171269731...	0	0	1	0	

30083 rows × 13 columns

```
In [ ]: #extracting just the CVC Labels, and just Normal and Abnormal catheters
CVC_train = train_data[["StudyInstanceUID", "CVC - Abnormal", "CVC - Normal"]]
CVC_train.set_index('StudyInstanceUID', inplace=True)
CVC_train = CVC_train.drop(CVC_train[(CVC_train["CVC - Abnormal"] == 0) & (CVC_train["CVC - Nor
CVC_train.drop("CVC - Abnormal", axis = 'columns', inplace = True)
CVC_train["CVC - Normal"].sum()
```

Out[]: 21324

```
In [ ]: CVC_normal = CVC_train[CVC_train["CVC - Normal"] == 1].sample(n = 2000)
CVC_abnormal = CVC_train[CVC_train["CVC - Normal"] == 0].sample(n = 2000)
CVC_all = pd.concat([CVC_normal, CVC_abnormal])
CVC_all
```

Out[]:

	CVC - Normal
StudyInstanceUID	
1.2.826.0.1.3680043.8.498.83944916333526123502778490500928788639	1
1.2.826.0.1.3680043.8.498.23933321106006088567170276662551480315	1
1.2.826.0.1.3680043.8.498.84400982664153368598738672971053132124	1
1.2.826.0.1.3680043.8.498.59376680813742484063095010151880137373	1
1.2.826.0.1.3680043.8.498.91751632618204442295160422886897847632	1
...	...
1.2.826.0.1.3680043.8.498.83097595799508746778446434996490113777	0
1.2.826.0.1.3680043.8.498.11333894316143324337529593966002928477	0
1.2.826.0.1.3680043.8.498.19229796633151935648327076303774354430	0
1.2.826.0.1.3680043.8.498.61413848185130431709941470197603810809	0
1.2.826.0.1.3680043.8.498.30103294359252823294128112316699100138	0

4000 rows × 1 columns

```
In [ ]: len(CVC_all)
```

Out[]: 4000

```
In [ ]: CVC_train = CVC_all
```

```
In [ ]: from PIL import Image
size = 600
train_imgs = np.empty((0, size, size))
FileIDs = []
lst = []

count = 0
for file in filenames:
    if count == 300:
        break

    #only importing the image if the image is part of the subsetted data
    if file.replace(".jpg", "") in CVC_train.index:

        #Making sure the images are the same pixel size
        img = Image.open(f"/projects/sc73/ranzcr-clip-catheter-line-classification/train/{file}")
        img = img.resize((size, size))
```

```

img = img.reshape((255,255))
img = np.array(img)

#add extra dimension so we can concatenate
img = np.expand_dims(img,(0))

#Save filename for later
FileIDs.append(file)

#adding to train_imgs array
train_imgs = np.concatenate((train_imgs, img), axis = 0)

count += 1

```

```

In [ ]: #cleaning up the labels
UID = []
for ID in FileIDs:
    ID = ID.replace(".jpg","")
    if ID in CVC_train.index:
        UID.append(ID)

```

```

In [ ]: data = []
for ID in UID:
    data.append(CVC_train.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)

train_labels = train_labels_df.values
train_labels_df

```

```

Out[ ]:

```

	CVC - Normal
1.2.826.0.1.3680043.8.498.20829430711461815077247364797774896990	0
1.2.826.0.1.3680043.8.498.35072888985370991799652296889578661624	0
1.2.826.0.1.3680043.8.498.10724541644358274215523122873926253955	1
1.2.826.0.1.3680043.8.498.12271878036664828678891914784344423473	0
1.2.826.0.1.3680043.8.498.81820017770914828024018162370001439126	1
...	...
1.2.826.0.1.3680043.8.498.33496817214155111186732586634241228007	1
1.2.826.0.1.3680043.8.498.76794125332716608263154717912129536285	1
1.2.826.0.1.3680043.8.498.10533863396518656580245826580989141214	0
1.2.826.0.1.3680043.8.498.46691559386432145944373733448410137409	0
1.2.826.0.1.3680043.8.498.70895478936511545330004775319430347870	1

300 rows × 1 columns

```

In [ ]: from sklearn.model_selection import train_test_split

normalized_images = train_imgs / 255.0 # Normalising the pixel of the images

X = normalized_images
Y = train_labels

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

```

ANNOTATED DATA

ANNOTATED DATA

```
In [ ]: anno_df = pd.read_csv("/projects/sc73/ranzcr-clip-catheter-line-classification/train_annotation")
anno_df = anno_df[(anno_df['label'] == 'CVC - Normal') | (anno_df['label'] == 'CVC - Abnormal')]
anno_df["label"] = anno_df["label"].replace({"CVC - Normal": 1, "CVC - Abnormal": 0})
anno_normal = anno_df[anno_df["label"] == 1].sample(n = 350)
anno_abnormal = anno_df[anno_df["label"] == 0].sample(n = 350)
anno_df = pd.concat([anno_normal, anno_abnormal])
anno_df = anno_df.reset_index(drop=True)
anno_df
```

```
Out [ ]:
```

	StudyInstanceUID	label	data
0	1.2.826.0.1.3680043.8.498.37784761397507321564...	1	[[921, 901], [936, 860], [955, 836], [980, 815...
1	1.2.826.0.1.3680043.8.498.99639605094624784905...	1	[[2583, 1090], [2532, 1026], [2462, 957], [236...
2	1.2.826.0.1.3680043.8.498.48805713281949763010...	1	[[734, 1077], [727, 589], [712, 353], [648, 18...
3	1.2.826.0.1.3680043.8.498.25755803986279453282...	1	[[936, 871], [991, 710], [1111, 576], [1274, 4...
4	1.2.826.0.1.3680043.8.498.13054605655150115189...	1	[[1282, 966], [1291, 855], [1312, 745], [1343,...
...	...	...	...
695	1.2.826.0.1.3680043.8.498.22420917910405243013...	0	[[935, 769], [958, 784], [1002, 801], [1045, 8...
696	1.2.826.0.1.3680043.8.498.80694113169580785316...	0	[[1682, 1288], [1721, 1119], [1724, 940], [169...
697	1.2.826.0.1.3680043.8.498.98695105155776417580...	0	[[339, 629], [391, 699], [457, 768], [546, 827...
698	1.2.826.0.1.3680043.8.498.76647715259188312737...	0	[[998, 1217], [1002, 1144], [1007, 1035], [102...
699	1.2.826.0.1.3680043.8.498.81798411904710820810...	0	[[1644, 942], [1682, 883], [1691, 839], [1687,...

700 rows × 3 columns

```
In [ ]: def _parse_annotation(raw):
    annotation = ast.literal_eval(raw)
    annotation = np.array(annotation, dtype=np.int32)
    annotation = np.expand_dims(annotation, axis=0)
    return annotation

def scale_annotations(annotation, original_shape, target_shape):
    ratio_y = target_shape[0] / original_shape[0]
    ratio_x = target_shape[1] / original_shape[1]
    annotation_scaled = [[int(xratio_x), int(yratio_y)] for [x, y] in annotation]
    return annotation_scaled

def annotate(img, target, annotation):
    if len(img.shape) == 2:
        img = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)
    original_shape = img.shape[:2]
    annotation = _parse_annotation(annotation)
    annotation = annotation[0]
    # annotation = scale_annotations(annotation, original_shape, (256, 256))
    # print(annotation)
    img = cv2.polylines(img, [np.array(annotation)], False, color=(255, 0, 0), thickness=20)
    return img
```

```
In [ ]: from torch.utils.data import Dataset
import ast
import torch
```

```
In [ ]: class CustomDataset(Dataset):
    def __init__(self, anno_df):
        self.anno_df = anno_df
        self.anno_file_names = anno_df["StudyInstanceUID"].values
```

```

self.anno_file_names = anno_df[ 'StudyInstanceUID' ].values
self.labels = anno_df[ 'label' ].values

def __len__(self):
    return len(self.anno_df)

def __getitem__(self, idx):
    file_name = self.anno_file_names[idx]
    image_path = f"/projects/sc73/ranzcr-clip-catheter-line-classification/train/{file_name}

    image = cv2.imread(image_path)
    image = annotate(image, self.anno_df["label"][idx], self.anno_df["data"][idx])
    image = cv2.resize(image, (360,360))
    image = np.asarray(image, dtype=np.float32) / 255.0

    labels = self.labels[idx]
    return image, labels

```

```

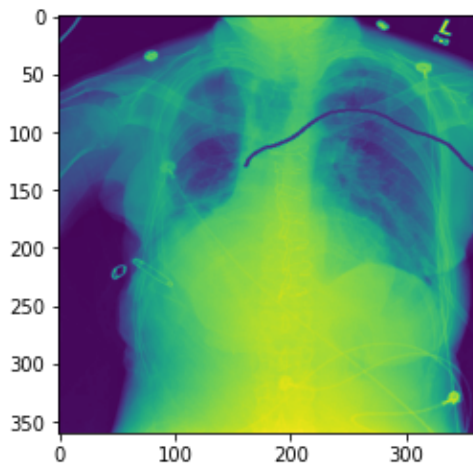
In [ ]: train_data = CustomDataset(anno_df)
gray_image = cv2.cvtColor(train_data[0][0], cv2.COLOR_BGR2GRAY)
plt.imshow(gray_image)

```

```

Out[ ]: <matplotlib.image.AxesImage at 0x7f885c9df0b8>

```



```

In [ ]: size = 360
train_imgs = np.empty((0,size, size))
train_labels = []

for i in range(len(train_data)):
    train_data = CustomDataset(anno_df)
    img, label = train_data[i]
    img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    img = np.expand_dims(img,(0))
    train_imgs = np.concatenate((train_imgs, img), axis = 0)
    train_labels.append(label)

    # plt.imshow(train_data[0], cmap=plt.cm.binary)

```

```

In [ ]: train_labels = np.array(train_labels)
train_imgs.shape

```

```

Out[ ]: (700, 360, 360)

```

```

In [ ]: from sklearn.model_selection import train_test_split

X = train_imgs
Y = train_labels

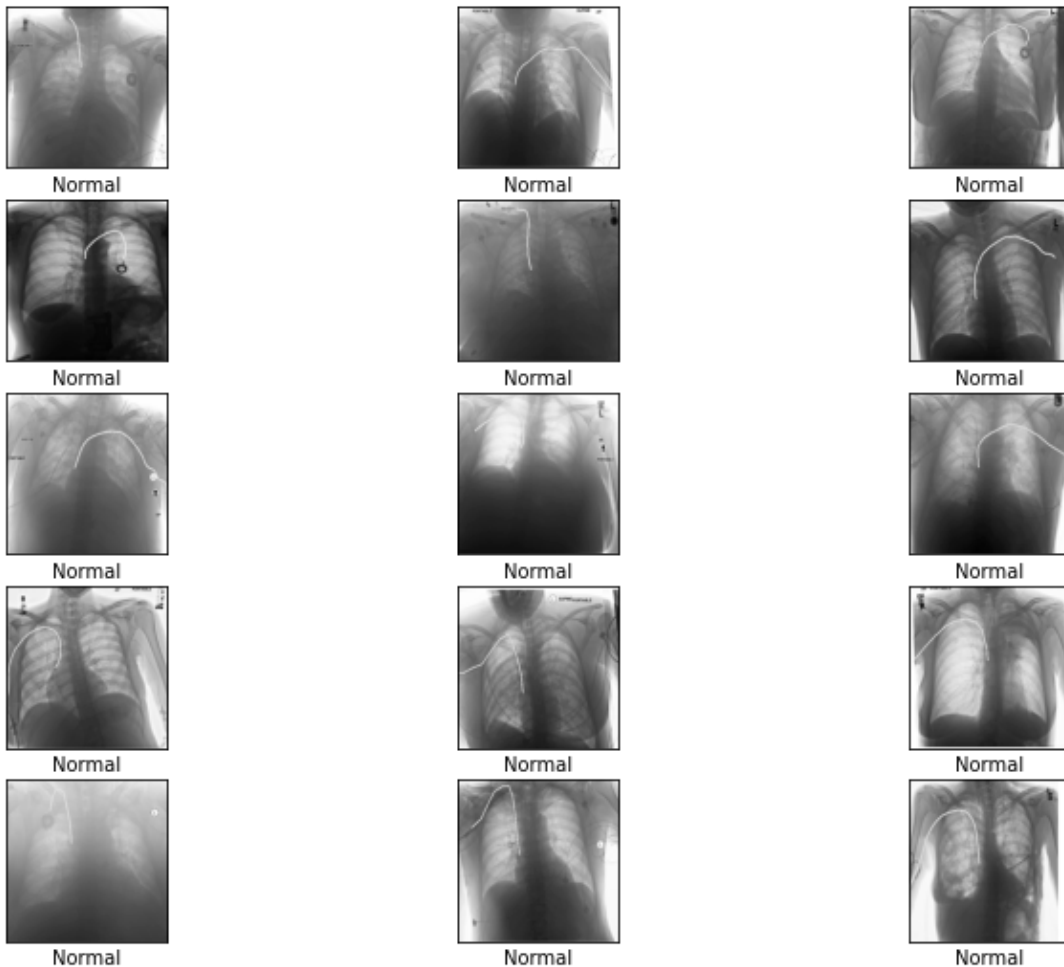
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

```

Making and training the model

```
In [ ]: def get_allignment(labels):  
        if labels == 1:  
            return "Normal"  
        else:  
            return "Abnormal"
```

```
In [ ]: #displaying 6 of the images in the train_imgs array  
plt.figure(figsize=(12,9))  
for i in range(15):  
    plt.subplot(5,3,i+1)  
    plt.xticks([])  
    plt.yticks([])  
    plt.grid(False)  
    plt.imshow(train_imgs[i+20], cmap=plt.cm.binary)  
    plt.xlabel(get_allignment(train_labels[i+20]))  
plt.show()
```



```
In [ ]: lr = 0.01  
epochs=70  
img_size = (size, size)  
model = tf.keras.Sequential([  
    tf.keras.layers.Flatten(input_shape=img_size),  
    tf.keras.layers.Dense(300, activation="relu"),  
    tf.keras.layers.Dense(100, activation="relu"),  
    tf.keras.layers.Dense(1),  
    tf.keras.layers.Activation('sigmoid')  
])
```

```
In [ ]: from tensorflow.keras.optimizers import SGD  
        from tensorflow.keras.callbacks import ReduceLROnPlateau  
  
opt = keras.optimizers.Adam(learning_rate=lr)  
# opt = SGD(learning_rate= lr)
```



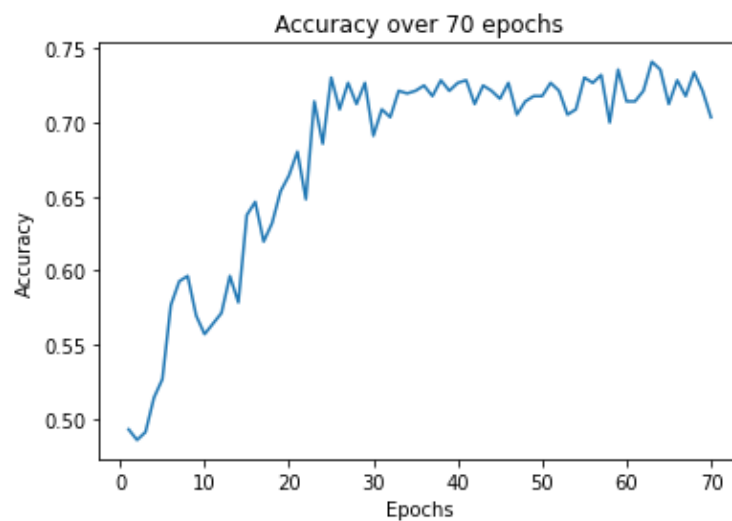
```
# opt = SGD(learning_rate=1e-4)
reducer_lr_cb = ReduceLROnPlateau(monitor='val_accuracy', factor=0.5, patience=4, min_lr = 0.0001)
model.compile(loss = "binary_crossentropy", optimizer = opt, metrics=['accuracy'])
```

```
In [ ]: history = model.fit(X_train, Y_train, epochs=epochs, validation_data = (X_test, Y_test), callba
```

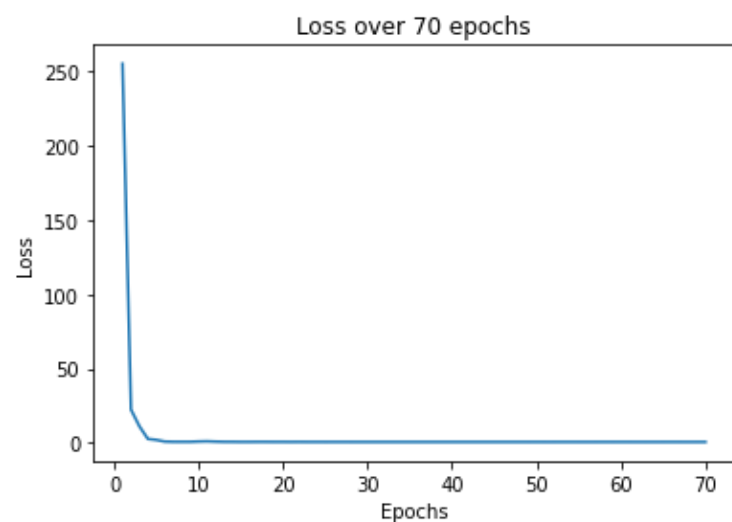
```
Epoch 1/70
18/18 [=====] - 2s 94ms/step - loss: 346.1730 - accuracy: 0.5010 - val_loss: 0.5010
Epoch 2/70
18/18 [=====] - 1s 65ms/step - loss: 25.3426 - accuracy: 0.4724 - val_loss: 0.4724
Epoch 3/70
18/18 [=====] - 1s 71ms/step - loss: 14.5813 - accuracy: 0.4815 - val_loss: 0.4815
Epoch 4/70
18/18 [=====] - 1s 68ms/step - loss: 2.8024 - accuracy: 0.5233 - val_loss: 0.5233
Epoch 5/70
18/18 [=====] - 1s 70ms/step - loss: 2.9123 - accuracy: 0.5126 - val_loss: 0.5126
Epoch 6/70
18/18 [=====] - 1s 66ms/step - loss: 0.8152 - accuracy: 0.5729 - val_loss: 0.5729
Epoch 7/70
18/18 [=====] - 1s 66ms/step - loss: 0.7059 - accuracy: 0.6035 - val_loss: 0.6035
Epoch 8/70
18/18 [=====] - 1s 70ms/step - loss: 0.6628 - accuracy: 0.5998 - val_loss: 0.5998
Epoch 9/70
18/18 [=====] - 1s 69ms/step - loss: 0.6751 - accuracy: 0.6015 - val_loss: 0.6015
Epoch 10/70
18/18 [=====] - 1s 67ms/step - loss: 0.8144 - accuracy: 0.5927 - val_loss: 0.5927
Epoch 11/70
18/18 [=====] - 1s 69ms/step - loss: 1.1473 - accuracy: 0.5460 - val_loss: 0.5460
Epoch 12/70
18/18 [=====] - 1s 69ms/step - loss: 0.8320 - accuracy: 0.5751 - val_loss: 0.5751
Epoch 13/70
18/18 [=====] - 1s 69ms/step - loss: 0.7039 - accuracy: 0.6056 - val_loss: 0.6056
Epoch 14/70
18/18 [=====] - 1s 68ms/step - loss: 0.6719 - accuracy: 0.5930 - val_loss: 0.5930
Epoch 15/70
18/18 [=====] - 1s 66ms/step - loss: 0.6167 - accuracy: 0.6342 - val_loss: 0.6342
Epoch 16/70
18/18 [=====] - 1s 66ms/step - loss: 0.6138 - accuracy: 0.6673 - val_loss: 0.6673
Epoch 17/70
18/18 [=====] - 1s 65ms/step - loss: 0.6051 - accuracy: 0.6491 - val_loss: 0.6491
Epoch 18/70
18/18 [=====] - 1s 61ms/step - loss: 0.6863 - accuracy: 0.6000 - val_loss: 0.6000
Epoch 19/70
18/18 [=====] - 1s 63ms/step - loss: 0.6253 - accuracy: 0.6289 - val_loss: 0.6289
Epoch 20/70
18/18 [=====] - 1s 64ms/step - loss: 0.6259 - accuracy: 0.6419 - val_loss: 0.6419
Epoch 21/70
18/18 [=====] - 1s 64ms/step - loss: 0.5843 - accuracy: 0.7118 - val_loss: 0.7118
Epoch 22/70
18/18 [=====] - 1s 62ms/step - loss: 0.6135 - accuracy: 0.6515 - val_loss: 0.6515
Epoch 23/70
18/18 [=====] - 1s 68ms/step - loss: 0.5894 - accuracy: 0.7164 - val_loss: 0.7164
Epoch 24/70
18/18 [=====] - 1s 69ms/step - loss: 0.5871 - accuracy: 0.7154 - val_loss: 0.7154
Epoch 25/70
18/18 [=====] - 1s 68ms/step - loss: 0.5826 - accuracy: 0.7346 - val_loss: 0.7346
Epoch 26/70
18/18 [=====] - 1s 71ms/step - loss: 0.5970 - accuracy: 0.7027 - val_loss: 0.7027
Epoch 27/70
18/18 [=====] - 1s 70ms/step - loss: 0.5871 - accuracy: 0.7202 - val_loss: 0.7202
Epoch 28/70
18/18 [=====] - 1s 66ms/step - loss: 0.6007 - accuracy: 0.6748 - val_loss: 0.6748
Epoch 29/70
18/18 [=====] - 1s 64ms/step - loss: 0.5890 - accuracy: 0.7145 - val_loss: 0.7145
Epoch 30/70
18/18 [=====] - 1s 64ms/step - loss: 0.5834 - accuracy: 0.7072 - val_loss: 0.7072
Epoch 31/70
18/18 [=====] - 1s 62ms/step - loss: 0.5831 - accuracy: 0.7159 - val_loss: 0.7159
Epoch 32/70
18/18 [=====] - 1s 62ms/step - loss: 0.5815 - accuracy: 0.7045 - val_loss: 0.7045
Epoch 33/70
18/18 [=====] - 1s 62ms/step - loss: 0.5818 - accuracy: 0.7107 - val_loss: 0.7107
```

```
18/18 [=====] - 1s 63ms/step - loss: 0.5818 - accuracy: 0.7407 - val_loss
Epoch 34/70
18/18 [=====] - 1s 65ms/step - loss: 0.5798 - accuracy: 0.7422 - val_loss
Epoch 35/70
18/18 [=====] - 1s 65ms/step - loss: 0.5719 - accuracy: 0.7280 - val_loss
Epoch 36/70
18/18 [=====] - 1s 65ms/step - loss: 0.5661 - accuracy: 0.7501 - val_loss
Epoch 37/70
18/18 [=====] - 1s 64ms/step - loss: 0.5910 - accuracy: 0.7021 - val_loss
Epoch 38/70
18/18 [=====] - 1s 63ms/step - loss: 0.5764 - accuracy: 0.7305 - val_loss
Epoch 39/70
18/18 [=====] - 1s 65ms/step - loss: 0.5753 - accuracy: 0.7463 - val_loss
Epoch 40/70
18/18 [=====] - 1s 78ms/step - loss: 0.5887 - accuracy: 0.7066 - val_loss
Epoch 41/70
18/18 [=====] - 1s 66ms/step - loss: 0.5802 - accuracy: 0.7325 - val_loss
Epoch 42/70
18/18 [=====] - 1s 67ms/step - loss: 0.5823 - accuracy: 0.7264 - val_loss
Epoch 43/70
18/18 [=====] - 1s 66ms/step - loss: 0.5817 - accuracy: 0.7301 - val_loss
Epoch 44/70
18/18 [=====] - 1s 69ms/step - loss: 0.5776 - accuracy: 0.7116 - val_loss
Epoch 45/70
18/18 [=====] - 1s 64ms/step - loss: 0.5788 - accuracy: 0.7125 - val_loss
Epoch 46/70
18/18 [=====] - 1s 70ms/step - loss: 0.5812 - accuracy: 0.7466 - val_loss
Epoch 47/70
18/18 [=====] - 1s 70ms/step - loss: 0.6019 - accuracy: 0.6795 - val_loss
Epoch 48/70
18/18 [=====] - 1s 74ms/step - loss: 0.5881 - accuracy: 0.6959 - val_loss
Epoch 49/70
18/18 [=====] - 1s 74ms/step - loss: 0.5612 - accuracy: 0.7392 - val_loss
Epoch 50/70
18/18 [=====] - 1s 74ms/step - loss: 0.5897 - accuracy: 0.7244 - val_loss
Epoch 51/70
18/18 [=====] - 1s 70ms/step - loss: 0.5743 - accuracy: 0.7316 - val_loss
Epoch 52/70
18/18 [=====] - 1s 70ms/step - loss: 0.5779 - accuracy: 0.7338 - val_loss
Epoch 53/70
18/18 [=====] - 1s 73ms/step - loss: 0.5890 - accuracy: 0.6887 - val_loss
Epoch 54/70
18/18 [=====] - 1s 66ms/step - loss: 0.5859 - accuracy: 0.6982 - val_loss
Epoch 55/70
18/18 [=====] - 1s 65ms/step - loss: 0.5653 - accuracy: 0.7575 - val_loss
Epoch 56/70
18/18 [=====] - 1s 71ms/step - loss: 0.5698 - accuracy: 0.7360 - val_loss
Epoch 57/70
18/18 [=====] - 1s 77ms/step - loss: 0.5654 - accuracy: 0.7346 - val_loss
Epoch 58/70
18/18 [=====] - 1s 72ms/step - loss: 0.5902 - accuracy: 0.7080 - val_loss
Epoch 59/70
18/18 [=====] - 1s 72ms/step - loss: 0.5756 - accuracy: 0.7434 - val_loss
Epoch 60/70
18/18 [=====] - 1s 68ms/step - loss: 0.5705 - accuracy: 0.7277 - val_loss
Epoch 61/70
18/18 [=====] - 1s 69ms/step - loss: 0.5855 - accuracy: 0.7084 - val_loss
Epoch 62/70
18/18 [=====] - 1s 68ms/step - loss: 0.5796 - accuracy: 0.7301 - val_loss
Epoch 63/70
18/18 [=====] - 1s 79ms/step - loss: 0.5634 - accuracy: 0.7568 - val_loss
Epoch 64/70
18/18 [=====] - 1s 73ms/step - loss: 0.5742 - accuracy: 0.7341 - val_loss
Epoch 65/70
18/18 [=====] - 1s 76ms/step - loss: 0.5885 - accuracy: 0.7059 - val_loss
Epoch 66/70
18/18 [=====] - 1s 72ms/step - loss: 0.5848 - accuracy: 0.7053 - val_loss
Epoch 67/70
18/18 [=====] - 1s 73ms/step - loss: 0.5773 - accuracy: 0.7176 - val_loss
Epoch 68/70
18/18 [=====] - 1s 70ms/step - loss: 0.5763 - accuracy: 0.7365 - val_loss
Epoch 69/70
18/18 [=====] - 1s 69ms/step - loss: 0.5867 - accuracy: 0.6958 - val_loss
```

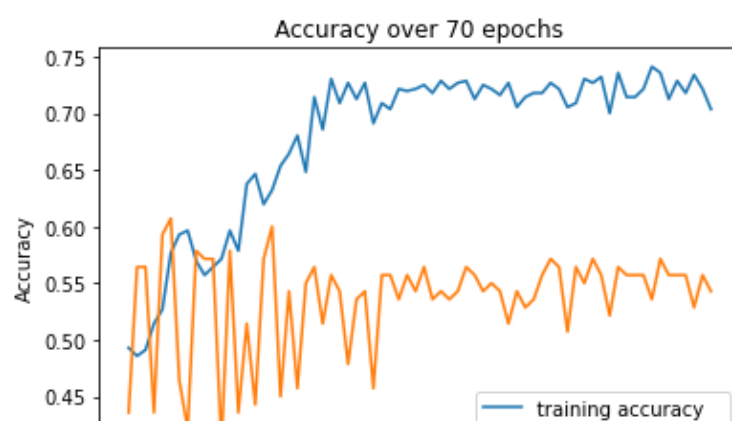
```
In [ ]: plt.plot(range(1,epochs+1), history.history["accuracy"])
plt.title(f"Accuracy over {epochs} epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy");
```

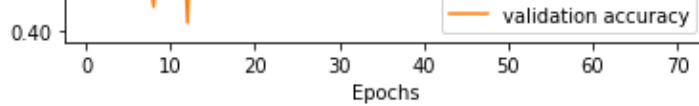


```
In [ ]: plt.plot(range(1,epochs+1), history.history["loss"])
plt.title(f"Loss over {epochs} epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss");
```



```
In [ ]: plt.plot(range(1,epochs+1), history.history["accuracy"])
plt.plot(range(1,epochs+1), history.history["val_accuracy"])
plt.legend(['training accuracy', 'validation accuracy'])
plt.title(f"Accuracy over {epochs} epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy");
```



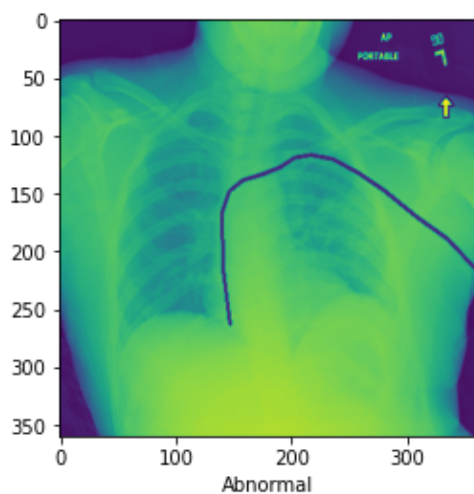


```
In [ ]: model.evaluate(X_test, Y_test)
```

```
In [ ]: plt.plot(range(1,epochs+1), history.history["loss"])
plt.plot(range(1,epochs+1), history.history["val_loss"])
plt.legend(['training loss', 'validation loss'])
plt.title(f"Loss over {epochs} epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss");
```

```
In [ ]: test_index = 9
plt.imshow(X_test[test_index])
plt.xlabel(get_alignment(Y_test[test_index]))
```

```
Out[ ]: Text(0.5, 0, 'Abnormal')
```



```
In [ ]: # probability_model = tf.keras.Sequential([model,
#                                              tf.keras.layers.Softmax()])
```

```
In [ ]: test = np.expand_dims(X_test[test_index],(0))
test.shape
```

```
Out[ ]: (1, 360, 360)
```

```
In [ ]: test_result = model.predict(test)
test_result[0][0]
```

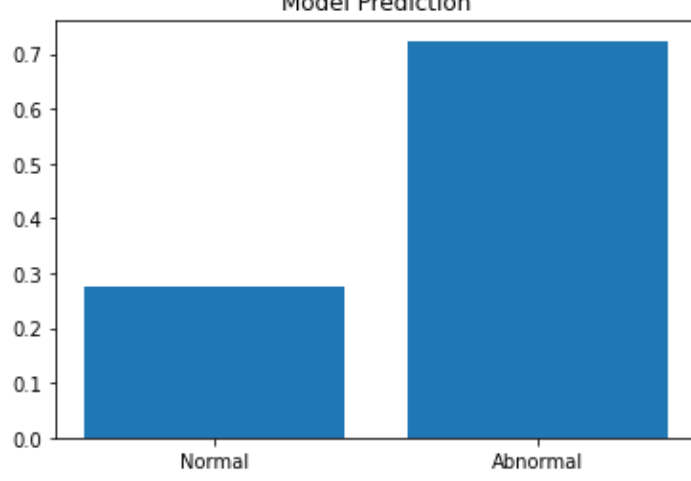
WARNING:tensorflow:5 out of the last 15 calls to <function Model.make_predict_function.<locals>.pr is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeat on objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (ument shapes that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/api_guides/python/tf/function for more details.

```
Out[ ]: 0.2762719
```

```
In [ ]: test_labels = ["Normal", "Abnormal"]
```

```
In [ ]: plt.bar(test_labels, [test_result[0][0], 1 -test_result[0][0]])
plt.title("Model Prediction")
```

```
Out[ ]: Text(0.5, 1.0, 'Model Prediction')
```



.

ResNet - CVC

```
In [ ]: import random
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
from PIL import Image
import ast
import os
import cv2
from tqdm import tqdm
```

/opt/conda/lib/python3.10/site-packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.3.5
warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")

```
In [ ]: train_data = pd.read_csv("/kaggle/input/ranzcr-clip-catheter-line-classification/train.csv")
# CVC_train = train_data[["StudyInstanceUID", "CVC - Abnormal", "CVC - Borderline", "CVC - Normal"]]
# CVC_train.set_index('StudyInstanceUID', inplace=True)
# CVC_train.head()

train_data[['CVC - Borderline', 'CVC - Normal', 'CVC - Abnormal']].value_counts()
train_data = train_data[train_data['CVC - Borderline'] != 1]
# train_data['CVC - Present'] = np.where((train_data['CVC - Abnormal'] == 0) & (train_data['CVC - Borderline'] == 1))

# train_data['CVC - Present'].value_counts()
train_data['CVC - Borderline'].value_counts()
# train_data
# train_data['CVC - Borderline'] = np.where((train_data['CVC - Normal'] == 1) | (train_data['CVC - Abnormal'] == 1))

train_data['CVC - Present'] = np.where((train_data['CVC - Abnormal'] == 0) & (train_data['CVC - Borderline'] == 1))

train_data['CVC - Present'].value_counts()
```

```
Out[ ]: CVC - Present
1      20873
0        750
Name: count, dtype: int64
```

```
In [ ]: sampled_data = pd.concat([train_data[train_data['CVC - Present'] == 1].sample(n=750), train_data[train_data['CVC - Present'] == 0].sample(n=750)])
sampled_data
```

```
Out[ ]:
```

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT - Abnormal	NGT - Borderline
28340	1.2.826.0.1.3680043.8.498.83320489135645999974...	0	0	0	0	0
4982	1.2.826.0.1.3680043.8.498.11682148749729194236...	0	0	1	0	0
11421	1.2.826.0.1.3680043.8.498.62983734602494481666...	0	0	0	0	0
22288	1.2.826.0.1.3680043.8.498.74056966962430552317...	0	0	1	0	0
15103	1.2.826.0.1.3680043.8.498.23791958383651330546...	0	0	0	0	0
...	...	...	...	...	...	...
25534	1.2.826.0.1.3680043.8.498.11255563749065952313...	0	1	0	0	0

15934	1.2.826.0.1.3680043.8.498.34645333555098820324...	0	0	0	1
6724	1.2.826.0.1.3680043.8.498.64781049488148307943...	0	0	1	0
9192	1.2.826.0.1.3680043.8.498.28261472510047998376...	0	0	0	0
17426	1.2.826.0.1.3680043.8.498.74456917827818957527...	0	0	1	0

1500 rows × 14 columns



```
In [ ]: def getFileNames(df):
        studyInstance = df['StudyInstanceUID'].tolist()
        studyInstance

        for i in range(len(studyInstance)):
            studyInstance[i] = f'{studyInstance[i]}.jpg'

        return studyInstance
```

```
In [ ]: studyInstance = getFileNames(sampled_data)
```

```
In [ ]: def processImages(filenamees,train_test):

        imgs = []
        FileIDs = []
        lst = []

        desired_channels = 3

        count = 0
        for file in tqdm(filenamees, position=0, leave=True):
            # Making sure the images are the same pixel size
            img = Image.open(f"/kaggle/input/ranzcr-clip-catheter-line-classification/{train_test}
            img = img.resize((300, 300))
            img = np.array(img)

            # Ensure that the image has 3 channels (RGB)
            if img.shape[-1] != 3:
                img = np.stack((img,) * 3, axis=-1) # Convert grayscale to RGB by duplicating ch

            imgs.append(img) # Append the 4D image to the list

            # Save filename for later
            FileIDs.append(file)

            count += 1

        # Convert the list of images to a numpy array
        imgs = np.array(imgs)
        return imgs, FileIDs
```

```
In [ ]: train_imgs, fileIDs = processImages(studyInstance, 'train')
```

100%|██████████| 1500/1500 [01:21<00:00, 18.44it/s]

```
In [ ]: #cleaning up the Labels
        UID = []
        for i in range(len(fileIDs)):
            UID.append(fileIDs[i].replace(".jpg",""))

        UID[0:10]
```

```
Out[ ]: ['1.2.826.0.1.3680043.8.498.83320489135645999974457109185017060587',
'1.2.826.0.1.3680043.8.498.11682148749729194236187083550083805585',
'1.2.826.0.1.3680043.8.498.62983734602494481666845016459657195529',
'1.2.826.0.1.3680043.8.498.74056966962430552317871031688278015762',
'1.2.826.0.1.3680043.8.498.23791958383651330546927001835650517198',
'1.2.826.0.1.3680043.8.498.89995812849337723285552454813516208797',
'1.2.826.0.1.3680043.8.498.65365629182698816190688886332537648480',
'1.2.826.0.1.3680043.8.498.29218976153043720655902319110213325079',
'1.2.826.0.1.3680043.8.498.18642853161045430356425011133359701508',
'1.2.826.0.1.3680043.8.498.97163600208343224793101223226948827431']
```

```
In [ ]: columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal', 'N
      "NGT - Incompletely Imaged", "Swan Ganz Catheter Present", "PatientID", "CV

CVCsampled_data = sampled_data.drop(columns=columns_to_drop, axis=1)

CVCsampled_data.set_index('StudyInstanceUID', inplace=True)
```

```
In [ ]: train_labels = np.empty((0,3))
data = []

for ID in UID:
    data.append(CVCsampled_data.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)
train_labels = train_labels_df['CVC - Present'].values
# train_labels = train_labels_df.values
train_labels
```

```
Out[ ]: array([1, 1, 1, ..., 0, 0, 0])
```

```
In [ ]: from sklearn.model_selection import train_test_split

# Split your data into training and temporary data (combined validation and test)
train_imgs, temp_imgs, train_labels, temp_labels = train_test_split(train_imgs, train_labels,

# Split the temporary data into validation and test sets
val_imgs, test_imgs, val_labels, test_labels = train_test_split(temp_imgs, temp_labels, test_
```

```
In [ ]: import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
```

NORMAL OR ABNORMAL

```
In [ ]: def buildTrainData(catheter, train_data, size):

    train_data = train_data[train_data[f'{catheter} - Present'] == 1]
    train_data = train_data[~((train_data[f'{catheter} - Normal'] == 1) & (train
    train_data[f'{catheter} - Normal/Abnormal'] = np.where(train_data[f'{catheter} - Normal']
    sampled_data = pd.concat([train_data[train_data[f'{catheter} - Normal'] == 1].sample(n=si

    return sampled_data
```

```
In [ ]: sample = buildTrainData('CVC', train_data, 750)
```


sample

Out[]:

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT - Abnormal	Bor
20366	1.2.826.0.1.3680043.8.498.12421859341733766169...	0	0	0	0	
18966	1.2.826.0.1.3680043.8.498.61954576091788767470...	0	0	0	0	
17661	1.2.826.0.1.3680043.8.498.12162154125048993968...	0	0	0	0	
21382	1.2.826.0.1.3680043.8.498.34100790242718134448...	0	0	0	0	
30001	1.2.826.0.1.3680043.8.498.39290688660746769862...	0	0	0	0	
...	...	...	...	...	...	
5834	1.2.826.0.1.3680043.8.498.24492665583305684486...	0	0	0	0	
27150	1.2.826.0.1.3680043.8.498.56720098445926282043...	0	0	0	0	
13887	1.2.826.0.1.3680043.8.498.97597003623982069445...	0	0	0	0	
15678	1.2.826.0.1.3680043.8.498.26758419850030547154...	0	0	0	0	
16568	1.2.826.0.1.3680043.8.498.10572345269368094686...	0	0	1	0	

1500 rows × 15 columns



In []:

```
nAb_data = getFileNames(sample)

cvc_imgs, fileIDs = processImages(nAb_data, 'train')
```

100%|██████████| 1500/1500 [01:21<00:00, 18.31it/s]

In []:

```
UID = []
for i in range(len(fileIDs)):
    UID.append(fileIDs[i].replace(".jpg",""))

columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal', 'N
                  "NGT - Incompletely Imaged", "Swan Ganz Catheter Present", "PatientID", "CV

CVCsample = sample.drop(columns=columns_to_drop, axis=1)

CVCsample.set_index('StudyInstanceUID', inplace=True)

CVCsample
```

Out[]:

	StudyInstanceUID	CVC - Normal/Abnormal
1.2.826.0.1.3680043.8.498.12421859341733766169493830337193607330		1
1.2.826.0.1.3680043.8.498.61954576091788767470760630851317720423		1
1.2.826.0.1.3680043.8.498.12162154125048993968767998953274132056		1
1.2.826.0.1.3680043.8.498.34100790242718134448072718905055902552		1
1.2.826.0.1.3680043.8.498.39290688660746769862459405228163320134		1
...		...
1.2.826.0.1.3680043.8.498.24492665583305684486011767114054826053		0

1.2.826.0.1.3680043.8.498.56720098445926282043775878721904198697	0
1.2.826.0.1.3680043.8.498.97597003623982069445062931389170388932	0
1.2.826.0.1.3680043.8.498.26758419850030547154723129883244762172	0
1.2.826.0.1.3680043.8.498.10572345269368094686053391847941465059	0

1500 rows × 1 columns

```
In [ ]: train_labels_df = np.empty((0,3))
data = []

for ID in UID:
    data.append(CVCsample.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)
cvc_labels = train_labels_df['CVC - Normal/Abnormal'].values
# train_labels = train_labels_df.values
cvc_labels
```

Out[]: array([1, 1, 1, ..., 0, 0, 0])

```
In [ ]: from sklearn.model_selection import train_test_split

# Split your data into training and temporary data (combined validation and test)
trainCVC_imgs, temp_imgs, trainCVC_labels, temp_labels = train_test_split(cvc_imgs, cvc_labels)

# Split the temporary data into validation and test sets
valCVC_imgs, testCVC_imgs, valCVC_labels, testCVC_labels = train_test_split(temp_imgs, temp_labels)
```

```
In [ ]: len(train_labels)
```

Out[]: 1200

```
In [ ]: from tensorflow.keras.layers import concatenate
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Model

from tensorflow.keras.layers import Dropout, BatchNormalization
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Load the ResNet50 model with pre-trained weights
base_model = ResNet50(weights='imagenet', include_top=False)

# Add dropout and batch normalization to custom fully connected layers
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.5)(x)

# First Output Branch: CVC Catheter Presence (Binary Classification)
output_CVCpresent = Dense(1, activation='sigmoid', name='output_CVCpresent')(x)

# Add a separate set of layers for the new task
y = base_model.output
y = GlobalAveragePooling2D()(y)
y = Dense(1024, activation='relu')(y)
```

```

y = Dense(1024, activation='relu')(y)
y = BatchNormalization()(y)
y = Dropout(0.5)(y)

# Second Output Branch: CVC Placement (Binary Classification)
output_cvc_placement = Dense(1, activation='sigmoid', name='output_CVCnormalAbnormal')(y)

# Combine the two output branches
combined_model = Model(inputs=base_model.input, outputs=[output_CVCpresent, output_cvc_placement])

# Compile the combined model with optimizer and loss functions for both tasks
combined_model.compile(optimizer='SGD',
                        loss={'output_CVCpresent': 'binary_crossentropy', 'output_CVCnormalAbnormal': 'binary_crossentropy'},
                        metrics={'output_CVCpresent': 'accuracy', 'output_CVCnormalAbnormal': 'accuracy'})

# Reduce learning rate when the validation loss plateaus
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=1e-6)

# Early stopping to prevent overfitting
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Save the best model during training
model_checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True)

history = combined_model.fit(
    {'input_1': train_imgs},
    {'output_CVCpresent': train_labels, 'output_CVCnormalAbnormal': trainCVC_labels},
    epochs=30,
    validation_data=(
        {'input_1': val_imgs},
        {'output_CVCpresent': val_labels, 'output_CVCnormalAbnormal': valCVC_labels}
    ),
    callbacks=[reduce_lr, early_stop, model_checkpoint]
)

# Evaluate the combined model
# test_loss, [test_accuracy_catheter, test_accuracy_cvc_placement] = combined_model.evaluate(
#     {'input_1': test_imgs},
#     {'output_catheter': test_labels, 'output_cvc_placement': testCVC_labels}
# )

# print(f'Test Accuracy (Catheter): {test_accuracy_catheter * 100:.2f}%')
# print(f'Test Accuracy (CVC Placement): {test_accuracy_cvc_placement * 100:.2f}%')

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet101_weights_tf_dim_ordering_tf_kernels_0.01_2017_10_25_tf.tar.gz 94765736/94765736 [=====] - 3s 0us/step

Epoch 1/30

38/38 [=====] - 42s 437ms/step - loss: 1.4872 - output_CVCpresent_loss: 0.6608 - output_CVCnormalAbnormal_accuracy: 0.6525 - val_loss: 1.4014 - val_output_CVCpresent_loss: 0.6133 - val_output_CVCnormalAbnormal_accuracy: 0.5667 - lr: 0.0100

Epoch 2/30

38/38 [=====] - 11s 298ms/step - loss: 0.9078 - output_CVCpresent_loss: 0.7958 - output_CVCnormalAbnormal_accuracy: 0.8050 - val_loss: 1.4315 - val_output_CVCpresent_loss: 0.6333 - val_output_CVCnormalAbnormal_accuracy: 0.5867 - lr: 0.0100

Epoch 3/30

38/38 [=====] - 12s 328ms/step - loss: 0.3334 - output_CVCpresent_loss: 0.9483 - output_CVCnormalAbnormal_accuracy: 0.9333 - val_loss: 1.3438 - val_output_CVCpresent_loss: 0.6400 - val_output_CVCnormalAbnormal_accuracy: 0.6600 - lr: 0.0100

Epoch 4/30

38/38 [=====] - 13s 334ms/step - loss: 0.1139 - output_CVCpresent_loss: 0.9792 - output_CVCnormalAbnormal_accuracy: 0.9850 - val_loss: 1.3135 - val_output_CVCpresent_loss: 0.7200 - val_output_CVCnormalAbnormal_accuracy: 0.6867 - lr: 0.0100

Epoch 5/30

38/38 [=====] - 11s 299ms/step - loss: 0.0538 - output_CVCpresent_loss: 0.9933 - output_CVCnormalAbnormal_accuracy: 0.9925 - val_loss: 1.8194 - val_output_CVCpresent_loss: 0.6933 - val_output_CVCnormalAbnormal_accuracy: 0.6933 - lr: 0.0100

Epoch 6/30

38/38 [=====] - 11s 299ms/step - loss: 0.0479 - output_CVCpresent_loss: 0.9942 - output_CVCnormalAbnormal_accuracy: 0.9925 - val_loss: 3.0691 - val_output_CVCpresent_loss: 0.6667 - val_output_CVCnormalAbnormal_accuracy: 0.6667 - lr: 0.0100

Epoch 7/30

```

38/38 [=====] - 11s 298ms/step - loss: 0.0247 - output_CVCpresent_loss: 0.9975 - output_CVCnormalAbnormal_accuracy: 0.9983 - val_loss: 1.9445 - val_output_CVCpresent_loss: 0.9983 - val_output_CVCnormalAbnormal_accuracy: 0.7667 - val_output_CVCpresent_loss: 0.9983 - lr: 0.0100
Epoch 8/30
38/38 [=====] - 11s 298ms/step - loss: 0.0244 - output_CVCpresent_loss: 0.9958 - output_CVCnormalAbnormal_accuracy: 0.9967 - val_loss: 2.2818 - val_output_CVCpresent_loss: 0.9967 - val_output_CVCnormalAbnormal_accuracy: 0.7600 - lr: 0.0100
Epoch 9/30
38/38 [=====] - 11s 298ms/step - loss: 0.0101 - output_CVCpresent_loss: 0.9992 - output_CVCnormalAbnormal_accuracy: 1.0000 - val_loss: 2.4004 - val_output_CVCpresent_loss: 0.9992 - val_output_CVCnormalAbnormal_accuracy: 0.7733 - lr: 0.0100
Epoch 10/30
38/38 [=====] - 11s 298ms/step - loss: 0.0130 - output_CVCpresent_loss: 1.0000 - output_CVCnormalAbnormal_accuracy: 0.9992 - val_loss: 2.3051 - val_output_CVCpresent_loss: 1.0000 - val_output_CVCnormalAbnormal_accuracy: 0.7733 - lr: 0.0020
Epoch 11/30
38/38 [=====] - 11s 298ms/step - loss: 0.0125 - output_CVCpresent_loss: 0.9983 - output_CVCnormalAbnormal_accuracy: 0.9992 - val_loss: 2.2831 - val_output_CVCpresent_loss: 0.9983 - val_output_CVCnormalAbnormal_accuracy: 0.7600 - lr: 0.0020
Epoch 12/30
38/38 [=====] - 11s 298ms/step - loss: 0.0035 - output_CVCpresent_loss: 1.0000 - output_CVCnormalAbnormal_accuracy: 1.0000 - val_loss: 2.3098 - val_output_CVCpresent_loss: 1.0000 - val_output_CVCnormalAbnormal_accuracy: 0.7667 - lr: 0.0020
Epoch 13/30
38/38 [=====] - 11s 298ms/step - loss: 0.0067 - output_CVCpresent_loss: 1.0000 - output_CVCnormalAbnormal_accuracy: 0.9992 - val_loss: 2.3227 - val_output_CVCpresent_loss: 1.0000 - val_output_CVCnormalAbnormal_accuracy: 0.7533 - lr: 0.0020
Epoch 14/30
38/38 [=====] - 11s 302ms/step - loss: 0.0075 - output_CVCpresent_loss: 1.0000 - output_CVCnormalAbnormal_accuracy: 1.0000 - val_loss: 2.3344 - val_output_CVCpresent_loss: 1.0000 - val_output_CVCnormalAbnormal_accuracy: 0.7667 - lr: 0.0020

```

```

In [ ]: # Evaluate accuracy for the 'output_catheter' branch
evaluation = combined_model.evaluate(
    {'input_1': test_imgs},
    {'output_CVCpresent': test_labels, 'output_CVCnormalAbnormal': testCVC_labels},
)

# Extract the accuracy values from the evaluation results
test_accuracy_catheter = evaluation[3] # Index 3 corresponds to 'output_catheter_accuracy'
test_accuracy_cvc_placement = evaluation[4] # Index 4 corresponds to 'output_cvc_placement_accuracy'

print(f'Test Accuracy (CVC Present): {test_accuracy_catheter * 100:.2f}%')
print(f'Test Accuracy (CVC Normal/Abnormal): {test_accuracy_cvc_placement * 100:.2f}%')

5/5 [=====] - 0s 86ms/step - loss: 1.0981 - output_CVCpresent_loss: 0.5467 - output_CVCnormalAbnormal_accuracy: 0.7200
Test Accuracy (CVC Present): 74.67%
Test Accuracy (CVC Normal/Abnormal): 72.00%

```

```

In [ ]: # Make predictions
predictions = combined_model.predict(test_imgs)

# Access predictions for the first task
cvc_present_predictions = predictions[0]
cvc_normal_abnormal_predictions = predictions[1]

# Create two separate DataFrames
df1 = pd.DataFrame(cvc_present_predictions, columns=["CVC - Present"])
df2 = pd.DataFrame(cvc_normal_abnormal_predictions, columns=["CVC - Normal/Abnormal"])

# Concatenate the DataFrames horizontally to create one with two columns
CVCpredictions = pd.concat([df1, df2], axis=1)

CVCpredictions['CVC - Present'] = np.where(CVCpredictions['CVC - Present'] > 0.5, 1, 0)

```

```

CVCpredictions['CVC - Normal/Abnormal'] = np.where(CVCpredictions['CVC - Normal/Abnormal'] >
CVCpredictions

from sklearn.metrics import confusion_matrix

# Create the confusion matrix
confusion_matrix_present = confusion_matrix(test_labels, CVCpredictions['CVC - Present'])

plt.gca().patch.set_facecolor('none')

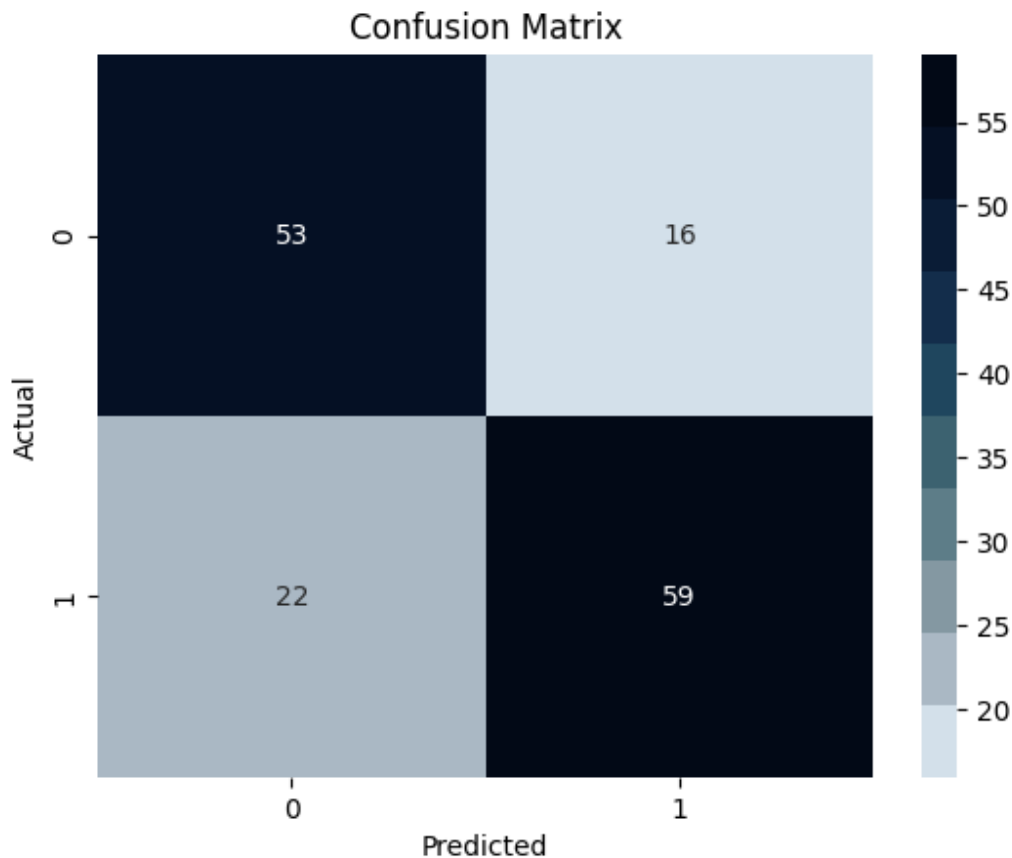
custom_palette = ['#D3E0EA', '#AAB8C4', '#8498A2', '#5D7D88', '#3C6270',
                  '#1F465E', '#132D4B', '#0A1C36', '#051024', '#020916']

# Create a heatmap plot of the confusion matrix
sns.heatmap(confusion_matrix_present, annot=True, cmap=custom_palette)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

plt.savefig('confusionmatrix_CVCexistence.png', bbox_inches='tight', transparent=True)

```

5/5 [=====] - 0s 89ms/step



<Figure size 640x480 with 0 Axes>

```

In [ ]: from sklearn.metrics import confusion_matrix
confusion_matrix_normal = confusion_matrix(testCVC_labels, CVCpredictions['CVC - Normal/Abnor

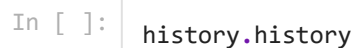
plt.gca().patch.set_facecolor('none')

custom_palette = ['#D3E0EA', '#AAB8C4', '#8498A2', '#5D7D88', '#3C6270',
                  '#1F465E', '#132D4B', '#0A1C36', '#051024', '#020916']

# Create a heatmap plot of the confusion matrix
sns.heatmap(confusion_matrix_normal, annot=True, cmap=custom_palette)
plt.xlabel('Predicted')
plt.ylabel('Actual')

```

```
# plt.savefig('confusionmatrix_CVCexistence.png', bbox_inches='tight', transparent=True)
```



CONCLUSIONS

```
0.0281941257417202,  
0.024158215150237083,  
0.010774710215628147,  
0.013118798844516277,  
0.0035529148299247026,  
0.006694619543850422,  
0.0052595664747059345,  
0.0015953542897477746,  
0.003741305321455002,  
0.003173540346324444],  
'output_CVCpresent_accuracy': [0.6608333587646484,  
0.7958333492279053,  
0.9483333230018616,  
0.9791666865348816,  
0.9933333396911621,  
0.9941666722297668,  
0.9975000023841858,  
0.9958333373069763,  
0.9991666674613953,  
1.0,  
0.9983333349227905,  
1.0,  
1.0,  
1.0],  
'output_CVCnormalAbnormal_accuracy': [0.6524999737739563,  
0.8050000071525574,  
0.9333333373069763,  
0.9850000143051147,  
0.9925000071525574,  
0.9925000071525574,  
0.9983333349227905,  
0.996666669845581,  
1.0,  
0.9991666674613953,  
0.9991666674613953,  
1.0,  
0.9991666674613953,  
1.0],  
'val_loss': [1.4014250040054321,  
1.4315134286880493,  
1.3437877893447876,  
1.3135241270065308,  
1.8194067478179932,  
3.069105863571167,  
1.9445444345474243,  
2.281766176223755,  
2.4004459381103516,  
2.3050899505615234,  
2.283097267150879,  
2.309751272201538,  
2.322709321975708,  
2.3343868255615234],  
'val_output_CVCpresent_loss': [0.6472787261009216,  
0.6646806597709656,  
0.6806280016899109,  
0.6570603251457214,  
0.8859343528747559,  
1.4903645515441895,  
0.9537824392318726,  
1.1207962036132812,  
1.183921217918396,  
1.1524124145507812,  
1.1485023498535156,  
1.164050579071045,  
1.1724931001663208,  
1.181626319885254],  
'val_output_CVCnormalAbnormal_loss': [0.7541463375091553,  
0.7668327689170837,  
0.6631596684455872,  
0.6564637422561646,  
0.9334725141525269,
```

```

1.5787413120269775,
0.9907619953155518,
1.1609700918197632,
1.2165248394012451,
1.1526775360107422,
1.1345950365066528,
1.1457009315490723,
1.1502161026000977,
1.1527605056762695],
'val_output_CVCpresent_accuracy': [0.6133333444595337,
0.6333333253860474,
0.6399999856948853,
0.7200000286102295,
0.6933333277702332,
0.6666666865348816,
0.7666666507720947,
0.7666666507720947,
0.7866666913032532,
0.7866666913032532,
0.753333330154419,
0.7599999904632568,
0.7599999904632568,
0.7599999904632568],
'val_output_CVCnormalAbnormal_accuracy': [0.5666666626930237,
0.5866666436195374,
0.6600000262260437,
0.6866666674613953,
0.6933333277702332,
0.6666666865348816,
0.7799999713897705,
0.7599999904632568,
0.7733333110809326,
0.7733333110809326,
0.7599999904632568,
0.7666666507720947,
0.753333330154419,
0.7666666507720947],
'lr': [0.01,
0.01,
0.01,
0.01,
0.01,
0.01,
0.01,
0.01,
0.01,
0.0019999999,
0.0019999999,
0.0019999999,
0.0019999999,
0.0019999999]}

```

```

In [ ]: import matplotlib.pyplot as plt

# Plot training & validation accuracy values
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['output_CVCpresent_accuracy'])
plt.plot(history.history['val_output_CVCpresent_accuracy'])
plt.title('CVC Present? - Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.figure(figsize=(12, 5))

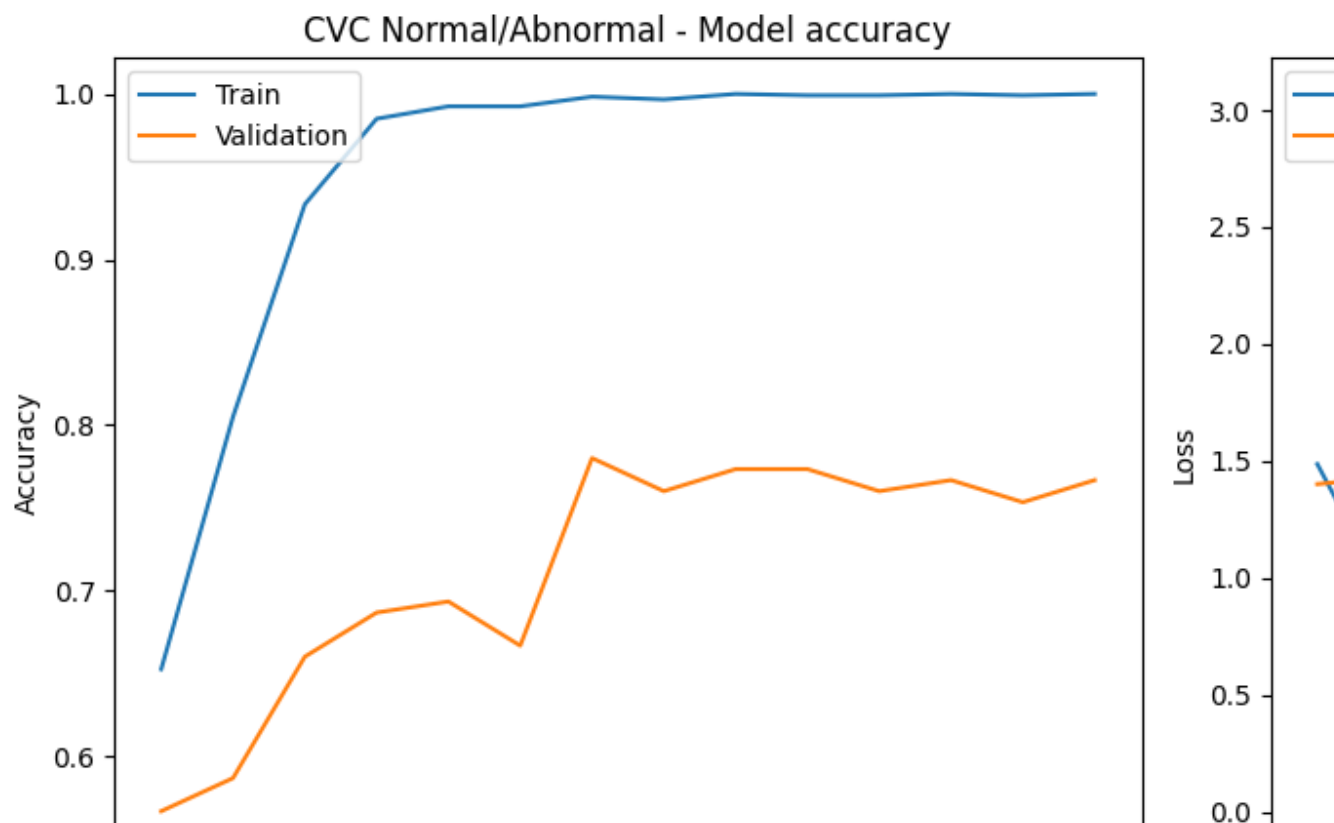
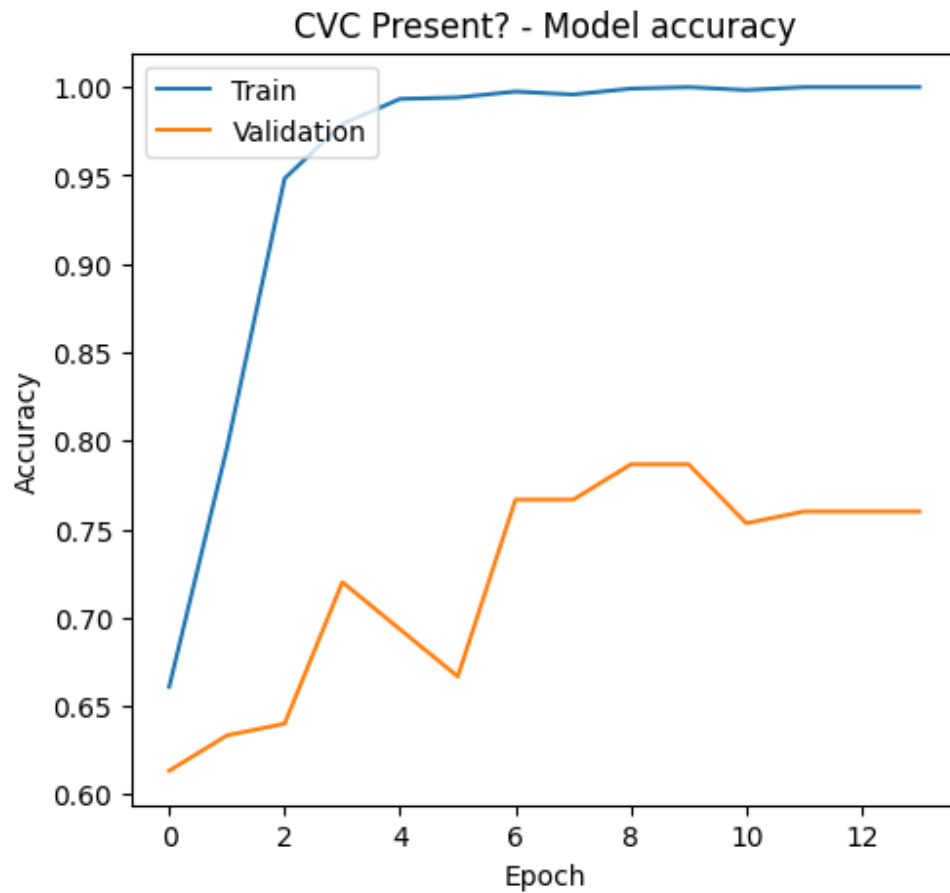
```

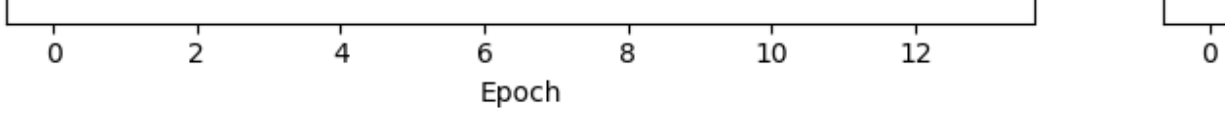


```
plt.legend(['Train', 'Validation'], loc='upper left')

# Plot training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.tight_layout()
plt.show()
```





In []:

ResNet - NGT

```
In [ ]: import random
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
from PIL import Image
import ast
import os
import cv2
from tqdm import tqdm
```

```
In [ ]: train_data = pd.read_csv("/kaggle/input/ranzcr-clip-catheter-line-classification/train.csv")
train_data
```

```
Out[ ]:
```

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT - Abnormal	NGT - Borderline	NGT - Normal
0	1.2.826.0.1.3680043.8.498.26697628953273228189...	0	0	0	0	0	0
1	1.2.826.0.1.3680043.8.498.46302891597398758759...	0	0	1	0	0	0
2	1.2.826.0.1.3680043.8.498.23819260719748494858...	0	0	0	0	0	0
3	1.2.826.0.1.3680043.8.498.68286643202323212801...	0	0	0	0	0	0
4	1.2.826.0.1.3680043.8.498.10050203009225938259...	0	0	0	0	0	0
...	...	...	...	...	...	...	...
30078	1.2.826.0.1.3680043.8.498.74257566841157531124...	0	0	1	0	0	0
30079	1.2.826.0.1.3680043.8.498.46510939987173529969...	0	0	0	0	0	0
30080	1.2.826.0.1.3680043.8.498.43173270582850645437...	0	0	1	0	0	0
30081	1.2.826.0.1.3680043.8.498.95092491950130838685...	0	0	0	0	0	0
30082	1.2.826.0.1.3680043.8.498.99518162226171269731...	0	0	1	0	0	0

30083 rows × 13 columns

```
In [ ]: # CVC_train = train_data[["StudyInstanceUID", "CVC - Abnormal", "CVC - Borderline", "CVC - Normal", "CVC - Present"]]
# CVC_train.set_index('StudyInstanceUID', inplace=True)
# CVC_train.head()

train_data = train_data[(train_data['NGT - Incompletely Imaged'] != 1) & (train_data['NGT - Borderline'] != 1)]

train_data[['NGT - Normal', 'NGT - Abnormal', 'NGT - Borderline', 'NGT - Incompletely Imaged']]
train_data['NGT - Present'] = np.where((train_data['NGT - Normal'] == 0) & (train_data['NGT - Abnormal'] == 0) & (train_data['NGT - Borderline'] == 0) & (train_data['NGT - Incompletely Imaged'] == 0), 1, 0)

train_data[['NGT - Normal', 'NGT - Abnormal']].value_counts()
# train_data['CVC - Borderline'].value_counts()
# # train_data
# # train_data['CVC - Borderline'] = np.where((train_data['CVC - Normal'] == 1) | (train_data['CVC - Abnormal'] == 1) | (train_data['CVC - Borderline'] == 1), 1, 0)
```

```
# train_data['CVC - Present'] = np.where((train_data['CVC - Abnormal'] == 0) & (train_data['CVC - Normal'] == 1), 1, 0)

# train_data['CVC - Present'].value_counts

# train_data = train_data[train_data[~((train_data[f'{catheter} - Normal'] == 1) & (train_data[f'{catheter} - Abnormal'] == 0))]]
# train_data[f'{catheter} - Normal/Abnormal'] = np.where(train_data[f'{catheter} - Normal'] == 1, 'Normal', 'Abnormal')
# sampled_data = pd.concat([train_data[train_data[f'{catheter} - Normal'] == 1].sample(n=size), train_data[train_data[f'{catheter} - Abnormal'] == 1].sample(n=size)])
```

/tmp/ipykernel_32/7930340.py:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html
train_data['NGT - Present'] = np.where((train_data['NGT - Normal'] == 0) & (train_data['NGT - Abnormal'] == 1), 1, 0)

```
Out[ ]: NGT - Normal  NGT - Abnormal
0              0          21775
1              0          4768
0              1           259
1              1            9
Name: count, dtype: int64
```

```
In [ ]: normalAb = pd.concat([train_data[train_data['NGT - Normal'] == 1].sample(n=125), train_data[train_data['NGT - Abnormal'] == 1].sample(n=125)], axis=1)
sampled_data = pd.concat([normalAb, train_data[train_data['NGT - Present'] == 0].sample(n=250), train_data[train_data['NGT - Present'] == 1].sample(n=250)])

# sampled_data = pd.concat([train_data[train_data['CVC - Present'] == 1].sample(n=750), train_data[train_data['CVC - Present'] == 0].sample(n=750)], axis=1)
sampled_data[['NGT - Abnormal', 'NGT - Normal']].value_counts()
```

```
Out[ ]: NGT - Abnormal  NGT - Normal
0              0          250
          1          125
1              0          122
          1            3
Name: count, dtype: int64
```

```
In [ ]: def getFileNames(df):
        studyInstance = df['StudyInstanceUID'].tolist()

        for i in range(len(studyInstance)):
            studyInstance[i] = f'{studyInstance[i]}.jpg'

        return studyInstance
```

```
In [ ]: studyInstance = getFileNames(sampled_data)
```

```
In [ ]: def processImages(filenamees,train_test):

        imgs = []
        FileIDs = []
        lst = []

        desired_channels = 3

        count = 0
        for file in tqdm(filenamees, position=0, leave=True):
            # Making sure the images are the same pixel size
            img = Image.open(f"/kaggle/input/ranzcr-clip-catheter-line-classification/{train_test}/{file}")
            img = img.resize((300, 300))
            img = np.array(img)

            # Ensure that the image has 3 channels (RGB)
            if img.shape[-1] != 3:
                img = np.stack((img,) * 3, axis=-1) # Convert grayscale to RGB by duplicating channels

            imgs.append(img) # Append the 4D image to the list
```

```
# Save filename for later
FileIDs.append(file)

count += 1

# Convert the list of images to a numpy array
imgs = np.array(imgs)
return imgs, FileIDs
```

```
train_imgs, fileIDs = processImages(studyInstance, 'train')
```

```
00%|██████████| 500/500 [00:28<00:00, 17.68it/s]
```

```
#cleaning up the labels
UID = []
for i in range(len(fileIDs)):
    UID.append(fileIDs[i].replace(".jpg", ""))

UID[0:10]
```

```
[ '1.2.826.0.1.3680043.8.498.65108787638800618504526868718911956744',  
  '1.2.826.0.1.3680043.8.498.10994531668075363818266044591102527717',  
  '1.2.826.0.1.3680043.8.498.49807016252879260088849140880053788065',  
  '1.2.826.0.1.3680043.8.498.75445875862496758679136471899539633375',  
  '1.2.826.0.1.3680043.8.498.89369371707034087254309411362762932453',  
  '1.2.826.0.1.3680043.8.498.44476689473309133836581960962296326162',  
  '1.2.826.0.1.3680043.8.498.11671797616816873403428410268840599989',  
  '1.2.826.0.1.3680043.8.498.80301292569484402631034426000610039631',  
  '1.2.826.0.1.3680043.8.498.10153667166658606707063887247369405550',  
  '1.2.826.0.1.3680043.8.498.10018307055250055778245370273733850307']
```

```
columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal', 'M  
"NGT - Incompletely Imaged", "Swan Ganz Catheter Present", "PatientID", "CV  
NGTsampld_data = sampled_data.drop(columns=columns_to_drop, axis=1)  
NGTsampld_data.set_index('StudyInstanceUID', inplace=True)
```

```
train_labels_df = np.empty((0,3))
data = []

for ID in UID:
    data.append(NGTsampled_data.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)
train_labels = train_labels_df['NGT - Present'].values
# train_labels = train_labels_df.values
train_labels
```

[illegible]

```
In [ ]:
UID = []
for i in range(len(fileIDs)):
    UID.append(fileIDs[i].replace(".jpg", ""))

columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal', 'N
                  "NGT - Incompletely Imaged", "Swan Ganz Catheter Present", "PatientID", "CV

NGTsample = sample.drop(columns=columns_to_drop, axis=1)

NGTsample.set_index('StudyInstanceUID', inplace=True)
```

Out[]:

NGT - Normal/Abnormal

StudyInstanceUID

1.2.826.0.1.3680043.8.498.10361726213889360297221617147966588121	1
1.2.826.0.1.3680043.8.498.31878102211936522453660823173608769080	1
1.2.826.0.1.3680043.8.498.45827680246405599842114977421380316000	1
1.2.826.0.1.3680043.8.498.12695704282968142311826592823035716895	1
1.2.826.0.1.3680043.8.498.90515750401474061451165676214491278451	1
...	...
1.2.826.0.1.3680043.8.498.67118987555969342287786734708355895624	0
1.2.826.0.1.3680043.8.498.10006237848798781176388878065969229683	0
1.2.826.0.1.3680043.8.498.53464106532734321247813859991912797211	0
1.2.826.0.1.3680043.8.498.91995629898587201008665176521265547339	0
1.2.826.0.1.3680043.8.498.26788309165352161244571600104054436515	0

500 rows $\times$ 1 columns

In []:

```
train_labels_df = np.empty((0,3))
data = []

for ID in UID:
    data.append(NGTsample.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)
ngt_labels = train_labels_df['NGT - Normal/Abnormal'].values
# train_labels = train_labels_df.values
ngt_labels
```

Out[]:

[illegible]

In []:

```

from sklearn.model_selection import train_test_split

# Split your data into training and temporary data (combined validation and test)
trainNGT_imgs, temp_imgs, trainNGT_labels, temp_labels = train_test_split(ngt_imgs, ngt_labels,
                                  test_size=0.2, random_state=42)

# Split the temporary data into validation and test sets
valNGT_imgs, testNGT_imgs, valNGT_labels, testNGT_labels = train_test_split(temp_imgs, temp_labels,
                                  test_size=0.5, random_state=42)

```

```
In [ ]: len(train_labels)
```

```
Out[ ]: 400
```

```

In [ ]: from tensorflow.keras.layers import concatenate
        from tensorflow.keras.preprocessing.image import ImageDataGenerator
        from tensorflow.keras.models import Model

        from tensorflow.keras.layers import Dropout, BatchNormalization
        from tensorflow.keras.callbacks import ReduceLROnPlateau
        from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

        # Load the ResNet50 model with pre-trained weights
        base_model = ResNet50(weights='imagenet', include_top=False)

        # Add dropout and batch normalization to custom fully connected layers
        x = base_model.output
        x = GlobalAveragePooling2D()(x)
        x = Dense(1024, activation='relu')(x)
        x = BatchNormalization()(x)
        x = Dropout(0.5)(x)

        # First Output Branch: CVC Catheter Presence (Binary Classification)
        output_CVCpresent = Dense(1, activation='sigmoid', name='output_NGTpresent')(x)

        # Add a separate set of layers for the new task
        y = base_model.output
        y = GlobalAveragePooling2D()(y)
        y = Dense(1024, activation='relu')(y)
        y = BatchNormalization()(y)
        y = Dropout(0.5)(y)

        # Second Output Branch: CVC Placement (Binary Classification)
        output_cvc_placement = Dense(1, activation='sigmoid', name='output_NGTnormalAbnormal')(y)

        # Combine the two output branches
        combined_model = Model(inputs=base_model.input, outputs=[output_CVCpresent, output_cvc_placement])

        # Compile the combined model with optimizer and loss functions for both tasks
        combined_model.compile(optimizer='SGD',
                               loss={'output_NGTpresent': 'binary_crossentropy', 'output_NGTnormalAbnormal': 'binary_crossentropy'},
                               metrics={'output_NGTpresent': 'accuracy', 'output_NGTnormalAbnormal': 'accuracy'})

        # Reduce learning rate when the validation loss plateaus
        reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=1e-6)

        # Early stopping to prevent overfitting
        early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

        # Save the best model during training
        model_checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True)

        history = combined_model.fit(
            {'input_1': train_imgs},
            {'output_NGTpresent': train_labels, 'output_NGTnormalAbnormal': trainNGT_labels},
            epochs=30,
            validation_data=(
                {'input_1': val_imgs},
                {'output_NGTpresent': val_labels, 'output_NGTnormalAbnormal': valNGT_labels}
            )
        )

```



```

),
callbacks=[reduce_lr, early_stop, model_checkpoint]
)

# # Evaluate the combined model
# test_loss, [test_accuracy_catheter, test_accuracy_cvc_placement] = combined_model.evaluate(
#     {'input_1': test_imgs},
#     {'output_catheter': test_labels, 'output_cvc_placement': testCVC_labels}
# )

# print(f'Test Accuracy (Catheter): {test_accuracy_catheter * 100:.2f}%')
# print(f'Test Accuracy (CVC Placement): {test_accuracy_cvc_placement * 100:.2f}%')

```

Downloading data from <https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet94765736/94765736> [=====] - 1s 0us/step

Epoch 1/30

13/13 [=====] - 37s 769ms/step - loss: 1.5461 - output_NGTpresent_loss: 0.6650 - output_NGTnormalAbnormal_accuracy: 0.6425 - val_loss: 1.4715 - val_output_NGTpresent_loss: 0.6650 - val_output_NGTnormalAbnormal_accuracy: 0.5000 - lr: 0.0100

Epoch 2/30

13/13 [=====] - 5s 393ms/step - loss: 0.8097 - output_NGTpresent_loss: 0.8275 - output_NGTnormalAbnormal_accuracy: 0.8225 - val_loss: 1.3660 - val_output_NGTpresent_loss: 0.8275 - val_output_NGTnormalAbnormal_accuracy: 0.6000 - lr: 0.0100

Epoch 3/30

13/13 [=====] - 5s 406ms/step - loss: 0.3624 - output_NGTpresent_loss: 0.9150 - output_NGTnormalAbnormal_accuracy: 0.9275 - val_loss: 1.2387 - val_output_NGTpresent_loss: 0.9150 - val_output_NGTnormalAbnormal_accuracy: 0.7000 - lr: 0.0100

Epoch 4/30

13/13 [=====] - 5s 392ms/step - loss: 0.1259 - output_NGTpresent_loss: 0.9850 - output_NGTnormalAbnormal_accuracy: 0.9800 - val_loss: 1.1865 - val_output_NGTpresent_loss: 0.9850 - val_output_NGTnormalAbnormal_accuracy: 0.6200 - lr: 0.0100

Epoch 5/30

13/13 [=====] - 5s 392ms/step - loss: 0.0520 - output_NGTpresent_loss: 0.9925 - output_NGTnormalAbnormal_accuracy: 0.9950 - val_loss: 1.0720 - val_output_NGTpresent_loss: 0.9925 - val_output_NGTnormalAbnormal_accuracy: 0.7200 - lr: 0.0100

Epoch 6/30

13/13 [=====] - 5s 396ms/step - loss: 0.0616 - output_NGTpresent_loss: 0.9950 - output_NGTnormalAbnormal_accuracy: 0.9925 - val_loss: 1.0078 - val_output_NGTpresent_loss: 0.9950 - val_output_NGTnormalAbnormal_accuracy: 0.7600 - lr: 0.0100

Epoch 7/30

13/13 [=====] - 5s 395ms/step - loss: 0.0192 - output_NGTpresent_loss: 1.0000 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.9449 - val_output_NGTpresent_loss: 1.0000 - val_output_NGTnormalAbnormal_accuracy: 0.7800 - lr: 0.0100

Epoch 8/30

13/13 [=====] - 5s 396ms/step - loss: 0.0179 - output_NGTpresent_loss: 1.0000 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8751 - val_output_NGTpresent_loss: 1.0000 - val_output_NGTnormalAbnormal_accuracy: 0.8400 - lr: 0.0100

Epoch 9/30

13/13 [=====] - 5s 394ms/step - loss: 0.0242 - output_NGTpresent_loss: 1.0000 - output_NGTnormalAbnormal_accuracy: 0.9950 - val_loss: 0.8477 - val_output_NGTpresent_loss: 1.0000 - val_output_NGTnormalAbnormal_accuracy: 0.8000 - lr: 0.0100

Epoch 10/30

13/13 [=====] - 5s 392ms/step - loss: 0.0098 - output_NGTpresent_loss: 1.0000 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8374 - val_output_NGTpresent_loss: 1.0000 - val_output_NGTnormalAbnormal_accuracy: 0.8000 - lr: 0.0100

Epoch 11/30

13/13 [=====] - 5s 390ms/step - loss: 0.0074 - output_NGTpresent_loss: 1.0000 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8135 - val_output_NGTpresent_loss: 1.0000 - val_output_NGTnormalAbnormal_accuracy: 0.8400 - lr: 0.0100

Epoch 12/30

13/13 [=====] - 4s 296ms/step - loss: 0.0077 - output_NGTpresent_loss: 1.0000 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8318 - val_output_NGTpresent_loss: 1.0000 - val_output_NGTnormalAbnormal_accuracy: 0.8200 - lr: 0.0100

Epoch 13/30

13/13 [=====] - 4s 296ms/step - loss: 0.0050 - output_NGTpresent_loss: 1.0000 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8303 - val_output_NGTpresent_loss: 1.0000 - val_output_NGTnormalAbnormal_accuracy: 0.8200 - lr: 0.0100

Epoch 14/30

13/13 [=====] - 4s 295ms/step - loss: 0.0067 - output_NGTpresent_loss: 1.0000 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8489 - val_output_NGTpresent_loss: 1.0000 - val_output_NGTnormalAbnormal_accuracy: 0.8200 - lr: 0.0100

```

Epoch 15/30
13/13 [=====] - 4s 295ms/step - loss: 0.0149 - output_NGTpresent_loss:
0.9975 - output_NGTnormalAbnormal_accuracy: 0.9975 - val_loss: 1.6993 - val_output_NGTpresent_lo
t_accuracy: 0.5400 - val_output_NGTnormalAbnormal_accuracy: 0.6800 - lr: 0.0100
Epoch 16/30
13/13 [=====] - 4s 297ms/step - loss: 0.0212 - output_NGTpresent_loss:
0.9925 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.8095 - val_output_NGTpresent_lo
t_accuracy: 0.5600 - val_output_NGTnormalAbnormal_accuracy: 0.7200 - lr: 0.0100
Epoch 17/30
13/13 [=====] - 4s 296ms/step - loss: 0.0040 - output_NGTpresent_loss:
1.0000 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.7148 - val_output_NGTpresent_lo
t_accuracy: 0.6000 - val_output_NGTnormalAbnormal_accuracy: 0.7000 - lr: 0.0020
Epoch 18/30
13/13 [=====] - 4s 296ms/step - loss: 0.0074 - output_NGTpresent_loss:
1.0000 - output_NGTnormalAbnormal_accuracy: 0.9975 - val_loss: 1.5938 - val_output_NGTpresent_lo
t_accuracy: 0.6400 - val_output_NGTnormalAbnormal_accuracy: 0.6800 - lr: 0.0020
Epoch 19/30
13/13 [=====] - 4s 296ms/step - loss: 0.0038 - output_NGTpresent_loss:
1.0000 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.4643 - val_output_NGTpresent_lo
t_accuracy: 0.6400 - val_output_NGTnormalAbnormal_accuracy: 0.6600 - lr: 0.0020
Epoch 20/30
13/13 [=====] - 4s 297ms/step - loss: 0.0079 - output_NGTpresent_loss:
1.0000 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.3442 - val_output_NGTpresent_lo
t_accuracy: 0.7000 - val_output_NGTnormalAbnormal_accuracy: 0.7200 - lr: 0.0020
Epoch 21/30
13/13 [=====] - 4s 311ms/step - loss: 0.0049 - output_NGTpresent_loss:
1.0000 - output_NGTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.2776 - val_output_NGTpresent_lo
t_accuracy: 0.7200 - val_output_NGTnormalAbnormal_accuracy: 0.7600 - lr: 0.0020

```

```

In [ ]: # Evaluate accuracy for the 'output_catheter' branch
evaluation = combined_model.evaluate(
    {'input_1': test_imgs},
    {'output_NGTpresent': test_labels, 'output_NGTnormalAbnormal': testNGT_labels},
)

# Extract the accuracy values from the evaluation results
test_accuracy_catheter = evaluation[3] # Index 3 corresponds to 'output_catheter_accuracy'
test_accuracy_cvc_placement = evaluation[4] # Index 4 corresponds to 'output_cvc_placement_a

print(f'Test Accuracy (NGT Present): {test_accuracy_catheter * 100:.2f}%')
print(f'Test Accuracy (NGT Normal/Abnormal): {test_accuracy_cvc_placement * 100:.2f}%')

2/2 [=====] - 0s 65ms/step - loss: 1.2740 - output_NGTpresent_loss: 0.6
600 - output_NGTnormalAbnormal_accuracy: 0.8400
Test Accuracy (NGT Present): 76.00%
Test Accuracy (NGT Normal/Abnormal): 84.00%

```

```

In [ ]: # Make predictions
predictions = combined_model.predict(test_imgs)
predictions

# Access predictions for the first task
ngt_present_predictions = predictions[0]
ngt_present_predictions
# Access predictions for the second task
ngt_normal_abnormal_predictions = predictions[1]

ngt_present_predictions
# Create two separate DataFrames
df1 = pd.DataFrame(ngt_present_predictions, columns=["NGT - Present"])
df2 = pd.DataFrame(ngt_normal_abnormal_predictions, columns=["NGT - Normal/Abnormal"])

# # Concatenate the DataFrames horizontally to create one with two columns
NGTpredictions = pd.concat([df1, df2], axis=1)
NGTpredictions

NGTpredictions['NGT - Present'] = np.where(NGTpredictions['NGT - Present'] > 0.5, 1, 0)
NGTpredictions['NGT - Normal/Abnormal'] = np.where(NGTpredictions['NGT - Normal/Abnormal'] >
NGTpredictions

```

```

from sklearn.metrics import confusion_matrix

# Create the confusion matrix
confusion_matrix_present = confusion_matrix(test_labels, NGTpredictions['NGT - Present'])

plt.gca().patch.set_facecolor('none')

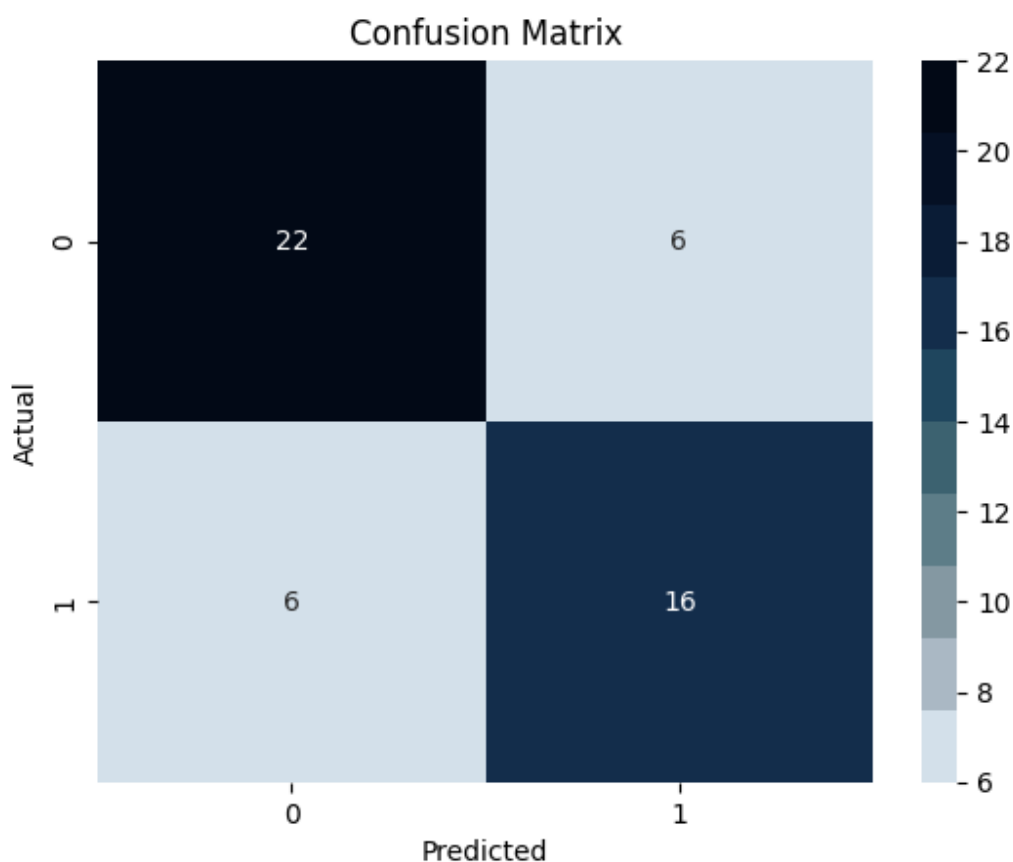
custom_palette = ['#D3E0EA', '#AAB8C4', '#8498A2', '#5D7D88', '#3C6270',
                  '#1F465E', '#132D4B', '#0A1C36', '#051024', '#020916']

# Create a heatmap plot of the confusion matrix
sns.heatmap(confusion_matrix_present, annot=True, cmap=custom_palette)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

plt.savefig('confusionmatrix_NGTexistence.png', bbox_inches='tight', transparent=True)

```

2/2 [=====] - 1s 85ms/step



<Figure size 640x480 with 0 Axes>

```

In [ ]: from sklearn.metrics import confusion_matrix
confusion_matrix_normal = confusion_matrix(testNGT_labels, NGTpredictions['NGT - Normal/Abnor

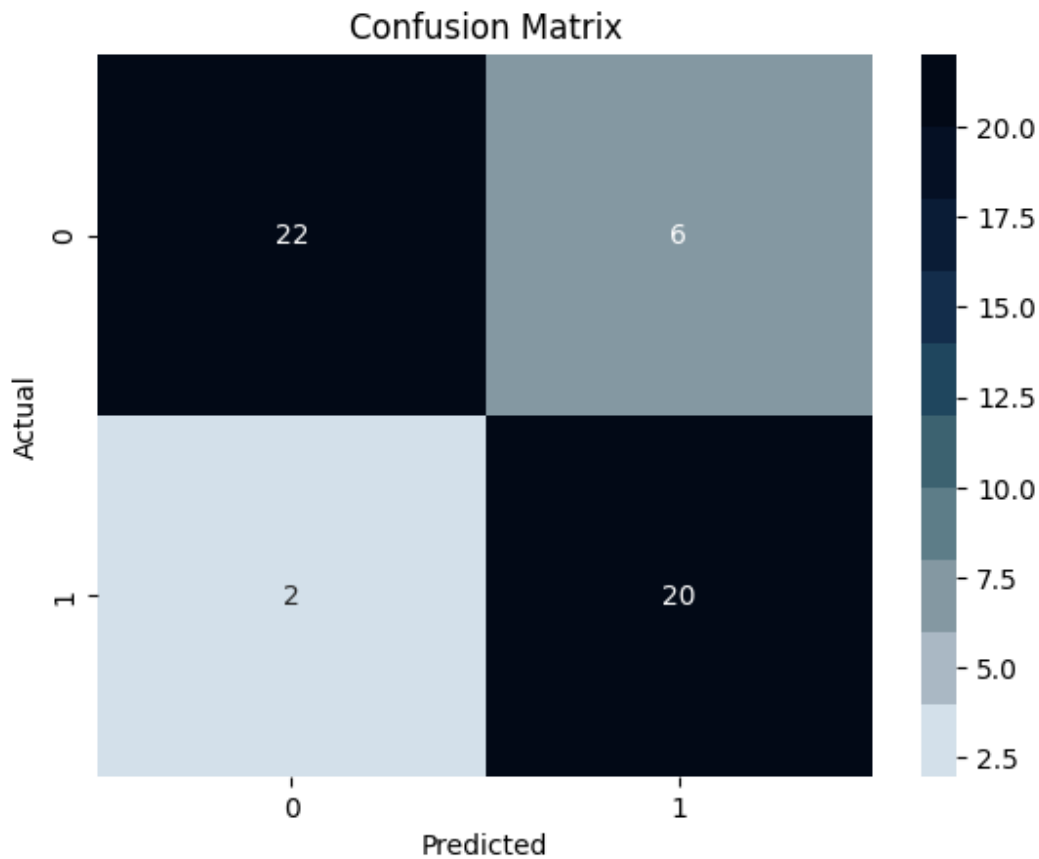
plt.gca().patch.set_facecolor('none')

custom_palette = ['#D3E0EA', '#AAB8C4', '#8498A2', '#5D7D88', '#3C6270',
                  '#1F465E', '#132D4B', '#0A1C36', '#051024', '#020916']

# Create a heatmap plot of the confusion matrix
sns.heatmap(confusion_matrix_normal, annot=True, cmap=custom_palette)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

```

```
# plt.savefig('confusionmatrix_CVCexistence.png', bbox_inches='tight', transparent=True)
```



```
In [ ]: history.history
```

```
Out[ ]: {'loss': [1.5461363792419434,  
0.8096561431884766,  
0.3624252378940582,  
0.12588241696357727,  
0.05198943614959717,  
0.06162864714860916,  
0.019237948581576347,  
0.017928985878825188,  
0.02415994368493557,  
0.009764919057488441,  
0.007427262607961893,  
0.007716720923781395,  
0.0050194538198411465,  
0.006688305176794529,  
0.014903539791703224,  
0.021240904927253723,  
0.0040471721440553665,  
0.007422173395752907,  
0.003778771497309208,  
0.007875504903495312,  
0.004931443836539984],  
'output_NGTpresent_loss': [0.7812063097953796,  
0.40417519211769104,  
0.19146518409252167,  
0.05916266515851021,  
0.028535082936286926,  
0.03126232326030731,  
0.009800111874938011,  
0.008465532213449478,  
0.009975463151931763,  
0.005508806090801954,  
0.0036987445782870054,  
0.004519784357398748,  
0.0022200404200702906,  
0.003765641711652279,  
0.007505080730875226,
```

```
0.007595089729875326,  
0.013587277382612228,  
0.0014799943892285228,  
0.0034188288263976574,  
0.0023902880493551493,  
0.003336479654535651,  
0.002694761147722602],  
'output_NGTnormalAbnormal_loss': [0.7649299502372742,  
0.4054809808731079,  
0.1709599494934082,  
0.06671973317861557,  
0.023454349488019943,  
0.0303663183003664,  
0.009437836706638336,  
0.009463454596698284,  
0.014184479601681232,  
0.0042561125010252,  
0.0037285182625055313,  
0.0031969365663826466,  
0.0027994131669402122,  
0.00292266346514225,  
0.007308450527489185,  
0.007653627544641495,  
0.0025671774055808783,  
0.004003344569355249,  
0.001388483215123415,  
0.004539025016129017,  
0.002236683154478669],  
'output_NGTpresent_accuracy': [0.6650000214576721,  
0.8274999856948853,  
0.9150000214576721,  
0.9850000143051147,  
0.9925000071525574,  
0.9950000047683716,  
1.0,  
1.0,  
1.0,  
1.0,  
1.0,  
1.0,  
1.0,  
1.0,  
0.9975000023841858,  
0.9925000071525574,  
1.0,  
1.0,  
1.0,  
1.0,  
1.0],  
'output_NGTnormalAbnormal_accuracy': [0.6424999833106995,  
0.8224999904632568,  
0.9275000095367432,  
0.9800000190734863,  
0.9950000047683716,  
0.9925000071525574,  
1.0,  
1.0,  
0.9950000047683716,  
1.0,  
1.0,  
1.0,  
1.0,  
1.0,  
0.9975000023841858,  
1.0,  
1.0,  
0.9975000023841858,  
1.0,  
1.0,  
1.0],  
'val_loss': [1.4714758396148682,
```

1.3660330772399902,
1.2387200593948364,
1.1864720582962036,
1.0719622373580933,
1.007774829864502,
0.9448912143707275,
0.8750890493392944,
0.847679615020752,
0.8374039530754089,
0.8134861588478088,
0.8318486213684082,
0.8302871584892273,
0.8489276766777039,
1.6992698907852173,
1.8095061779022217,
1.7148480415344238,
1.5937925577163696,
1.4643398523330688,
1.3442195653915405,
1.2776433229446411],
'val_output_NGTpresent_loss': [0.7254487872123718,
0.6729957461357117,
0.6296237707138062,
0.5807766914367676,
0.537304699420929,
0.49708831310272217,
0.47406136989593506,
0.4486723244190216,
0.4147377908229828,
0.41546034812927246,
0.40087997913360596,
0.41584154963493347,
0.40809136629104614,
0.40820378065109253,
1.1377323865890503,
1.1762562990188599,
1.0702706575393677,
0.9503426551818848,
0.8329941034317017,
0.7397612929344177,
0.6948841214179993],
'val_output_NGTnormalAbnormal_loss': [0.7460271716117859,
0.6930372714996338,
0.609096348285675,
0.6056952476501465,
0.5346575379371643,
0.5106863975524902,
0.4708298444747925,
0.42641666531562805,
0.432941734790802,
0.4219435751438141,
0.4126061201095581,
0.41600701212882996,
0.42219582200050354,
0.4407239556312561,
0.5615374445915222,
0.6332499980926514,
0.6445773243904114,
0.6434499621391296,
0.6313459277153015,
0.604458212852478,
0.5827592611312866],
'val_output_NGTpresent_accuracy': [0.47999998927116394,
0.6200000047683716,
0.6200000047683716,
0.699999988079071,
0.7200000286102295,
0.7400000095367432,
0.7799999713897705,
0.7799999713897705,
0.8399999737739563,

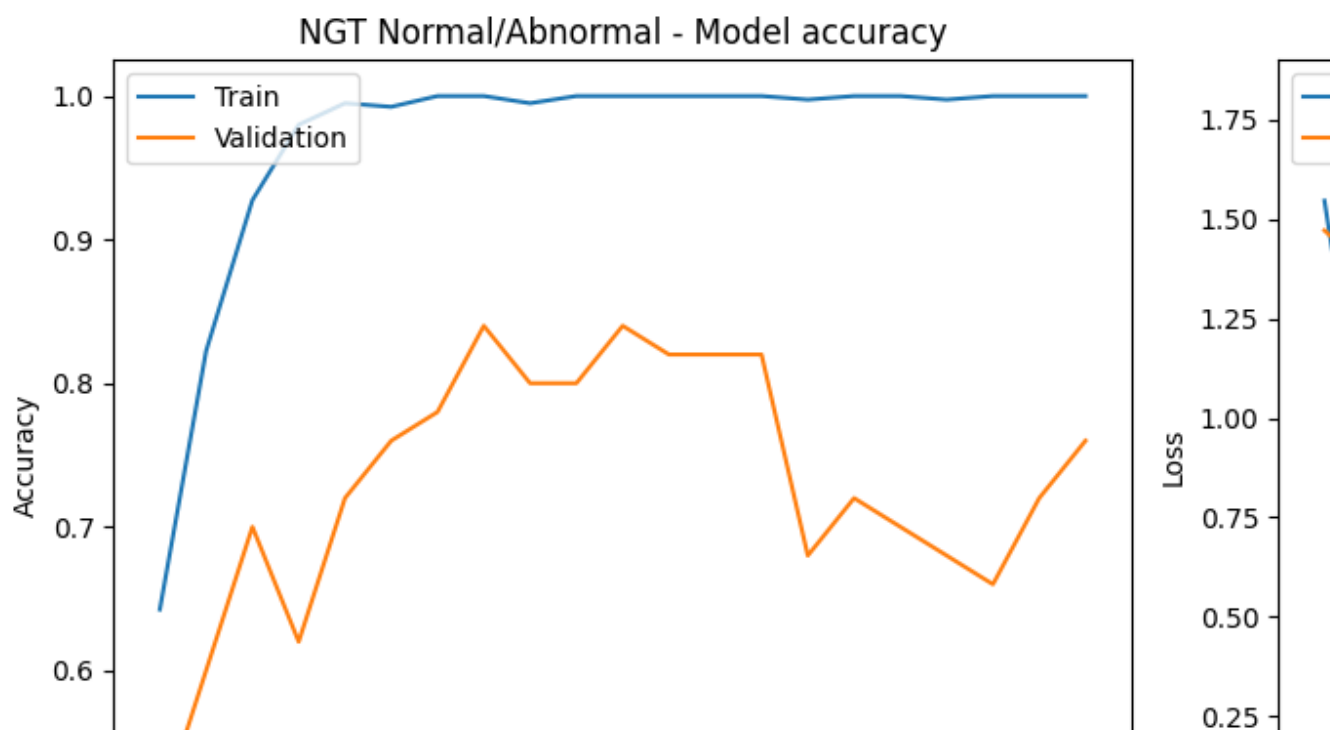
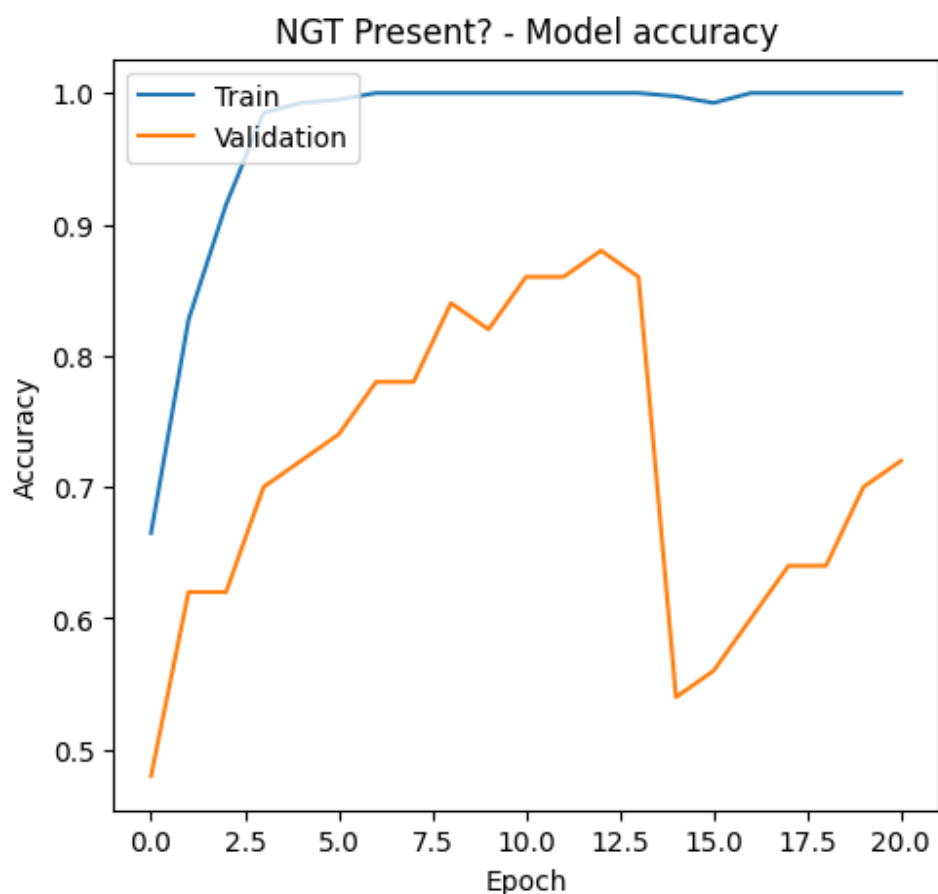

```

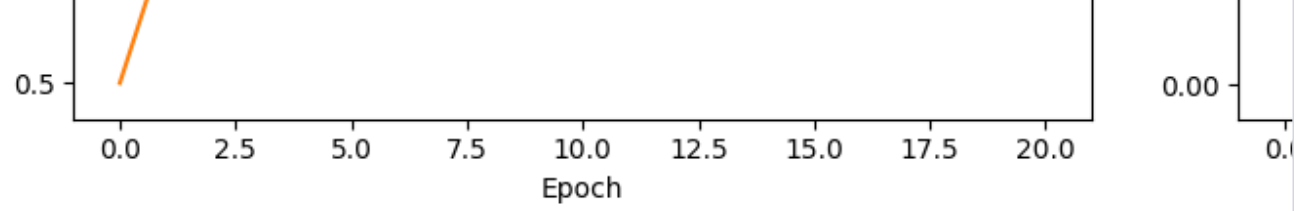
plt.plot(history.history['val_output_normal/abnormal_accuracy'])
plt.title('NGT Normal/Abnormal - Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

# Plot training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.tight_layout()
plt.show()

```





ResNet - ETT

```
In [ ]: import random
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
from PIL import Image
import ast
import os
import cv2
from tqdm import tqdm
```

```
In [ ]: train_data = pd.read_csv("/kaggle/input/ranzcr-clip-catheter-line-classification/train.csv")
train_data = train_data[(train_data['ETT - Borderline'] != 1)]

train_data[['ETT - Normal', 'ETT - Abnormal', 'ETT - Borderline']].value_counts()
train_data['ETT - Present'] = np.where((train_data['ETT - Normal'] == 0) & (train_data['ETT - A

train_data[['ETT - Normal', 'ETT - Abnormal']].value_counts()
```

```
Out[ ]: ETT - Normal  ETT - Abnormal
0                0                21626
1                0                7240
0                1                 79
Name: count, dtype: int64
```

```
In [ ]: normalAb = pd.concat([train_data[train_data['ETT - Normal'] == 1].sample(n=39), train_data[trai
sampled_data = pd.concat([normalAb, train_data[train_data['ETT - Present'] == 0].sample(n=78)])

sampled_data
```

```
Out[ ]:
```

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT - Abnormal	N
18335	1.2.826.0.1.3680043.8.498.12808251684309131301...	0	0	1	0	
16818	1.2.826.0.1.3680043.8.498.64545344928474329458...	0	0	1	0	
2370	1.2.826.0.1.3680043.8.498.95430425383132004650...	0	0	1	0	
25211	1.2.826.0.1.3680043.8.498.10480044806634047819...	0	0	1	0	
27722	1.2.826.0.1.3680043.8.498.13373062361633258041...	0	0	1	0	
...	...	...	...	...	...	
23460	1.2.826.0.1.3680043.8.498.10383526461057115729...	0	0	0	0	
11622	1.2.826.0.1.3680043.8.498.53392761555531684084...	0	0	0	0	
12896	1.2.826.0.1.3680043.8.498.12835091123305319564...	0	0	0	0	
15818	1.2.826.0.1.3680043.8.498.75375541699770932705...	0	0	0	0	
15021	1.2.826.0.1.3680043.8.498.30271781979432929522...	0	0	0	0	

156 rows × 14 columns

```
In [ ]: def getFileNames(df):
        studyInstance = df['StudyInstanceUID'].tolist()
        studyInstance

        for i in range(len(studyInstance)):
            studyInstance[i] = f'{studyInstance[i]}.jpg'

        return studyInstance
```

```
In [ ]: studyInstance = getFileNames(sampled_data)
```

```
In [ ]: def processImages(filenamees,train_test):

        imgs = []
        FileIDs = []
        lst = []

        desired_channels = 3

        count = 0
        for file in tqdm(filenamees, position=0, leave=True):
            # Making sure the images are the same pixel size
            img = Image.open(f"/kaggle/input/ranzcr-clip-catheter-line-classification/{train_test}/
            img = img.resize((300, 300))
            img = np.array(img)

            # Ensure that the image has 3 channels (RGB)
            if img.shape[-1] != 3:
                img = np.stack((img,) * 3, axis=-1) # Convert grayscale to RGB by duplicating chan

            imgs.append(img) # Append the 4D image to the list

            # Save filename for later
            FileIDs.append(file)

            count += 1

        # Convert the List of images to a numpy array
        imgs = np.array(imgs)
        return imgs, FileIDs
```

```
In [ ]: train_imgs, fileIDs = processImages(studyInstance, 'train')
```

```
100%|██████████| 156/156 [00:09<00:00, 16.39it/s]
```

```
In [ ]: #cleaning up the labels
        UID = []
        for i in range(len(fileIDs)):
            UID.append(fileIDs[i].replace(".jpg",""))

        UID[0:10]
```

```
Out[ ]: ['1.2.826.0.1.3680043.8.498.12808251684309131301583755654824541840',
        '1.2.826.0.1.3680043.8.498.64545344928474329458810362248833710064',
        '1.2.826.0.1.3680043.8.498.95430425383132004650191433607630505062',
        '1.2.826.0.1.3680043.8.498.10480044806634047819626681796785825533',
        '1.2.826.0.1.3680043.8.498.13373062361633258041199854131744962522',
        '1.2.826.0.1.3680043.8.498.28300964548658894395744362436309385737',
        '1.2.826.0.1.3680043.8.498.59297302149458774284162472548980015632',
        '1.2.826.0.1.3680043.8.498.27638524618188151587003689152335998105',
        '1.2.826.0.1.3680043.8.498.13028010645410161954037386970470767950',
        '1.2.826.0.1.3680043.8.498.92346044940481053670973987722867427355']
```

```
In [ ]: columns to drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal', 'NGT -
```

```

columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal', 'NGT - Borderline', 'NGT - Normal', 'ETT - Present', 'NGT - Present', 'ETT - Incompletely Imaged', 'Swan Ganz Catheter Present', 'PatientID', 'CVC']
ETTsampled_data = sampled_data.drop(columns=columns_to_drop, axis=1)
ETTsampled_data.set_index('StudyInstanceUID', inplace=True)

```

```

In [ ]: train_labels_df = np.empty((0,3))
data = []

for ID in UID:
    data.append(ETTsampled_data.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)
train_labels = train_labels_df['ETT - Present'].values
train_labels

```

```

Out[ ]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0])

```

```

In [ ]: from sklearn.model_selection import train_test_split

# Split your data into training and temporary data (combined validation and test)
train_imgs, temp_imgs, train_labels, temp_labels = train_test_split(train_imgs, train_labels, temp_imgs, temp_labels)

# Split the temporary data into validation and test sets
val_imgs, test_imgs, val_labels, test_labels = train_test_split(temp_imgs, temp_labels, test_imgs, test_labels)

```

```

In [ ]: import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout

```

```

In [ ]: def buildTrainData(catheter, train_data, size):

    train_data = train_data[train_data[f'{catheter} - Present'] == 1]
    train_data = train_data[~((train_data[f'{catheter} - Normal'] == 1) & (train_data[f'{catheter} - Normal/Abnormal'] == 1))]
    train_data[f'{catheter} - Normal/Abnormal'] = np.where(train_data[f'{catheter} - Normal'] == 1, train_data[f'{catheter} - Normal/Abnormal'], train_data[f'{catheter} - Normal'])
    sampled_data = pd.concat([train_data[train_data[f'{catheter} - Normal'] == 1].sample(n=size), train_data[train_data[f'{catheter} - Present'] == 1].sample(n=size)])

    return sampled_data

```

```

In [ ]: sample = buildTrainData('ETT', train_data, 78)
sample

```

```

Out[ ]:

```

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT - Abnormal	NGT - Borderline
29887	1.2.826.0.1.3680043.8.498.36518875504968416477...	0	0	1	0	
6375	1.2.826.0.1.3680043.8.498.13181424540386334611...	0	0	1	0	
29672	1.2.826.0.1.3680043.8.498.10815368909177631498...	0	0	1	0	
14805	1.2.826.0.1.3680043.8.498.13221541665973093351...	0	0	1	0	
5126	1.2.826.0.1.3680043.8.498.52626675650328112126...	0	0	1	0	

...	...	...	...	...	...
20701	1.2.826.0.1.3680043.8.498.10479615105716260873...	1	0	0	0
18286	1.2.826.0.1.3680043.8.498.45630226598251928362...	1	0	0	0
23435	1.2.826.0.1.3680043.8.498.80937139423520710449...	1	0	0	0
5457	1.2.826.0.1.3680043.8.498.30836180807940123518...	1	0	0	0
20200	1.2.826.0.1.3680043.8.498.28578895328900384008...	1	0	0	0

156 rows × 15 columns



```
In [ ]: nAb_data = getFileNames(sample)
        ett_imgs, fileIDs = processImages(nAb_data, 'train')
```

100%|██████████| 156/156 [00:08<00:00, 18.49it/s]

```
In [ ]: UID = []
        for i in range(len(fileIDs)):
            UID.append(fileIDs[i].replace(".jpg",""))

        columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal', 'NGT - Incompletely Imaged', "Swan Ganz Catheter Present", "PatientID", "CVC"]

        ETTsample = sample.drop(columns=columns_to_drop, axis=1)
        ETTsample.set_index('StudyInstanceUID', inplace=True)

        ETTsample
```

	ETT - Normal/Abnormal
StudyInstanceUID	
1.2.826.0.1.3680043.8.498.36518875504968416477207111692632539562	1
1.2.826.0.1.3680043.8.498.13181424540386334611112176122756730652	1
1.2.826.0.1.3680043.8.498.10815368909177631498970268684837832051	1
1.2.826.0.1.3680043.8.498.13221541665973093351408354838592188322	1
1.2.826.0.1.3680043.8.498.50626675605939419196753481286642152534	1
...	...
1.2.826.0.1.3680043.8.498.10479615105716260873080198321864360531	0
1.2.826.0.1.3680043.8.498.45630226598251928362902318654119069919	0
1.2.826.0.1.3680043.8.498.80937139423520710449087164918249582817	0
1.2.826.0.1.3680043.8.498.30836180807940123518012807355584281014	0
1.2.826.0.1.3680043.8.498.28578895328900384008640781750393178159	0

156 rows × 1 columns

```
In [ ]: train_labels_df = np.empty((0,3))
        data = []

        for ID in UID:
            data.append(ETTsample.loc[ID])

        train_labels_df = pd.DataFrame(data)
```

```
train_labels_df = train_labels_df.reindex(UID)
ett_labels = train_labels_df['ETT - Normal/Abnormal'].values
ett_labels
```

```
Out[ ]: array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
          1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
          1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
          1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
          0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          0, 0]])
```

```
In [ ]: from sklearn.model_selection import train_test_split

# Split your data into training and temporary data (combined validation and test)
trainETT_imgs, temp_imgs, trainETT_labels, temp_labels = train_test_split(ett_imgs, ett_labels,

# Split the temporary data into validation and test sets
valETT_imgs, testETT_imgs, valETT_labels, testETT_labels = train_test_split(temp_imgs, temp_labels,
```

```
In [ ]: from tensorflow.keras.layers import concatenate
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dropout, BatchNormalization
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Load the ResNet50 model with pre-trained weights
base_model = ResNet50(weights='imagenet', include_top=False)

# Add dropout and batch normalization to custom fully connected layers
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.5)(x)

# First Output Branch: CVC Catheter Presence (Binary Classification)
output_ETTpresent = Dense(1, activation='sigmoid', name='output_ETTpresent')(x)

# Add a separate set of layers for the new task
y = base_model.output
y = GlobalAveragePooling2D()(y)
y = Dense(1024, activation='relu')(y)
y = BatchNormalization()(y)
y = Dropout(0.5)(y)

# Second Output Branch: CVC Placement (Binary Classification)
output_ett_placement = Dense(1, activation='sigmoid', name='output_ETTnormalAbnormal')(y)

# Combine the two output branches
combined_model = Model(inputs=base_model.input, outputs=[output_ETTpresent, output_ett_placement])

# Compile the combined model with optimizer and Loss functions for both tasks
combined_model.compile(optimizer='SGD',
                      loss={'output_ETTpresent': 'binary_crossentropy', 'output_ETTnormalAbnormal': 'binary_crossentropy'},
                      metrics={'output_ETTpresent': 'accuracy', 'output_ETTnormalAbnormal': 'accuracy'})

# Reduce Learning rate when the validation loss plateaus
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=1e-6)

# Early stopping to prevent overfitting
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Save the best model during training
model_checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True)

history = combined_model.fit(
    (input_1, train_imgs),
    (train_labels, train_labels),
    validation_data=(val_imgs, val_labels),
    validation_labels=val_labels,
    callbacks=[early_stop, model_checkpoint, reduce_lr],
    epochs=100)
```

```

        {'input_1': train_imgs},
        {'output_ETTpresent': train_labels, 'output_ETTnormalAbnormal': trainETT_labels},
        epochs=30,
        validation_data=(
            {'input_1': val_imgs},
            {'output_ETTpresent': val_labels, 'output_ETTnormalAbnormal': valETT_labels}
        ),
        callbacks=[reduce_lr, early_stop, model_checkpoint]
    )

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_94765736/94765736 [=====] - 4s 0us/step

Epoch 1/30

4/4 [=====] - 96s 21s/step - loss: 1.5294 - output_ETTpresent_loss: 0.781048 - output_ETTnormalAbnormal_accuracy: 0.6290 - val_loss: 1.4038 - val_output_ETTpresent_loss: 0.781048 - val_output_ETTnormalAbnormal_accuracy: 0.5625 - lr: 0.0100

Epoch 2/30

4/4 [=====] - 83s 21s/step - loss: 0.5241 - output_ETTpresent_loss: 0.253952 - output_ETTnormalAbnormal_accuracy: 0.8710 - val_loss: 1.3186 - val_output_ETTpresent_loss: 0.253952 - val_output_ETTnormalAbnormal_accuracy: 0.6250 - lr: 0.0100

Epoch 3/30

4/4 [=====] - 84s 21s/step - loss: 0.2495 - output_ETTpresent_loss: 0.121435 - output_ETTnormalAbnormal_accuracy: 0.9355 - val_loss: 1.2999 - val_output_ETTpresent_loss: 0.121435 - val_output_ETTnormalAbnormal_accuracy: 0.6250 - lr: 0.0100

Epoch 4/30

4/4 [=====] - 81s 20s/step - loss: 0.0957 - output_ETTpresent_loss: 0.063758 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.3016 - val_output_ETTpresent_loss: 0.063758 - val_output_ETTnormalAbnormal_accuracy: 0.6250 - lr: 0.0100

Epoch 5/30

4/4 [=====] - 87s 22s/step - loss: 0.0547 - output_ETTpresent_loss: 0.029919 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.2631 - val_output_ETTpresent_loss: 0.029919 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0100

Epoch 6/30

4/4 [=====] - 83s 21s/step - loss: 0.0277 - output_ETTpresent_loss: 0.012000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.2225 - val_output_ETTpresent_loss: 0.012000 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0100

Epoch 7/30

4/4 [=====] - 82s 21s/step - loss: 0.0184 - output_ETTpresent_loss: 0.010000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.1466 - val_output_ETTpresent_loss: 0.010000 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0100

Epoch 8/30

4/4 [=====] - 82s 21s/step - loss: 0.0414 - output_ETTpresent_loss: 0.016000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0619 - val_output_ETTpresent_loss: 0.016000 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0100

Epoch 9/30

4/4 [=====] - 83s 21s/step - loss: 0.0092 - output_ETTpresent_loss: 0.004000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.9877 - val_output_ETTpresent_loss: 0.004000 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0100

Epoch 10/30

4/4 [=====] - 82s 21s/step - loss: 0.0098 - output_ETTpresent_loss: 0.004000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.9460 - val_output_ETTpresent_loss: 0.004000 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0100

Epoch 11/30

4/4 [=====] - 81s 20s/step - loss: 0.0450 - output_ETTpresent_loss: 0.025919 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8813 - val_output_ETTpresent_loss: 0.025919 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0100

Epoch 12/30

4/4 [=====] - 94s 24s/step - loss: 0.0108 - output_ETTpresent_loss: 0.005000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8621 - val_output_ETTpresent_loss: 0.005000 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0100

Epoch 13/30

4/4 [=====] - 83s 21s/step - loss: 0.0039 - output_ETTpresent_loss: 0.002000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8457 - val_output_ETTpresent_loss: 0.002000 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0100

Epoch 14/30

4/4 [=====] - 83s 21s/step - loss: 0.0093 - output_ETTpresent_loss: 0.004000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8343 - val_output_ETTpresent_loss: 0.004000 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100

Epoch 15/30

4/4 [=====] - 83s 21s/step - loss: 0.0309 - output_ETTpresent_loss: 0.014000 - output_ETTnormalAbnormal_accuracy: 0.9919 - val_loss: 0.8222 - val_output_ETTpresent_loss: 0.014000 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100

Epoch 16/30

```

Epoch 16/30
4/4 [=====] - 85s 21s/step - loss: 0.0094 - output_ETTpresent_loss: 0.005
000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8135 - val_output_ETTpresent_loss: 0
ccuracy: 0.8750 - val_output_ETTnormalAbnormal_accuracy: 0.8750 - lr: 0.0100
Epoch 17/30
4/4 [=====] - 81s 20s/step - loss: 0.0098 - output_ETTpresent_loss: 0.007
000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8621 - val_output_ETTpresent_loss: 0
ccuracy: 0.8125 - val_output_ETTnormalAbnormal_accuracy: 0.8750 - lr: 0.0100
Epoch 18/30
4/4 [=====] - 82s 20s/step - loss: 0.0155 - output_ETTpresent_loss: 0.007
000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.9171 - val_output_ETTpresent_loss: 0
ccuracy: 0.6875 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0100
Epoch 19/30
4/4 [=====] - 101s 27s/step - loss: 0.0037 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.9482 - val_output_ETTpresent_loss:
accuracy: 0.6875 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0100
Epoch 20/30
4/4 [=====] - 83s 20s/step - loss: 0.0042 - output_ETTpresent_loss: 0.002
000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.9695 - val_output_ETTpresent_loss: 0
ccuracy: 0.6875 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0100
Epoch 21/30
4/4 [=====] - 82s 20s/step - loss: 0.0042 - output_ETTpresent_loss: 0.001
000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.9895 - val_output_ETTpresent_loss: 0
ccuracy: 0.6250 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0100
Epoch 22/30
4/4 [=====] - 84s 21s/step - loss: 0.0040 - output_ETTpresent_loss: 0.001
000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0014 - val_output_ETTpresent_loss: 0
ccuracy: 0.6250 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0020
Epoch 23/30
4/4 [=====] - 82s 21s/step - loss: 0.0072 - output_ETTpresent_loss: 0.003
000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0160 - val_output_ETTpresent_loss: 0
ccuracy: 0.6250 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0020
Epoch 24/30
4/4 [=====] - 82s 21s/step - loss: 0.0027 - output_ETTpresent_loss: 9.376
1.0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0227 - val_output_ETTpresent_loss
t_accuracy: 0.6250 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0020
Epoch 25/30
4/4 [=====] - 82s 20s/step - loss: 0.0042 - output_ETTpresent_loss: 0.002
000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0329 - val_output_ETTpresent_loss: 0
ccuracy: 0.6250 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0020
Epoch 26/30
4/4 [=====] - 82s 21s/step - loss: 0.0065 - output_ETTpresent_loss: 0.004
000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0353 - val_output_ETTpresent_loss: 0
ccuracy: 0.6250 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0020

```

```

In [ ]: # Evaluate accuracy for the 'output_catheter' branch
evaluation = combined_model.evaluate(
    {'input_1': test_imgs},
    {'output_ETTpresent': test_labels, 'output_ETTnormalAbnormal': testETT_labels},
)

# Extract the accuracy values from the evaluation results
test_accuracy_catheter = evaluation[3] # Index 3 corresponds to 'output_catheter_accuracy'
test_accuracy_cvc_placement = evaluation[4] # Index 4 corresponds to 'output_cvc_placement_acc

print(f'Test Accuracy (ETT Present): {test_accuracy_catheter * 100:.2f}%')
print(f'Test Accuracy (ETT Normal/Abnormal): {test_accuracy_cvc_placement * 100:.2f}%')

```

```

1/1 [=====] - 2s 2s/step - loss: 1.1538 - output_ETTpresent_loss: 0.5100
0 - output_ETTnormalAbnormal_accuracy: 0.6250
Test Accuracy (ETT Present): 75.00%
Test Accuracy (ETT Normal/Abnormal): 62.50%

```

```

In [ ]: test_path = "/kaggle/input/ranzcr-clip-catheter-line-classification/test"

all_files = os.listdir(test_path)

test_files = all_files[:300]

```

```

In [ ]: test_img, testFiles = processImages(test_files, "test")

```


In []:

```

predictions = combined_model.predict(test_imgs)
predictions

# Access predictions for the first task
ngt_present_predictions = predictions[0]
ngt_present_predictions
# Access predictions for the second task
ngt_normal_abnormal_predictions = predictions[1]

ngt_present_predictions
# Create two separate DataFrames
df1 = pd.DataFrame(ngt_present_predictions, columns=["ETT - Present"])
df2 = pd.DataFrame(ngt_normal_abnormal_predictions, columns=['ETT - Normal/Abnormal'])

# Concatenate the DataFrames horizontally to create one with two columns
NGTpredictions = pd.concat([df1, df2], axis=1)
NGTpredictions

NGTpredictions['ETT - Present'] = np.where(NGTpredictions['ETT - Present'] > 0.5, 1, 0)
NGTpredictions['ETT - Normal/Abnormal'] = np.where(NGTpredictions['ETT - Normal/Abnormal'] > 0.
NGTpredictions

from sklearn.metrics import confusion_matrix

# Create the confusion matrix
confusion_matrix_present = confusion_matrix(test_labels, NGTpredictions['ETT - Present'])

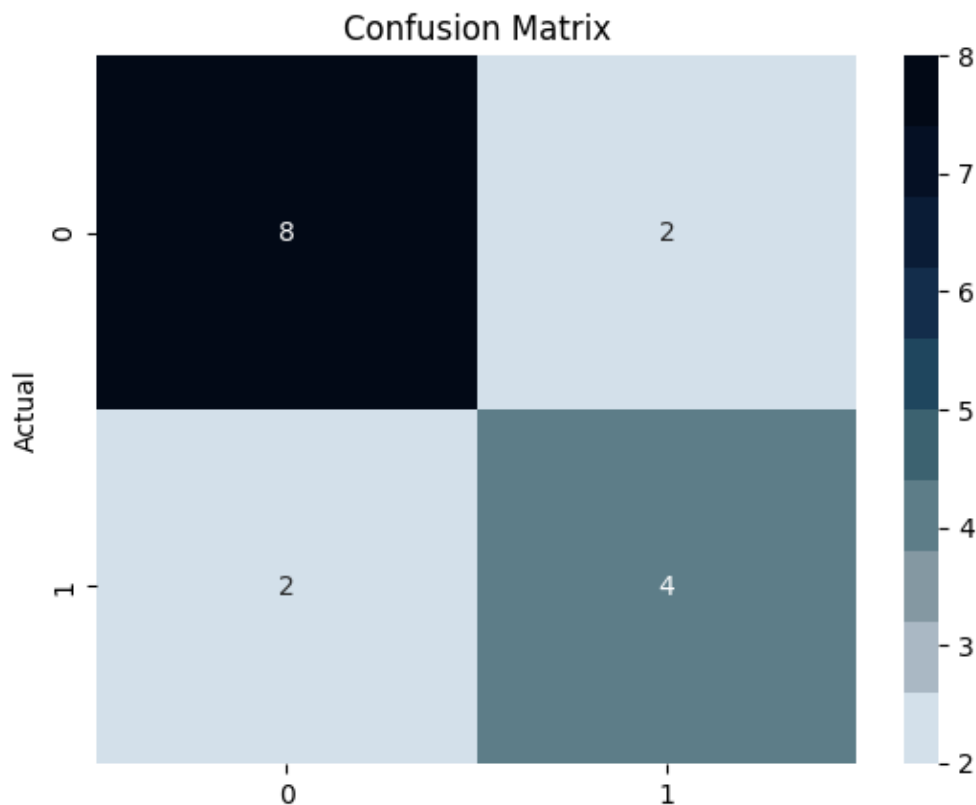
plt.gca().patch.set_facecolor('none')

custom_palette = ['#D3E0EA', '#AAB8C4', '#8498A2', '#5D7D88', '#3C6270',
                  '#1F465E', '#132D4B', '#0A1C36', '#051024', '#020916']

# Create a heatmap plot of the confusion matrix
sns.heatmap(confusion_matrix_present, annot=True, cmap=custom_palette)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

```

1/1 [=====] - 7s 7s/step



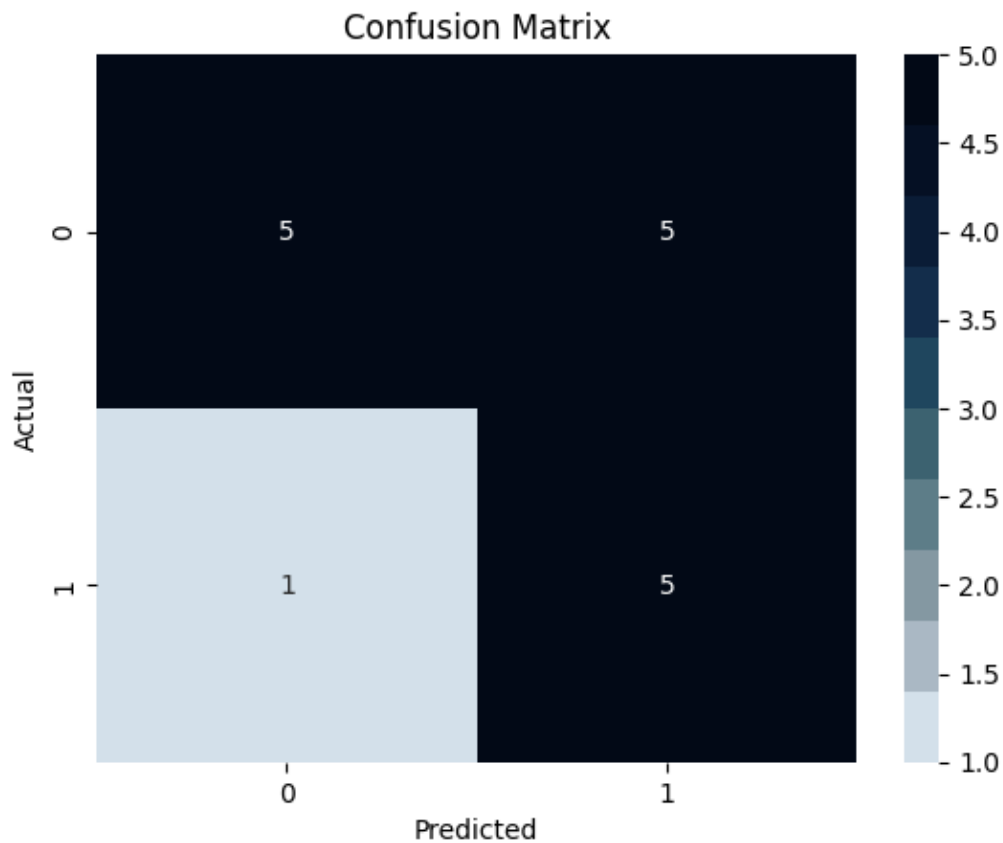
Predicted

```
In [ ]: confusion_matrix_present = confusion_matrix(test_labels, NGTpredictions['ETT - Normal/Abnormal'])

plt.gca().patch.set_facecolor('none')

custom_palette = ['#D3E0EA', '#AAB8C4', '#8498A2', '#5D7D88', '#3C6270',
                  '#1F465E', '#132D4B', '#0A1C36', '#051024', '#020916']

# Create a heatmap plot of the confusion matrix
sns.heatmap(confusion_matrix_present, annot=True, cmap=custom_palette)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



DenseNet - CVC

```
In [ ]: import random
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import json
from PIL import Image
import ast
import os

from tqdm import tqdm
```

```
In [ ]: import pandas as pd

filenames = os.listdir("/projects/sc73/ranzcr-clip-catheter-line-classification/train")
```

```
In [ ]: #extracting just the CVC Labels
train_data = pd.read_csv("/projects/sc73/ranzcr-clip-catheter-line-classification/train_data.csv")
train_data = train_data[train_data['CVC - Borderline'] != 1]
train_data['CVC - Present'] = np.where((train_data['CVC - Abnormal'] == 0) & (train_data['CVC - Borderline'] == 0), 1, 0)

train_data['CVC - Present'].value_counts()
```

```
Out[ ]: 1    20873
0       750
Name: CVC - Present, dtype: int64
```

```
In [ ]: sampled_data = pd.concat([train_data[train_data['CVC - Present'] == 1].sample(n=750),
                                train_data[train_data['CVC - Present'] == 0].sample(n=750)],
                                axis=0)
```

```
Out[ ]:
```

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT Abnorm
5125	1.2.826.0.1.3680043.8.498.29798135944425181867...	0	0	0	
18625	1.2.826.0.1.3680043.8.498.56917573254477591662...	0	0	0	
29064	1.2.826.0.1.3680043.8.498.11749599424730443758...	0	0	1	
25183	1.2.826.0.1.3680043.8.498.84533062276529819766...	1	0	0	
15555	1.2.826.0.1.3680043.8.498.49761949951848218455...	0	0	0	
...	...	...	...	...	
22809	1.2.826.0.1.3680043.8.498.93902612986195069275...	0	0	0	
241	1.2.826.0.1.3680043.8.498.73088700281917814818...	0	0	1	
812	1.2.826.0.1.3680043.8.498.85658065578219937558...	0	0	1	
5106	1.2.826.0.1.3680043.8.498.84390265238997870942...	0	0	1	
1969	1.2.826.0.1.3680043.8.498.11976136065771571983...	0	0	1	



```
studyInstance = sampled_data['StudyInstanceUID'].tolist()
studyInstance

for i in range(len(studyInstance)):
    studyInstance[i] = f'{studyInstance[i]}.jpg'

studyInstance
```

```
Out[ ]: ['1.2.826.0.1.3680043.8.498.29798135944425181867741764889671646820.jpg',
'1.2.826.0.1.3680043.8.498.56917573254477591662381927833438589539.jpg',
'1.2.826.0.1.3680043.8.498.11749599424730443758948253678195377134.jpg',
'1.2.826.0.1.3680043.8.498.84533062276529819766171648406000766083.jpg',
'1.2.826.0.1.3680043.8.498.49761949951848218455749059001453442446.jpg',
'1.2.826.0.1.3680043.8.498.40981519393347637453680278096781297052.jpg',
'1.2.826.0.1.3680043.8.498.82309444747241213141372618882835382602.jpg',
'1.2.826.0.1.3680043.8.498.31007111818249500067228144131625734696.jpg',
'1.2.826.0.1.3680043.8.498.45258993479143480195404274416141355225.jpg',
'1.2.826.0.1.3680043.8.498.81361445628248936097480449911449382182.jpg',
'1.2.826.0.1.3680043.8.498.56362571928002495216786513544266233241.jpg',
'1.2.826.0.1.3680043.8.498.11531534547316263713311684295632234083.jpg',
'1.2.826.0.1.3680043.8.498.15841117153179839120329046729534619540.jpg',
'1.2.826.0.1.3680043.8.498.10198749153295666130764917543606749567.jpg',
'1.2.826.0.1.3680043.8.498.58752002338269439452712494232863718508.jpg',
'1.2.826.0.1.3680043.8.498.40949483489317081252702261809411864307.jpg',
'1.2.826.0.1.3680043.8.498.39887544675136422772390772424461700599.jpg',
'1.2.826.0.1.3680043.8.498.61592830077212194262382391947832579612.jpg',
'1.2.826.0.1.3680043.8.498.61547437489721642299827542453334131706.jpg',
'1.2.826.0.1.3680043.8.498.40295823631429811842801631139279035349.jpg',
'1.2.826.0.1.3680043.8.498.11927519055879651497879927840865892209.jpg',
'1.2.826.0.1.3680043.8.498.60589852072552480175985641084974448525.jpg',
'1.2.826.0.1.3680043.8.498.39682826396416829375253103181898223541.jpg',
'1.2.826.0.1.3680043.8.498.21409838160375144993328632178539861175.jpg',
'1.2.826.0.1.3680043.8.498.17056128638826264484137256626208982490.jpg',
'1.2.826.0.1.3680043.8.498.57143770666308355869192606514186313200.jpg',
'1.2.826.0.1.3680043.8.498.12905402717932419151886459977674607175.jpg',
'1.2.826.0.1.3680043.8.498.47473975469338420874861620441889009425.jpg',
'1.2.826.0.1.3680043.8.498.17066944198380357543203427950526008746.jpg',
'1.2.826.0.1.3680043.8.498.11528141281799724454403644769035885129.jpg',
'1.2.826.0.1.3680043.8.498.33436731379479851110760935401840563806.jpg',
'1.2.826.0.1.3680043.8.498.53884958436324639454047401816306616929.jpg',
'1.2.826.0.1.3680043.8.498.33683604904976354868975496714435170374.jpg',
'1.2.826.0.1.3680043.8.498.11833705601078128847897390516413050779.jpg',
'1.2.826.0.1.3680043.8.498.59796883382597218032859159614500290030.jpg',
'1.2.826.0.1.3680043.8.498.28024759847988905831417209038313203046.jpg',
'1.2.826.0.1.3680043.8.498.22729505514236797340364045195277979092.jpg',
'1.2.826.0.1.3680043.8.498.46114955096234955051372963533837209303.jpg',
'1.2.826.0.1.3680043.8.498.93996674799645988481951427980491263942.jpg',
'1.2.826.0.1.3680043.8.498.12958799345979695687303792506702694526.jpg',
'1.2.826.0.1.3680043.8.498.73960663000693211773844351741305260154.jpg',
'1.2.826.0.1.3680043.8.498.75011483175188248204305638032766280192.jpg',
'1.2.826.0.1.3680043.8.498.10607584632225170250956643027962473911.jpg',
'1.2.826.0.1.3680043.8.498.59277764276174987637118258927045514665.jpg',
'1.2.826.0.1.3680043.8.498.71834519845169647251301091121432121219.jpg',
'1.2.826.0.1.3680043.8.498.56621396182777363230024500734936671793.jpg',
'1.2.826.0.1.3680043.8.498.37113660038147024402986813487687695758.jpg',
'1.2.826.0.1.3680043.8.498.52393525259921370427065436316641843043.jpg',
'1.2.826.0.1.3680043.8.498.71983654258710568062425937413333819640.jpg',
'1.2.826.0.1.3680043.8.498.47732941800851998214388873297302992155.jpg',
'1.2.826.0.1.3680043.8.498.10467855402103430711470175178835758653.jpg',
'1.2.826.0.1.3680043.8.498.32414175172882724168130319262584333943.jpg',
'1.2.826.0.1.3680043.8.498.64887497188160097943501689029993429677.jpg',
'1.2.826.0.1.3680043.8.498.88554420391193718276492906728625716692.jpg',
'1.2.826.0.1.3680043.8.498.38003901769454345073561825816922538291.jpg',
'1.2.826.0.1.3680043.8.498.52165595692988545865844692026445220692.jpg',
'1.2.826.0.1.3680043.8.498.51242343675506530674471759767296481329.jpg',
'1.2.826.0.1.3680043.8.498.10102145462980903773024100390892159036.jpg',
'1.2.826.0.1.3680043.8.498.74548358423150416738419732336303114551.jpg',
'1.2.826.0.1.3680043.8.498.52544884986870333153110267952780658863.jpg',
```

'1.2.826.0.1.3680043.8.498.94032436850109510769671826713688632145.jpg',
'1.2.826.0.1.3680043.8.498.12432086470486835151817637839871458930.jpg',
'1.2.826.0.1.3680043.8.498.10304910088744435718808753496597431161.jpg',
'1.2.826.0.1.3680043.8.498.94625911385910624169048503198242567241.jpg',
'1.2.826.0.1.3680043.8.498.85497215653436426876460534604601759069.jpg',
'1.2.826.0.1.3680043.8.498.74656854855767376731520470446024408044.jpg',
'1.2.826.0.1.3680043.8.498.28282486449725806009993670787932734379.jpg',
'1.2.826.0.1.3680043.8.498.12792948363385530170229353413228008481.jpg',
'1.2.826.0.1.3680043.8.498.92887252575654338321109180011869398178.jpg',
'1.2.826.0.1.3680043.8.498.52829771355056785526910581978914323655.jpg',
'1.2.826.0.1.3680043.8.498.10447229620745029192628441615226650974.jpg',
'1.2.826.0.1.3680043.8.498.41247672018860819344245563630114291427.jpg',
'1.2.826.0.1.3680043.8.498.26071677976121126094841588408325876543.jpg',
'1.2.826.0.1.3680043.8.498.10233483199428800495852150397933116117.jpg',
'1.2.826.0.1.3680043.8.498.95507333872881773304903945865066366262.jpg',
'1.2.826.0.1.3680043.8.498.86556990915562641335160987905196873078.jpg',
'1.2.826.0.1.3680043.8.498.20168149160025262725849887926714182220.jpg',
'1.2.826.0.1.3680043.8.498.10884614372274716166638895515695716519.jpg',
'1.2.826.0.1.3680043.8.498.28361472631689286486596775258278669525.jpg',
'1.2.826.0.1.3680043.8.498.40183653686440598805310560675446424277.jpg',
'1.2.826.0.1.3680043.8.498.50020215050322989273778013713515235663.jpg',
'1.2.826.0.1.3680043.8.498.11560842996253223681355125548775417454.jpg',
'1.2.826.0.1.3680043.8.498.10251926765095417906724146170052570377.jpg',
'1.2.826.0.1.3680043.8.498.98694750212105016604354908118338717177.jpg',
'1.2.826.0.1.3680043.8.498.67547100183894812219169431602326589691.jpg',
'1.2.826.0.1.3680043.8.498.11799794208784263847398153870237672486.jpg',
'1.2.826.0.1.3680043.8.498.12052436520311970300442536791234577449.jpg',
'1.2.826.0.1.3680043.8.498.43588620924272019787709616817095820394.jpg',
'1.2.826.0.1.3680043.8.498.37716693029953619445976921430920729153.jpg',
'1.2.826.0.1.3680043.8.498.10962666347147599736851008656479263658.jpg',
'1.2.826.0.1.3680043.8.498.75170563215094752038895405766230179882.jpg',
'1.2.826.0.1.3680043.8.498.32665013930528750130301395098139968929.jpg',
'1.2.826.0.1.3680043.8.498.32545360805072399405234820441872423413.jpg',
'1.2.826.0.1.3680043.8.498.12943735962780073791844944001123094906.jpg',
'1.2.826.0.1.3680043.8.498.82650805005497425786849124442660567603.jpg',
'1.2.826.0.1.3680043.8.498.73131695979135829823733106153213596940.jpg',
'1.2.826.0.1.3680043.8.498.88786990116615088936460729929312040723.jpg',
'1.2.826.0.1.3680043.8.498.45171530708732397441319217266713595463.jpg',
'1.2.826.0.1.3680043.8.498.57110944635821449507567128815869071414.jpg',
'1.2.826.0.1.3680043.8.498.88230784304989213439114738223963753923.jpg',
'1.2.826.0.1.3680043.8.498.71853035197486588807333011766191698529.jpg',
'1.2.826.0.1.3680043.8.498.62072023324268520225725991093243462721.jpg',
'1.2.826.0.1.3680043.8.498.17197802609903129007093573905718965376.jpg',
'1.2.826.0.1.3680043.8.498.54133835386464372262025748706014467272.jpg',
'1.2.826.0.1.3680043.8.498.75365920664679925212112897129896936492.jpg',
'1.2.826.0.1.3680043.8.498.55935946569866113867101920323850990215.jpg',
'1.2.826.0.1.3680043.8.498.72635199587960252625310678044669691694.jpg',
'1.2.826.0.1.3680043.8.498.26323554106924660475932061311984322616.jpg',
'1.2.826.0.1.3680043.8.498.41945305818337225154058453989441160271.jpg',
'1.2.826.0.1.3680043.8.498.14892540893857338565499372702510851706.jpg',
'1.2.826.0.1.3680043.8.498.10201435294858971900683459703999810483.jpg',
'1.2.826.0.1.3680043.8.498.35910127910306163938375969785161495840.jpg',
'1.2.826.0.1.3680043.8.498.34253051376305622302016913320150430074.jpg',
'1.2.826.0.1.3680043.8.498.51787441927186915503454322161148523315.jpg',
'1.2.826.0.1.3680043.8.498.68684431859651126164340752233548526494.jpg',
'1.2.826.0.1.3680043.8.498.13247235397828796725341339700734470952.jpg',
'1.2.826.0.1.3680043.8.498.19461507373197010744827752590058085345.jpg',
'1.2.826.0.1.3680043.8.498.83671803822557628856940979263337747964.jpg',
'1.2.826.0.1.3680043.8.498.55116125656035304819696307876072495939.jpg',
'1.2.826.0.1.3680043.8.498.85658012966840037375232726057655753117.jpg',
'1.2.826.0.1.3680043.8.498.12331060388117642694992751936409671831.jpg',
'1.2.826.0.1.3680043.8.498.12170678037590424280716301082830933304.jpg',
'1.2.826.0.1.3680043.8.498.55329724133125691266263309962728084176.jpg',
'1.2.826.0.1.3680043.8.498.90890095997847895849835202568282146016.jpg',
'1.2.826.0.1.3680043.8.498.81182381242584634263270526118775967574.jpg',
'1.2.826.0.1.3680043.8.498.11006008351489316067187562165895994056.jpg',
'1.2.826.0.1.3680043.8.498.7414162700797635253224205572396621256.jpg',
'1.2.826.0.1.3680043.8.498.50340126387437066363541495556187241133.jpg',
'1.2.826.0.1.3680043.8.498.28503179508014886971737723478508155332.jpg',
'1.2.826.0.1.3680043.8.498.58937013601376079209875900984267696219.jpg',
'1.2.826.0.1.3680043.8.498.71920369474410656524909921957625762885.jpg',

'1.2.826.0.1.3680043.8.498.52598938316352959274470128126749866289.jpg',
'1.2.826.0.1.3680043.8.498.78320306814557672660977685213119727738.jpg',
'1.2.826.0.1.3680043.8.498.8189495000077502932181260936044912348.jpg',
'1.2.826.0.1.3680043.8.498.28194228094257568346696314030506302007.jpg',
'1.2.826.0.1.3680043.8.498.11218082940148118592504002542799663891.jpg',
'1.2.826.0.1.3680043.8.498.23510926850520348791539647313604608355.jpg',
'1.2.826.0.1.3680043.8.498.46602761217226224919786015042313729013.jpg',
'1.2.826.0.1.3680043.8.498.77829051102467770886526031357278894045.jpg',
'1.2.826.0.1.3680043.8.498.10515972011913833378465817826643025107.jpg',
'1.2.826.0.1.3680043.8.498.19503133181889072738858544200993952695.jpg',
'1.2.826.0.1.3680043.8.498.22435382038849037398034722925575987992.jpg',
'1.2.826.0.1.3680043.8.498.13325708416006522465740052193499192139.jpg',
'1.2.826.0.1.3680043.8.498.39886298078881020410508601655005815242.jpg',
'1.2.826.0.1.3680043.8.498.25807096714208879326694128851604424906.jpg',
'1.2.826.0.1.3680043.8.498.35889693886956094834495596780045620753.jpg',
'1.2.826.0.1.3680043.8.498.86695666948239303892127807794988881784.jpg',
'1.2.826.0.1.3680043.8.498.55105543609776490386552026542876907361.jpg',
'1.2.826.0.1.3680043.8.498.94426455015930926979909524203606841676.jpg',
'1.2.826.0.1.3680043.8.498.10945158382577206349111245192157590414.jpg',
'1.2.826.0.1.3680043.8.498.18111985350557695358986536377437772259.jpg',
'1.2.826.0.1.3680043.8.498.11177684714965936933535628539309359088.jpg',
'1.2.826.0.1.3680043.8.498.95274996622197814994531627608956374903.jpg',
'1.2.826.0.1.3680043.8.498.70209723308080429173849264460073614154.jpg',
'1.2.826.0.1.3680043.8.498.11565029358139705191423124454594062534.jpg',
'1.2.826.0.1.3680043.8.498.12696950626963190187862220338834250122.jpg',
'1.2.826.0.1.3680043.8.498.20255231767246063773351695329494565877.jpg',
'1.2.826.0.1.3680043.8.498.94795540059621744084348139978063668299.jpg',
'1.2.826.0.1.3680043.8.498.11107504311750004651063423759552800910.jpg',
'1.2.826.0.1.3680043.8.498.55320296361099065518766124324033564573.jpg',
'1.2.826.0.1.3680043.8.498.62375760809446630048434901221690188239.jpg',
'1.2.826.0.1.3680043.8.498.19802779689819985092428059012051400260.jpg',
'1.2.826.0.1.3680043.8.498.62594343334823946985121964885820569925.jpg',
'1.2.826.0.1.3680043.8.498.97408246164051408782249343957433614144.jpg',
'1.2.826.0.1.3680043.8.498.62106087784447094738626641507308117032.jpg',
'1.2.826.0.1.3680043.8.498.94187729119431142447001156104308654331.jpg',
'1.2.826.0.1.3680043.8.498.38144189811451868454570240647297664274.jpg',
'1.2.826.0.1.3680043.8.498.66869767807772093174251650368944817912.jpg',
'1.2.826.0.1.3680043.8.498.10214733722328213639799519292896669691.jpg',
'1.2.826.0.1.3680043.8.498.19085075491833883798847197486330363033.jpg',
'1.2.826.0.1.3680043.8.498.15953401697051048742583436931034768944.jpg',
'1.2.826.0.1.3680043.8.498.11372641415829492501705434348636911631.jpg',
'1.2.826.0.1.3680043.8.498.91014417889968684241536009977468220446.jpg',
'1.2.826.0.1.3680043.8.498.11075843296049306074335627004334244805.jpg',
'1.2.826.0.1.3680043.8.498.90739419434369781616912405397379361163.jpg',
'1.2.826.0.1.3680043.8.498.59751820394879602489477370646391792712.jpg',
'1.2.826.0.1.3680043.8.498.96178954658016700643840321060472935008.jpg',
'1.2.826.0.1.3680043.8.498.12362344323626420494429949494925573601.jpg',
'1.2.826.0.1.3680043.8.498.71437656045381360253357553161519362047.jpg',
'1.2.826.0.1.3680043.8.498.13069390431676915051655073887942797432.jpg',
'1.2.826.0.1.3680043.8.498.12441107294193883855648191211474556692.jpg',
'1.2.826.0.1.3680043.8.498.29217002470508428811329721384027594649.jpg',
'1.2.826.0.1.3680043.8.498.22983336814340357553739563119976705380.jpg',
'1.2.826.0.1.3680043.8.498.10977718807120386582035780090368443868.jpg',
'1.2.826.0.1.3680043.8.498.37517558237481967062888482194631088767.jpg',
'1.2.826.0.1.3680043.8.498.30586284703416913320965628960253239506.jpg',
'1.2.826.0.1.3680043.8.498.33445715436261464983476895328101911165.jpg',
'1.2.826.0.1.3680043.8.498.89196490618523984490929181338106581812.jpg',
'1.2.826.0.1.3680043.8.498.69825262250029992212074685783282692808.jpg',
'1.2.826.0.1.3680043.8.498.12856178691551464092183114902015454708.jpg',
'1.2.826.0.1.3680043.8.498.52230099487781495929570266003824992134.jpg',
'1.2.826.0.1.3680043.8.498.83015949213405379642577654491348396486.jpg',
'1.2.826.0.1.3680043.8.498.95511622272845677714093030950638899340.jpg',
'1.2.826.0.1.3680043.8.498.10307938324543225962157913103348443468.jpg',
'1.2.826.0.1.3680043.8.498.63802954090811485101053226570555813062.jpg',
'1.2.826.0.1.3680043.8.498.43606986028072021482943057717500385540.jpg',
'1.2.826.0.1.3680043.8.498.86067386409593251625370025254288493417.jpg',
'1.2.826.0.1.3680043.8.498.84565206039488635985561004406585153335.jpg',
'1.2.826.0.1.3680043.8.498.11749450062462039558160488062568916854.jpg',
'1.2.826.0.1.3680043.8.498.27017817914326411722555942551856044508.jpg',
'1.2.826.0.1.3680043.8.498.12543574692866107367565198967175875365.jpg',
'1.2.826.0.1.3680043.8.498.11806543356813341688887463915359735837.jpg'

1.2.826.0.1.3680043.8.498.11244974927295521685970365558886658605.jpg',
'1.2.826.0.1.3680043.8.498.12407594085941905062690726953593784998.jpg',
'1.2.826.0.1.3680043.8.498.87385704988926694289623562878626796107.jpg',
'1.2.826.0.1.3680043.8.498.39243391380280313171246353015717990303.jpg',
'1.2.826.0.1.3680043.8.498.93287196294729517934576369814860188072.jpg',
'1.2.826.0.1.3680043.8.498.49336156336604641263855308032147030947.jpg',
'1.2.826.0.1.3680043.8.498.56888473055384190968966597937115189179.jpg',
'1.2.826.0.1.3680043.8.498.70159611245185569474807274143880851085.jpg',
'1.2.826.0.1.3680043.8.498.70796822518512365481022724365668161623.jpg',
'1.2.826.0.1.3680043.8.498.18324301212185589689553016930520589670.jpg',
'1.2.826.0.1.3680043.8.498.11244974927295521685970365558886658605.jpg',
'1.2.826.0.1.3680043.8.498.10582484326088591883556106608757372719.jpg',
'1.2.826.0.1.3680043.8.498.89719499392304088820868449202869164526.jpg',
'1.2.826.0.1.3680043.8.498.42281996467902955997849329585744888460.jpg',
'1.2.826.0.1.3680043.8.498.16639518127735021947738088272379701701.jpg',
'1.2.826.0.1.3680043.8.498.29677516638084167928732268870871092629.jpg',
'1.2.826.0.1.3680043.8.498.55683574389227182141155722417808780313.jpg',
'1.2.826.0.1.3680043.8.498.48446263555138605692293994525661706995.jpg',
'1.2.826.0.1.3680043.8.498.14347040000247510728221267271787290501.jpg',
'1.2.826.0.1.3680043.8.498.27919183910420702782472097588922516221.jpg',
'1.2.826.0.1.3680043.8.498.67385917605307750221768804701797668565.jpg',
'1.2.826.0.1.3680043.8.498.49497276842317967626815301010532086144.jpg',
'1.2.826.0.1.3680043.8.498.41112139469322176513036070794630219726.jpg',
'1.2.826.0.1.3680043.8.498.57562706261298256656000346211938315370.jpg',
'1.2.826.0.1.3680043.8.498.42518200656750055378467842682413233928.jpg',
'1.2.826.0.1.3680043.8.498.13290625661610333858940531061906078201.jpg',
'1.2.826.0.1.3680043.8.498.78631334867713202750540625086792591124.jpg',
'1.2.826.0.1.3680043.8.498.55687143497926025637123457065348083133.jpg',
'1.2.826.0.1.3680043.8.498.14143569260130103712849891984652803398.jpg',
'1.2.826.0.1.3680043.8.498.51474952759847244775822730211215869617.jpg',
'1.2.826.0.1.3680043.8.498.51103704356973821836718684599359294414.jpg',
'1.2.826.0.1.3680043.8.498.10383134060241605643929589665735924463.jpg',
'1.2.826.0.1.3680043.8.498.34908450415232161801906270932919695006.jpg',
'1.2.826.0.1.3680043.8.498.11971167339007442571865303761634905107.jpg',
'1.2.826.0.1.3680043.8.498.61142245547621587025387714081477802812.jpg',
'1.2.826.0.1.3680043.8.498.40730751694451250401800763583692765452.jpg',
'1.2.826.0.1.3680043.8.498.76166230996805861973860509153814855474.jpg',
'1.2.826.0.1.3680043.8.498.24487435506030871712438835177525051141.jpg',
'1.2.826.0.1.3680043.8.498.13013102368054240724898950819559010373.jpg',
'1.2.826.0.1.3680043.8.498.12922058983569196431186087563337052328.jpg',
'1.2.826.0.1.3680043.8.498.46362424044526536005287795153396574665.jpg',
'1.2.826.0.1.3680043.8.498.14499239017554312322112095477958636422.jpg',
'1.2.826.0.1.3680043.8.498.68320073741972584352391439047754474196.jpg',
'1.2.826.0.1.3680043.8.498.13359717473057483947650927033594377860.jpg',
'1.2.826.0.1.3680043.8.498.51247742386761546835802940225758984123.jpg',
'1.2.826.0.1.3680043.8.498.10946315660146482634981959873943814294.jpg',
'1.2.826.0.1.3680043.8.498.45798006531333108466716393386227122798.jpg',
'1.2.826.0.1.3680043.8.498.10836557587492377579396054009436986711.jpg',
'1.2.826.0.1.3680043.8.498.47189172054514574409147503165974428818.jpg',
'1.2.826.0.1.3680043.8.498.10686260729298994864461343059724464660.jpg',
'1.2.826.0.1.3680043.8.498.12060733217842120259756770380336807236.jpg',
'1.2.826.0.1.3680043.8.498.10083227715434855265344997618519145359.jpg',
'1.2.826.0.1.3680043.8.498.17097319013071776775603906453186583173.jpg',
'1.2.826.0.1.3680043.8.498.65428547845001998381573764351016415738.jpg',
'1.2.826.0.1.3680043.8.498.90180575226368594897044389485132482717.jpg',
'1.2.826.0.1.3680043.8.498.51577949955932347890374500244961910346.jpg',
'1.2.826.0.1.3680043.8.498.99377784633373958896673962801486767508.jpg',
'1.2.826.0.1.3680043.8.498.61809571976452768857702615778934131064.jpg',
'1.2.826.0.1.3680043.8.498.73594026743130377079398573172331130837.jpg',
'1.2.826.0.1.3680043.8.498.20610782794298517391685637953407405698.jpg',
'1.2.826.0.1.3680043.8.498.17660253991397906822070673386290497017.jpg',
'1.2.826.0.1.3680043.8.498.84939523501306668183596702990832330453.jpg',
'1.2.826.0.1.3680043.8.498.19978862773839461464494835208810864569.jpg',
'1.2.826.0.1.3680043.8.498.64276174690205728728613674556123746223.jpg',
'1.2.826.0.1.3680043.8.498.12617878736694688314418470000862199217.jpg',
'1.2.826.0.1.3680043.8.498.24310276724132257153009243084755460068.jpg',
'1.2.826.0.1.3680043.8.498.84339841979790176797370542316285340621.jpg',
'1.2.826.0.1.3680043.8.498.12561121489578487614358439168037868522.jpg',
'1.2.826.0.1.3680043.8.498.82843875331234299681225077619365930768.jpg',
'1.2.826.0.1.3680043.8.498.99419329617359404390003185487056516884.jpg',
'1.2.826.0.1.3680043.8.498.66853917004458792532726987807994690556.jpg',
'1.2.826.0.1.3680043.8.498.76402314726065631164220528758811621102.jpg'

1.2.826.0.1.3680043.8.498.7049251472090305110422052873881021192.jpg',
'1.2.826.0.1.3680043.8.498.13000193898689297445218164919507468966.jpg',
'1.2.826.0.1.3680043.8.498.1024650193399593648104469760943580143.jpg',
'1.2.826.0.1.3680043.8.498.25011331801509963824078936711905812577.jpg',
'1.2.826.0.1.3680043.8.498.77301303214162257666942081356402843634.jpg',
'1.2.826.0.1.3680043.8.498.39514974896604162584027232086994947693.jpg',
'1.2.826.0.1.3680043.8.498.11795763953605519412045570326455733649.jpg',
'1.2.826.0.1.3680043.8.498.91930412341541147308916611618348558005.jpg',
'1.2.826.0.1.3680043.8.498.12234665206761198402210881485512551492.jpg',
'1.2.826.0.1.3680043.8.498.10488105078395252036779152715685992874.jpg',
'1.2.826.0.1.3680043.8.498.44205706476209808676742675602236960581.jpg',
'1.2.826.0.1.3680043.8.498.93556865109531283984463583507477022796.jpg',
'1.2.826.0.1.3680043.8.498.11694181190147435116042196979444505946.jpg',
'1.2.826.0.1.3680043.8.498.21605449270637685743559880263826452708.jpg',
'1.2.826.0.1.3680043.8.498.57387072299912009275381816774872558818.jpg',
'1.2.826.0.1.3680043.8.498.28132143085938494574521815068574327657.jpg',
'1.2.826.0.1.3680043.8.498.43894304054275951364424526293865194206.jpg',
'1.2.826.0.1.3680043.8.498.12240364620248061611974869145427391135.jpg',
'1.2.826.0.1.3680043.8.498.12081115207703122108774156212292333476.jpg',
'1.2.826.0.1.3680043.8.498.50654708560376255679730085653371718019.jpg',
'1.2.826.0.1.3680043.8.498.11724692884031176912721971652114873495.jpg',
'1.2.826.0.1.3680043.8.498.12475129002860593592565743956886726560.jpg',
'1.2.826.0.1.3680043.8.498.98788098254769981224076685025427383565.jpg',
'1.2.826.0.1.3680043.8.498.94998327893098816019420089901570497451.jpg',
'1.2.826.0.1.3680043.8.498.63026467393675520181120250231332684888.jpg',
'1.2.826.0.1.3680043.8.498.77078061458437898960488256084111808445.jpg',
'1.2.826.0.1.3680043.8.498.43148634253652776687624932397200988456.jpg',
'1.2.826.0.1.3680043.8.498.25991027013391423564970730543831656518.jpg',
'1.2.826.0.1.3680043.8.498.48499280780546560746131556350357958387.jpg',
'1.2.826.0.1.3680043.8.498.20406162780802463823028877837568112626.jpg',
'1.2.826.0.1.3680043.8.498.13239527774225681629857225213214290215.jpg',
'1.2.826.0.1.3680043.8.498.65754199346368650891662283276690179531.jpg',
'1.2.826.0.1.3680043.8.498.12259824389722274702565411161655182045.jpg',
'1.2.826.0.1.3680043.8.498.11367425297228489335892439413680772313.jpg',
'1.2.826.0.1.3680043.8.498.25758687135489310042999489878609727384.jpg',
'1.2.826.0.1.3680043.8.498.78854210900521920993671950084046948625.jpg',
'1.2.826.0.1.3680043.8.498.17176966024620183831992965868503684024.jpg',
'1.2.826.0.1.3680043.8.498.91230271087491797711534324447688261948.jpg',
'1.2.826.0.1.3680043.8.498.36245295633243500638269064067814258091.jpg',
'1.2.826.0.1.3680043.8.498.12058127799362411657312006542085094404.jpg',
'1.2.826.0.1.3680043.8.498.27448044256912869856499607935988804850.jpg',
'1.2.826.0.1.3680043.8.498.92346044940481053670973987722867427355.jpg',
'1.2.826.0.1.3680043.8.498.12569078568497047920609849461137943331.jpg',
'1.2.826.0.1.3680043.8.498.71220172190076666584059125496194010134.jpg',
'1.2.826.0.1.3680043.8.498.12244051162278090404061254117230458330.jpg',
'1.2.826.0.1.3680043.8.498.12937780815096081996210348483435339134.jpg',
'1.2.826.0.1.3680043.8.498.39633406267509395619841518449472239688.jpg',
'1.2.826.0.1.3680043.8.498.10410475934472775843850641678697833261.jpg',
'1.2.826.0.1.3680043.8.498.44556769822545610605212497643309291369.jpg',
'1.2.826.0.1.3680043.8.498.10572345269368094686053391847941465059.jpg',
'1.2.826.0.1.3680043.8.498.21020531488565003637814381748998396251.jpg',
'1.2.826.0.1.3680043.8.498.83141372692903598497419223477117049931.jpg',
'1.2.826.0.1.3680043.8.498.34324378643957919089495074087760279897.jpg',
'1.2.826.0.1.3680043.8.498.18514815973148180545440475593287617024.jpg',
'1.2.826.0.1.3680043.8.498.13213190373596768321015219639773796107.jpg',
'1.2.826.0.1.3680043.8.498.54273241946569477201684918069530133356.jpg',
'1.2.826.0.1.3680043.8.498.29716902516908056798429978810209406358.jpg',
'1.2.826.0.1.3680043.8.498.10519357295420963066550224701249670588.jpg',
'1.2.826.0.1.3680043.8.498.82588480314564262034994265388006104589.jpg',
'1.2.826.0.1.3680043.8.498.11544385224224386275145616929439714019.jpg',
'1.2.826.0.1.3680043.8.498.76226396277106322926308395874549068443.jpg',
'1.2.826.0.1.3680043.8.498.21509396634037353465740384956882187079.jpg',
'1.2.826.0.1.3680043.8.498.20807190125845525764144591010909707989.jpg',
'1.2.826.0.1.3680043.8.498.11390326940092428383106428715286983124.jpg',
'1.2.826.0.1.3680043.8.498.60015525892850324210341290207687402956.jpg',
'1.2.826.0.1.3680043.8.498.52562215801106011190336505002627769338.jpg',
'1.2.826.0.1.3680043.8.498.48851434714301661659230522406439614952.jpg',
'1.2.826.0.1.3680043.8.498.20471817776811779838164792845607735881.jpg',
'1.2.826.0.1.3680043.8.498.21339215442803159672020412040390440359.jpg',
'1.2.826.0.1.3680043.8.498.18860634360723131458306793608663941686.jpg',
'1.2.826.0.1.3680043.8.498.10002446304107330308555550280339793610.jpg',
'1.2.826.0.1.3680043.8.498.12202887016615480252786101080477088024.jpg'

'1.2.826.0.1.3680043.8.498.13303887916615489353786181989477088924.jpg',
'1.2.826.0.1.3680043.8.498.10720117773668356272716426927429812321.jpg',
'1.2.826.0.1.3680043.8.498.12834937973850403791779394935997897611.jpg',
'1.2.826.0.1.3680043.8.498.22440070393391898613838133912396978244.jpg',
'1.2.826.0.1.3680043.8.498.51920777764905355713525506683397278691.jpg',
'1.2.826.0.1.3680043.8.498.21655544945131420727951567779351222742.jpg',
'1.2.826.0.1.3680043.8.498.58838775968032446425053982934275813140.jpg',
'1.2.826.0.1.3680043.8.498.60384765310483869449265839075722432710.jpg',
'1.2.826.0.1.3680043.8.498.59397415867803324495230424961939191950.jpg',
'1.2.826.0.1.3680043.8.498.89607677461738605013621356197180642892.jpg',
'1.2.826.0.1.3680043.8.498.10266271498334692666948478101682200297.jpg',
'1.2.826.0.1.3680043.8.498.91811302886617077892676020349944884309.jpg',
'1.2.826.0.1.3680043.8.498.11467889609833817207718260709647304670.jpg',
'1.2.826.0.1.3680043.8.498.11385757035151592493494471667892958381.jpg',
'1.2.826.0.1.3680043.8.498.90740149107735388017833598019160835236.jpg',
'1.2.826.0.1.3680043.8.498.13519000077114406220142392460877013014.jpg',
'1.2.826.0.1.3680043.8.498.40876515065869520936740772756308395819.jpg',
'1.2.826.0.1.3680043.8.498.10474262768464057796993863400337705357.jpg',
'1.2.826.0.1.3680043.8.498.57504843181807509148702460925223511336.jpg',
'1.2.826.0.1.3680043.8.498.43475917559674895734056863378511389173.jpg',
'1.2.826.0.1.3680043.8.498.62649176908847408796521409686293774601.jpg',
'1.2.826.0.1.3680043.8.498.12232937689022030488592427427712420880.jpg',
'1.2.826.0.1.3680043.8.498.69431064406314409503053990306006154317.jpg',
'1.2.826.0.1.3680043.8.498.12823119512103013576055323428358225606.jpg',
'1.2.826.0.1.3680043.8.498.25625109318696735303184708651507874302.jpg',
'1.2.826.0.1.3680043.8.498.49904538010912003444599985472597047706.jpg',
'1.2.826.0.1.3680043.8.498.10257347683526910556921218614296145978.jpg',
'1.2.826.0.1.3680043.8.498.26231711464364418992262100315034948700.jpg',
'1.2.826.0.1.3680043.8.498.12616703394901928073154828052071586966.jpg',
'1.2.826.0.1.3680043.8.498.44371910343069045083421726894218863768.jpg',
'1.2.826.0.1.3680043.8.498.20741375466052885670981323480722712503.jpg',
'1.2.826.0.1.3680043.8.498.12918412846626507592663383169481893873.jpg',
'1.2.826.0.1.3680043.8.498.11312029807809455256963584757252975540.jpg',
'1.2.826.0.1.3680043.8.498.10979893498647793922394130404890700345.jpg',
'1.2.826.0.1.3680043.8.498.64127060557361018142091700215411765684.jpg',
'1.2.826.0.1.3680043.8.498.58529203791589360084734185571785065707.jpg',
'1.2.826.0.1.3680043.8.498.11847654012393015714449448401286737085.jpg',
'1.2.826.0.1.3680043.8.498.60377594322603664909746176474261851319.jpg',
'1.2.826.0.1.3680043.8.498.43629640400789508883687064635120011102.jpg',
'1.2.826.0.1.3680043.8.498.25187213031738553025223253290151118178.jpg',
'1.2.826.0.1.3680043.8.498.98260853803315563907163727912117944849.jpg',
'1.2.826.0.1.3680043.8.498.12524676795566048844326252696261393733.jpg',
'1.2.826.0.1.3680043.8.498.30151510159973506801936265562837377373.jpg',
'1.2.826.0.1.3680043.8.498.22169853977971863607603123652731952758.jpg',
'1.2.826.0.1.3680043.8.498.61298782917958948344245461662493563720.jpg',
'1.2.826.0.1.3680043.8.498.13131315294502745778968068567429692539.jpg',
'1.2.826.0.1.3680043.8.498.83308638166764076582462949669227770996.jpg',
'1.2.826.0.1.3680043.8.498.71600316399877539266827074860093892073.jpg',
'1.2.826.0.1.3680043.8.498.88326240807506948712429568815457262762.jpg',
'1.2.826.0.1.3680043.8.498.73197192979181450686524535002923455129.jpg',
'1.2.826.0.1.3680043.8.498.10097665613196014916879910659987090537.jpg',
'1.2.826.0.1.3680043.8.498.58499812405365585486405747420432752294.jpg',
'1.2.826.0.1.3680043.8.498.11200525975420464178411349155263142378.jpg',
'1.2.826.0.1.3680043.8.498.66344124022147823786116377160779281536.jpg',
'1.2.826.0.1.3680043.8.498.74857361010807442497279697191516556759.jpg',
'1.2.826.0.1.3680043.8.498.88619734558286397694712749343519550704.jpg',
'1.2.826.0.1.3680043.8.498.93128839017968672425060624949480489756.jpg',
'1.2.826.0.1.3680043.8.498.13434348370889823502473217527168399444.jpg',
'1.2.826.0.1.3680043.8.498.98815288002933434290642075419231800065.jpg',
'1.2.826.0.1.3680043.8.498.10870045259235031764959503824056806538.jpg',
'1.2.826.0.1.3680043.8.498.87007989088351854183504297354829526924.jpg',
'1.2.826.0.1.3680043.8.498.11111270699385727384099484271535615506.jpg',
'1.2.826.0.1.3680043.8.498.12656928813467801197498597575923338945.jpg',
'1.2.826.0.1.3680043.8.498.38466803339627913875409718128344692030.jpg',
'1.2.826.0.1.3680043.8.498.11539645593046462866693416266406515599.jpg',
'1.2.826.0.1.3680043.8.498.47425394958331616747477785636847932948.jpg',
'1.2.826.0.1.3680043.8.498.66941565807538361326745536331742680154.jpg',
'1.2.826.0.1.3680043.8.498.12216645945914319847480978290394454811.jpg',
'1.2.826.0.1.3680043.8.498.36734737982312382833302573253917471184.jpg',
'1.2.826.0.1.3680043.8.498.55764757528039257870349587105244672050.jpg',
'1.2.826.0.1.3680043.8.498.38918435774500601695568572218755016298.jpg',
'1.2.826.0.1.3680043.8.498.13303887916615489353786181989477088924.jpg',
'1.2.826.0.1.3680043.8.498.10720117773668356272716426927429812321.jpg',
'1.2.826.0.1.3680043.8.498.12834937973850403791779394935997897611.jpg',
'1.2.826.0.1.3680043.8.498.22440070393391898613838133912396978244.jpg',
'1.2.826.0.1.3680043.8.498.51920777764905355713525506683397278691.jpg',
'1.2.826.0.1.3680043.8.498.21655544945131420727951567779351222742.jpg',
'1.2.826.0.1.3680043.8.498.58838775968032446425053982934275813140.jpg',
'1.2.826.0.1.3680043.8.498.60384765310483869449265839075722432710.jpg',
'1.2.826.0.1.3680043.8.498.59397415867803324495230424961939191950.jpg',
'1.2.826.0.1.3680043.8.498.89607677461738605013621356197180642892.jpg',
'1.2.826.0.1.3680043.8.498.10266271498334692666948478101682200297.jpg',
'1.2.826.0.1.3680043.8.498.91811302886617077892676020349944884309.jpg',
'1.2.826.0.1.3680043.8.498.11467889609833817207718260709647304670.jpg',
'1.2.826.0.1.3680043.8.498.11385757035151592493494471667892958381.jpg',
'1.2.826.0.1.3680043.8.498.90740149107735388017833598019160835236.jpg',
'1.2.826.0.1.3680043.8.498.13519000077114406220142392460877013014.jpg',
'1.2.826.0.1.3680043.8.498.40876515065869520936740772756308395819.jpg',
'1.2.826.0.1.3680043.8.498.10474262768464057796993863400337705357.jpg',
'1.2.826.0.1.3680043.8.498.57504843181807509148702460925223511336.jpg',
'1.2.826.0.1.3680043.8.498.43475917559674895734056863378511389173.jpg',
'1.2.826.0.1.3680043.8.498.62649176908847408796521409686293774601.jpg',
'1.2.826.0.1.3680043.8.498.12232937689022030488592427427712420880.jpg',
'1.2.826.0.1.3680043.8.498.69431064406314409503053990306006154317.jpg',
'1.2.826.0.1.3680043.8.498.12823119512103013576055323428358225606.jpg',
'1.2.826.0.1.3680043.8.498.25625109318696735303184708651507874302.jpg',
'1.2.826.0.1.3680043.8.498.49904538010912003444599985472597047706.jpg',
'1.2.826.0.1.3680043.8.498.10257347683526910556921218614296145978.jpg',
'1.2.826.0.1.3680043.8.498.26231711464364418992262100315034948700.jpg',
'1.2.826.0.1.3680043.8.498.12616703394901928073154828052071586966.jpg',
'1.2.826.0.1.3680043.8.498.44371910343069045083421726894218863768.jpg',
'1.2.826.0.1.3680043.8.498.20741375466052885670981323480722712503.jpg',
'1.2.826.0.1.3680043.8.498.12918412846626507592663383169481893873.jpg',
'1.2.826.0.1.3680043.8.498.11312029807809455256963584757252975540.jpg',
'1.2.826.0.1.3680043.8.498.10979893498647793922394130404890700345.jpg',
'1.2.826.0.1.3680043.8.498.64127060557361018142091700215411765684.jpg',
'1.2.826.0.1.3680043.8.498.58529203791589360084734185571785065707.jpg',
'1.2.826.0.1.3680043.8.498.11847654012393015714449448401286737085.jpg',
'1.2.826.0.1.3680043.8.498.60377594322603664909746176474261851319.jpg',
'1.2.826.0.1.3680043.8.498.43629640400789508883687064635120011102.jpg',
'1.2.826.0.1.3680043.8.498.25187213031738553025223253290151118178.jpg',
'1.2.826.0.1.3680043.8.498.98260853803315563907163727912117944849.jpg',
'1.2.826.0.1.3680043.8.498.12524676795566048844326252696261393733.jpg',
'1.2.826.0.1.3680043.8.498.30151510159973506801936265562837377373.jpg',
'1.2.826.0.1.3680043.8.498.22169853977971863607603123652731952758.jpg',
'1.2.826.0.1.3680043.8.498.61298782917958948344245461662493563720.jpg',
'1.2.826.0.1.3680043.8.498.13131315294502745778968068567429692539.jpg',
'1.2.826.0.1.3680043.8.498.83308638166764076582462949669227770996.jpg',
'1.2.826.0.1.3680043.8.498.71600316399877539266827074860093892073.jpg',
'1.2.826.0.1.3680043.8.498.88326240807506948712429568815457262762.jpg',
'1.2.826.0.1.3680043.8.498.73197192979181450686524535002923455129.jpg',
'1.2.826.0.1.3680043.8.498.10097665613196014916879910659987090537.jpg',
'1.2.826.0.1.3680043.8.498.58499812405365585486405747420432752294.jpg',
'1.2.826.0.1.3680043.8.498.11200525975420464178411349155263142378.jpg',
'1.2.826.0.1.3680043.8.498.66344124022147823786116377160779281536.jpg',
'1.2.826.0.1.3680043.8.498.74857361010807442497279697191516556759.jpg',
'1.2.826.0.1.3680043.8.498.88619734558286397694712749343519550704.jpg',
'1.2.826.0.1.3680043.8.498.93128839017968672425060624949480489756.jpg',
'1.2.826.0.1.3680043.8.498.13434348370889823502473217527168399444.jpg',
'1.2.826.0.1.3680043.8.498.98815288002933434290642075419231800065.jpg',
'1.2.826.0.1.3680043.8.498.10870045259235031764959503824056806538.jpg',
'1.2.826.0.1.3680043.8.498.87007989088351854183504297354829526924.jpg',
'1.2.826.0.1.3680043.8.498.11111270699385727384099484271535615506.jpg',
'1.2.826.0.1.3680043.8.498.12656928813467801197498597575923338945.jpg',
'1.2.826.0.1.3680043.8.498.38466803339627913875409718128344692030.jpg',
'1.2.826.0.1.3680043.8.498.11539645593046462866693416266406515599.jpg',
'1.2.826.0.1.3680043.8.498.47425394958331616747477785636847932948.jpg',
'1.2.826.0.1.3680043.8.498.66941565807538361326745536331742680154.jpg',
'1.2.826.0.1.3680043.8.498.12216645945914319847480978290394454811.jpg',
'1.2.826.0.1.3680043.8.498.36734737982312382833302573253917471184.jpg',
'1.2.826.0.1.3680043.8.498.55764757528039257870349587105244672050.jpg',
'1.2.826.0.1.3680043.8.498.38918435774500601695568572218755016298.jpg',
'1.2.826.0.1.3680043.8.498.13303887916615489353786181989477088924.jpg',
'1.2.826.0.1.3680043.8.498.10720117773668356272716426927429812321.jpg',
'1.2.826.0.1.3680043.8.498.12834937973850403791779394935997897611.jpg',
'1.2.826.0.1.3680043.8.498.22440070393391898613838133912396978244.jpg',
'1.2.826.0.1.3680043.8.498.51920777764905355713525506683397278691.jpg',
'1.2.826.0.1.3680043.8.498.21655544945131420727951567779351222742.jpg',
'1.2.826.0.1.3680043.8.498.58838775968032446425053982934275813140.jpg',
'1.2.826.0.1.3680043.8.498.60384765310483869449265839075722432710.jpg',
'1.2.826.0.1.3680043.8.498.59397415867803324495230424961939191950.jpg',
'1.2.826.0.1.3680043.8.498.89607677461738605013621356197180642892.jpg',
'1.2.826.0.1.3680043.8.498.10266271498334692666948478101682200297.jpg',
'1.2.826.0.1.3680043.8.498.91811302886617077892676020349944884309.jpg',
'1.2.826.0.1.3680043.8.498.11467889609833817207718260709647304670.jpg',
'1.2.826.0.1.3680043.8.498.11385757035151592493494471667892958381.jpg',
'1.2.826.0.1.3680043.8.498.90740149107735388017833598019160835236.jpg',
'1.2.826.0.1.3680043.8.498.13519000077114406220142392460877013014.jpg',
'1.2.826.0.1.3680043.8.498.40876515065869520936740772756308395819.jpg',
'1.2.826.0.1.3680043.8.498.10474262768464057796993863400337705357.jpg',
'1.2.826.0.1.3680043.8.498.57504843181807509148702460925223511336.jpg',
'1.2.826.0.1.3680043.8.498.43475917559674895734056863378511389173.jpg',
'1.2.826.0.1.3680043.8.498.62649176908847408796521409686293774601.jpg',
'1.2.826.0.1.3680043.8.498.12232937689022030488592427427712420880.jpg',
'1.2.826.0.1.3680043.8.498.69431064406314409503053990306006154317.jpg',
'1.2.826.0.1.3680043.8.498.12823119512103013576055323428358225606.jpg',
'1.2.826.0.1.3680043.8.498.25625109318696735303184708651507874302.jpg',
'1.2.826.0.1.3680043.8.498.49904538010912003444599985472597047706.jpg',
'1.2.826.0.1.3680043.8.498.10257347683526910556921218614296145978.jpg',
'1.2.826.0.1.3680043.8.498.26231711464364418992262100315034948700.jpg',
'1.2.826.0.1.3680043.8.498.12616703394901928073154828052071586966.jpg',
'1.2.826.0.1.3680043.8.498.44371910343069045083421726894218863768.jpg',
'1.2.826.0.1.3680043.8.498.20741375466052885670981323480722712503.jpg',
'1.2.826.0.1.3680043.8.498.12918412846626507592663383169481893873.jpg',
'1.2.826.0.1.3680043.8.498.11312029807809455256963584757252975540.jpg',
'1.2.826.0.1.3680043.8.498.10979893498647793922394130404890700345.jpg',
'1.2.826.0.1.3680043.8.498.641270605573

1.2.826.0.1.3680043.8.498.15931312148291914712240378922308466852.jpg',
'1.2.826.0.1.3680043.8.498.68947530961431327995685809909216702822.jpg',
'1.2.826.0.1.3680043.8.498.53972838567758557924633425561175170990.jpg',
'1.2.826.0.1.3680043.8.498.29741239000332475170638163118364245882.jpg',
'1.2.826.0.1.3680043.8.498.50677882474739133485233341143548323254.jpg',
'1.2.826.0.1.3680043.8.498.80574306898149045942518692007219873480.jpg',
'1.2.826.0.1.3680043.8.498.10735812453426800406977113811971042185.jpg',
'1.2.826.0.1.3680043.8.498.11168413473675485525431970689181276493.jpg',
'1.2.826.0.1.3680043.8.498.12894873953982055258284931678466300883.jpg',
'1.2.826.0.1.3680043.8.498.12697468334484127910694665733313262980.jpg',
'1.2.826.0.1.3680043.8.498.38109098949985981736613969334693229493.jpg',
'1.2.826.0.1.3680043.8.498.13315204945129975526376734089652313269.jpg',
'1.2.826.0.1.3680043.8.498.76559671062023300251661629560732089635.jpg',
'1.2.826.0.1.3680043.8.498.19923702811484883288782948939832269481.jpg',
'1.2.826.0.1.3680043.8.498.10198811575816374012076202215712905926.jpg',
'1.2.826.0.1.3680043.8.498.52359369730856697834711102873212898959.jpg',
'1.2.826.0.1.3680043.8.498.18937818371940465440963031535417415340.jpg',
'1.2.826.0.1.3680043.8.498.71015619396910727206799985183991133558.jpg',
'1.2.826.0.1.3680043.8.498.51389622482633003466716195457359210999.jpg',
'1.2.826.0.1.3680043.8.498.42681844534848074059015399377551184954.jpg',
'1.2.826.0.1.3680043.8.498.11039176369512210331637148073185893661.jpg',
'1.2.826.0.1.3680043.8.498.80058141917130703975834534533058558900.jpg',
'1.2.826.0.1.3680043.8.498.81418065673026746066238372325524734983.jpg',
'1.2.826.0.1.3680043.8.498.11340751421886204548699441534989158287.jpg',
'1.2.826.0.1.3680043.8.498.97444427792035375441391986386524029654.jpg',
'1.2.826.0.1.3680043.8.498.10758448290157477120455044067195682930.jpg',
'1.2.826.0.1.3680043.8.498.69324912005988735062015666170775686431.jpg',
'1.2.826.0.1.3680043.8.498.47152742927349286796091202256925910535.jpg',
'1.2.826.0.1.3680043.8.498.10525734373863100548327671495507197396.jpg',
'1.2.826.0.1.3680043.8.498.10972428118463139207171250260364898269.jpg',
'1.2.826.0.1.3680043.8.498.81534736579289279921670994558702647620.jpg',
'1.2.826.0.1.3680043.8.498.39947551746122458589630674328138454312.jpg',
'1.2.826.0.1.3680043.8.498.13382753622100514390530610038120944510.jpg',
'1.2.826.0.1.3680043.8.498.54308003497429441219078968469868119805.jpg',
'1.2.826.0.1.3680043.8.498.11622885097278601053720034625355797318.jpg',
'1.2.826.0.1.3680043.8.498.67798274902194944247854906270081058289.jpg',
'1.2.826.0.1.3680043.8.498.68905613786950624932357839352114147643.jpg',
'1.2.826.0.1.3680043.8.498.87804971431900060391989554149850083900.jpg',
'1.2.826.0.1.3680043.8.498.74190045333565570017428818377245264093.jpg',
'1.2.826.0.1.3680043.8.498.15140219985596387264131363506522015559.jpg',
'1.2.826.0.1.3680043.8.498.81433270261453474452306837934953232693.jpg',
'1.2.826.0.1.3680043.8.498.94913069401380596298704095517435874582.jpg',
'1.2.826.0.1.3680043.8.498.10020144805733220809675178506169339046.jpg',
'1.2.826.0.1.3680043.8.498.39837938167953014003149142705075211033.jpg',
'1.2.826.0.1.3680043.8.498.10424558672010570346473840895233424637.jpg',
'1.2.826.0.1.3680043.8.498.56963597078538105813500234584506159085.jpg',
'1.2.826.0.1.3680043.8.498.26776784290468536104531525590724289504.jpg',
'1.2.826.0.1.3680043.8.498.12769254282572995712804395725636328346.jpg',
'1.2.826.0.1.3680043.8.498.79581992205627026618193794909366846264.jpg',
'1.2.826.0.1.3680043.8.498.70332450543674022614719924407628495164.jpg',
'1.2.826.0.1.3680043.8.498.13261084033434209362112671441132682228.jpg',
'1.2.826.0.1.3680043.8.498.12548878681742953521618647571821848099.jpg',
'1.2.826.0.1.3680043.8.498.82446052265170667084608618174732706079.jpg',
'1.2.826.0.1.3680043.8.498.40087594870190704875808445770367253034.jpg',
'1.2.826.0.1.3680043.8.498.80075763966903465802988353073871382997.jpg',
'1.2.826.0.1.3680043.8.498.42072605886034933767606576473953712613.jpg',
'1.2.826.0.1.3680043.8.498.12965172326722814424720180263675220119.jpg',
'1.2.826.0.1.3680043.8.498.47406940617398055783931564486066577034.jpg',
'1.2.826.0.1.3680043.8.498.39455408555283891094727931915886223924.jpg',
'1.2.826.0.1.3680043.8.498.11113705966995108397899971335588347515.jpg',
'1.2.826.0.1.3680043.8.498.63441207389617607608803535296811572314.jpg',
'1.2.826.0.1.3680043.8.498.66725076500298357765208291144256922546.jpg',
'1.2.826.0.1.3680043.8.498.92419709942581198072159836382110480577.jpg',
'1.2.826.0.1.3680043.8.498.48309512752259813431983520766133432267.jpg',
'1.2.826.0.1.3680043.8.498.14820104998871120229486671929353597090.jpg',
'1.2.826.0.1.3680043.8.498.10117706278084856399014858720725184831.jpg',
'1.2.826.0.1.3680043.8.498.98471802486274130357290440122883501478.jpg',
'1.2.826.0.1.3680043.8.498.42419165276185525761261699890386725202.jpg',
'1.2.826.0.1.3680043.8.498.99239526861661932223480718831802524607.jpg',
'1.2.826.0.1.3680043.8.498.47463930704154478021063311914497102284.jpg',
'1.2.826.0.1.3680043.8.498.15649072787495613711344474851728962871.jpg',

'1.2.826.0.1.3680043.8.498.11078024989580826220856845021804379024.jpg',
'1.2.826.0.1.3680043.8.498.39184639522873645728960457534766585811.jpg',
'1.2.826.0.1.3680043.8.498.10814374807313534439338105365614091084.jpg',
'1.2.826.0.1.3680043.8.498.10439448554537796997251242047106750172.jpg',
'1.2.826.0.1.3680043.8.498.44330101010727304616612534529372543480.jpg',
'1.2.826.0.1.3680043.8.498.81708454948738191558533422701223894086.jpg',
'1.2.826.0.1.3680043.8.498.90235447763131452799722661384469134213.jpg',
'1.2.826.0.1.3680043.8.498.13839250663796719986792230722609481978.jpg',
'1.2.826.0.1.3680043.8.498.12473031475309922659169504546744454953.jpg',
'1.2.826.0.1.3680043.8.498.22094507760933946944282987377477530036.jpg',
'1.2.826.0.1.3680043.8.498.81727313586288439162499604047325759611.jpg',
'1.2.826.0.1.3680043.8.498.12035496198910189553182768431231025462.jpg',
'1.2.826.0.1.3680043.8.498.54141070107733427792645350316474009709.jpg',
'1.2.826.0.1.3680043.8.498.90705357574177820870396426126859924153.jpg',
'1.2.826.0.1.3680043.8.498.21822774183930987680442935704705392029.jpg',
'1.2.826.0.1.3680043.8.498.11610016449788130163176304801449497134.jpg',
'1.2.826.0.1.3680043.8.498.10222683376039301560941940410824203329.jpg',
'1.2.826.0.1.3680043.8.498.67619649138693277648412348972171062294.jpg',
'1.2.826.0.1.3680043.8.498.98597202070890408502455675642430140286.jpg',
'1.2.826.0.1.3680043.8.498.96881461622506111902649405770272396165.jpg',
'1.2.826.0.1.3680043.8.498.46477314832315639160115283142402533193.jpg',
'1.2.826.0.1.3680043.8.498.73530218507609695400395126952702316515.jpg',
'1.2.826.0.1.3680043.8.498.55786818369238581159207938419965753355.jpg',
'1.2.826.0.1.3680043.8.498.16890932664036028305140485960352461722.jpg',
'1.2.826.0.1.3680043.8.498.80164399745291754538360656486711366075.jpg',
'1.2.826.0.1.3680043.8.498.72646697080370419963141342977324038476.jpg',
'1.2.826.0.1.3680043.8.498.68872042113456303992363480069940211359.jpg',
'1.2.826.0.1.3680043.8.498.67752822041970665855415706021150112996.jpg',
'1.2.826.0.1.3680043.8.498.51401636150694465027548691147190972612.jpg',
'1.2.826.0.1.3680043.8.498.85887293669850363397834085367878785326.jpg',
'1.2.826.0.1.3680043.8.498.56736332803287254782815412447727557898.jpg',
'1.2.826.0.1.3680043.8.498.11750504527122124160918961938536152456.jpg',
'1.2.826.0.1.3680043.8.498.12064491823385324471625346721703417328.jpg',
'1.2.826.0.1.3680043.8.498.10271099777698188466653716551944869017.jpg',
'1.2.826.0.1.3680043.8.498.23864420702382820871159615797354095924.jpg',
'1.2.826.0.1.3680043.8.498.37597116583175510535079213261857502521.jpg',
'1.2.826.0.1.3680043.8.498.23660756376926689093791496280854557859.jpg',
'1.2.826.0.1.3680043.8.498.14608891556538620321972966240365252518.jpg',
'1.2.826.0.1.3680043.8.498.97898517431692420201214809980286091995.jpg',
'1.2.826.0.1.3680043.8.498.78157744043845712068253354386577753631.jpg',
'1.2.826.0.1.3680043.8.498.98510573816561719642104662914830291596.jpg',
'1.2.826.0.1.3680043.8.498.43254836743778316549797396408275802972.jpg',
'1.2.826.0.1.3680043.8.498.74199743439338923092877609445041031483.jpg',
'1.2.826.0.1.3680043.8.498.12198074630516664831091515756690894548.jpg',
'1.2.826.0.1.3680043.8.498.52677577817985380649316161608894652374.jpg',
'1.2.826.0.1.3680043.8.498.92574966420276716414097941675326650066.jpg',
'1.2.826.0.1.3680043.8.498.19266170093754171925829738833926726452.jpg',
'1.2.826.0.1.3680043.8.498.78851348262676480351066045263999245838.jpg',
'1.2.826.0.1.3680043.8.498.28770911096404663949601724412153169072.jpg',
'1.2.826.0.1.3680043.8.498.17140835188607356171938302667736606673.jpg',
'1.2.826.0.1.3680043.8.498.12668583918549413729614042427490356678.jpg',
'1.2.826.0.1.3680043.8.498.92214565786453420373913506268676895891.jpg',
'1.2.826.0.1.3680043.8.498.86671816594522472921437309429001056520.jpg',
'1.2.826.0.1.3680043.8.498.12408098681851682481985000790037614068.jpg',
'1.2.826.0.1.3680043.8.498.96734002239095661987027532364900693190.jpg',
'1.2.826.0.1.3680043.8.498.49228274508019517407152542064582167550.jpg',
'1.2.826.0.1.3680043.8.498.81864059525802417726289264019734609330.jpg',
'1.2.826.0.1.3680043.8.498.11550531826677380700263976849104439388.jpg',
'1.2.826.0.1.3680043.8.498.44414380630436735637535362652829935040.jpg',
'1.2.826.0.1.3680043.8.498.85798461004802263886488355135210734576.jpg',
'1.2.826.0.1.3680043.8.498.61406352116624405274362223324973143177.jpg',
'1.2.826.0.1.3680043.8.498.76856151269385666914190125858546562220.jpg',
'1.2.826.0.1.3680043.8.498.49818895103545161083662593970268836167.jpg',
'1.2.826.0.1.3680043.8.498.96288992824088941155047804959520382815.jpg',
'1.2.826.0.1.3680043.8.498.12494716084926326587581136239976675142.jpg',
'1.2.826.0.1.3680043.8.498.28918218295057161635077338836980466876.jpg',
'1.2.826.0.1.3680043.8.498.11245459627194481071423349770702911662.jpg',
'1.2.826.0.1.3680043.8.498.62231838292704551369968716805706794942.jpg',
'1.2.826.0.1.3680043.8.498.11518810910546278225718335906698010523.jpg',
'1.2.826.0.1.3680043.8.498.49325657494038471931768761924475387667.jpg',
'1.2.826.0.1.3680043.8.498.13795009936447085576558135262118950224.jpg',

'1.2.826.0.1.3680043.8.498.58010035098816868908505466159230606286.jpg',
'1.2.826.0.1.3680043.8.498.11129696639319701381585134974114248289.jpg',
'1.2.826.0.1.3680043.8.498.95745833750574310851379896375695778722.jpg',
'1.2.826.0.1.3680043.8.498.87252429674445355082110555897592649920.jpg',
'1.2.826.0.1.3680043.8.498.86925394054343816856580575230864267838.jpg',
'1.2.826.0.1.3680043.8.498.14812260466837537048832178464899934784.jpg',
'1.2.826.0.1.3680043.8.498.11437406748617066836469925204820231294.jpg',
'1.2.826.0.1.3680043.8.498.57724222218344448509691821370290772844.jpg',
'1.2.826.0.1.3680043.8.498.35762392761957651553181286870593608706.jpg',
'1.2.826.0.1.3680043.8.498.61568104110168313556095749848893263352.jpg',
'1.2.826.0.1.3680043.8.498.63899596215378697815675019964994707517.jpg',
'1.2.826.0.1.3680043.8.498.71336949093450099705416524721701144532.jpg',
'1.2.826.0.1.3680043.8.498.10684856675305775353953997263320567821.jpg',
'1.2.826.0.1.3680043.8.498.11374399746030343954906529276969234675.jpg',
'1.2.826.0.1.3680043.8.498.12249731978921299395102086427615711941.jpg',
'1.2.826.0.1.3680043.8.498.96764327362117228035287157841500780388.jpg',
'1.2.826.0.1.3680043.8.498.85809077656626255523361871181970865548.jpg',
'1.2.826.0.1.3680043.8.498.73215360544587945735978569570710445280.jpg',
'1.2.826.0.1.3680043.8.498.92905989267114815328283399869206243373.jpg',
'1.2.826.0.1.3680043.8.498.57964054194073285015024649761378402794.jpg',
'1.2.826.0.1.3680043.8.498.13355204868953725002445809577576496892.jpg',
'1.2.826.0.1.3680043.8.498.87871403233671050585223188271294314655.jpg',
'1.2.826.0.1.3680043.8.498.74469355101717731204450242911631246173.jpg',
'1.2.826.0.1.3680043.8.498.41883087427347094220506795608534035794.jpg',
'1.2.826.0.1.3680043.8.498.12431924418500391106472106857463395358.jpg',
'1.2.826.0.1.3680043.8.498.27960076109879313037734647928825472871.jpg',
'1.2.826.0.1.3680043.8.498.88236168117917883101536756025332726142.jpg',
'1.2.826.0.1.3680043.8.498.47030190045488210757143905206195883485.jpg',
'1.2.826.0.1.3680043.8.498.96583893231781564127075410275833320760.jpg',
'1.2.826.0.1.3680043.8.498.64578463849584714225989789438984798426.jpg',
'1.2.826.0.1.3680043.8.498.72191600158331484431489915235571345981.jpg',
'1.2.826.0.1.3680043.8.498.21626314997148109795190655913418965646.jpg',
'1.2.826.0.1.3680043.8.498.76563724867929440143198464236615836643.jpg',
'1.2.826.0.1.3680043.8.498.35233392316305415219868200468916654167.jpg',
'1.2.826.0.1.3680043.8.498.39188970453889321875482079142241345223.jpg',
'1.2.826.0.1.3680043.8.498.88269436517309599194375692514847193635.jpg',
'1.2.826.0.1.3680043.8.498.12978055090804518090096136702600288041.jpg',
'1.2.826.0.1.3680043.8.498.79548727783552383479110305481983665014.jpg',
'1.2.826.0.1.3680043.8.498.79583397337684636624675832421852887971.jpg',
'1.2.826.0.1.3680043.8.498.99038739563520886893330602528937238279.jpg',
'1.2.826.0.1.3680043.8.498.64643677658389119798269011958523545413.jpg',
'1.2.826.0.1.3680043.8.498.21847916031617684383947756524678477707.jpg',
'1.2.826.0.1.3680043.8.498.91853142063786622039125706998081544384.jpg',
'1.2.826.0.1.3680043.8.498.34573664982029740966216709847921043436.jpg',
'1.2.826.0.1.3680043.8.498.84703078783795249058278928201612907205.jpg',
'1.2.826.0.1.3680043.8.498.60181337572007385202288983758502060724.jpg',
'1.2.826.0.1.3680043.8.498.44654110520143517442684962065039067394.jpg',
'1.2.826.0.1.3680043.8.498.33970751517070368699811044163651939949.jpg',
'1.2.826.0.1.3680043.8.498.12769822764359741852576220030643089495.jpg',
'1.2.826.0.1.3680043.8.498.81015550761365103003947833288359604657.jpg',
'1.2.826.0.1.3680043.8.498.10111413832301737777500631887044787675.jpg',
'1.2.826.0.1.3680043.8.498.45197923752124128061537373509995229245.jpg',
'1.2.826.0.1.3680043.8.498.11316209953375691386354260827223225996.jpg',
'1.2.826.0.1.3680043.8.498.12300559672915671783963815131993664466.jpg',
'1.2.826.0.1.3680043.8.498.55260237650588247757071429645308393046.jpg',
'1.2.826.0.1.3680043.8.498.49414921675174113235788639521671111697.jpg',
'1.2.826.0.1.3680043.8.498.56783851573641752333982240507004368239.jpg',
'1.2.826.0.1.3680043.8.498.11112021771342236980428933149046718436.jpg',
'1.2.826.0.1.3680043.8.498.71715499018318673543158802157233565877.jpg',
'1.2.826.0.1.3680043.8.498.42093280948492732918474844392595643481.jpg',
'1.2.826.0.1.3680043.8.498.10969266877403386295241306761970625703.jpg',
'1.2.826.0.1.3680043.8.498.73319340904978868159058913026172271855.jpg',
'1.2.826.0.1.3680043.8.498.10678969622825692875248934036314654360.jpg',
'1.2.826.0.1.3680043.8.498.12376766380846553914811942507952332186.jpg',
'1.2.826.0.1.3680043.8.498.42332509795893008620618790050264827911.jpg',
'1.2.826.0.1.3680043.8.498.25858378340005804790129597717579920559.jpg',
'1.2.826.0.1.3680043.8.498.42196006246298938270012448240806216387.jpg',
'1.2.826.0.1.3680043.8.498.89176845563501225900249190784759827268.jpg',
'1.2.826.0.1.3680043.8.498.87595682175987025078222793749918953600.jpg',
'1.2.826.0.1.3680043.8.498.12164597187723723656799585228599222873.jpg',
'1.2.826.0.1.3680043.8.498.50069752788951166005957096189395950091.jpg',

'1.2.826.0.1.3680043.8.498.10710746490077274100315449353799614206.jpg',
'1.2.826.0.1.3680043.8.498.11587553311969771713948535079287077133.jpg',
'1.2.826.0.1.3680043.8.498.43658727069068649524345974141885318712.jpg',
'1.2.826.0.1.3680043.8.498.25032666441290222283867646163265893457.jpg',
'1.2.826.0.1.3680043.8.498.80201610996740563779710986726605083177.jpg',
'1.2.826.0.1.3680043.8.498.12561601421453391143393792961101960509.jpg',
'1.2.826.0.1.3680043.8.498.37608918916414248074604121273285694055.jpg',
'1.2.826.0.1.3680043.8.498.79952300015546112879746691324605416700.jpg',
'1.2.826.0.1.3680043.8.498.82039419103756253313235547265138035432.jpg',
'1.2.826.0.1.3680043.8.498.10824296541330631603253258388826800525.jpg',
'1.2.826.0.1.3680043.8.498.12308843311058252389667517078236269889.jpg',
'1.2.826.0.1.3680043.8.498.96158355049284579089677031356904656910.jpg',
'1.2.826.0.1.3680043.8.498.73146785334889132038827215582126783795.jpg',
'1.2.826.0.1.3680043.8.498.12276964431481217628438982812532368177.jpg',
'1.2.826.0.1.3680043.8.498.16564694969906988590490166771243138045.jpg',
'1.2.826.0.1.3680043.8.498.64695053419975696462375926169847459574.jpg',
'1.2.826.0.1.3680043.8.498.76947872445114113193576091628816599022.jpg',
'1.2.826.0.1.3680043.8.498.75375138619901196640698787674215422160.jpg',
'1.2.826.0.1.3680043.8.498.69581712680166862192987660408573989198.jpg',
'1.2.826.0.1.3680043.8.498.91149527422263328246221283351944373265.jpg',
'1.2.826.0.1.3680043.8.498.63237938123653620588453044280036826789.jpg',
'1.2.826.0.1.3680043.8.498.17080279434906746657843271247097378394.jpg',
'1.2.826.0.1.3680043.8.498.11570656132730709593463338474813599139.jpg',
'1.2.826.0.1.3680043.8.498.72368228381990399550293850835625431548.jpg',
'1.2.826.0.1.3680043.8.498.10328278303464020327951410030331303934.jpg',
'1.2.826.0.1.3680043.8.498.49572486652159468633367986789162007417.jpg',
'1.2.826.0.1.3680043.8.498.83567305802261875961079569728279572336.jpg',
'1.2.826.0.1.3680043.8.498.80240670347243354816700806145165209600.jpg',
'1.2.826.0.1.3680043.8.498.30435010245728794238752191589694177911.jpg',
'1.2.826.0.1.3680043.8.498.51677831986993270067828121152096933699.jpg',
'1.2.826.0.1.3680043.8.498.15803062392445733234290478507456240331.jpg',
'1.2.826.0.1.3680043.8.498.10857611179649529482694783291782721373.jpg',
'1.2.826.0.1.3680043.8.498.12422470538907238866378421142367895935.jpg',
'1.2.826.0.1.3680043.8.498.61797250716863349240424072936509626421.jpg',
'1.2.826.0.1.3680043.8.498.10486595294565392512899034773841090902.jpg',
'1.2.826.0.1.3680043.8.498.18869555080701304415024962764340315146.jpg',
'1.2.826.0.1.3680043.8.498.11720085682119016889836870564260163777.jpg',
'1.2.826.0.1.3680043.8.498.69418086162799059321875887848701987988.jpg',
'1.2.826.0.1.3680043.8.498.67242128764309129480882007367812779609.jpg',
'1.2.826.0.1.3680043.8.498.50146387142959002506237608655417456575.jpg',
'1.2.826.0.1.3680043.8.498.13605819361678404662539362558451446746.jpg',
'1.2.826.0.1.3680043.8.498.12739314658160716099047991925446495680.jpg',
'1.2.826.0.1.3680043.8.498.41866585391692122713071756460148229067.jpg',
'1.2.826.0.1.3680043.8.498.22675672242487038441464483461707191295.jpg',
'1.2.826.0.1.3680043.8.498.14840486993105356546549586387951480700.jpg',
'1.2.826.0.1.3680043.8.498.14003577210507616908102877824554154805.jpg',
'1.2.826.0.1.3680043.8.498.27465329122233469870689433167100854836.jpg',
'1.2.826.0.1.3680043.8.498.78851445969313527278449258220338602878.jpg',
'1.2.826.0.1.3680043.8.498.39636543257319180713682255821572619982.jpg',
'1.2.826.0.1.3680043.8.498.22603933280423024012656726107949106377.jpg',
'1.2.826.0.1.3680043.8.498.11921120804662576662755843317589994977.jpg',
'1.2.826.0.1.3680043.8.498.10198440752922915501323015609348643762.jpg',
'1.2.826.0.1.3680043.8.498.27814539480524984904777037440711403361.jpg',
'1.2.826.0.1.3680043.8.498.11606186336987177111675075502139923546.jpg',
'1.2.826.0.1.3680043.8.498.11159536818095175340083658369985005736.jpg',
'1.2.826.0.1.3680043.8.498.86329990956696458607443726512927192079.jpg',
'1.2.826.0.1.3680043.8.498.53756032699087681611633359025104015317.jpg',
'1.2.826.0.1.3680043.8.498.27239863735603536595285476641102542705.jpg',
'1.2.826.0.1.3680043.8.498.35054769143117620018353508751778667887.jpg',
'1.2.826.0.1.3680043.8.498.28495518124504893582980909214928556618.jpg',
'1.2.826.0.1.3680043.8.498.33708208254825295613708569843300508579.jpg',
'1.2.826.0.1.3680043.8.498.53546912683297856379122960201862902942.jpg',
'1.2.826.0.1.3680043.8.498.79418044269257998158820452096734922831.jpg',
'1.2.826.0.1.3680043.8.498.90330114153375934875084872567385121550.jpg',
'1.2.826.0.1.3680043.8.498.54166850380402629298329676239463338387.jpg',
'1.2.826.0.1.3680043.8.498.66238787061000743102931903538004735534.jpg',
'1.2.826.0.1.3680043.8.498.88959557036272687422679111599714307129.jpg',
'1.2.826.0.1.3680043.8.498.55070942335414408523893062021213526430.jpg',
'1.2.826.0.1.3680043.8.498.15820083988888929405175355802318021833.jpg',
'1.2.826.0.1.3680043.8.498.53981067269201305021014807627791468968.jpg',
'1.2.826.0.1.3680043.8.498.13247047302373932077069111993428752176.jpg',

'1.2.826.0.1.3680043.8.498.89665238180237553489273453820466146881.jpg',
'1.2.826.0.1.3680043.8.498.29239229328123989997551765421801520268.jpg',
'1.2.826.0.1.3680043.8.498.10473409061856177510574249430168423442.jpg',
'1.2.826.0.1.3680043.8.498.56095557588195171619730712348364435742.jpg',
'1.2.826.0.1.3680043.8.498.20575848684139417734558431163076353512.jpg',
'1.2.826.0.1.3680043.8.498.33241713380600829488390713928087438817.jpg',
'1.2.826.0.1.3680043.8.498.12212069198217768070588865702851339751.jpg',
'1.2.826.0.1.3680043.8.498.10958355061875820699827828669420206504.jpg',
'1.2.826.0.1.3680043.8.498.97863901569633288028864738347382224007.jpg',
'1.2.826.0.1.3680043.8.498.82030693571162250993341474603525165996.jpg',
'1.2.826.0.1.3680043.8.498.10092126960536614253862379880491750809.jpg',
'1.2.826.0.1.3680043.8.498.95806441305547998596015464255464100203.jpg',
'1.2.826.0.1.3680043.8.498.48353850655312071896701581794268965791.jpg',
'1.2.826.0.1.3680043.8.498.40223862755196768046218884884774162153.jpg',
'1.2.826.0.1.3680043.8.498.69639703766958619272977076158646308485.jpg',
'1.2.826.0.1.3680043.8.498.10171899161570208352656933618428664572.jpg',
'1.2.826.0.1.3680043.8.498.76356815045089154586487061431299294823.jpg',
'1.2.826.0.1.3680043.8.498.92973902578030245542713793205662473422.jpg',
'1.2.826.0.1.3680043.8.498.79289110832255343228589570659162678088.jpg',
'1.2.826.0.1.3680043.8.498.20079318868059516038949285700922725415.jpg',
'1.2.826.0.1.3680043.8.498.21949773638481591305689778112613388190.jpg',
'1.2.826.0.1.3680043.8.498.95247470835568001496834482861104764622.jpg',
'1.2.826.0.1.3680043.8.498.38849391559259863045578182323226781472.jpg',
'1.2.826.0.1.3680043.8.498.75475284521444722834728709829763556622.jpg',
'1.2.826.0.1.3680043.8.498.51016746524410364660513720342120052460.jpg',
'1.2.826.0.1.3680043.8.498.69230475044069917309172367248984420993.jpg',
'1.2.826.0.1.3680043.8.498.65941942125668869198583576153231833502.jpg',
'1.2.826.0.1.3680043.8.498.57664629540453990231758902976077222516.jpg',
'1.2.826.0.1.3680043.8.498.55479817136940213068810675684362641673.jpg',
'1.2.826.0.1.3680043.8.498.66282853620844100114798185141399438782.jpg',
'1.2.826.0.1.3680043.8.498.91689943123544859305734500891909483223.jpg',
'1.2.826.0.1.3680043.8.498.13320522277371450367516939135350097464.jpg',
'1.2.826.0.1.3680043.8.498.78567503470943633523191163424533080082.jpg',
'1.2.826.0.1.3680043.8.498.71952530618036681962442483923177497530.jpg',
'1.2.826.0.1.3680043.8.498.10229201673887959453375823349990497198.jpg',
'1.2.826.0.1.3680043.8.498.19254713702494896076930580783756932926.jpg',
'1.2.826.0.1.3680043.8.498.12933710057522815347299868890760574140.jpg',
'1.2.826.0.1.3680043.8.498.12604037263018452388498742040141411551.jpg',
'1.2.826.0.1.3680043.8.498.11202158984829786599879578022190673268.jpg',
'1.2.826.0.1.3680043.8.498.26483567019279259788024155674912944628.jpg',
'1.2.826.0.1.3680043.8.498.49069387549144530069853422352626237537.jpg',
'1.2.826.0.1.3680043.8.498.36343735211840713232409468859662477668.jpg',
'1.2.826.0.1.3680043.8.498.61050228492884028661730486694438444594.jpg',
'1.2.826.0.1.3680043.8.498.12384401624859909419039251988758384910.jpg',
'1.2.826.0.1.3680043.8.498.10533863396518656580245826580989141214.jpg',
'1.2.826.0.1.3680043.8.498.60893077507298255959383945935212161672.jpg',
'1.2.826.0.1.3680043.8.498.32164405475472575302228121138129715070.jpg',
'1.2.826.0.1.3680043.8.498.66328539023153767433813659007209755679.jpg',
'1.2.826.0.1.3680043.8.498.38741207512214785704203365503701957178.jpg',
'1.2.826.0.1.3680043.8.498.10452679147412500522552545486435104415.jpg',
'1.2.826.0.1.3680043.8.498.21727786100057032798048691720553952287.jpg',
'1.2.826.0.1.3680043.8.498.65148015415261732780626679886304067247.jpg',
'1.2.826.0.1.3680043.8.498.46289766348303047209649355036405722334.jpg',
'1.2.826.0.1.3680043.8.498.11300711336285275196560425669345677119.jpg',
'1.2.826.0.1.3680043.8.498.33496188291797486047850721414862968664.jpg',
'1.2.826.0.1.3680043.8.498.33281358252335075648531792505847887395.jpg',
'1.2.826.0.1.3680043.8.498.88743865108143008053345723016525426202.jpg',
'1.2.826.0.1.3680043.8.498.97681842091854566531334710480597716395.jpg',
'1.2.826.0.1.3680043.8.498.13527088783226582069746460702787450957.jpg',
'1.2.826.0.1.3680043.8.498.58417910281420266209374814060140535952.jpg',
'1.2.826.0.1.3680043.8.498.58115249384625178330183585351233948038.jpg',
'1.2.826.0.1.3680043.8.498.93293933448974028944777431831925469483.jpg',
'1.2.826.0.1.3680043.8.498.24607933983023495906580292991017270277.jpg',
'1.2.826.0.1.3680043.8.498.44425072218269019798744055134966360284.jpg',
'1.2.826.0.1.3680043.8.498.71429034639912009365217312905767164327.jpg',
'1.2.826.0.1.3680043.8.498.95744511881568964292652719900397668834.jpg',
'1.2.826.0.1.3680043.8.498.96311228469094629679821868712032248786.jpg',
'1.2.826.0.1.3680043.8.498.31390007914241662617693430523150312177.jpg',
'1.2.826.0.1.3680043.8.498.22224819670922746306896642234289575719.jpg',
'1.2.826.0.1.3680043.8.498.11947414383094977021120146998340264825.jpg',
'1.2.826.0.1.3680043.8.498.50295740239605680172724572893032724480.jpg',

'1.2.826.0.1.3680043.8.498.40478688820218019919554524179572721957.jpg',
'1.2.826.0.1.3680043.8.498.60949448156706238573173030075346205234.jpg',
'1.2.826.0.1.3680043.8.498.37621239492777651856971636207486753066.jpg',
'1.2.826.0.1.3680043.8.498.10616046117395675894796822789034600108.jpg',
'1.2.826.0.1.3680043.8.498.24799022519598382523325643690210404881.jpg',
'1.2.826.0.1.3680043.8.498.59448383364577609036637803738166121326.jpg',
'1.2.826.0.1.3680043.8.498.47790293155471499656895385861424785518.jpg',
'1.2.826.0.1.3680043.8.498.87569243548987476311032499287778579028.jpg',
'1.2.826.0.1.3680043.8.498.57332665974596512394313887326999373311.jpg',
'1.2.826.0.1.3680043.8.498.13381127336797898792841623526633394481.jpg',
'1.2.826.0.1.3680043.8.498.66891790906331431480970027904799740721.jpg',
'1.2.826.0.1.3680043.8.498.11310570221748863300638357151713540495.jpg',
'1.2.826.0.1.3680043.8.498.67810475638153448553533618534413783631.jpg',
'1.2.826.0.1.3680043.8.498.11963574639490848767438261672153471206.jpg',
'1.2.826.0.1.3680043.8.498.43131407064108816831447238381575714618.jpg',
'1.2.826.0.1.3680043.8.498.93924999416204973310706175342910448380.jpg',
'1.2.826.0.1.3680043.8.498.56369775866076293212442380321358920510.jpg',
'1.2.826.0.1.3680043.8.498.11061501234069068265253166056710021147.jpg',
'1.2.826.0.1.3680043.8.498.55163528300241867959503728693351689974.jpg',
'1.2.826.0.1.3680043.8.498.98684843337067795444414415305013839819.jpg',
'1.2.826.0.1.3680043.8.498.68217255698367432931486358042911886092.jpg',
'1.2.826.0.1.3680043.8.498.19196663464537437830301739049078212196.jpg',
'1.2.826.0.1.3680043.8.498.66293833077752245646669229730903433908.jpg',
'1.2.826.0.1.3680043.8.498.32018898057199540220995219509749333939.jpg',
'1.2.826.0.1.3680043.8.498.19697778336163620890163688905960381471.jpg',
'1.2.826.0.1.3680043.8.498.12081555449560607487646381923653562395.jpg',
'1.2.826.0.1.3680043.8.498.86482546113749715250037448887665474278.jpg',
'1.2.826.0.1.3680043.8.498.12446971389385006108573503695299633882.jpg',
'1.2.826.0.1.3680043.8.498.16685145848089373870698736004452376249.jpg',
'1.2.826.0.1.3680043.8.498.59809397121680249433728152529027849303.jpg',
'1.2.826.0.1.3680043.8.498.63965203979576731536024986286446925480.jpg',
'1.2.826.0.1.3680043.8.498.12240328596881032975862691977952544561.jpg',
'1.2.826.0.1.3680043.8.498.12388633920354965905972865429126173885.jpg',
'1.2.826.0.1.3680043.8.498.31011632360427328050457077884757457727.jpg',
'1.2.826.0.1.3680043.8.498.12101332540821162609531366213085632834.jpg',
'1.2.826.0.1.3680043.8.498.93030267282177828038590581719531097650.jpg',
'1.2.826.0.1.3680043.8.498.46666874685792777442868391673775049627.jpg',
'1.2.826.0.1.3680043.8.498.12752023674046086003000117257423027122.jpg',
'1.2.826.0.1.3680043.8.498.86197050744949128884872063529641034396.jpg',
'1.2.826.0.1.3680043.8.498.12535698779349487859104293604949096264.jpg',
'1.2.826.0.1.3680043.8.498.91110265471057804252745055915947225008.jpg',
'1.2.826.0.1.3680043.8.498.34645333555098820324356222947648009619.jpg',
'1.2.826.0.1.3680043.8.498.86119268496299670984694512696762979875.jpg',
'1.2.826.0.1.3680043.8.498.13151791663205648387416534465678032054.jpg',
'1.2.826.0.1.3680043.8.498.12823233979365639425836249903668888847.jpg',
'1.2.826.0.1.3680043.8.498.54725767978179985703919604383751319541.jpg',
'1.2.826.0.1.3680043.8.498.51937895029908619504310036260062633059.jpg',
'1.2.826.0.1.3680043.8.498.65614839772435983918953137130086924135.jpg',
'1.2.826.0.1.3680043.8.498.38668096836650220283789036360846112837.jpg',
'1.2.826.0.1.3680043.8.498.23605470341386605415033594389409440956.jpg',
'1.2.826.0.1.3680043.8.498.26341849425533378961171223556546097315.jpg',
'1.2.826.0.1.3680043.8.498.30107666456418936029508442405867573157.jpg',
'1.2.826.0.1.3680043.8.498.24825148313322166268936314354747747775.jpg',
'1.2.826.0.1.3680043.8.498.10127685402277965387261857228403285937.jpg',
'1.2.826.0.1.3680043.8.498.10544514288749160092453883077946388728.jpg',
'1.2.826.0.1.3680043.8.498.91454360780236888121221088965040182716.jpg',
'1.2.826.0.1.3680043.8.498.98060347071250186022893582097923328426.jpg',
'1.2.826.0.1.3680043.8.498.49197692672249488448453371216300646069.jpg',
'1.2.826.0.1.3680043.8.498.44115252344656427769689937573516279206.jpg',
'1.2.826.0.1.3680043.8.498.16081891806636510134189256638503603786.jpg',
'1.2.826.0.1.3680043.8.498.26408042106929721384058518370280693304.jpg',
'1.2.826.0.1.3680043.8.498.93847151738325809602164912030916859013.jpg',
'1.2.826.0.1.3680043.8.498.11612045493397591301814152037120636457.jpg',
'1.2.826.0.1.3680043.8.498.13345261439151579909236159587380063131.jpg',
'1.2.826.0.1.3680043.8.498.27712629326006897776702087184956551406.jpg',
'1.2.826.0.1.3680043.8.498.43609558835156870201174292025042525817.jpg',
'1.2.826.0.1.3680043.8.498.12301519614089043712368832761704750930.jpg',
'1.2.826.0.1.3680043.8.498.10295029511908851248060738144836999382.jpg',
'1.2.826.0.1.3680043.8.498.13963908218521519204150506393495366449.jpg',
'1.2.826.0.1.3680043.8.498.72598527577197250033387270423700920577.jpg',
'1.2.826.0.1.3680043.8.498.15786016706005165685171845319694168431.jpg',

1.2.826.0.1.3680043.8.498.81244217669483846008365074325194401948.jpg',
'1.2.826.0.1.3680043.8.498.86762470549186067857241308344630562034.jpg',
'1.2.826.0.1.3680043.8.498.48896647897116453064825058846325835504.jpg',
'1.2.826.0.1.3680043.8.498.68719963003505803481866191402322549382.jpg',
'1.2.826.0.1.3680043.8.498.21716138197740594100861810473161658111.jpg',
'1.2.826.0.1.3680043.8.498.61668872384081681726758499391093260160.jpg',
'1.2.826.0.1.3680043.8.498.93490598043133896025831521446596919342.jpg',
'1.2.826.0.1.3680043.8.498.37993403715775332082386269174865378332.jpg',
'1.2.826.0.1.3680043.8.498.49629713444944619796383606310797208241.jpg',
'1.2.826.0.1.3680043.8.498.48568052079011626028672217193540643327.jpg',
'1.2.826.0.1.3680043.8.498.67401305161594750839414957419032656583.jpg',
'1.2.826.0.1.3680043.8.498.18623899246027475134940862388409388823.jpg',
'1.2.826.0.1.3680043.8.498.43761479769148946736350731020969045175.jpg',
'1.2.826.0.1.3680043.8.498.70556928637314908190043753509080639346.jpg',
'1.2.826.0.1.3680043.8.498.54177684731454047476558941981746949474.jpg',
'1.2.826.0.1.3680043.8.498.10528556319778106540839661998539270956.jpg',
'1.2.826.0.1.3680043.8.498.12739245905507956943010054821450997339.jpg',
'1.2.826.0.1.3680043.8.498.11407048237084297178818294987567035881.jpg',
'1.2.826.0.1.3680043.8.498.96406955712119661656412685705445497931.jpg',
'1.2.826.0.1.3680043.8.498.50544507505522519089938740252109638745.jpg',
'1.2.826.0.1.3680043.8.498.68123597500155034353447254616832026438.jpg',
'1.2.826.0.1.3680043.8.498.12329918139293792657312101967803411034.jpg',
'1.2.826.0.1.3680043.8.498.98686531715047887673135488255304575036.jpg',
'1.2.826.0.1.3680043.8.498.60790664871631102230196884772808723907.jpg',
'1.2.826.0.1.3680043.8.498.14116654485638912148065272663789678618.jpg',
'1.2.826.0.1.3680043.8.498.77768376993936699357126750370843583855.jpg',
'1.2.826.0.1.3680043.8.498.82083592403007294584692264392726221431.jpg',
'1.2.826.0.1.3680043.8.498.81725679897290047835412347284019448803.jpg',
'1.2.826.0.1.3680043.8.498.51773940720706077257764509345477025369.jpg',
'1.2.826.0.1.3680043.8.498.15045820810654928384161257291611931429.jpg',
'1.2.826.0.1.3680043.8.498.78715330304045487459143788276776992885.jpg',
'1.2.826.0.1.3680043.8.498.12489439272589979128670058332904197203.jpg',
'1.2.826.0.1.3680043.8.498.28794136327509149789054455455195680707.jpg',
'1.2.826.0.1.3680043.8.498.11150176698544281023867780204381990477.jpg',
'1.2.826.0.1.3680043.8.498.11338572426973411314520277675162307063.jpg',
'1.2.826.0.1.3680043.8.498.12559760467099865666447578235333442548.jpg',
'1.2.826.0.1.3680043.8.498.10247835758178837578498859836156655953.jpg',
'1.2.826.0.1.3680043.8.498.11371029377455089388773090275178050889.jpg',
'1.2.826.0.1.3680043.8.498.59000837536872259040577141592245957659.jpg',
'1.2.826.0.1.3680043.8.498.11321967066764250923024784295742078244.jpg',
'1.2.826.0.1.3680043.8.498.11662436381922560394971858542854294460.jpg',
'1.2.826.0.1.3680043.8.498.11916229899462321328242335428906700205.jpg',
'1.2.826.0.1.3680043.8.498.86627002349579697587948490370395234058.jpg',
'1.2.826.0.1.3680043.8.498.43584820782558220859624721830629706618.jpg',
'1.2.826.0.1.3680043.8.498.43340424479611237895060478106689360500.jpg',
'1.2.826.0.1.3680043.8.498.13139807261977398213368515262447880745.jpg',
'1.2.826.0.1.3680043.8.498.28261472510047998376677126660658555551.jpg',
'1.2.826.0.1.3680043.8.498.80538780277526130691616420695781860773.jpg',
'1.2.826.0.1.3680043.8.498.62015824992988632734732028510878678058.jpg',
'1.2.826.0.1.3680043.8.498.85869114025602181438504943771475480375.jpg',
'1.2.826.0.1.3680043.8.498.31047302115150632984629993493088626412.jpg',
'1.2.826.0.1.3680043.8.498.28924322411012007756657474550402323157.jpg',
'1.2.826.0.1.3680043.8.498.19030034097808120761370471402749663104.jpg',
'1.2.826.0.1.3680043.8.498.11255534358810904840956733321539095700.jpg',
'1.2.826.0.1.3680043.8.498.32098448387372378461848014434334228174.jpg',
'1.2.826.0.1.3680043.8.498.44733940494710602815998768409533488692.jpg',
'1.2.826.0.1.3680043.8.498.12435141977560755717299919823745956382.jpg',
'1.2.826.0.1.3680043.8.498.12309121102402800688953778164046216892.jpg',
'1.2.826.0.1.3680043.8.498.10400437761587805367345098585741710689.jpg',
'1.2.826.0.1.3680043.8.498.53703874666111275576231619376274428969.jpg',
'1.2.826.0.1.3680043.8.498.10621379484581898840178485065281300027.jpg',
'1.2.826.0.1.3680043.8.498.12803458838869899243038004939785276402.jpg',
'1.2.826.0.1.3680043.8.498.34123604304139833436999040997949902421.jpg',
'1.2.826.0.1.3680043.8.498.10113069419621189926597289360854261721.jpg',
'1.2.826.0.1.3680043.8.498.85556167790573310759892143055692684515.jpg',
'1.2.826.0.1.3680043.8.498.13607750581556559583685502299522242243.jpg',
'1.2.826.0.1.3680043.8.498.74639971649942960289768769516583790950.jpg',
'1.2.826.0.1.3680043.8.498.15487329970770394031066908357774726588.jpg',
'1.2.826.0.1.3680043.8.498.44357541655684212436076434652092371424.jpg',
'1.2.826.0.1.3680043.8.498.13301596340518173871182723211743792808.jpg',
'1.2.826.0.1.3680043.8.498.05546792158796626387432830218263315850.jpg'

'1.2.826.0.1.3680043.8.498.30273147828303300738881695478190086535.jpg',
'1.2.826.0.1.3680043.8.498.30543459998639687388034609264482612122.jpg',
'1.2.826.0.1.3680043.8.498.11787103062280694791203943354675984745.jpg',
'1.2.826.0.1.3680043.8.498.11078581759199796317347973845260728646.jpg',
'1.2.826.0.1.3680043.8.498.11856484281647175410986616807399788295.jpg',
'1.2.826.0.1.3680043.8.498.38199716279202665980657990919201136600.jpg',
'1.2.826.0.1.3680043.8.498.12319654582363902330879923162505812500.jpg',
'1.2.826.0.1.3680043.8.498.85532799680825387534302013447068585944.jpg',
'1.2.826.0.1.3680043.8.498.89699382582993493976854778963855488290.jpg',
'1.2.826.0.1.3680043.8.498.11200313068647305637967260428059257565.jpg',
'1.2.826.0.1.3680043.8.498.38377166135523367103094712299328581092.jpg',
'1.2.826.0.1.3680043.8.498.79189032758179310257417568396486019086.jpg',
'1.2.826.0.1.3680043.8.498.65875221093031599126670803587226435854.jpg',
'1.2.826.0.1.3680043.8.498.27157728902741073243225350823934673745.jpg',
'1.2.826.0.1.3680043.8.498.12142439878193071127673336216387516568.jpg',
'1.2.826.0.1.3680043.8.498.89719766018852106378366570897526553138.jpg',
'1.2.826.0.1.3680043.8.498.12030615153092170111727705465470754325.jpg',
'1.2.826.0.1.3680043.8.498.45281710185115138948880069137500526868.jpg',
'1.2.826.0.1.3680043.8.498.48498248972779095092320964527895502352.jpg',
'1.2.826.0.1.3680043.8.498.17848594642056898227161770052344663095.jpg',
'1.2.826.0.1.3680043.8.498.10790594484024825770230975138368952833.jpg',
'1.2.826.0.1.3680043.8.498.22320705680148774104574675541453580288.jpg',
'1.2.826.0.1.3680043.8.498.13105973284201838236703511967561155596.jpg',
'1.2.826.0.1.3680043.8.498.64781049488148307943679318270258123176.jpg',
'1.2.826.0.1.3680043.8.498.10291859228136695804198917442021382630.jpg',
'1.2.826.0.1.3680043.8.498.15294851103858209344876656333002451286.jpg',
'1.2.826.0.1.3680043.8.498.45610693976612793338130037407624294119.jpg',
'1.2.826.0.1.3680043.8.498.71464019112015394546851186903588647508.jpg',
'1.2.826.0.1.3680043.8.498.26024795691872432995024987906425586557.jpg',
'1.2.826.0.1.3680043.8.498.13442403538558979602420895528883462813.jpg',
'1.2.826.0.1.3680043.8.498.91678626024317484450487039860869294931.jpg',
'1.2.826.0.1.3680043.8.498.46079420276241325209580144649632111030.jpg',
'1.2.826.0.1.3680043.8.498.10031131575745854404126410511882847279.jpg',
'1.2.826.0.1.3680043.8.498.97317555641609251051711537440437103519.jpg',
'1.2.826.0.1.3680043.8.498.99235555025604097345765385457950845794.jpg',
'1.2.826.0.1.3680043.8.498.12233799567524846197502279061758590998.jpg',
'1.2.826.0.1.3680043.8.498.97869615760241827644212427520711177056.jpg',
'1.2.826.0.1.3680043.8.498.88113051894593421623463430273866185821.jpg',
'1.2.826.0.1.3680043.8.498.11637594688773276129126093164815472402.jpg',
'1.2.826.0.1.3680043.8.498.11541485524684594115246178504483511880.jpg',
'1.2.826.0.1.3680043.8.498.31972584983735553983323980725713839789.jpg',
'1.2.826.0.1.3680043.8.498.26151423710154697232251757856857521572.jpg',
'1.2.826.0.1.3680043.8.498.63845213550568297841677452091509427127.jpg',
'1.2.826.0.1.3680043.8.498.54882896655544505960369453100927929985.jpg',
'1.2.826.0.1.3680043.8.498.85166690739409035609563269625914847874.jpg',
'1.2.826.0.1.3680043.8.498.49531959241536977905245333088874267777.jpg',
'1.2.826.0.1.3680043.8.498.12906054598574301635449305477183493302.jpg',
'1.2.826.0.1.3680043.8.498.64414359924793107284852101336273092720.jpg',
'1.2.826.0.1.3680043.8.498.83008074908391147546630420103750216994.jpg',
'1.2.826.0.1.3680043.8.498.11658774926362789148875916576955990416.jpg',
'1.2.826.0.1.3680043.8.498.11272656076858257923471352206045877700.jpg',
'1.2.826.0.1.3680043.8.498.95744954492795153056764962106971527208.jpg',
'1.2.826.0.1.3680043.8.498.89218218542010864953025355308835862110.jpg',
'1.2.826.0.1.3680043.8.498.44991943803674980416574921225602688770.jpg',
'1.2.826.0.1.3680043.8.498.50912117691319755731206739915395481674.jpg',
'1.2.826.0.1.3680043.8.498.25711356688801615846005095690956876590.jpg',
'1.2.826.0.1.3680043.8.498.70611462604949822757451521855923175044.jpg',
'1.2.826.0.1.3680043.8.498.18501223283964244804870499128522747009.jpg',
'1.2.826.0.1.3680043.8.498.13286009455957559946265623540680445623.jpg',
'1.2.826.0.1.3680043.8.498.27726392091303420945027835049284045729.jpg',
'1.2.826.0.1.3680043.8.498.35726237080821073394537164446772789957.jpg',
'1.2.826.0.1.3680043.8.498.65215102454660067501859372955840809887.jpg',
'1.2.826.0.1.3680043.8.498.10244889160083993493696708462043336818.jpg',
'1.2.826.0.1.3680043.8.498.38211941419027590865915594437207346240.jpg',
'1.2.826.0.1.3680043.8.498.76873981093833909666100955199059328203.jpg',
'1.2.826.0.1.3680043.8.498.10861905136245694758396214165062414915.jpg',
'1.2.826.0.1.3680043.8.498.85962787771949599992136126868228366854.jpg',
'1.2.826.0.1.3680043.8.498.52116827008525466090806093076695677315.jpg',
'1.2.826.0.1.3680043.8.498.12553597402571332356580522500160184997.jpg',
'1.2.826.0.1.3680043.8.498.12077238215101433714089460182781377356.jpg',
'1.2.826.0.1.3680043.8.498.126680365085555517507703580450155718.jpg'

```

1.2.826.0.1.3680043.8.498.71368026058855565517597792580450155718.jpg',
'1.2.826.0.1.3680043.8.498.97539960468810238603880460205026657866.jpg',
'1.2.826.0.1.3680043.8.498.60080956841618870940968575868106407717.jpg',
'1.2.826.0.1.3680043.8.498.91412349700684046393147309411782226312.jpg',
'1.2.826.0.1.3680043.8.498.78986155428128276539832738050654514914.jpg',
'1.2.826.0.1.3680043.8.498.29165846044217811244200097292738336669.jpg',
'1.2.826.0.1.3680043.8.498.89154888024684546463007152284604791721.jpg',
'1.2.826.0.1.3680043.8.498.27489903932468105536960721671013981761.jpg',
'1.2.826.0.1.3680043.8.498.12564911732465349258096364938161968039.jpg',
'1.2.826.0.1.3680043.8.498.69770233351320266790231127014365510967.jpg',
'1.2.826.0.1.3680043.8.498.12739571388889329673266321505609390944.jpg',
'1.2.826.0.1.3680043.8.498.65382238816347938357470427928072870018.jpg',
'1.2.826.0.1.3680043.8.498.87060544214618912677358800066372917759.jpg',
'1.2.826.0.1.3680043.8.498.66124446863564042991078573141965710930.jpg',
'1.2.826.0.1.3680043.8.498.10357109821121236745328604877052939511.jpg',
'1.2.826.0.1.3680043.8.498.22681438984244725251540286783146340207.jpg',
'1.2.826.0.1.3680043.8.498.86485102999781626038426497763377392220.jpg',
'1.2.826.0.1.3680043.8.498.11839203381250258831327796961034559204.jpg',
'1.2.826.0.1.3680043.8.498.68162226723023573539516580147162531088.jpg',
...]

```

```

In [ ]: def processImages(filenamees,train_test):

    imgs = []
    FileIDs = []
    lst = []

    desired_channels = 3

    count = 0
    for file in tqdm(filenamees, position=0, leave=True):
        # Making sure the images are the same pixel size
        img = Image.open(f"/projects/sc73/ranzcr-clip-catheter-line-classification/train/{file}")
        img = img.resize((300, 300))
        img = np.array(img)

        # Ensure that the image has 3 channels (RGB)
        if img.shape[-1] != 3:
            img = np.stack((img,) * 3, axis=-1) # Convert grayscale to RGB by duplicating

        imgs.append(img) # Append the 4D image to the list

        # Save filename for later
        FileIDs.append(file)

        count += 1

    # Convert the list of images to a numpy array
    imgs = np.array(imgs)
    return imgs, FileIDs

```

```

In [ ]: train_imgs, fileIDs = processImages(studyInstance, 'train')

```

```

100%|██████████| 1500/1500 [01:27<00:00, 17.08it/s]

```

```

In [ ]: #cleaning up the labels
        UID = []
        for i in range(len(fileIDs)):
            UID.append(fileIDs[i].replace(".jpg", ""))

        UID[0:10]

```

```

Out[ ]: ['1.2.826.0.1.3680043.8.498.29798135944425181867741764889671646820',
        '1.2.826.0.1.3680043.8.498.56917573254477591662381927833438589539',
        '1.2.826.0.1.3680043.8.498.11749599424730443758948253678195377134',
        '1.2.826.0.1.3680043.8.498.84533062276529819766171648406000766083',
        '1.2.826.0.1.3680043.8.498.49761949951848218455749059001453442446',

```

```
'1.2.826.0.1.3680043.8.498.40981519393347637453680278096781297052',
'1.2.826.0.1.3680043.8.498.82309444747241213141372618882835382602',
'1.2.826.0.1.3680043.8.498.31007111818249500067228144131625734696',
'1.2.826.0.1.3680043.8.498.45258993479143480195404274416141355225',
'1.2.826.0.1.3680043.8.498.81361445628248936097480449911449382182']
```

```
In [ ]: columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnorm
                        "NGT - Incompletely Imaged", "Swan Ganz Catheter Present", "PatientI

CVCsampled_data = sampled_data.drop(columns=columns_to_drop, axis=1)

CVCsampled_data.set_index('StudyInstanceUID', inplace=True)
```

```
In [ ]: CVCsampled_data = CVCsampled_data.reindex(UID)
```

```
In [ ]: train_labels = np.empty((0,3))
data = []

for ID in UID:
    data.append(CVCsampled_data.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)
train_labels = train_labels_df['CVC - Present'].values
# train_labels = train_labels_df.values
train_labels
```

```
Out[ ]: array([1, 1, 1, ..., 0, 0, 0])
```

```
In [ ]:
```

```
In [ ]: from sklearn.model_selection import train_test_split

# Split your data into training and temporary data (combined validation and test)
train_imgs, temp_imgs, train_labels, temp_labels = train_test_split(train_imgs, train_

# Split the temporary data into validation and test sets
val_imgs, test_imgs, val_labels, test_labels = train_test_split(temp_imgs, temp_labels
```

```
In [ ]: print(len(train_labels))
print(len(train_imgs))
```

```
1050
1050
```

```
In [ ]: from tensorflow.keras.applications.densenet import DenseNet121
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Dropout, BatchNormalization
# Load the DenseNet121 base model without its top layers
base_model = DenseNet121(weights='imagenet', include_top=False)

# Add dropout and batch normalization to custom fully connected layers
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.5)(x)
predictions = Dense(1, activation='sigmoid')(x)
```

```
model = Model(inputs=base_model.input, outputs=predictions)
```

In []:

```
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Reduce Learning rate when the validation loss plateaus
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=1e-6)

# Compile the model with optimizer and loss function
model.compile(optimizer='SGD', loss='binary_crossentropy', metrics=['accuracy'])

# Early stopping to prevent overfitting
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Save the best model during training
model_checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True)

# Model Training with callbacks
history = model.fit(train_imgs, train_labels, epochs=30, validation_data=(val_imgs, va

# Evaluate the model
test_loss, test_accuracy = model.evaluate(test_imgs, test_labels)
print(f'Test Accuracy: {test_accuracy * 100:.2f}%')
```

Epoch 1/30

33/33 [=====] - 335s 10s/step - loss: 0.8092 - accuracy: 0.6341
- val_loss: 0.6901 - val_accuracy: 0.5200

Epoch 2/30

33/33 [=====] - 327s 10s/step - loss: 0.5765 - accuracy: 0.7454
- val_loss: 0.6442 - val_accuracy: 0.6089

Epoch 3/30

33/33 [=====] - 328s 10s/step - loss: 0.4111 - accuracy: 0.8242
- val_loss: 0.5992 - val_accuracy: 0.6889

Epoch 4/30

33/33 [=====] - 326s 10s/step - loss: 0.1796 - accuracy: 0.9254
- val_loss: 0.7282 - val_accuracy: 0.6222

Epoch 5/30

33/33 [=====] - 327s 10s/step - loss: 0.1319 - accuracy: 0.9567
- val_loss: 0.6402 - val_accuracy: 0.6622

Epoch 6/30

33/33 [=====] - 326s 10s/step - loss: 0.0619 - accuracy: 0.9753
- val_loss: 0.6933 - val_accuracy: 0.6978

Epoch 7/30

33/33 [=====] - 326s 10s/step - loss: 0.0262 - accuracy: 0.9970
- val_loss: 0.7280 - val_accuracy: 0.7244

Epoch 8/30

33/33 [=====] - 326s 10s/step - loss: 0.0143 - accuracy: 0.9996
- val_loss: 0.6578 - val_accuracy: 0.7778

Epoch 9/30

33/33 [=====] - 326s 10s/step - loss: 0.0101 - accuracy: 1.0000
- val_loss: 0.6884 - val_accuracy: 0.7956

Epoch 10/30

33/33 [=====] - 326s 10s/step - loss: 0.0156 - accuracy: 0.9949
- val_loss: 0.6923 - val_accuracy: 0.8089

Epoch 11/30

33/33 [=====] - 325s 10s/step - loss: 0.0112 - accuracy: 0.9975
- val_loss: 0.7251 - val_accuracy: 0.7956

Epoch 12/30

33/33 [=====] - 326s 10s/step - loss: 0.0133 - accuracy: 0.9981
- val_loss: 0.7263 - val_accuracy: 0.7867

Epoch 13/30

33/33 [=====] - 325s 10s/step - loss: 0.0074 - accuracy: 0.9999
- val_loss: 0.7217 - val_accuracy: 0.7911

8/8 [=====] - 8s 913ms/step - loss: 0.6211 - accuracy: 0.6489

Test Accuracy: 64.89%

In []:

```
def buildTrainData(catheter, train_data, size):
```

```

train_data = train_data[train_data[f'{catheter} - Present'] == 1]
train_data = train_data[train_data[~((train_data[f'{catheter} - Normal'] == 1) &
train_data[f'{catheter} - Normal/Abnormal'] = np.where(train_data[f'{catheter} - N
sampled_data = pd.concat([train_data[train_data[f'{catheter} - Normal'] == 1].samp

return sampled_data

```

```

In [ ]: sample = buildTrainData('CVC', train_data, 750)
sample

```

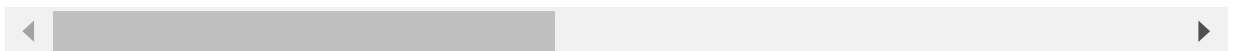
```

Out[ ]:

```

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT Abnorm
25212	1.2.826.0.1.3680043.8.498.11108043051434216161...	0	0	1	
23056	1.2.826.0.1.3680043.8.498.81914401367896518348...	0	0	0	
27015	1.2.826.0.1.3680043.8.498.34415144012741888520...	0	0	0	
22320	1.2.826.0.1.3680043.8.498.88754391070038246530...	0	0	0	
13863	1.2.826.0.1.3680043.8.498.44441453615466617228...	0	0	0	
...	...	...	...	...	
18281	1.2.826.0.1.3680043.8.498.82030693571162250993...	0	0	0	
11246	1.2.826.0.1.3680043.8.498.82166619974013732691...	0	0	0	
5834	1.2.826.0.1.3680043.8.498.24492665583305684486...	0	0	0	
24983	1.2.826.0.1.3680043.8.498.95363365709766072176...	0	0	0	
28831	1.2.826.0.1.3680043.8.498.12685876309309753864...	0	0	0	

1500 rows × 15 columns



```

In [ ]: def getFileNames(df):
        studyInstance = df['StudyInstanceUID'].tolist()
        studyInstance

        for i in range(len(studyInstance)):
            studyInstance[i] = f'{studyInstance[i]}.jpg'

        return studyInstance

```

```

In [ ]: nAb_data = getFileNames(sample)

cvc_imgs, fileIDs = processImages(nAb_data, 'train')

```

100%|██████████| 1500/1500 [01:29<00:00, 16.67it/s]

```

In [ ]: UID = []
        for i in range(len(fileIDs)):
            UID.append(fileIDs[i].replace(".jpg", ""))

        columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnorm
                            "NGT - Incompletely Imaged", "Swan Ganz Catheter Present", "PatientI

        CVCsample = sample.drop(columns=columns_to_drop, axis=1)

        CVCsample.set_index('StudyInstanceUID', inplace=True)

```

```
CVCsample
```

Out[]:

CVC - Normal/Abnormal	
StudyInstanceUID	
1.2.826.0.1.3680043.8.498.11108043051434216161830699489737681751	1
1.2.826.0.1.3680043.8.498.81914401367896518348642216432851373095	1
1.2.826.0.1.3680043.8.498.34415144012741888520360296227836680446	1
1.2.826.0.1.3680043.8.498.88754391070038246530878003342444547273	1
1.2.826.0.1.3680043.8.498.44441453615466617228387474073459217208	1
...	...
1.2.826.0.1.3680043.8.498.82030693571162250993341474603525165996	0
1.2.826.0.1.3680043.8.498.82166619974013732691987468699733186093	0
1.2.826.0.1.3680043.8.498.24492665583305684486011767114054826053	0
1.2.826.0.1.3680043.8.498.95363365709766072176296908908865687897	0
1.2.826.0.1.3680043.8.498.12685876309309753864148952685783845265	0

1500 rows × 1 columns

In []:

```
train_labels = np.empty((0,3))
data = []

for ID in UID:
    data.append(CVCsample.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)
cvc_labels = train_labels_df['CVC - Normal/Abnormal'].values
# train_labels = train_labels_df.values
cvc_labels
```

Out[]: array([1, 1, 1, ..., 0, 0, 0])

In []:

```
from sklearn.model_selection import train_test_split

# Split your data into training and temporary data (combined validation and test)
trainCVC_imgs, temp_imgs, trainCVC_labels, temp_labels = train_test_split(cvc_imgs, cv

# Split the temporary data into validation and test sets
val_imgs, testCVC_imgs, val_labels, testCVC_labels = train_test_split(temp_imgs, temp_
```

In []:

```
from tensorflow.keras.applications.densenet import DenseNet121
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Dropout, BatchNormalization
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
base_model = DenseNet121(weights='imagenet', include_top=False)
```

```

# Add dropout and batch normalization to custom fully connected layers
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.5)(x)
predictions = Dense(1, activation='sigmoid')(x)
model = Model(inputs=base_model.input, outputs=predictions)

# Reduce Learning rate when the validation loss plateaus
reduce_lr = ReduceLRonPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=1e-6)

# Compile the model with optimizer and loss function
model.compile(optimizer='SGD', loss='binary_crossentropy', metrics=['accuracy'])

# Early stopping to prevent overfitting
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Save the best model during training
model_checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True)

# Model Training with callbacks
history = model.fit(trainCVC_imgs, trainCVC_labels, epochs=30, validation_data=(val_im

# Evaluate the model
test_loss, test_accuracy = model.evaluate(test_imgs, test_labels)
print(f'Test Accuracy: {test_accuracy * 100:.2f}%')

```

```

Epoch 1/30
33/33 [=====] - 332s 10s/step - loss: 0.9921 - accuracy: 0.4954
- val_loss: 0.7441 - val_accuracy: 0.5111
Epoch 2/30
33/33 [=====] - 323s 10s/step - loss: 0.7399 - accuracy: 0.6192
- val_loss: 1.0775 - val_accuracy: 0.5156
Epoch 3/30
33/33 [=====] - 321s 10s/step - loss: 0.6228 - accuracy: 0.6812
- val_loss: 0.9324 - val_accuracy: 0.5156
Epoch 4/30
33/33 [=====] - 323s 10s/step - loss: 0.4569 - accuracy: 0.7864
- val_loss: 0.8534 - val_accuracy: 0.5378
Epoch 5/30
33/33 [=====] - 322s 10s/step - loss: 0.3085 - accuracy: 0.8748
- val_loss: 1.2451 - val_accuracy: 0.5378
Epoch 6/30
33/33 [=====] - 323s 10s/step - loss: 0.1728 - accuracy: 0.9304
- val_loss: 1.1353 - val_accuracy: 0.5244
Epoch 7/30
33/33 [=====] - 323s 10s/step - loss: 0.0992 - accuracy: 0.9670
- val_loss: 1.1810 - val_accuracy: 0.5244
Epoch 8/30
33/33 [=====] - 323s 10s/step - loss: 0.0545 - accuracy: 0.9893
- val_loss: 1.1564 - val_accuracy: 0.5600
Epoch 9/30
33/33 [=====] - 326s 10s/step - loss: 0.0441 - accuracy: 0.9899
- val_loss: 1.1502 - val_accuracy: 0.5644
Epoch 10/30
33/33 [=====] - 323s 10s/step - loss: 0.0343 - accuracy: 0.9949
- val_loss: 1.0994 - val_accuracy: 0.5689
Epoch 11/30
33/33 [=====] - 322s 10s/step - loss: 0.0306 - accuracy: 0.9976
- val_loss: 1.0800 - val_accuracy: 0.5689
8/8 [=====] - 10s 1s/step - loss: 0.7520 - accuracy: 0.4800
Test Accuracy: 48.00%

```

In []:

DenseNet - NGT

```
In [ ]: import random
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import json
from PIL import Image
import ast
import os

from tqdm import tqdm
```

```
In [ ]: #extracting just the CVC labels
train_data = pd.read_csv("/kaggle/input/ranzcr-clip-catheter-line-classification/train_data.csv")
train_data = train_data[train_data['NGT - Borderline'] != 1]
train_data['NGT - Present'] = np.where((train_data['NGT - Abnormal'] == 0) & (train_data['NGT - Borderline'] == 0), 1, 0)
train_data['NGT - Present'].value_counts()
```

```
Out[ ]: NGT - Present
0      24501
1       5053
Name: count, dtype: int64
```

```
In [ ]: sampled_data = pd.concat([train_data[train_data['NGT - Present'] == 1].sample(n=750),
                                sampled_data])
```

```
Out[ ]:
```

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NG Abnorm
2381	1.2.826.0.1.3680043.8.498.72027842236740426828...	0	0	1	
21218	1.2.826.0.1.3680043.8.498.17124285924673288354...	0	1	0	
9458	1.2.826.0.1.3680043.8.498.83787296854153893198...	0	0	1	
4115	1.2.826.0.1.3680043.8.498.87430326611380193974...	0	0	1	
27341	1.2.826.0.1.3680043.8.498.11939772432256969564...	0	1	0	
...	...	...	...	...	
29678	1.2.826.0.1.3680043.8.498.90915831742574604863...	0	0	0	
29690	1.2.826.0.1.3680043.8.498.92686998100433418697...	0	0	1	
3397	1.2.826.0.1.3680043.8.498.11438096043898267210...	0	0	0	
7600	1.2.826.0.1.3680043.8.498.95329529106960247096...	0	0	0	
16690	1.2.826.0.1.3680043.8.498.10819922478064523573...	0	0	0	

1500 rows × 14 columns



```
In [ ]: studyInstance = sampled_data['StudyInstanceUID'].tolist()
studyInstance
```



```

for i in range(len(studyInstance)):
    studyInstance[i] = f'{studyInstance[i]}.jpg'

```

```

In [ ]: def processImages(filenamees,train_test):

    imgs = []
    FileIDs = []
    lst = []

    desired_channels = 3

    count = 0
    for file in tqdm(filenamees, position=0, leave=True):
        # Making sure the images are the same pixel size
        img = Image.open(f"/kaggle/input/ranzcr-clip-catheter-line-classification/{tr
        img = img.resize((300, 300))
        img = np.array(img)

        # Ensure that the image has 3 channels (RGB)
        if img.shape[-1] != 3:
            img = np.stack((img,) * 3, axis=-1) # Convert grayscale to RGB by duplic

        imgs.append(img) # Append the 4D image to the list

        # Save filename for later
        FileIDs.append(file)

        count += 1

    # Convert the List of images to a numpy array
    imgs = np.array(imgs)
    return imgs, FileIDs

```

```

In [ ]: train_imgs, fileIDs = processImages(studyInstance, 'train')

```

```

100%|██████████| 1500/1500 [01:07<00:00, 22.09it/s]

```

```

In [ ]: #cleaning up the labels
        UID = []
        for i in range(len(fileIDs)):
            UID.append(fileIDs[i].replace(".jpg",""))

        UID[0:10]

```

```

Out[ ]: ['1.2.826.0.1.3680043.8.498.72027842236740426828450197490947975428',
        '1.2.826.0.1.3680043.8.498.17124285924673288354603519153366166318',
        '1.2.826.0.1.3680043.8.498.83787296854153893198021375584075109904',
        '1.2.826.0.1.3680043.8.498.87430326611380193974662642793143433568',
        '1.2.826.0.1.3680043.8.498.11939772432256969564077532046371198564',
        '1.2.826.0.1.3680043.8.498.44820679417635582340602856104412843092',
        '1.2.826.0.1.3680043.8.498.11151413709327750697141242070265662852',
        '1.2.826.0.1.3680043.8.498.85042569865520881877173875324351728516',
        '1.2.826.0.1.3680043.8.498.27474037990075738604723665323633255657',
        '1.2.826.0.1.3680043.8.498.12149340551138111408992017658320866089']

```

```

In [ ]: columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnor
        "NGT - Incompletely Imaged", "Swan Ganz Catheter Present", "Patient

        CVCsampled_data = sampled_data.drop(columns=columns_to_drop, axis=1)

        CVCsampled_data.set_index('StudyInstanceUID', inplace=True)

```

```

In [ ]: CVCsampled_data = CVCsampled_data.reindex(UID)

```

```

In [ ]:

```

```

train_labels = np.empty((0,3))
data = []

for ID in UID:
    data.append(CVCsampled_data.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)
train_labels = train_labels_df['NGT - Present'].values
# train_labels = train_labels_df.values
train_labels

```

Out[]: array([1, 1, 1, ..., 0, 0, 0])

In []:

```

In [ ]: from sklearn.model_selection import train_test_split

# Split your data into training and temporary data (combined validation and test)
train_imgs, temp_imgs, train_labels, temp_labels = train_test_split(train_imgs, train_labels,
                             temp_imgs, temp_labels)

# Split the temporary data into validation and test sets
val_imgs, test_imgs, val_labels, test_labels = train_test_split(temp_imgs, temp_labels,

```

```

In [ ]: print(len(train_labels))
        print(len(train_imgs))

```

1050

1050

```

In [ ]: from tensorflow.keras.applications.densenet import DenseNet121
        from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
        from tensorflow.keras.models import Model
        from tensorflow.keras.optimizers import Adam
        from tensorflow.keras.layers import Dropout, BatchNormalization
        # Load the DenseNet121 base model without its top layers
        base_model = DenseNet121(weights='imagenet', include_top=False)

        # Add dropout and batch normalization to custom fully connected layers
        x = base_model.output
        x = GlobalAveragePooling2D()(x)
        x = Dense(1024, activation='relu')(x)
        x = BatchNormalization()(x)
        x = Dropout(0.5)(x)
        predictions = Dense(1, activation='sigmoid')(x)
        model = Model(inputs=base_model.input, outputs=predictions)

```

D1019 06:58:44.395660674	15 config.cc:119]	gRPC EXPERIMENT
tcp_frame_size_tuning	OFF (default:OFF)	
D1019 06:58:44.395685658	15 config.cc:119]	gRPC EXPERIMENT
tcp_rcv_lowat	OFF (default:OFF)	
D1019 06:58:44.395689062	15 config.cc:119]	gRPC EXPERIMENT
peer_state_based_framing	OFF (default:OFF)	
D1019 06:58:44.395691555	15 config.cc:119]	gRPC EXPERIMENT
flow_control_fixes	ON (default:ON)	
D1019 06:58:44.395693788	15 config.cc:119]	gRPC EXPERIMENT
memory_pressure_controller	OFF (default:OFF)	
D1019 06:58:44.395696331	15 config.cc:119]	gRPC EXPERIMENT
unconstrained_max_quota_buffer_size	OFF (default:OFF)	
D1019 06:58:44.395699749	15 config.cc:119]	gRPC EXPERIMENT
new_hpack_huffman_decoder	ON (default:ON)	
D1019 06:58:44.395702059	15 config.cc:119]	gRPC EXPERIMENT
event_engine_client	OFF (default:OFF)	
D1019 06:58:44.395704328	15 config.cc:119]	gRPC EXPERIMENT

```

monitoring_experiment ON (default:ON)
D1019 06:58:44.395706529 15 config.cc:119] gRPC EXPERIMENT
promise_based_client_call OFF (default:OFF)
D1019 06:58:44.395708820 15 config.cc:119] gRPC EXPERIMENT
free_large_allocator OFF (default:OFF)
D1019 06:58:44.395711017 15 config.cc:119] gRPC EXPERIMENT
promise_based_server_call OFF (default:OFF)
D1019 06:58:44.395713352 15 config.cc:119] gRPC EXPERIMENT
transport_supplies_client_latency OFF (default:OFF)
D1019 06:58:44.395721850 15 config.cc:119] gRPC EXPERIMENT
event_engine_listener OFF (default:OFF)
I1019 06:58:44.395978094 15 ev_epoll1_linux.cc:122] grpc epoll fd: 7
0
D1019 06:58:44.395992442 15 ev_posix.cc:144] Using polling engine: epoll1
D1019 06:58:44.396033705 15 dns_resolver_ares.cc:822] Using ares dns resolver
D1019 06:58:44.396593778 15 lb_policy_registry.cc:46] registering LB policy factory for "priority_experimental"
D1019 06:58:44.396605542 15 lb_policy_registry.cc:46] registering LB policy factory for "outlier_detection_experimental"
D1019 06:58:44.396609010 15 lb_policy_registry.cc:46] registering LB policy factory for "weighted_target_experimental"
D1019 06:58:44.396612077 15 lb_policy_registry.cc:46] registering LB policy factory for "pick_first"
D1019 06:58:44.396615243 15 lb_policy_registry.cc:46] registering LB policy factory for "round_robin"
D1019 06:58:44.396618338 15 lb_policy_registry.cc:46] registering LB policy factory for "weighted_round_robin_experimental"
D1019 06:58:44.396626244 15 lb_policy_registry.cc:46] registering LB policy factory for "ring_hash_experimental"
D1019 06:58:44.396645756 15 lb_policy_registry.cc:46] registering LB policy factory for "grpclb"
D1019 06:58:44.396680004 15 lb_policy_registry.cc:46] registering LB policy factory for "rls_experimental"
D1019 06:58:44.396696472 15 lb_policy_registry.cc:46] registering LB policy factory for "xds_cluster_manager_experimental"
D1019 06:58:44.396700356 15 lb_policy_registry.cc:46] registering LB policy factory for "xds_cluster_impl_experimental"
D1019 06:58:44.396703670 15 lb_policy_registry.cc:46] registering LB policy factory for "cds_experimental"
D1019 06:58:44.396728266 15 lb_policy_registry.cc:46] registering LB policy factory for "xds_cluster_resolver_experimental"
D1019 06:58:44.396731979 15 lb_policy_registry.cc:46] registering LB policy factory for "xds_override_host_experimental"
D1019 06:58:44.396735762 15 lb_policy_registry.cc:46] registering LB policy factory for "xds_wrr_locality_experimental"
D1019 06:58:44.396740979 15 certificate_provider_registry.cc:35] registering certificate provider factory for "file_watcher"
I1019 06:58:44.400642685 15 socket_utils_common_posix.cc:408] Disabling AF_INET6 sockets because ::1 is not available.
I1019 06:58:44.416041354 377 socket_utils_common_posix.cc:337] TCP_USER_TIMEOUT is available. TCP_USER_TIMEOUT will be used thereafter
E1019 06:58:44.421656447 377 oauth2_credentials.cc:236] oauth_fetch: UNKNOWN:C-ares status is not ARES_SUCCESS qtype=A name=metadata.google.internal. is_balancer=0: Domain name not found {grpc_status:2, created_time:"2023-10-19T06:58:44.421640715+00:00"}
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/densenet/densenet121_weights_tf_dim_ordering_tf_kernels_notop.h5
29084464/29084464 [=====] - 0s 0us/step

```

```

In [ ]: from tensorflow.keras.callbacks import ReduceLROnPlateau
        from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

        # Reduce learning rate when the validation loss plateaus
        reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=1e-6)

        # Compile the model with optimizer and loss function
        model.compile(optimizer='SGD', loss='binary_crossentropy', metrics=['accuracy'])

        # Early stopping to prevent overfitting

```

```

early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Save the best model during training
model_checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True)

# Model Training with callbacks
history = model.fit(train_imgs, train_labels, epochs=30, validation_data=(val_imgs, val_labels))

# Evaluate the model
test_loss, test_accuracy = model.evaluate(test_imgs, test_labels)
print(f'Test Accuracy: {test_accuracy * 100:.2f}%')

```

```

Epoch 1/30
33/33 [=====] - 242s 7s/step - loss: 0.6585 - accuracy: 0.7000
- val_loss: 0.5851 - val_accuracy: 0.7289 - lr: 0.0100
Epoch 2/30
33/33 [=====] - 219s 7s/step - loss: 0.4641 - accuracy: 0.8133
- val_loss: 0.4836 - val_accuracy: 0.8044 - lr: 0.0100
Epoch 3/30
33/33 [=====] - 220s 7s/step - loss: 0.3136 - accuracy: 0.8705
- val_loss: 0.5736 - val_accuracy: 0.7111 - lr: 0.0100
Epoch 4/30
33/33 [=====] - 217s 7s/step - loss: 0.1800 - accuracy: 0.9286
- val_loss: 0.5125 - val_accuracy: 0.8089 - lr: 0.0100
Epoch 5/30
33/33 [=====] - 218s 7s/step - loss: 0.0995 - accuracy: 0.9581
- val_loss: 0.4295 - val_accuracy: 0.8222 - lr: 0.0100
Epoch 6/30
33/33 [=====] - 219s 7s/step - loss: 0.0711 - accuracy: 0.9781
- val_loss: 0.5158 - val_accuracy: 0.8356 - lr: 0.0100
Epoch 7/30
33/33 [=====] - 216s 7s/step - loss: 0.0191 - accuracy: 0.9971
- val_loss: 0.5504 - val_accuracy: 0.8711 - lr: 0.0100
Epoch 8/30
33/33 [=====] - 220s 7s/step - loss: 0.0216 - accuracy: 0.9924
- val_loss: 0.6313 - val_accuracy: 0.8489 - lr: 0.0100
Epoch 9/30
33/33 [=====] - 220s 7s/step - loss: 0.0154 - accuracy: 0.9971
- val_loss: 0.5175 - val_accuracy: 0.8533 - lr: 0.0100
Epoch 10/30
33/33 [=====] - 218s 7s/step - loss: 0.0086 - accuracy: 1.0000
- val_loss: 0.5334 - val_accuracy: 0.8667 - lr: 0.0100
Epoch 11/30
33/33 [=====] - 216s 7s/step - loss: 0.0087 - accuracy: 0.9971
- val_loss: 0.5484 - val_accuracy: 0.8533 - lr: 0.0020
Epoch 12/30
33/33 [=====] - 218s 7s/step - loss: 0.0061 - accuracy: 0.9990
- val_loss: 0.5643 - val_accuracy: 0.8622 - lr: 0.0020
Epoch 13/30
33/33 [=====] - 214s 6s/step - loss: 0.0085 - accuracy: 0.9990
- val_loss: 0.6009 - val_accuracy: 0.8756 - lr: 0.0020
Epoch 14/30
33/33 [=====] - 222s 7s/step - loss: 0.0053 - accuracy: 0.9990
- val_loss: 0.6065 - val_accuracy: 0.8756 - lr: 0.0020
Epoch 15/30
33/33 [=====] - 219s 7s/step - loss: 0.0040 - accuracy: 1.0000
- val_loss: 0.6124 - val_accuracy: 0.8667 - lr: 0.0020
8/8 [=====] - 8s 1s/step - loss: 0.4234 - accuracy: 0.7956
Test Accuracy: 79.56%

```

```

In [ ]: def buildTrainData(catheter, train_data, size):

        train_data = train_data[train_data[f'{catheter} - Present'] == 1]
        train_data = train_data[~((train_data[f'{catheter} - Normal'] == 1) |
        train_data[f'{catheter} - Normal/Abnormal'])
        sampled_data = pd.concat([train_data[train_data[f'{catheter} - Normal'] == 1].sample(
        size, random_state=0),
        train_data[train_data[f'{catheter} - Present'] == 1].sample(
        size, random_state=0)])

        return sampled_data

```

```

In [ ]: sample = buildTrainData('NGT', train_data, 250)
sample[['NGT - Normal', 'NGT - Abnormal']].value_counts()

```

```
sample[['NGT - Normal', 'NGT - Abnormal']].value_counts()
```

```
Out[ ]: NGT - Normal  NGT - Abnormal
0           1           250
1           0           250
Name: count, dtype: int64
```

```
In [ ]: nAb_data = getFileNames(sample)

ngt_imgs, fileIDs = processImages(nAb_data, 'train')
```

100%|██████████| 500/500 [00:19<00:00, 26.20it/s]

```
In [ ]: UID = []
for i in range(len(fileIDs)):
    UID.append(fileIDs[i].replace(".jpg", ""))

columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal',
                  'NGT - Incompletely Imaged', 'Swan Ganz Catheter Present', 'Patient']

NGTsample = sample.drop(columns=columns_to_drop, axis=1)

NGTsample.set_index('StudyInstanceUID', inplace=True)

NGTsample
```

```
Out[ ]:
```

	NGT - Normal/Abnormal
--	--------------------------

StudyInstanceUID	
1.2.826.0.1.3680043.8.498.83557702852578344152590975521603931676	1
1.2.826.0.1.3680043.8.498.33475188387649615865841783605310668224	1
1.2.826.0.1.3680043.8.498.75705881484822777811302505593938591112	1
1.2.826.0.1.3680043.8.498.82830037186816678378898651543905905890	1
1.2.826.0.1.3680043.8.498.12422333356367561670298118556489412076	1
...	...
1.2.826.0.1.3680043.8.498.77213305346319900514294160115528863198	0
1.2.826.0.1.3680043.8.498.12707801410845250972678487173558836809	0
1.2.826.0.1.3680043.8.498.64848197645682080966935746451622376111	0
1.2.826.0.1.3680043.8.498.49651108389255271593507462962326144550	0
1.2.826.0.1.3680043.8.498.82069748487050441478024818899239025360	0

500 rows × 1 columns

```
In [ ]: # Extract UIDs from fileIDs
UID = [fileID.replace(".jpg", "") for fileID in fileIDs]

# List of columns to drop
columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal',
                  'NGT - Incompletely Imaged', 'Swan Ganz Catheter Present', 'Patient',
                  'CVC - Abnormal', 'CVC - Normal', 'CVC - Present']

# Drop the columns and set index
NGTsample = sample.drop(columns=columns_to_drop, errors='ignore') # 'errors' parameter
NGTsample.set_index('StudyInstanceUID', inplace=True)

NGTsample
```

NGT - Present	NG Normal/Abnormal
---------------	--------------------

500 rows × 2 columns

```
train_labels_df = np.empty((0,3))
data = []

for ID in UID:
    data.append(NGTsample.loc[ID])

train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)
ngt_labels = train_labels_df['NGT - Normal/Abnormal'].values
# train_labels = train_labels_df.values
ngt_labels
```

```
from sklearn.model_selection import train_test_split

# Split your data into training and temporary data (combined validation and test)
```

```

# Split your data into training and temporary data (combined validation and test)
trainNGT_imgs, temp_imgs, trainNGT_labels, temp_labels = train_test_split(ngt_imgs, n

# Split the temporary data into validation and test sets
valNGT_imgs, testNGT_imgs, valNGT_labels, testNGT_labels = train_test_split(temp_imgs

```

In []:

```

from tensorflow.keras.applications.densenet import DenseNet121
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Dropout, BatchNormalization
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
base_model = DenseNet121(weights='imagenet', include_top=False)

# Add dropout and batch normalization to custom fully connected layers
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.5)(x)
predictions = Dense(1, activation='sigmoid')(x)
model = Model(inputs=base_model.input, outputs=predictions)

# Reduce learning rate when the validation loss plateaus
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=1e-6)

# Compile the model with optimizer and loss function
model.compile(optimizer='SGD', loss='binary_crossentropy', metrics=['accuracy'])

# Early stopping to prevent overfitting
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Save the best model during training
model_checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True)

# Model Training with callbacks
history = model.fit(trainNGT_imgs, trainNGT_labels, epochs=30, validation_data=(val_i

# Evaluate the model
test_loss, test_accuracy = model.evaluate(test_imgs, test_labels)
print(f'Test Accuracy: {test_accuracy * 100:.2f}%')

```

```

Epoch 1/30
13/13 [=====] - 106s 7s/step - loss: 0.8169 - accuracy: 0.6050
- val_loss: 0.7350 - val_accuracy: 0.4844 - lr: 0.0100
Epoch 2/30
13/13 [=====] - 87s 7s/step - loss: 0.6494 - accuracy: 0.6725
- val_loss: 0.6992 - val_accuracy: 0.4800 - lr: 0.0100
Epoch 3/30
13/13 [=====] - 88s 7s/step - loss: 0.4995 - accuracy: 0.7775
- val_loss: 0.7756 - val_accuracy: 0.4667 - lr: 0.0100
Epoch 4/30
13/13 [=====] - 89s 7s/step - loss: 0.2937 - accuracy: 0.8875
- val_loss: 0.6905 - val_accuracy: 0.5422 - lr: 0.0100
Epoch 5/30
13/13 [=====] - 87s 7s/step - loss: 0.2128 - accuracy: 0.9225
- val_loss: 0.6453 - val_accuracy: 0.6089 - lr: 0.0100
Epoch 6/30
13/13 [=====] - 86s 7s/step - loss: 0.0854 - accuracy: 0.9675
- val_loss: 0.6639 - val_accuracy: 0.5778 - lr: 0.0100
Epoch 7/30
13/13 [=====] - 88s 7s/step - loss: 0.0450 - accuracy: 0.9875
- val_loss: 0.6268 - val_accuracy: 0.6578 - lr: 0.0100
Epoch 8/30
13/13 [=====] - 86s 7s/step - loss: 0.0308 - accuracy: 0.9950
- val_loss: 0.6437 - val_accuracy: 0.6267 - lr: 0.0100
Epoch 9/30
13/13 [=====] - 86s 7s/step - loss: 0.0207 - accuracy: 1.0000

```



```

- val_loss: 0.6352 - val_accuracy: 0.6622 - lr: 0.0100
Epoch 10/30
13/13 [=====] - 86s 7s/step - loss: 0.0073 - accuracy: 1.0000
- val_loss: 0.6646 - val_accuracy: 0.6756 - lr: 0.0100
Epoch 11/30
13/13 [=====] - 86s 7s/step - loss: 0.0092 - accuracy: 1.0000
- val_loss: 0.6716 - val_accuracy: 0.6978 - lr: 0.0100
Epoch 12/30
13/13 [=====] - 87s 7s/step - loss: 0.0074 - accuracy: 1.0000
- val_loss: 0.6829 - val_accuracy: 0.6844 - lr: 0.0100
Epoch 13/30
13/13 [=====] - 87s 7s/step - loss: 0.0076 - accuracy: 1.0000
- val_loss: 0.7033 - val_accuracy: 0.7067 - lr: 0.0020
Epoch 14/30
13/13 [=====] - 87s 7s/step - loss: 0.0059 - accuracy: 1.0000
- val_loss: 0.7239 - val_accuracy: 0.6978 - lr: 0.0020
Epoch 15/30
13/13 [=====] - 88s 7s/step - loss: 0.0055 - accuracy: 1.0000
- val_loss: 0.7463 - val_accuracy: 0.6889 - lr: 0.0020
Epoch 16/30
13/13 [=====] - 88s 7s/step - loss: 0.0060 - accuracy: 1.0000
- val_loss: 0.7640 - val_accuracy: 0.6933 - lr: 0.0020
Epoch 17/30
13/13 [=====] - 88s 7s/step - loss: 0.0043 - accuracy: 1.0000
- val_loss: 0.7804 - val_accuracy: 0.7067 - lr: 0.0020
8/8 [=====] - 8s 959ms/step - loss: 0.5868 - accuracy: 0.7289
Test Accuracy: 72.89%

```

```
In [ ]: pip install seaborn
```

Collecting seaborn

Downloading seaborn-0.13.0-py3-none-any.whl (294 kB)

294.6/294.6 kB 6.1 MB/s eta 0:00:00a 0:00:01

Requirement already satisfied: matplotlib!=3.6.1,>=3.3 in /usr/local/lib/python3.8/site-packages (from seaborn) (3.7.2)

Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.8/site-packages (from seaborn) (2.0.3)

Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.8/site-packages (from seaborn) (1.23.5)

Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.8/site-packages (from matplotlib!=3.6.1,>=3.3->seaborn) (1.4.4)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.8/site-packages (from matplotlib!=3.6.1,>=3.3->seaborn) (0.11.0)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.8/site-packages (from matplotlib!=3.6.1,>=3.3->seaborn) (2.8.2)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.8/site-packages (from matplotlib!=3.6.1,>=3.3->seaborn) (23.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.8/site-packages (from matplotlib!=3.6.1,>=3.3->seaborn) (4.40.0)

Requirement already satisfied: importlib-resources>=3.2.0 in /usr/local/lib/python3.8/site-packages (from matplotlib!=3.6.1,>=3.3->seaborn) (6.0.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.8/site-packages (from matplotlib!=3.6.1,>=3.3->seaborn) (1.1.0)

Requirement already satisfied: pyparsing<3.1,>=2.3.1 in /usr/local/lib/python3.8/site-packages (from matplotlib!=3.6.1,>=3.3->seaborn) (3.0.9)

Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.8/site-packages (from matplotlib!=3.6.1,>=3.3->seaborn) (10.0.0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.8/site-packages (from pandas>=1.2->seaborn) (2023.3)

Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.8/site-packages (from pandas>=1.2->seaborn) (2023.3)

Requirement already satisfied: zipp>=3.1.0 in /usr/local/lib/python3.8/site-packages (from importlib-resources>=3.2.0->matplotlib!=3.6.1,>=3.3->seaborn) (3.15.0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.8/site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.3->seaborn) (1.16.0)

Installing collected packages: seaborn

Successfully installed seaborn-0.13.0

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip is available: 23.0.1 -> 23.3

[notice] To update, run: `pip install --upgrade pip`

Note: you may need to restart the kernel to use updated packages.

In []:

```
import seaborn as sns

# Make predictions
predictions = model.predict(test_imgs)

# Convert the predictions to a binary array
predictions_binary = np.where(predictions > 0.5, 1, 0)

from sklearn.metrics import confusion_matrix

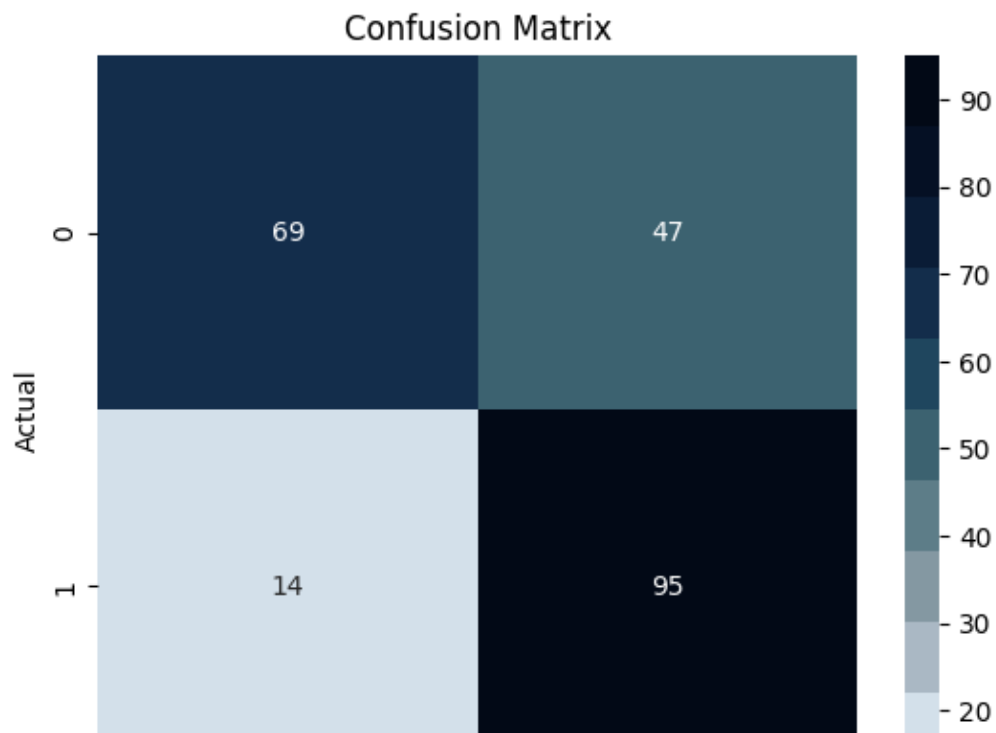
# Create the confusion matrix
confusion_matrix = confusion_matrix(test_labels, predictions_binary)

plt.gca().patch.set_facecolor('none')

custom_palette = ['#D3E0EA', '#AAB8C4', '#8498A2', '#5D7D88', '#3C6270',
                  '#1F465E', '#132D4B', '#0A1C36', '#051024', '#020916']

# Create a heatmap plot of the confusion matrix
sns.heatmap(confusion_matrix, annot=True, cmap=custom_palette)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

8/8 [=====] - 10s 961ms/step



DenseNet - ETT

```
In [ ]: import random
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
from PIL import Image
import ast
import os
import cv2
from tqdm import tqdm
```

/opt/conda/lib/python3.10/site-packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.16.3.5
warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")

```
In [ ]: train_data = pd.read_csv("/kaggle/input/ranzcr-clip-catheter-line-classification/train.csv")
train_data = train_data[(train_data['ETT - Borderline'] != 1)]

train_data[['ETT - Normal', 'ETT - Abnormal', 'ETT - Borderline']].value_counts()
train_data['ETT - Present'] = np.where((train_data['ETT - Normal'] == 0) & (train_data['ETT - A

train_data[['ETT - Normal', 'ETT - Abnormal']].value_counts()
```

```
Out[ ]: ETT - Normal  ETT - Abnormal
0                0          21626
1                0          7240
0                1           79
Name: count, dtype: int64
```

```
In [ ]: normalAb = pd.concat([train_data[train_data['ETT - Normal'] == 1].sample(n=39), train_data[trai
sampled_data = pd.concat([normalAb, train_data[train_data['ETT - Present'] == 0].sample(n=78)])

sampled_data
```

```
Out[ ]:
```

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT - Abnormal	NGT - Borderline
114	1.2.826.0.1.3680043.8.498.58978399544484725012...	0	0	1	0	
29215	1.2.826.0.1.3680043.8.498.34414786983144528822...	0	0	1	0	
27577	1.2.826.0.1.3680043.8.498.13206394033751660209...	0	0	1	0	
15863	1.2.826.0.1.3680043.8.498.12943991364992320755...	0	0	1	0	
11959	1.2.826.0.1.3680043.8.498.11818325818713851605...	0	0	1	0	
...	...	...	...	...	...	
15055	1.2.826.0.1.3680043.8.498.12942904850809349583...	0	0	0	0	
21807	1.2.826.0.1.3680043.8.498.88179133086149977138...	0	0	0	0	
1272	1.2.826.0.1.3680043.8.498.17783739974925320326...	0	0	0	0	
11015	1.2.826.0.1.3680043.8.498.11326606389259498883...	0	0	0	0	
6567	1.2.826.0.1.3680043.8.498.13680799914598090743...	0	0	0	0	

Preview

Code

Blame

886 lines (886 loc) · 97.2 KB

Raw



```
def getFileNames(df):
    studyInstance = df['StudyInstanceUID'].tolist()
    studyInstance

    for i in range(len(studyInstance)):
        studyInstance[i] = f'{studyInstance[i]}.jpg'

    return studyInstance
```

```
In [ ]: studyInstance = getFileNames(sampled_data)
```

```
In [ ]: def processImages(filenamees,train_test):

    imgs = []
    FileIDs = []
    lst = []

    desired_channels = 3

    count = 0
    for file in tqdm(filenamees, position=0, leave=True):
        # Making sure the images are the same pixel size
        img = Image.open(f"/kaggle/input/ranzcr-clip-catheter-line-classification/{train_test}/"
                        + file)
        img = img.resize((300, 300))
        img = np.array(img)

        # Ensure that the image has 3 channels (RGB)
        if img.shape[-1] != 3:
            img = np.stack((img,) * 3, axis=-1) # Convert grayscale to RGB by duplicating chan

        imgs.append(img) # Append the 4D image to the list

        # Save filename for later
        FileIDs.append(file)

        count += 1

    # Convert the List of images to a numpy array
    imgs = np.array(imgs)
    return imgs, FileIDs
```

```
In [ ]: train_imgs, fileIDs = processImages(studyInstance, 'train')
```

```
100%|██████████| 156/156 [00:08<00:00, 18.96it/s]
```

```
In [ ]: #cleaning up the labels
        UID = []
        for i in range(len(fileIDs)):
            UID.append(fileIDs[i].replace(".jpg",""))

        UID[0:10]
```

```
Out[ ]: ['1.2.826.0.1.3680043.8.498.58978399544484725012990490099074395827',
        '1.2.826.0.1.3680043.8.498.34414786983144528822093886496110328396',
        '1.2.826.0.1.3680043.8.498.13206394033751660209376456532079454754',
        '1.2.826.0.1.3680043.8.498.12943991364992320755105760439294630123',
        '1.2.826.0.1.3680043.8.498.11818325818713851605005356351281851103',
        '1.2.826.0.1.3680043.8.498.10310363550730341037663245917488147977',
        '1.2.826.0.1.3680043.8.498.75345280158369398004405022398954691780',
        '1.2.826.0.1.3680043.8.498.96692014265830615336985041998566357144',
        '1.2.826.0.1.3680043.8.498.57562706261208256656080246211038215270',
        '1.2.826.0.1.3680043.8.498.57562706261208256656080246211038215270']
```

```
1.2.826.0.1.3680043.8.498.57562706261298256656000346211938315370 ;  
'1.2.826.0.1.3680043.8.498.53556272721280942362238528028253111101']
```

```
In [ ]: columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal', 'NGT  
"NGT - Incompletely Imaged", "Swan Ganz Catheter Present", "PatientID", "CVC  
  
ETTsampled_data = sampled_data.drop(columns=columns_to_drop, axis=1)  
  
ETTsampled_data.set_index('StudyInstanceUID', inplace=True)
```

```
In [ ]: train_labels_df = np.empty((0,3))  
data = []  
  
for ID in UID:  
    data.append(ETTsampled_data.loc[ID])  
  
train_labels_df = pd.DataFrame(data)  
  
train_labels_df = train_labels_df.reindex(UID)  
train_labels = train_labels_df['ETT - Present'].values  
train_labels
```

```
Out[ ]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
0, 0])
```

```
In [ ]: from sklearn.model_selection import train_test_split  
  
# Split your data into training and temporary data (combined validation and test)  
train_imgs, temp_imgs, train_labels, temp_labels = train_test_split(train_imgs, train_labels, t  
  
# Split the temporary data into validation and test sets  
val_imgs, test_imgs, val_labels, test_labels = train_test_split(temp_imgs, temp_labels, test_si
```

```
In [ ]: import numpy as np  
import tensorflow as tf  
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
```

```
In [ ]: def buildTrainData(catheter, train_data, size):  
  
    train_data = train_data[train_data[f'{catheter} - Present'] == 1]  
    train_data = train_data[~((train_data[f'{catheter} - Normal'] == 1) & (train_d  
    train_data[f'{catheter} - Normal/Abnormal'] = np.where(train_data[f'{catheter} - Normal'] =  
    sampled_data = pd.concat([train_data[train_data[f'{catheter} - Normal'] == 1].sample(n=size  
  
    return sampled_data
```

```
In [ ]: sample = buildTrainData('ETT', train_data, 78)  
sample
```

```
Out[ ]:
```

	StudyInstanceUID	ETT - Abnormal	ETT - Borderline	ETT - Normal	NGT - Abnormal	N Border
5514	1.2.826.0.1.3680043.8.498.62482621549066450941...	0	0	1	0	
21633	1.2.826.0.1.3680043.8.498.11143364354882692909...	0	0	1	0	
24111	1.2.826.0.1.3680043.8.498.78715330304045487459...	0	0	1	0	

24235	1.2.826.0.1.3680043.8.498.12355228832479391064...	0	0	1	0
3004	1.2.826.0.1.3680043.8.498.64225013817365417818...	0	0	1	0
...	...	...	...	...	...
25183	1.2.826.0.1.3680043.8.498.84533062276529819766...	1	0	0	0
10679	1.2.826.0.1.3680043.8.498.15739917130183288582...	1	0	0	0
11092	1.2.826.0.1.3680043.8.498.66853777496301033011...	1	0	0	0
15399	1.2.826.0.1.3680043.8.498.61889973816225513932...	1	0	0	0
7044	1.2.826.0.1.3680043.8.498.64944381826947752542...	1	0	0	0

156 rows × 15 columns



```
In [ ]: nAb_data = getFileNames(sample)
        ett_imgs, fileIDs = processImages(nAb_data, 'train')
```

100%|██████████| 156/156 [00:07<00:00, 20.95it/s]

```
In [ ]: UID = []
        for i in range(len(fileIDs)):
            UID.append(fileIDs[i].replace(".jpg", ""))

        columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal', 'NGT
                           "NGT - Incompletely Imaged", "Swan Ganz Catheter Present", "PatientID", "CVC

        ETTsample = sample.drop(columns=columns_to_drop, axis=1)
        ETTsample.set_index('StudyInstanceUID', inplace=True)

        ETTsample
```

	ETT - Normal/Abnormal
StudyInstanceUID	
1.2.826.0.1.3680043.8.498.62482621549066450941742616968051043277	1
1.2.826.0.1.3680043.8.498.11143364354882692909222821800764262312	1
1.2.826.0.1.3680043.8.498.78715330304045487459143788276776992885	1
1.2.826.0.1.3680043.8.498.12355228832479391064189272904637360930	1
1.2.826.0.1.3680043.8.498.64225013817365417818047236662954334719	1
...	...
1.2.826.0.1.3680043.8.498.84533062276529819766171648406000766083	0
1.2.826.0.1.3680043.8.498.15739917130183288582495374522364822377	0
1.2.826.0.1.3680043.8.498.66853777496301033011786823921410375380	0
1.2.826.0.1.3680043.8.498.61889973816225513932209370012107580731	0
1.2.826.0.1.3680043.8.498.64944381826947752542630851918799645855	0

156 rows × 1 columns

```
In [ ]: train_labels_df = np.empty((0,3))
        data = []

        for ID in UID:
            data.append(ETTsample.loc[ID])
```

```
train_labels_df = pd.DataFrame(data)

train_labels_df = train_labels_df.reindex(UID)
ett_labels = train_labels_df['ETT - Normal/Abnormal'].values
ett_labels
```

[illegible]

```
In [ ]: from sklearn.model_selection import train_test_split

# Split your data into training and temporary data (combined validation and test)
trainETT_imgs, temp_imgs, trainETT_labels, temp_labels = train_test_split(ett_imgs, ett_labels,

# Split the temporary data into validation and test sets
valETT_imgs, testETT_imgs, valETT_labels, testETT_labels = train_test_split(temp_imgs, temp_labels,
```

```
In [ ]:
from tensorflow.keras.layers import concatenate
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dropout, BatchNormalization
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Load the DenseNet121 model with pre-trained weights
base_model = DenseNet121(weights='imagenet', include_top=False)

# Add dropout and batch normalization to custom fully connected layers
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.5)(x)

# First Output Branch: CVC Catheter Presence (Binary Classification)
output_ETTpresent = Dense(1, activation='sigmoid', name='output_ETTpresent')(x)

# Add a separate set of layers for the new task
y = base_model.output
y = GlobalAveragePooling2D()(y)
y = Dense(1024, activation='relu')(y)
y = BatchNormalization()(y)
y = Dropout(0.5)(y)

# Second Output Branch: CVC Placement (Binary Classification)
output_ett_placement = Dense(1, activation='sigmoid', name='output_ETTnormalAbnormal')(y)

# Combine the two output branches
combined_model = Model(inputs=base_model.input, outputs=[output_ETTpresent, output_ett_placemen

# Compile the combined model with optimizer and loss functions for both tasks
combined_model.compile(optimizer='SGD',
                        loss={'output_ETTpresent': 'binary_crossentropy', 'output_ETTnormalAbnor
                        metrics={'output_ETTpresent': 'accuracy', 'output_ETTnormalAbnormal': 'a

# # Reduce learning rate when the validation loss plateaus
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=1e-6)

# Early stopping to prevent overfitting
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Save the best model during training
model_checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True)
```

```

history = combined_model.fit(
    {'input_1': train_imgs},
    {'output_ETTpresent': train_labels, 'output_ETTnormalAbnormal': trainETT_labels},
    epochs=30,
    validation_data=(
        {'input_1': val_imgs},
        {'output_ETTpresent': val_labels, 'output_ETTnormalAbnormal': valETT_labels}
    ),
    callbacks=[reduce_lr, early_stop, model_checkpoint]
)

```

Downloading data from <https://storage.googleapis.com/tensorflow/keras-applications/densenet/denser29084464/29084464> [=====] - 1s 0us/step

Epoch 1/30

4/4 [=====] - 52s 4s/step - loss: 1.4512 - output_ETTpresent_loss: 0.7217
13 - output_ETTnormalAbnormal_accuracy: 0.6532 - val_loss: 1.5578 - val_output_ETTpresent_loss: 0.
accuracy: 0.5625 - val_output_ETTnormalAbnormal_accuracy: 0.5625 - lr: 0.0100

Epoch 2/30

4/4 [=====] - 2s 680ms/step - loss: 0.6651 - output_ETTpresent_loss: 0.33
8710 - output_ETTnormalAbnormal_accuracy: 0.8468 - val_loss: 1.3220 - val_output_ETTpresent_loss:
accuracy: 0.5625 - val_output_ETTnormalAbnormal_accuracy: 0.5625 - lr: 0.0100

Epoch 3/30

4/4 [=====] - 2s 680ms/step - loss: 0.3179 - output_ETTpresent_loss: 0.14
9516 - output_ETTnormalAbnormal_accuracy: 0.9516 - val_loss: 1.2560 - val_output_ETTpresent_loss:
accuracy: 0.6250 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0100

Epoch 4/30

4/4 [=====] - 1s 316ms/step - loss: 0.1800 - output_ETTpresent_loss: 0.11
9435 - output_ETTnormalAbnormal_accuracy: 0.9839 - val_loss: 1.2838 - val_output_ETTpresent_loss:
accuracy: 0.8125 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0100

Epoch 5/30

4/4 [=====] - 1s 315ms/step - loss: 0.0880 - output_ETTpresent_loss: 0.03
0000 - output_ETTnormalAbnormal_accuracy: 0.9919 - val_loss: 1.2812 - val_output_ETTpresent_loss:
accuracy: 0.6875 - val_output_ETTnormalAbnormal_accuracy: 0.6250 - lr: 0.0100

Epoch 6/30

4/4 [=====] - 2s 686ms/step - loss: 0.0413 - output_ETTpresent_loss: 0.01
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.2468 - val_output_ETTpresent_loss:
accuracy: 0.6875 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100

Epoch 7/30

4/4 [=====] - 2s 694ms/step - loss: 0.0280 - output_ETTpresent_loss: 0.01
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.1881 - val_output_ETTpresent_loss:
accuracy: 0.7500 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100

Epoch 8/30

4/4 [=====] - 3s 739ms/step - loss: 0.0241 - output_ETTpresent_loss: 0.01
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.1472 - val_output_ETTpresent_loss:
accuracy: 0.8125 - val_output_ETTnormalAbnormal_accuracy: 0.8750 - lr: 0.0100

Epoch 9/30

4/4 [=====] - 2s 686ms/step - loss: 0.0344 - output_ETTpresent_loss: 0.01
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0780 - val_output_ETTpresent_loss:
accuracy: 0.8125 - val_output_ETTnormalAbnormal_accuracy: 0.9375 - lr: 0.0100

Epoch 10/30

4/4 [=====] - 2s 693ms/step - loss: 0.0163 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0328 - val_output_ETTpresent_loss:
accuracy: 0.8750 - val_output_ETTnormalAbnormal_accuracy: 0.9375 - lr: 0.0100

Epoch 11/30

4/4 [=====] - 2s 683ms/step - loss: 0.0100 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0060 - val_output_ETTpresent_loss:
accuracy: 0.8750 - val_output_ETTnormalAbnormal_accuracy: 0.8750 - lr: 0.0100

Epoch 12/30

4/4 [=====] - 2s 719ms/step - loss: 0.0094 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.9590 - val_output_ETTpresent_loss:
accuracy: 0.9375 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100

Epoch 13/30

4/4 [=====] - 2s 672ms/step - loss: 0.0090 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.9321 - val_output_ETTpresent_loss:
accuracy: 0.8750 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100

Epoch 14/30

4/4 [=====] - 1s 316ms/step - loss: 0.0133 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.9348 - val_output_ETTpresent_loss:
accuracy: 0.8125 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0100

Epoch 15/30

4/4 [=====] - 1s 317ms/step - loss: 0.0271 - output_ETTpresent_loss: 0.01

```

0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0147 - val_output_ETTpresent_loss:
accuracy: 0.6250 - val_output_ETTnormalAbnormal_accuracy: 0.6875 - lr: 0.0100
Epoch 16/30
4/4 [=====] - 1s 317ms/step - loss: 0.0057 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0168 - val_output_ETTpresent_loss:
accuracy: 0.6250 - val_output_ETTnormalAbnormal_accuracy: 0.6250 - lr: 0.0100
Epoch 17/30
4/4 [=====] - 1s 316ms/step - loss: 0.0069 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 1.0158 - val_output_ETTpresent_loss:
accuracy: 0.6250 - val_output_ETTnormalAbnormal_accuracy: 0.6250 - lr: 0.0100
Epoch 18/30
4/4 [=====] - 3s 794ms/step - loss: 0.0319 - output_ETTpresent_loss: 0.01
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8981 - val_output_ETTpresent_loss:
accuracy: 0.9375 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0100
Epoch 19/30
4/4 [=====] - 2s 693ms/step - loss: 0.0108 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8792 - val_output_ETTpresent_loss:
accuracy: 0.8750 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0100
Epoch 20/30
4/4 [=====] - 2s 695ms/step - loss: 0.0048 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8523 - val_output_ETTpresent_loss:
accuracy: 0.9375 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0100
Epoch 21/30
4/4 [=====] - 1s 317ms/step - loss: 0.0059 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8555 - val_output_ETTpresent_loss:
accuracy: 0.8750 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0100
Epoch 22/30
4/4 [=====] - 1s 317ms/step - loss: 0.0036 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8544 - val_output_ETTpresent_loss:
accuracy: 0.8750 - val_output_ETTnormalAbnormal_accuracy: 0.7500 - lr: 0.0100
Epoch 23/30
4/4 [=====] - 2s 694ms/step - loss: 0.0087 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.8033 - val_output_ETTpresent_loss:
accuracy: 0.8750 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100
Epoch 24/30
4/4 [=====] - 2s 706ms/step - loss: 0.0041 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.7882 - val_output_ETTpresent_loss:
accuracy: 0.8750 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100
Epoch 25/30
4/4 [=====] - 2s 687ms/step - loss: 0.0034 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.7779 - val_output_ETTpresent_loss:
accuracy: 0.8750 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100
Epoch 26/30
4/4 [=====] - 2s 696ms/step - loss: 0.0040 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.7697 - val_output_ETTpresent_loss:
accuracy: 0.9375 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100
Epoch 27/30
4/4 [=====] - 2s 732ms/step - loss: 0.0030 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.7639 - val_output_ETTpresent_loss:
accuracy: 0.9375 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100
Epoch 28/30
4/4 [=====] - 1s 323ms/step - loss: 0.0078 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.7781 - val_output_ETTpresent_loss:
accuracy: 0.9375 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100
Epoch 29/30
4/4 [=====] - 2s 691ms/step - loss: 0.0037 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.7534 - val_output_ETTpresent_loss:
accuracy: 0.9375 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100
Epoch 30/30
4/4 [=====] - 2s 682ms/step - loss: 0.0050 - output_ETTpresent_loss: 0.00
0000 - output_ETTnormalAbnormal_accuracy: 1.0000 - val_loss: 0.7332 - val_output_ETTpresent_loss:
accuracy: 0.9375 - val_output_ETTnormalAbnormal_accuracy: 0.8125 - lr: 0.0100

```

In []:

```

# Evaluate accuracy for the 'output_catheter' branch
evaluation = combined_model.evaluate(
    {'input_1': test_imgs},
    {'output_ETTpresent': test_labels, 'output_ETTnormalAbnormal': testETT_labels},
)

# Extract the accuracy values from the evaluation results
test_accuracy_catheter = evaluation[3] # Index 3 corresponds to 'output_catheter_accuracy'
test_accuracy_cvc_placement = evaluation[4] # Index 4 corresponds to 'output cvc placement acc

```



```
print(f'Test Accuracy (ETT Present): {test_accuracy_catheter * 100:.2f}%')
print(f'Test Accuracy (ETT Normal/Abnormal): {test_accuracy_cvc_placement * 100:.2f}%')
```

```
1/1 [=====] - 0s 82ms/step - loss: 0.6107 - output_ETTpresent_loss: 0.281
375 - output_ETTnormalAbnormal_accuracy: 0.8750
Test Accuracy (ETT Present): 93.75%
Test Accuracy (ETT Normal/Abnormal): 87.50%
```

```
In [ ]: test_path = "/kaggle/input/ranzcr-clip-catheter-line-classification/test"

all_files = os.listdir(test_path)

test_files = all_files[:300]
```

```
In [ ]: test_img, testFiles = processImages(test_files, "test")
```

```
100%|██████████| 300/300 [00:15<00:00, 19.78it/s]
```

```
In [ ]: predictions = combined_model.predict(test_imgs)
predictions

# Access predictions for the first task
ngt_present_predictions = predictions[0]
ngt_present_predictions
# Access predictions for the second task
ngt_normal_abnormal_predictions = predictions[1]

ngt_present_predictions
# Create two separate DataFrames
df1 = pd.DataFrame(ngt_present_predictions, columns=["ETT - Present"])
df2 = pd.DataFrame(ngt_normal_abnormal_predictions, columns=['ETT - Normal/Abnormal'])

# Concatenate the DataFrames horizontally to create one with two columns
NGTpredictions = pd.concat([df1, df2], axis=1)
NGTpredictions

NGTpredictions['ETT - Present'] = np.where(NGTpredictions['ETT - Present'] > 0.5, 1, 0)
NGTpredictions['ETT - Normal/Abnormal'] = np.where(NGTpredictions['ETT - Normal/Abnormal'] > 0.5, 1, 0)
NGTpredictions

from sklearn.metrics import confusion_matrix

# Create the confusion matrix
confusion_matrix_present = confusion_matrix(test_labels, NGTpredictions['ETT - Present'])

plt.gca().patch.set_facecolor('none')

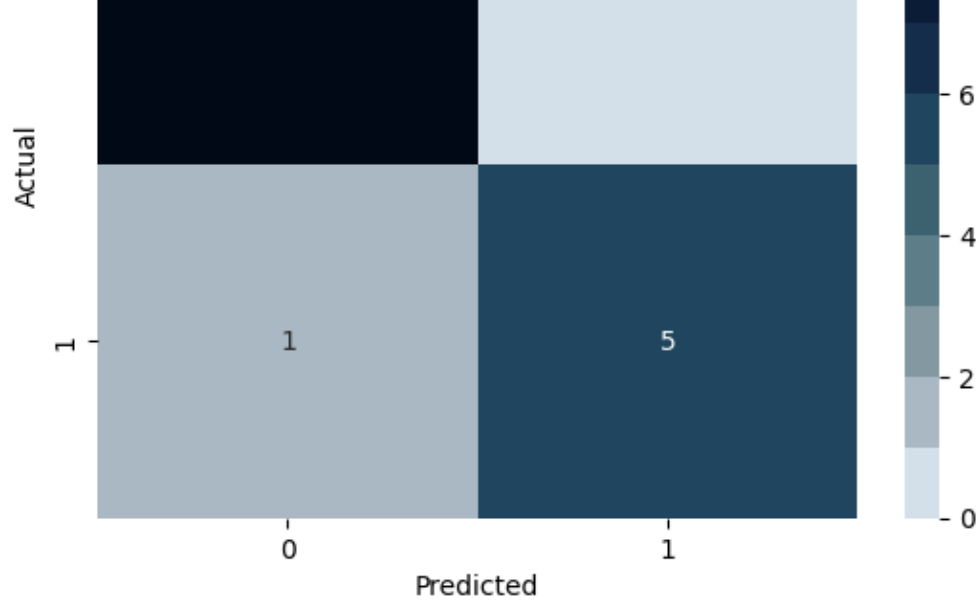
custom_palette = ['#D3E0EA', '#AAB8C4', '#8498A2', '#5D7D88', '#3C6270',
                  '#1F465E', '#132D4B', '#0A1C36', '#051024', '#020916']

# Create a heatmap plot of the confusion matrix
sns.heatmap(confusion_matrix_present, annot=True, cmap=custom_palette)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

```
1/1 [=====] - 2s 2s/step
```

Confusion Matrix



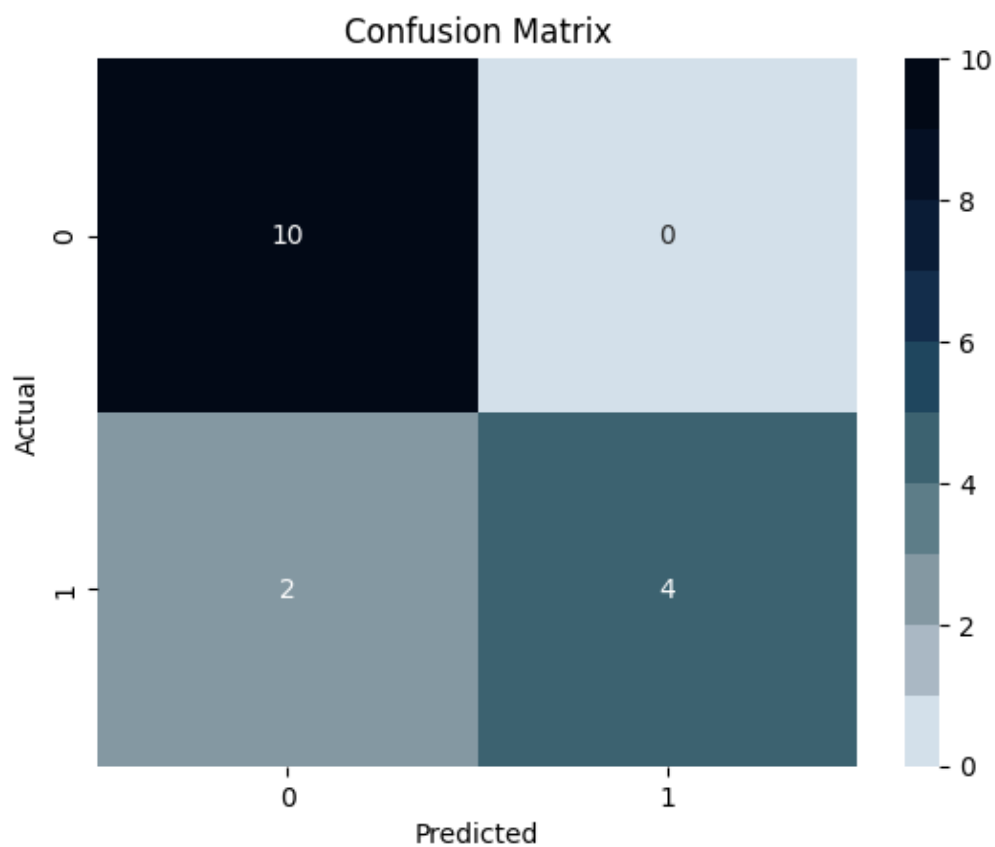


```
In [ ]: confusion_matrix_present = confusion_matrix(test_labels, NGTpredictions['ETT - Normal/Abnormal'])

plt.gca().patch.set_facecolor('none')

custom_palette = ['#D3E0EA', '#AAB8C4', '#8498A2', '#5D7D88', '#3C6270',
                  '#1F465E', '#132D4B', '#0A1C36', '#051024', '#020916']

# Create a heatmap plot of the confusion matrix
sns.heatmap(confusion_matrix_present, annot=True, cmap=custom_palette)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



UNet

```
In [ ]: %pip install segmentation_models_pytorch
```

```
Requirement already satisfied: segmentation_models_pytorch in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: torchvision>=0.5.0 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: pretrainedmodels==0.7.4 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: efficientnet-pytorch==0.7.1 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: timm==0.9.2 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: pillow in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: torch in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: munch in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: huggingface-hub in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: safetensors in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: sympy in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: triton==2.0.0 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: cmake in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: lit in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
Requirement already satisfied: mpmath>=0.19 in /usr/local/lib/python3.10/dist-packages (from segmentation_models_pytorch)
```

```
In [ ]: import tensorflow as tf
import ast
import numpy as np
import matplotlib.pyplot as plt
import os
import pandas as pd
from sklearn.model_selection import train_test_split
import torch
import torch.nn as nn
from torch.nn.functional import relu
from torch.utils.data import Dataset
from PIL import Image
from tqdm import tqdm
import segmentation_models_pytorch as smp
import warnings
warnings.filterwarnings("ignore")
from sklearn.metrics import confusion_matrix
import seaborn as sns
import cv2
from scipy import interpolate

print(tf.__version__)
```

2.13.0

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/conter

```
In [ ]: # checking if gpu is working
device_name = tf.test.gpu_device_name()

if "GPU" not in device_name:
    print("GPU device not found")

print('Found GPU at: {}'.format(device_name))

print("GPU", "available (YESS!!!!)" if tf.config.list_physical_devices("GPU") else "not availab
```

GPU device not found

Found GPU at:

GPU not available :(

```
In [ ]: df = pd.read_csv("/content/drive/MyDrive/ADS2002-Project-CatheterPlacement/Data/train.csv")
TRAIN_PATH = "/content/drive/MyDrive/ADS2002-Project-CatheterPlacement/Data/train"
anno_df = pd.read_csv("/content/drive/MyDrive/ADS2002-Project-CatheterPlacement/Data/train_anno
```

```
In [ ]: anno_df = anno_df[(anno_df['label'] == 'CVC - Normal') | (anno_df['label'] == 'CVC - Abnormal')]
anno_df["label"] = anno_df["label"].replace({"CVC - Normal": 1, "CVC - Abnormal": 0})
anno_df = anno_df.reset_index(drop=True)
```

```
In [ ]: anno_df["label"].value_counts()
```

```
Out[ ]: 1    7437
0     1206
Name: label, dtype: int64
```

```
In [ ]: anno_df
```

```
Out[ ]:
```

	StudyInstanceUID	label	data
0	1.2.826.0.1.3680043.8.498.12616281126973421762...	1	[[1487, 1279], [1477, 1168], [1472, 1052], [14...
1	1.2.826.0.1.3680043.8.498.12616281126973421762...	1	[[1328, 7], [1347, 101], [1383, 193], [1400, 2...
2	1.2.826.0.1.3680043.8.498.11697104485452001927...	1	[[1366, 961], [1411, 861], [1453, 751], [1508,...
3	1.2.826.0.1.3680043.8.498.87704688663091069148...	1	[[906, 604], [1103, 578], [1242, 607], [1459, ...
4	1.2.826.0.1.3680043.8.498.53113362093090654004...	1	[[1152, 938], [1193, 856], [1265, 795], [1362,...
...	...	...	...
8638	1.2.826.0.1.3680043.8.498.88885675487819792225...	1	[[1384, 1098], [1480, 899], [1554, 774], [1630...
8639	1.2.826.0.1.3680043.8.498.18590666552942008211...	1	[[1250, 1105], [1247, 1031], [1248, 970], [126...
8640	1.2.826.0.1.3680043.8.498.66774579023475418162...	1	[[1379, 1402], [1389, 1233], [1386, 1137], [13...
8641	1.2.826.0.1.3680043.8.498.18560506324214035535...	1	[[1302, 1543], [1308, 1488], [1317, 1355], [13...
8642	1.2.826.0.1.3680043.8.498.98119637144473510638...	0	[[1144, 1022], [1162, 934], [1179, 783], [1188...

8643 rows × 3 columns

```
In [ ]: df_1 = anno_df[anno_df["label"] == 1]
df_0 = anno_df[anno_df["label"] == 0]

# Sample 300 rows from each DataFrame
sampled_1 = df_1.sample(n=1000, random_state=42) # Using random_state for reproducibility
sampled_0 = df_0.sample(n=1000, random_state=42)
```

```

# Concatenate the samples to get a balanced subset
anno_df = pd.concat([sampled_1, sampled_0]).reset_index(drop=True)

test = anno_df.sample(frac=0.15, random_state=42)
anno_df = anno_df.drop(test.index).reset_index(drop=True)
test = test.reset_index(drop=True)
valid = anno_df.sample(frac=0.1, random_state=42)
anno_df = anno_df.drop(valid.index).reset_index(drop=True)
valid = valid.reset_index(drop=True)

```

```

In [ ]: # columns_to_drop = ['ETT - Abnormal', 'ETT - Borderline', 'ETT - Normal', 'NGT - Abnormal', 'N
#
# "NGT - Incompletely Imaged", "Swan Ganz Catheter Present", "PatientID", "CV

# df = df.drop(columns=columns_to_drop, axis = 1)

# df_0 = df[df["CVC - Normal"] == 0]
# df_1 = df[df["CVC - Normal"] == 1]

# # Sample 300 rows from each DataFrame
# sampled_0 = df_0.sample(n=350, random_state=42) # Using random_state for reproducibility
# sampled_1 = df_1.sample(n=350, random_state=42)

# # Concatenate the samples to get a balanced subset
# df = pd.concat([sampled_0, sampled_1]).reset_index(drop=True)

# test = df.sample(frac=0.15, random_state=42)
# df = df.drop(test.index).reset_index(drop=True)
# test = test.reset_index(drop=True)
# valid = df.sample(frac=0.1, random_state=42)
# df = df.drop(valid.index).reset_index(drop=True)
# valid = valid.reset_index(drop=True)

```

```

In [ ]: # multiple cathetrers per patient
matches_in_test = anno_df['StudyInstanceUID'].isin(test['StudyInstanceUID']).sum()
print("test matches:", matches_in_test)

matches_in_val = anno_df['StudyInstanceUID'].isin(valid['StudyInstanceUID']).sum()
print("valid matches:", matches_in_val)

```

test matches: 22
valid matches: 13

```

In [ ]: print("train", anno_df["label"].value_counts())
print("valid", valid["label"].value_counts())
print("test", test["label"].value_counts())

```

```

train 1    768
0     762
Name: label, dtype: int64
valid 0     87
1      83
Name: label, dtype: int64
test  0    151
1     149
Name: label, dtype: int64

```

```

In [ ]: imagenet_mean = np.array([0.485, 0.456, 0.406])
imagenet_std = np.array([0.229, 0.224, 0.225])

```

```

In [ ]: def _parse_annotation(raw):
    annotation = ast.literal_eval(raw)
    annotation = np.array(annotation, dtype=np.int32)
    annotation = np.expand_dims(annotation, axis=0)
    return annotation

```

```

In [ ]: def scale_annotations(annotation, original_shape, target_shape):

```

```

def scale_annotations(annotation, original_shape, target_shape):
    ratio_y = target_shape[0] / original_shape[0]
    ratio_x = target_shape[1] / original_shape[1]
    annotation_scaled = [[int(x*ratio_x), int(y*ratio_y)] for [x, y] in annotation]
    return annotation_scaled

def annotate(img, target, annotation):
    if len(img.shape) == 2:
        img = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)
    original_shape = img.shape[:2]
    annotation = _parse_annotation(annotation)
    annotation = annotation[0]
    # annotation = scale_annotations(annotation, original_shape, (256, 256))
    # print(annotation)
    img = cv2.polylines(img, [np.array(annotation)], False, color=(255, 0, 0), thickness=10)
    return img

```

In []:

```

class CustomDataset(Dataset):
    def __init__(self, anno_df):
        self.anno_df = anno_df
        self.anno_file_names = anno_df["StudyInstanceUID"].values
        self.labels = anno_df['label'].values

    def __len__(self):
        return len(self.anno_df)

    def __getitem__(self, idx):
        file_name = self.anno_file_names[idx]
        image_path = f"{TRAIN_PATH}/{file_name}.jpg"

        image = cv2.imread(image_path)
        image = annotate(image, self.anno_df["label"][idx], self.anno_df["data"][idx])
        image = cv2.resize(image, (256,256))
        image = np.asarray(image, dtype=np.float32) / 255.0
        image = (image - imagenet_mean) / imagenet_std

        labels = torch.tensor(self.labels[idx]).float()
        return image, torch.tensor(labels)

```

In []:

```

class TestDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df
        self.file_names = df['StudyInstanceUID'].values
        self.labels = df['label'].values

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        file_name = self.file_names[idx]
        image_path = f"{TRAIN_PATH}/{file_name}.jpg"

        image = cv2.imread(image_path)
        image = cv2.resize(image, (256,256))
        image = np.asarray(image, dtype=np.float32) / 255.0
        image = (image - imagenet_mean) / imagenet_std

        labels = torch.tensor(self.labels[idx]).float()
        return image, torch.tensor(labels)

```

In []:

```

train_dataset = CustomDataset(anno_df)
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=8, shuffle=True)

valid_dataset = CustomDataset(valid)
valid_loader = torch.utils.data.DataLoader(valid_dataset, batch_size=8, shuffle=True)

test_dataset = CustomDataset(test)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=8, shuffle=False)

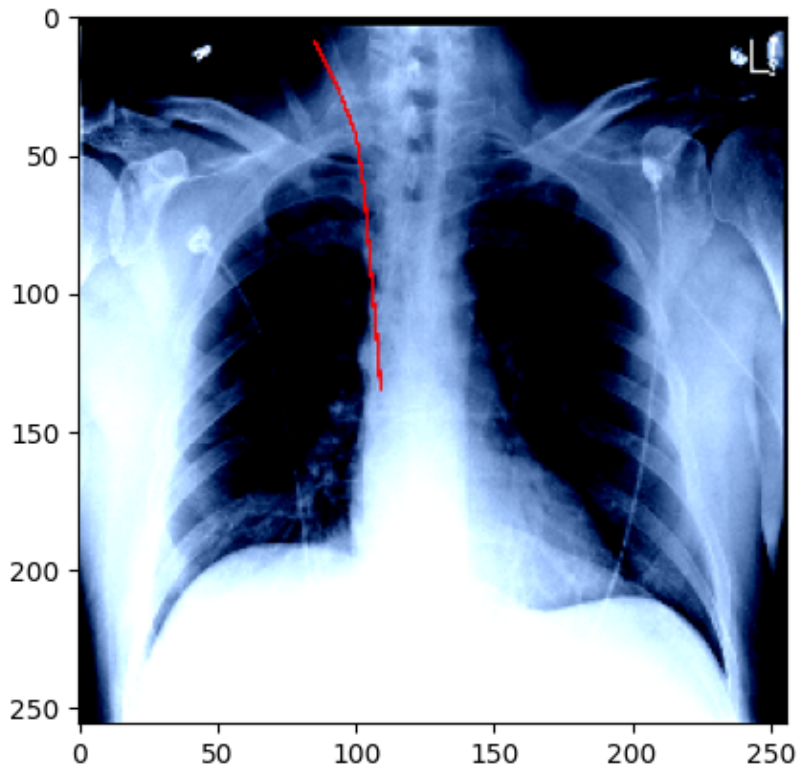
```

```
In [ ]: test_no_annot = TestDataset(test)
        test_loader_no_annot = torch.utils.data.DataLoader(test_no_annot, batch_size=8, shuffle=False)
```

```
In [ ]: plt.imshow(train_dataset[5][0])
```

WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] f

```
Out[ ]: <matplotlib.image.AxesImage at 0x7a069cda1ab0>
```



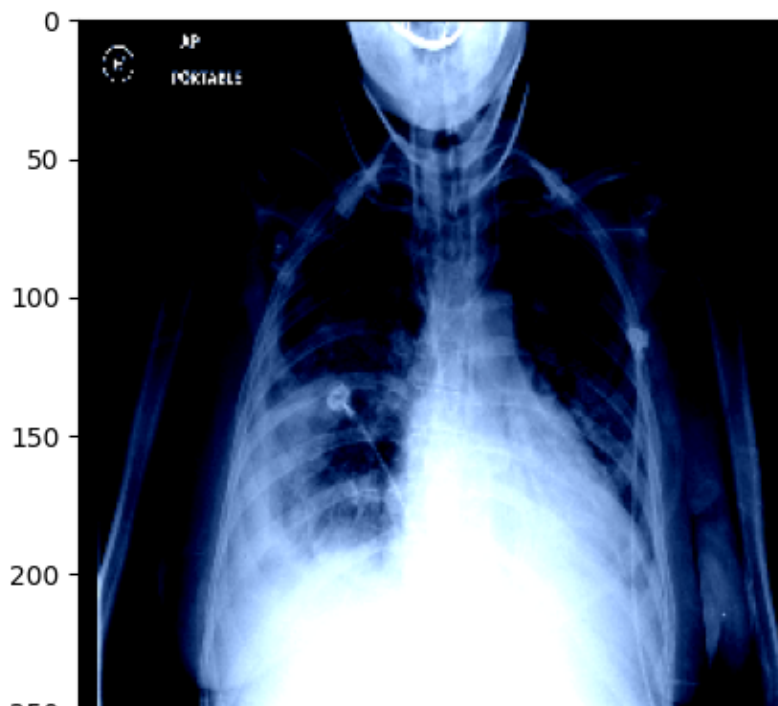
```
In [ ]: train_dataset[15][0].shape
```

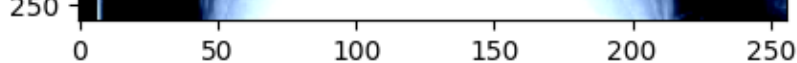
```
Out[ ]: (256, 256, 3)
```

```
In [ ]: plt.imshow(test_no_annot[10][0])
```

WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] f

```
Out[ ]: <matplotlib.image.AxesImage at 0x7a069d2c89d0>
```





```
In [ ]: img_test = train_dataset[2][0]
img_test = img_test.astype(np.uint8)
img_test.shape
```

```
Out[ ]: (256, 256, 3)
```

```
In [ ]: device = "cuda"
```

```
In [ ]: model = smp.Unet("efficientnet-b3", encoder_weights="imagenet", classes=3, in_channels=3, aux_p
model = model.to(device)
```

```
In [ ]: optim = torch.optim.Adam(model.parameters(), lr=1e-5)
criterion = torch.nn.BCEWithLogitsLoss()
sigmoid = torch.nn.Sigmoid()
```

```
In [ ]: def train(model, trainloader, epochs: int, valloader):
    model.train()
    train_loss_list = []
    train_accuracy_list = []
    val_loss_list = []
    val_accuracy_list = []

    for epoch in tqdm(range(epochs), position=0, leave=True, desc="Epochs"):
        total_samples = 0
        epoch_loss = 0
        num_matches = 0

        for data in trainloader:
            inputs, labels = data
            inputs = inputs.unsqueeze(1) # adds channel dimension to work with unet input
            inputs = inputs.permute(0, 4, 2, 3, 1)
            inputs = inputs.squeeze(-1) # this removes the unnecessary channel dimension
            inputs = inputs.to(device)
            inputs = inputs.to(torch.float32)
            labels = labels.to(device)

            optim.zero_grad()
            _, outputs = model(inputs)
            loss = criterion(outputs, labels.unsqueeze(1))

            loss.backward()
            optim.step()

            labels = labels.long()
            probabilities = sigmoid(outputs)
            # print("Labels", labels)
            threshold = 0.50
            predicted = (probabilities > threshold).float()
            predicted = predicted.view(-1,1)
            predicted = predicted.squeeze()
            # print("preds", predicted)
            matching_indices = (labels == predicted)
            # print("matching_indices", matching_indices)
            num_matches += matching_indices.sum().item()
            # print("num_matches", num_matches)
            total_samples += labels.size(0)
            # print("total_samples", total_samples)

            epoch_loss += loss.item()

    train_accuracy = num_matches / total_samples
    train_accuracy_list.append(train_accuracy)
```

```

train_accuracy_list.append(train_accuracy)
train_loss_list.append(epoch_loss)
#     print("Training - epoch num", epoch + 1, "done!")

# Validation
model.eval() # Set the model to evaluation mode
with torch.no_grad():
    val_loss = 0
    val_matches = 0
    val_samples = 0

    for data in valloader:
        inputs, labels = data
        inputs = inputs.unsqueeze(1) # adds channel dimension to work with unet input
        inputs = inputs.permute(0, 4, 2, 3, 1)
        inputs = inputs.squeeze(-1) # this removes the unnecessary channel dimension
        inputs = inputs.to(device)
        inputs = inputs.to(torch.float32)
        labels = labels.to(device)

        _, outputs = model(inputs)
        loss = criterion(outputs, labels.unsqueeze(1))

        labels = labels.long()
        #     print("Labels", labels)
        probabilities = sigmoid(outputs)
        threshold = 0.50
        predicted = (probabilities > threshold).float()
        predicted = predicted.view(-1,1)
        predicted = predicted.squeeze()
        matching_indices = (labels == predicted)

        val_matches += matching_indices.sum().item()
        val_samples += labels.size(0)
        val_loss += loss.item()

    val_accuracy = val_matches / val_samples
    val_accuracy_list.append(val_accuracy)
    val_loss_list.append(val_loss)

model.train()

return train_loss_list, train_accuracy_list, val_loss_list, val_accuracy_list

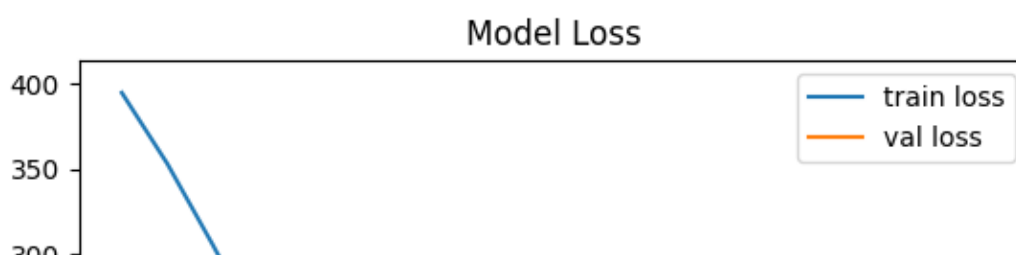
```

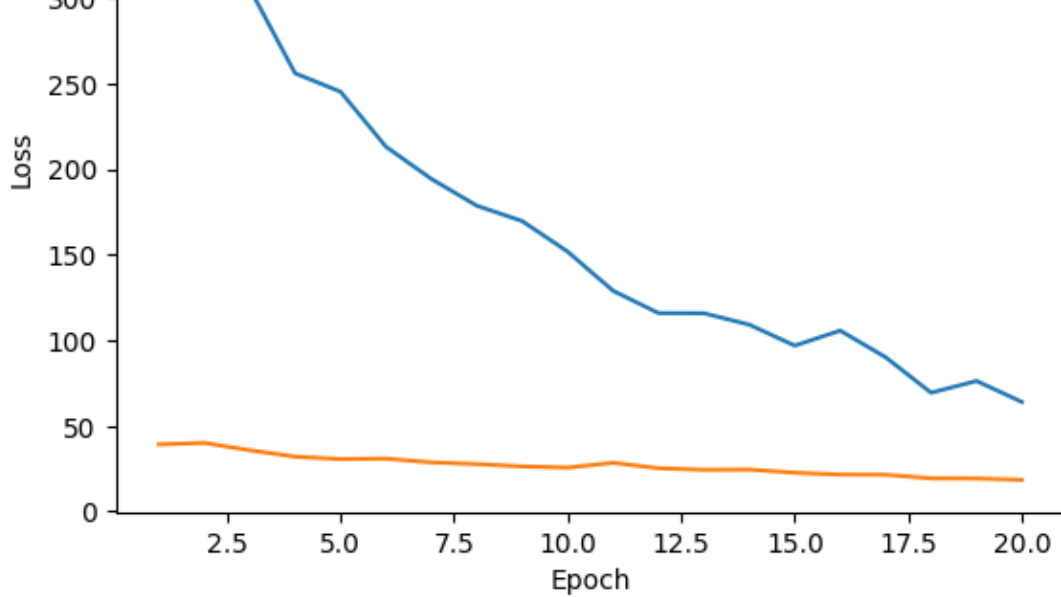
```
In [ ]: epochs = 25
```

```
In [ ]: train_loss, train_acc, val_loss, val_acc = train(model, train_loader, epochs, valid_loader)
```

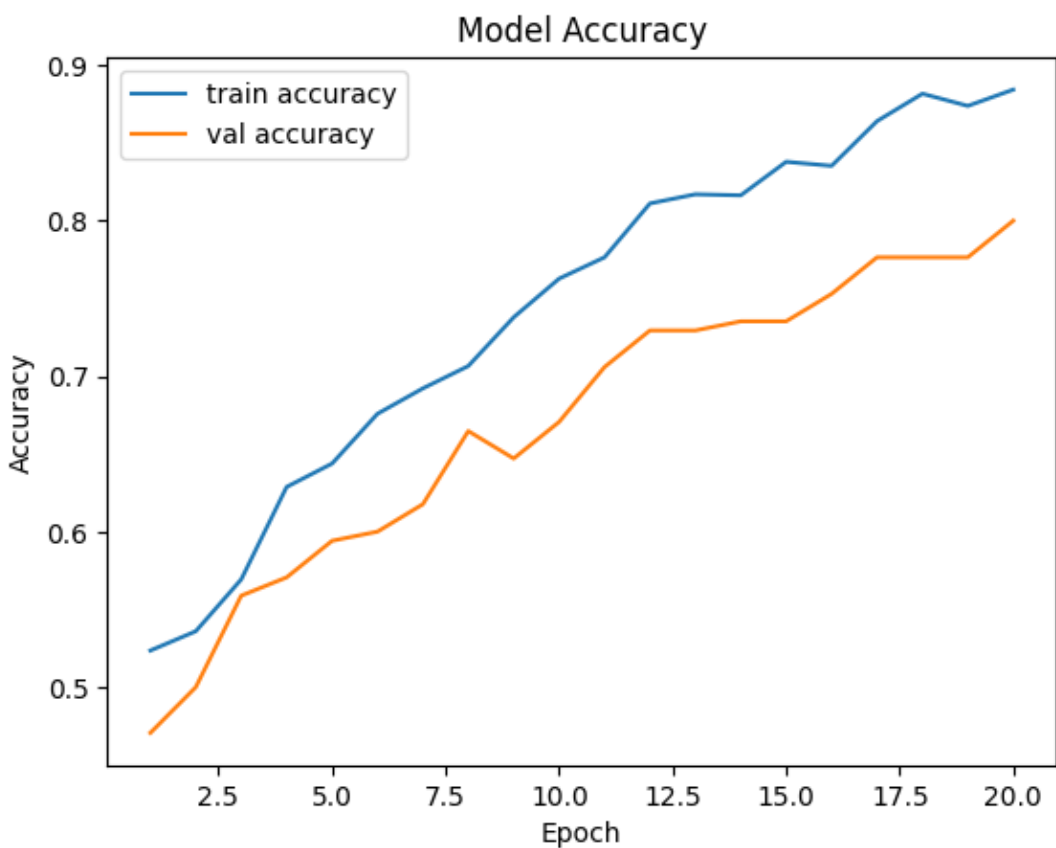
```
In [ ]: # custom_palette = ['#D3E0EA', '#AAB8C4', '#8498A2', '#5D7D88', '#3C6270',
#                       '#1F465E', '#132D4B', '#0A1C36', '#051024', '#020916']
```

```
In [ ]: plt.plot(range(1,epochs+1), train_loss, label="train loss")
plt.plot(range(1,epochs+1), val_loss, label="val loss")
plt.title("Model Loss")
plt.ylabel("Loss")
plt.xlabel("Epoch")
plt.legend();
```





```
In [ ]: plt.plot(range(1,epochs+1), train_acc, label = "train accuracy")
plt.plot(range(1,epochs+1), val_acc, label = "val accuracy")
plt.title("Model Accuracy")
plt.ylabel("Accuracy")
plt.xlabel("Epoch")
plt.legend();
```



```
In [ ]: print("train_loss", train_loss)
print("train_acc", train_acc)
print("valid_loss", val_loss)
print("valid_acc", val_acc)
```

```
train_loss [394.46800051629543, 353.11748134344816, 305.8045287504792, 256.15733351558447, 245.409
9.78691174834967, 151.90769063588232, 129.02073660842143, 115.99207521905191, 115.92130426433869,
8, 69.43906404697373, 76.3379799923132, 64.04926557370345]
train_acc [0.5235294117647059, 0.5359477124183006, 0.569281045751634, 0.6287581699346405, 0.643796
79084967320262, 0.7627450980392156, 0.7764705882352941, 0.8111111111111111, 0.8169934640522876, 0.
4, 0.8816993464052287, 0.8738562091503268, 0.884313725490196]
valid_loss [39.24534550309181, 40.13199880075455, 35.85023429989815, 32.084457870572805, 30.67246
413217663765, 25.68534190952778, 28.473002582788467, 25.334357224404812, 24.413095806259662, 24.52
9.409699855812505, 19.31693032954354, 18.540888933464885]
```

```
valid_acc [0.47058823529411764, 0.5, 0.5588235294117647, 0.5705882352941176, 0.5941176470588235, 0.62941176, 0.7058823529411765, 0.7294117647058823, 0.7294117647058823, 0.7352941176470589, 0.73529411764705882352941, 0.8]
```

```
In [ ]: torch.save(model.state_dict(), 'UNet_model_efficient_better.pth')
```

```
In [ ]: def evaluate(model, test_loader):
        """
        Evaluates the model on the given validation loader.

        Args:
        - model (torch.nn.Module): The model to evaluate.
        - val_loader (torch.utils.data.DataLoader): DataLoader for validation data.

        Returns:
        - Tuple[float, float]: Returns validation loss and accuracy.
        """

        model.eval()

        num_matches = 0
        total_samples = 0
        labels_list = []
        pred_list = []

        with torch.no_grad():
            for data in tqdm(test_loader):

                inputs, labels = data
                inputs = inputs.unsqueeze(1) # adds channel dimension to work with unet input
                inputs = inputs.permute(0, 4, 2, 3, 1)
                inputs = inputs.squeeze(-1) # this removes the unnecessary channel dimension
                inputs = inputs.to(device) # moved inputs to device
                inputs = inputs.to(torch.float32)
                labels = labels.to(device) # moved labels to device
                print("labels", labels)
                _, outputs = model(inputs)
                # print("labels", labels)

                labels = labels.long()
                threshold = 0.50
                probabilities = sigmoid(outputs)
                print(probabilities)
                predicted = (probabilities > threshold).float()
                predicted = predicted.view(-1,1)
                predicted = predicted.squeeze()
                labels_list.append(labels)
                pred_list.append(predicted)
                matching_indices = (labels == predicted)
                num_matches += matching_indices.sum().item()
                total_samples += labels.size(0)

        accuracy = num_matches / total_samples

        return accuracy, labels_list, pred_list
```

```
In [ ]: test_acc, labels, preds = evaluate(model, test_loader)
```

```
3%|          | 1/38 [00:00<00:16, 2.18it/s]
labels tensor([0., 1., 0., 1., 0., 0., 1., 0.], device='cuda:0')
tensor([[4.4658e-05],
        [9.9857e-01],
        [5.4186e-02],
        [9.7481e-01],
        [5.3744e-04],
        [3.5874e-04],
        [8.7535e-01],
        [1.2033e-08]], device='cuda:0')
```

```
5%|█| 2/38 [00:00<00:16, 2.24it/s]
labels tensor([1., 0., 1., 0., 0., 1., 1., 1.], device='cuda:0')
tensor([[9.5453e-01],
        [5.2274e-01],
        [9.9099e-01],
        [9.2101e-01],
        [2.0417e-07],
        [9.9381e-01],
        [9.9989e-01],
        [1.2935e-02]], device='cuda:0')

8%|█| 3/38 [00:01<00:15, 2.30it/s]
labels tensor([1., 1., 0., 0., 1., 0., 0., 0.], device='cuda:0')
tensor([[9.9345e-02],
        [9.4032e-01],
        [9.6463e-01],
        [1.6418e-01],
        [9.9916e-01],
        [2.7480e-05],
        [3.6969e-02],
        [3.4788e-11]], device='cuda:0')

11%|█| 4/38 [00:01<00:14, 2.39it/s]
labels tensor([0., 1., 0., 1., 1., 1., 1., 1.], device='cuda:0')
tensor([[0.9692],
        [0.9916],
        [0.0286],
        [0.9517],
        [1.0000],
        [1.0000],
        [0.4573],
        [0.9757]], device='cuda:0')

13%|█| 5/38 [00:02<00:14, 2.31it/s]
labels tensor([1., 1., 0., 1., 0., 0., 0., 1.], device='cuda:0')
tensor([[1.0000e+00],
        [9.9412e-01],
        [3.5836e-06],
        [9.9999e-01],
        [9.9879e-01],
        [7.5230e-05],
        [8.9477e-01],
        [4.8793e-01]], device='cuda:0')

16%|█| 6/38 [00:02<00:13, 2.31it/s]
labels tensor([1., 0., 1., 1., 0., 1., 1., 1.], device='cuda:0')
tensor([[0.9512],
        [0.0016],
        [0.9999],
        [0.9674],
        [0.3029],
        [0.9983],
        [0.8080],
        [0.0557]], device='cuda:0')

18%|█| 7/38 [00:03<00:13, 2.28it/s]
labels tensor([1., 0., 0., 1., 0., 1., 1., 0.], device='cuda:0')
tensor([[9.3817e-01],
        [9.5901e-01],
        [8.5369e-01],
        [7.1146e-01],
        [3.9855e-01],
        [9.9515e-01],
        [3.9995e-02],
        [8.5037e-07]], device='cuda:0')

21%|█| 8/38 [00:03<00:12, 2.33it/s]
labels tensor([0., 0., 0., 1., 0., 1., 0., 0.], device='cuda:0')
tensor([[1.8272e-07],
        [1.5846e-09],
        [2.1871e-01],
        [9.4465e-01],
        [9.5732e-01],
        [7.0730e-01],
        [2.9527e-01],
        [9.9198e-01]], device='cuda:0')
```

```
24% |███████| 9/38 [00:03<00:12, 2.35it/s]
labels tensor([0., 0., 1., 0., 1., 0., 0., 1.], device='cuda:0')
tensor([[1.1949e-03],
        [4.4541e-02],
        [6.8895e-01],
        [5.0905e-03],
        [1.0000e+00],
        [1.9970e-05],
        [4.7582e-03],
        [9.8273e-01]], device='cuda:0')
```

```
26% |███████| 10/38 [00:04<00:11, 2.35it/s]
labels tensor([0., 0., 1., 1., 0., 1., 1., 0.], device='cuda:0')
tensor([[9.5793e-01],
        [2.3935e-05],
        [8.4443e-01],
        [9.3879e-01],
        [2.3485e-05],
        [1.3555e-03],
        [9.7334e-01],
        [2.4188e-02]], device='cuda:0')
```

```
29% |███████| 11/38 [00:04<00:11, 2.37it/s]
labels tensor([1., 1., 0., 0., 0., 0., 0., 1.], device='cuda:0')
tensor([[9.9423e-01],
        [5.0071e-02],
        [1.0125e-03],
        [5.1735e-01],
        [6.0802e-01],
        [1.1345e-06],
        [9.9929e-01],
        [9.4550e-01]], device='cuda:0')
```

```
32% |███████| 12/38 [00:05<00:10, 2.38it/s]
labels tensor([1., 0., 0., 0., 1., 1., 0., 0.], device='cuda:0')
tensor([[9.0564e-01],
        [9.3656e-01],
        [9.5999e-01],
        [9.6099e-03],
        [9.5697e-01],
        [9.8457e-01],
        [2.0529e-05],
        [2.5029e-07]], device='cuda:0')
```

```
34% |███████| 13/38 [00:05<00:10, 2.35it/s]
labels tensor([1., 0., 0., 0., 1., 1., 1., 0.], device='cuda:0')
tensor([[6.8118e-01],
        [6.5264e-07],
        [2.4669e-05],
        [9.1142e-04],
        [1.7204e-01],
        [9.0784e-01],
        [8.2472e-01],
        [4.5900e-04]], device='cuda:0')
```

```
37% |███████| 14/38 [00:05<00:10, 2.35it/s]
labels tensor([1., 1., 1., 1., 1., 1., 1., 0.], device='cuda:0')
tensor([[0.3647],
        [1.0000],
        [0.9997],
        [1.0000],
        [0.9974],
        [0.0208],
        [0.9753],
        [0.0220]], device='cuda:0')
```

```
39% |███████| 15/38 [00:06<00:09, 2.32it/s]
labels tensor([0., 0., 1., 1., 0., 1., 1., 0.], device='cuda:0')
tensor([[1.2851e-10],
        [1.0503e-08],
        [9.9749e-01],
        [9.9951e-01],
        [1.9934e-04],
        [9.9197e-01],
        [1.2422e-01],
        [6.3782e-01]], device='cuda:0')
```

```
42% |███████| 16/38 [00:06<00:09, 2.42it/s]
```

```
42%|███████| 16/38 [00:06<00:09, 2.43it/s]
labels tensor([0., 0., 0., 1., 1., 0., 1., 1.], device='cuda:0')
tensor([[0.0048],
        [0.0019],
        [0.0014],
        [0.0344],
        [1.0000],
        [0.3420],
        [0.9999],
        [0.9682]], device='cuda:0')

45%|███████| 17/38 [00:07<00:08, 2.48it/s]
labels tensor([0., 1., 0., 0., 1., 1., 1., 0.], device='cuda:0')
tensor([[5.5632e-04],
        [6.8748e-01],
        [5.9475e-06],
        [3.1521e-05],
        [9.9956e-01],
        [8.2631e-01],
        [9.9826e-01],
        [8.3423e-08]], device='cuda:0')

47%|███████| 18/38 [00:07<00:08, 2.37it/s]
labels tensor([0., 0., 0., 0., 0., 1., 0., 1.], device='cuda:0')
tensor([[9.4541e-01],
        [2.1830e-04],
        [6.9344e-08],
        [5.1725e-04],
        [8.2750e-03],
        [9.9999e-01],
        [1.1015e-04],
        [9.9993e-01]], device='cuda:0')

50%|███████| 19/38 [00:08<00:09, 2.05it/s]
labels tensor([0., 1., 1., 0., 1., 1., 0., 1.], device='cuda:0')
tensor([[1.8347e-05],
        [1.0000e+00],
        [6.6601e-01],
        [8.4572e-05],
        [8.2895e-01],
        [9.9329e-01],
        [1.1250e-06],
        [9.9862e-01]], device='cuda:0')

53%|███████| 20/38 [00:08<00:09, 1.91it/s]
labels tensor([1., 0., 1., 1., 1., 0., 1., 0.], device='cuda:0')
tensor([[8.0150e-01],
        [1.1484e-04],
        [9.9994e-01],
        [5.2104e-01],
        [7.2037e-01],
        [3.0544e-04],
        [9.9644e-01],
        [8.6407e-01]], device='cuda:0')

55%|███████| 21/38 [00:09<00:09, 1.79it/s]
labels tensor([1., 1., 1., 0., 0., 0., 0., 0.], device='cuda:0')
tensor([[1.0000],
        [0.9941],
        [1.0000],
        [0.9979],
        [0.2506],
        [0.0014],
        [0.0315],
        [0.0316]], device='cuda:0')

58%|███████| 22/38 [00:10<00:09, 1.73it/s]
labels tensor([1., 0., 0., 1., 0., 0., 1., 0.], device='cuda:0')
tensor([[9.9928e-01],
        [4.1153e-02],
        [9.0676e-05],
        [9.9998e-01],
        [5.8613e-06],
        [2.6347e-01],
        [8.9885e-01],
        [2.7408e-02]], device='cuda:0')

61%|███████| 23/38 [00:10<00:09, 1.66it/s]
```

```
61%|███████| 25/38 [00:10<00:03, 1.66it/s]
labels tensor([1., 0., 1., 0., 0., 1., 0., 0.], device='cuda:0')
tensor([[9.9880e-01],
        [1.1260e-04],
        [9.9999e-01],
        [6.5173e-08],
        [6.1769e-01],
        [1.0000e+00],
        [2.1616e-03],
        [6.0722e-01]], device='cuda:0')

63%|███████| 24/38 [00:11<00:08, 1.66it/s]
labels tensor([1., 1., 0., 0., 0., 1., 1.], device='cuda:0')
tensor([[9.9981e-01],
        [6.2895e-01],
        [9.9736e-01],
        [9.8076e-04],
        [1.2091e-03],
        [3.9815e-03],
        [3.1611e-03],
        [9.9999e-01]], device='cuda:0')

66%|███████| 25/38 [00:11<00:07, 1.79it/s]
labels tensor([0., 1., 1., 1., 0., 1., 1., 0.], device='cuda:0')
tensor([[4.8342e-05],
        [2.9040e-05],
        [9.9957e-01],
        [1.9078e-02],
        [8.6270e-06],
        [9.9825e-01],
        [4.3052e-02],
        [3.5328e-05]], device='cuda:0')

68%|███████| 26/38 [00:12<00:06, 1.93it/s]
labels tensor([0., 1., 1., 1., 1., 1., 0., 0.], device='cuda:0')
tensor([[7.2543e-03],
        [6.8873e-02],
        [1.0000e+00],
        [9.9943e-01],
        [9.9208e-01],
        [1.0000e+00],
        [9.4536e-01],
        [1.9886e-04]], device='cuda:0')

71%|███████| 27/38 [00:12<00:05, 2.02it/s]
labels tensor([0., 1., 0., 1., 1., 1., 1., 0.], device='cuda:0')
tensor([[0.0038],
        [0.9993],
        [0.8635],
        [0.9560],
        [0.9997],
        [1.0000],
        [1.0000],
        [0.0105]], device='cuda:0')

74%|███████| 28/38 [00:13<00:04, 2.13it/s]
labels tensor([0., 1., 1., 0., 1., 1., 1., 0.], device='cuda:0')
tensor([[5.7911e-06],
        [8.5019e-01],
        [9.6399e-01],
        [5.2357e-02],
        [8.7799e-01],
        [9.9788e-01],
        [1.1748e-02],
        [9.7183e-01]], device='cuda:0')

76%|███████| 29/38 [00:13<00:04, 2.15it/s]
labels tensor([0., 0., 1., 1., 0., 1., 0., 1.], device='cuda:0')
tensor([[4.5862e-06],
        [2.5767e-02],
        [9.8773e-01],
        [6.2733e-01],
        [1.6830e-01],
        [9.9719e-01],
        [6.6997e-03],
        [9.9992e-01]], device='cuda:0')

79%|███████| 30/38 [00:14<00:03, 2.20it/s]
```



```
labels tensor([0., 0., 1., 0., 1., 0., 1.], device='cuda:0')
tensor([[3.7331e-04],
        [7.9426e-04],
        [3.1021e-01],
        [9.9850e-01],
        [2.8474e-06],
        [2.6180e-05],
        [9.5817e-01],
        [9.9964e-01]], device='cuda:0')

82% |██████████| 31/38 [00:14<00:03, 2.30it/s]
labels tensor([1., 0., 1., 1., 0., 1., 1., 0.], device='cuda:0')
tensor([[2.2391e-01],
        [7.8441e-03],
        [1.0000e+00],
        [9.9989e-01],
        [1.0317e-04],
        [9.9972e-01],
        [9.9999e-01],
        [3.3265e-02]], device='cuda:0')

84% |██████████| 32/38 [00:14<00:02, 2.26it/s]
labels tensor([1., 0., 0., 1., 0., 1., 0., 0.], device='cuda:0')
tensor([[1.4624e-01],
        [1.2944e-04],
        [6.0626e-01],
        [7.9606e-01],
        [3.3316e-06],
        [8.4300e-01],
        [9.3378e-06],
        [1.3236e-02]], device='cuda:0')

87% |██████████| 33/38 [00:15<00:02, 2.31it/s]
labels tensor([1., 1., 0., 0., 1., 0., 1., 0.], device='cuda:0')
tensor([[5.4946e-01],
        [9.6416e-01],
        [5.6577e-01],
        [6.7751e-05],
        [1.2860e-03],
        [1.1949e-08],
        [3.4548e-01],
        [9.9775e-01]], device='cuda:0')

89% |██████████| 34/38 [00:15<00:01, 2.32it/s]
labels tensor([0., 0., 1., 1., 1., 0., 0., 0.], device='cuda:0')
tensor([[1.6956e-01],
        [2.5364e-03],
        [9.9973e-01],
        [9.9464e-01],
        [9.9669e-01],
        [1.6650e-04],
        [5.2562e-05],
        [3.9553e-02]], device='cuda:0')

92% |██████████| 35/38 [00:16<00:01, 2.40it/s]
labels tensor([1., 0., 0., 0., 1., 1., 0., 1.], device='cuda:0')
tensor([[2.6670e-03],
        [1.2750e-04],
        [3.9263e-02],
        [4.1323e-05],
        [1.0000e+00],
        [9.9997e-01],
        [6.8956e-06],
        [9.5155e-01]], device='cuda:0')

95% |██████████| 36/38 [00:16<00:00, 2.41it/s]
labels tensor([0., 0., 1., 0., 0., 1., 1., 0.], device='cuda:0')
tensor([[9.8093e-01],
        [6.2354e-02],
        [9.9984e-01],
        [9.1158e-01],
        [6.7100e-05],
        [6.0237e-01],
        [9.9924e-01],
        [1.7054e-07]], device='cuda:0')

97% |██████████| 37/38 [00:16<00:00, 2.45it/s]
```

```

labels tensor([1., 1., 0., 0., 0., 0., 1., 1.], device='cuda:0')
tensor([[9.9997e-01],
        [9.5877e-01],
        [1.3543e-04],
        [5.5927e-03],
        [9.9966e-01],
        [3.0701e-01],
        [7.7974e-01],
        [9.4908e-01]], device='cuda:0')
100% ██████████ 38/38 [00:17<00:00, 2.21it/s]
labels tensor([1., 0., 0., 1.], device='cuda:0')
tensor([[0.0216],
        [0.4778],
        [0.2626],
        [0.0012]], device='cuda:0')

```

```
In [ ]: test_acc
```

```
Out[ ]: 0.8
```

```

In [ ]: preds_tensor = torch.cat(preds).cpu().numpy()
labels_tensor = torch.cat(labels).cpu().numpy()

# Compute the confusion matrix
cm = confusion_matrix(labels_tensor, preds_tensor)

# Visualize the confusion matrix
plt.figure(figsize=(5, 5))
sns.heatmap(cm, annot=True, fmt='g', cmap='Blues', cbar=False)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

```

